ПРОЕКТИРОВАНИЕ ВЫСОКОНАГРУЖЕННЫХ СИСТЕМ · СИСТЕМНЫЙ ДИЗАЙН

MeetFlow: подготовка к защите

Системный дизайн с нуля — и вся работа по шагам: откуда каждое число, куда идут данные, что говорить

Zoom-подобный сервис видеовстреч · 8 дата-центров · 16.9 млн участников онлайн в пик
Работа: README.md, ветка `review-fixes`, коммит `226cba1`
В приложениях: полный текст работы со всеми схемами, ревью, исходники схем БД, скрипт расчёта
11.10.2026

Содержание

Часть 0. Как пользоваться документом

0.1. Что внутри
0.2. Как проходит защита
0.3. Где лежат файлы работы

Часть 1. Системный дизайн простыми словами

1.1. Что это за дисциплина
1.2. Цепочка рассуждений
1.3. Шесть главных идей
1.4. Что требует методичка курса

Часть 2. Работа на одной странице

2.1. Что такое MeetFlow
2.2. Главные числа
2.3. Общая схема
2.4. Восемь главных идей архитектуры
2.5. Три самые тяжёлые вещи

Часть 3. Откуда что берётся

3.1. Какие бывают источники и почему им верят
3.2. Исходные числа аудитории
3.3. Технические исходные числа
3.4. Бенчмарки — пределы одного сервера
3.5. Цены
3.6. Проектные допущения

Часть 4. Путь данных: откуда куда что идёт

4.1. Как клиент находит дата-центр
4.2. Логин
4.3. Создание встречи
4.4. Вход во встречу (join)
4.5. Видео и звук (медиа)
4.6. Присутствие и heartbeat
4.7. Чат
4.8. Запись встречи
4.9. Список встреч пользователя
4.10. Что копируется между дата-центрами
4.11. Что происходит при поломках
4.12. Пользователь залогинился в США и улетел в Японию

Часть 5. Разбор работы по разделам

Раздел 1. Тема и целевая аудитория
Раздел 2. Расчёт нагрузки
Раздел 3. Глобальная балансировка
Раздел 4. Локальная балансировка
Раздел 5. Логическая схема БД
Раздел 6. Физическая схема БД

Раздел 7. Алгоритмы
Раздел 8. Технологии
Раздел 9. Надёжность
Раздел 10. Схема проекта
Раздел 11. Серверы и стоимость

Часть 6. Замечания ревьюера: что спросили и как отвечать

Часть 7. Словарь терминов

7.1. Нагрузка и расчёты
7.2. Сеть и балансировка
7.3. Медиа
7.4. Данные
7.5. Инфраструктура и надёжность
7.6. Источники и экономика

Часть 8. Вопросы и ответы

8.1. Нагрузка
8.2. Дата-центры и балансировка
8.3. Точки записи и консистентность
8.4. Медиа
8.5. Балансировщики
8.6. Базы данных
8.7. Деньги

Часть 9. Шпаргалка и речь на 5 минут

9.1. Числа на одну страницу
9.2. Речь на 5 минут
9.3. Как отвечать, если не знаете ответ

Источники

Приложения (после части 9): А — полный текст работы (README.md) со всеми схемами и источниками; Б — ревью: что исправлено; В — исходный код схем БД (DBML); Г — скрипт расчёта.

Системный дизайн с нуля — и вся работа по шагам: откуда каждое число, куда идут данные, что говорить

Документ для автора работы. Он написан для новичка: после него нужно понимать каждое решение работы и уметь объяснить его своими словами. Числа совпадают с README.md ветки `review-fixes` (коммит `226cba1`), все расчёты повторяет скрипт `meetflow_calc.py`.

Часть 0. Как пользоваться документом

0.1. Что внутри

Часть	О чём	Когда читать
1. Системный дизайн простыми словами	Что это за дисциплина, как устроено рассуждение, что требует методичка курса	Первой, целиком
2. Работа на одной странице	Что такое MeetFlow, главные числа, общая схема, главные идеи	Сразу после части 1
3. Откуда что берётся	Каждое исходное число: источник, кто его опубликовал, почему ему можно верить	Перед частью 5
4. Путь данных	Пошагово: что происходит при логине, создании встречи, входе, видео, чате, записи, отказе ДЦ, переезде пользователя	Главная часть для понимания
5. Разбор работы по разделам 1–11	Что требует методичка, что сделано, как посчитано, почему так, **что говорить**, какие вопросы зададут	С README рядом
6. Замечания ревьюера	Что спросили, что изменили, как отвечать	Перед защитой
7. Словарь терминов	Все термины работы простыми словами	По мере необходимости
8. Вопросы и ответы	Вопросы, которые вероятнее всего зададут, с готовыми ответами	Перед защитой
9. Шпаргалка и речь на 5 минут	Числа на одну страницу и текст выступления	В день защиты
Приложения (в PDF-версии)	Полный текст работы со всеми схемами, ревью, исходники схем БД, скрипт расчёта	Как справочник

0.2. Как проходит защита

По методичке курса (init/highload) защита — это:

- **5 минут презентации** (5–15 слайдов) и **5 минут ответов** на вопросы преподавателя;
- презентуете **другому преподавателю**, не тому, кто проверял работу;
- две оценки по 0–10 баллов: **качество проекта** (инженерные решения, глубина проработки, правильность расчётов) и **качество презентации** (понятная подача, правильный фокус, правильные данные на слайдах).

Отсюда главное правило подготовки: **проверяют понимание, а не память**. Если вы можете объяснить, *почему* число такое и *что будет, если оно изменится*, вы отвечаете на любой вариант вопроса.

0.3. Где лежат файлы работы

Файл	Что это
`README.md`	Сама работа — расчётно-пояснительная записка по разделам 1–11 и список источников
`resource/calc/meetflow_calc.py`	Скрипт, который считает все числа работы. Меняешь допущение — получаешь все таблицы заново
`resource/calc/imf_weo_2026apr_gdp_2025.json`	ВВП стран из базы МВФ — на нём построены доли дата-центров
`resource/schemes/full-scheme.drawio`	Исходник общей схемы проекта (раздел 10), открывается в diagrams.net
`resource/icons/images/full-scheme.svg`	Картинка общей схемы
`resource/logic-database-sql.md`, `resource/physical-database-sql.md`	Код логической и физической схем БД на языке DBML (для dbdiagram.io)

`resource/icons/images/logic-database.svg`, `physical-database.svg`	Картинки схем БД
`docs/ревью.md`	Что было неверно в прошлой версии и что исправлено
`docs/подготовка-к-защите.md`	Этот документ

Часть 1. Системный дизайн простыми словами

1.1. Что это за дисциплина

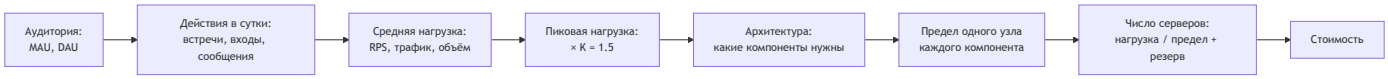
Системный дизайн — это проектирование системы, которая выдержит заданную нагрузку. Нужно пройти путь от вопроса «сколько у нас пользователей?» до ответа «какие нужны компоненты, сколько серверов, где они стоят, что будет при поломке и сколько это стоит».

Код в системном дизайне почти не пишут. Пишут **расчёты и обоснования**. Хорошая работа отличается от плохой тем, что каждое решение в ней следует из числа:

- слабо: «Возьмём Redis, потому что он быстрый»;
- сильно: «Нужно 738 тысяч операций в секунду с маленькими ключами, которые должны сами удаляться по таймеру. Один шард Redis даёт около 120 тысяч операций в секунду, значит, нужно не меньше 7 шардов на весь мир».

1.2. Цепочка рассуждений

Любая работа по системному дизайну устроена как одна цепочка. В MeetFlow она выглядит так:



Если на защите вы потеряли нить, вернитесь к этой цепочке: любой вопрос относится к одному из её звеньев.

1.3. Шесть главных идей

1. Нагрузку описывают четыре числа.

Число	Что значит	Пример в MeetFlow
RPS (requests per second)	Сколько запросов в секунду приходит к сервису	675 000 heartbeat-сообщений в секунду в пик
Трафик, Гбит/с	Сколько данных в секунду идёт по сети	78 131 Гбит/с видео и аудио в пик ≈ 78 Тбит/с
Объём хранения, ТБ/ПБ	Сколько данных лежит на дисках	685 ПБ записей встреч за 30 дней
Одновременные соединения	Сколько клиентов держат открытое соединение	16.9 млн WebSocket-соединений в пик

2. Считаем пик, а не среднее. Днём пользователей больше, ночью меньше. Серверы должны выдержать самый нагруженный час. Отношение пикового часа к среднему за сутки называют **коэффициентом пика K**. В работе K = 1.5: пиковый час в полтора раза нагруженнее среднего.

3. У каждого сервера есть предел, и он что-то одно. Сервер упирается либо в процессор (CPU), либо в память (RAM), либо в сеть, либо в диск. Этот предел называют **ограничителем**. Число серверов = нагрузка / предел одного сервера, округлённое вверх, плюс резерв. Главный навык — найти, во что именно упирается компонент. Пример из работы: балансировщик NGINX упирается не в число запросов, а в **память под одновременные соединения**; медиасервер SFU — не в сетевую карту, а в **процессор**.

4. Всё ломается, и у каждой поломки свой масштаб.

Что сломалось	Как переживаем
Один сервер	Запасной сервер (**N+1**)
Здание, стойка, зона доступности (AZ)	Серверы разнесены по 3 зонам; копии данных — в разных зонах

Целый регион (дата-центр)	Глобальная балансировка уводит пользователей в другие регионы

5. Данные — самое сложное. Сервис без данных можно просто запустить в двух экземплярах. С данными нужно решить: где они лежат, сколько копий, **кто может писать** (точка записи), насколько свежие копии видят читатели (**консистентность**), как разрезать данные на части, когда они не помещаются на один сервер (**шардирование**).

6. Всё — компромиссы. Надёжность стоит денег, скорость — сложности, свежесть данных — задержки. Сильный ответ на защите называет компромисс вслух: «Записи не копируем в другой регион: копия 685 ПБ удвоила бы самую дорогую статью. Цена решения — при гибели целого региона его записи потеряются».

1.4. Что требует методичка курса

Методичка — homework_architecture.md. Ниже — её требования своими словами и где они выполнены.

Раздел	Что требует методичка	Где в работе
Общее	README.md в публичном репозитории GitHub, пронумерованный список источников в конце	Весь README, 103 источника
1. Тема и аудитория	Тип сервиса, реальные аналоги, аудитория от 1 млн в день, её размер и местоположение, 3–7 ключевых функций MVP, ключевые продуктовые решения	1.1–1.5
2. Расчёт нагрузки	MAU, DAU, средний объём хранилища пользователя по типам (в штуках и ГБ), действия пользователя в день по типам — **сводной таблицей**. Объём хранения по типам (в штуках и ТБ), пиковый и суточный трафик по типам, RPS по методам — средний и пиковый, сводной таблицей. **SimilarWeb запрещён как первоисточник**, для перепечаток нужен оригинал	2.1–2.4
3. Глобальная балансировка	Разбиение по доменам, обоснование расположения ДЦ (влияние на продуктовые метрики), распределение запросов по ДЦ, схемы DNS- и Anycast-балансировки, механизм регулировки трафика между ДЦ	3.1–3.9
4. Локальная балансировка	Схема балансировки со всеми уровнями и механизмами резервирования, формула резервирования (N×2 или N+1), расчёт числа балансировщиков (ограничители — SSL termination и сеть)	4.1–4.6
5. Логическая схема БД	Картинка без привязки к СУБД, таблица с описанием таблиц, **все** данные — включая файлы, кеши и буферы, — размеры, QPS на чтение и запись, требования к консистентности, распределение нагрузки по ключам	5.1–5.4
6. Физическая схема БД	Картинка с привязкой к СУБД, индексы, денормализация, выбор СУБД по таблицам, шардирование и резервирование по таблицам, клиентские библиотеки, балансировка запросов и мультиплексирование подключений, **схема резервного копирования**	6.1–6.12
7. Алгоритмы	Необязательный раздел. Для каждого алгоритма: задача, входные данные, шаги, структуры данных, альтернативы, обоснование параметров, влияние на нагрузку и схему БД	7.1–7.3
8. Технологии	Сводная таблица: технология, где применяется, почему выбрана	8.1–8.2
9. Надёжность	Сводная таблица по всем компонентам со способами резервирования	9.1–9.2
10. Схема проекта	Картинка взаимодействия сервисов, пояснения, потоки данных, внутренняя и внешняя балансировка	10.1–10.5
11. Список серверов	Конфигурации, количество серверов, что на них работает; для Kubernetes — контейнеры с ресурсами	11.1–11.4

Часть 2. Работа на одной странице

2.1. Что такое MeetFlow

MeetFlow — учебный проект сервиса групповых видеовстреч, как Zoom, Microsoft Teams или Google Meet. Шесть функций MVP (минимального продукта):

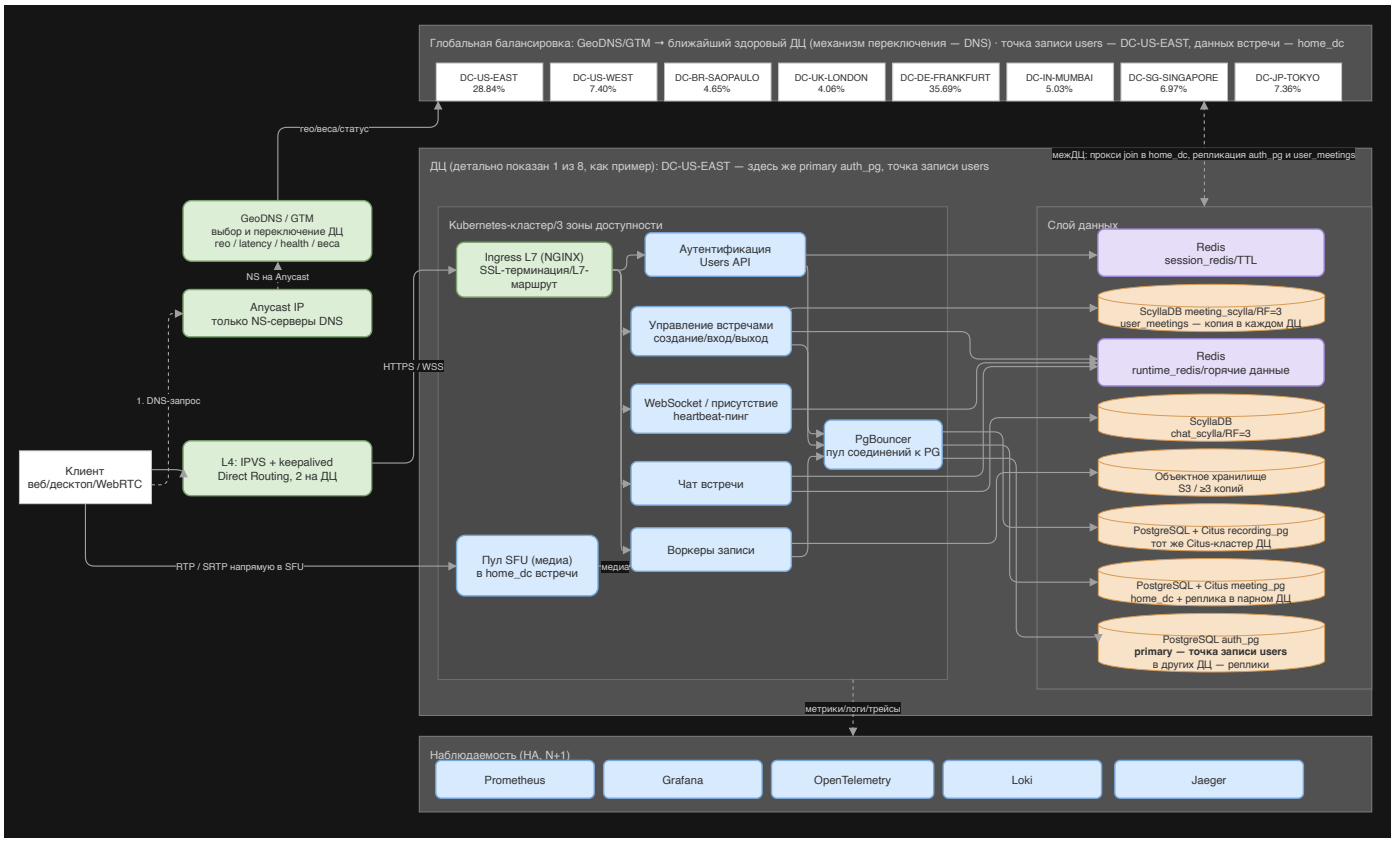
1. создать встречу — сразу или запланировать;
2. войти во встречу по ссылке-приглашению;
3. групповая аудио- и видеосвязь в качестве 720p;
4. демонстрация экрана;
5. чат во время встречи;
6. облачная запись встречи.

Аудитория — весь мир, кроме материкового Китая. Масштаб взят у реальных аналогов: **320 млн пользователей в месяц** (Teams), **115 млн в день** (Teams), **300 млн участий во встречах в день** (Zoom).

2.2. Главные числа

Что	Число	Откуда
Пользователей в месяц / в день	320 млн / 115 млн	Microsoft (Teams)
Участий во встречах в сутки	300 млн	Zoom, документ в SEC
Средняя встреча	10 человек, 54 минуты	Блог Zoom
Встреч в сутки	30 млн	300 млн / 10
Коэффициент пика K	1.5	Замечание ревьюера
Участников онлайн в среднем / в пик	11.25 млн / 16.9 млн	Расчёт, п. 2.2
Видео и аудио в пик	78 Тбит/с	Онлайн × битрейт по требованиям Zoom
Heartbeat в пик	675 тыс. в секунду	Сообщение «я ещё здесь» раз в 25 с
Записи встреч	685 ПБ за 30 дней	Худший случай: записывается каждая встреча
Дата-центров	8	Доли по ВВП стран (МВФ)
Серверов	30 047, из них 28 570 — медиасерверы SFU	Раздел 11
Стоимость	≈ 1 032 млн ₽/мес ≈ $12.2 млн	Цены Selectel, раздел 11.4

2.3. Общая схема



Как читать: **зелёное** — сеть и балансировка, **синее** — сервисы на C++23, **оранжевое** — постоянные хранилища, **фиолетовое** — Redis (быстрые данные в памяти). Подробно нарисован один дата-центр, `DC-US-EAST`; остальные 7 устроены так же. Клиент сначала спрашивает DNS, в какой ДЦ идти, затем идёт в балансировщик L4, оттуда в NGINX L7 и в сервисы. Видео идёт отдельно и напрямую — в медиасервер SFU.

2.4. Восемь главных идей архитектуры

Идея	Суть	Зачем
Control plane ≠ media plane	Управляющие запросы (вход, чат, heartbeat) идут через балансировщики, видео — напрямую в SFU	Балансировщикам не нужно пропускать 78 Тбит/с; встреча живёт на одном SFU
`home_dc` — «домашний» ДЦ встречи	Встреча закреплена за ДЦ, где её создали; код ДЦ зашит в `meeting_id`	Все участники встречи — на одном SFU, все её данные — в одном месте
Одна точка записи на объект	`users` пишутся в один основной сервер на весь мир, данные встречи — в `home_dc`	Не бывает двух одновременных записей одного объекта в разных ДЦ
ДЦ выбирает DNS	GeoDNS отвечает каждому пользователю адресом ближайшего здорового ДЦ; Anycast — только у самих DNS-серверов	Можно управлять весами и выводить ДЦ из работы
SFU, а не MCU	Сервер пересылает видеопотоки, а не склеивает их	Нет дорогого перекодирования на сервере
Simulcast и ABR	Клиент шлёт видео в нескольких качествах, сервер отдаёт каждому подходящее и снижает качество при плохой сети	Меньше трафика, звонок не рвётся на плохой сети
Своя СУБД под каждый тип данных	PostgreSQL — где нужны транзакции, ScyllaDB — где много записей, Redis — горячие данные, S3 — файлы	Каждое хранилище делает то, что умеет лучше всего

Резерв без двойного комплекта	При отказе ДЦ на время аварии снижаем качество видео вместо того, чтобы держать вторую копию всех серверов	Двойной комплект SFU стоил бы ещё ≈ 400 млн ₽ в месяц

2.5. Три самые тяжёлые вещи

Архитектуру и бюджет определяют три блока, а не API-запросы:

1. **Медиа**: 78 Тбит/с → 28 570 серверов SFU → 400 млн ₽ в месяц.
2. **Записи**: 685 ПБ → хранилище S3 → 610 млн ₽ в месяц.
3. **Присутствие (presence)**: 16.9 млн открытых соединений и 675 тыс. heartbeat в секунду → Redis, WebSocket-поды и NGINX, упирающийся в память.

Всё остальное — вход, чат, списки встреч — это тысячи или десятки тысяч запросов в секунду и меньше 1% бюджета.

Часть 3. Откуда что берётся

На защите почти любое число могут спросить: «А это откуда?». На этот вопрос есть всего три ответа:

- **из первоисточника** — официальный отчёт компании, официальная документация, стандарт, государственная статистика;
- **из бенчмарка** — опубликованный замер производительности;
- **из нашего допущения** — тогда нужно назвать его допущением и сказать, чем оно ограничено.

3.1. Какие бывают источники и почему им верят

Тип источника	Примеры в работе	Почему ему можно верить
Отчётность публичной компании	Стенограмма звонка Microsoft с инвесторами, проспект Zoom в SEC, годовой отчёт Zoom 10-K, пресс-релиз Zoom о выручке	За ложь в отчётности перед инвесторами компании отвечают по закону. SEC — комиссия по ценным бумагам США
Официальная документация продукта	Требования Zoom к сети, лимиты чата Zoom, документация Redis, PostgreSQL, ScyllaDB, Citus, NGINX, Kubernetes, mediasoup	Производитель описывает свой продукт
Стандарты	RFC (стандарты интернета), ITU-T G.114	Принятые международные документы
Государственная и межгосударственная статистика	МВФ (ВВП стран), перепись США, курс ЦБ РФ	Официальные данные
Бенчмарки	NGINX (запросы и TLS на ядро), Jitsi и LiveKit (пропускная способность SFU), redis-benchmark	Опубликованный замер с описанием стенда
Прайс-лист	Selectel: серверы, облако, S3, трафик	Реальные цены, по которым можно купить
Проектное допущение	K = 1.5, 5 сообщений чата на участие, 3 открытия списка встреч в день	Публичных данных нет — значение выбрано и подписано как допущение

Важное правило методички. Данные SimilarWeb запрещено использовать как первоисточник: у этого сервиса слишком большая погрешность. Перепечатки (новостные статьи, форумы) нужно заменять оригиналом. В прошлой версии работы доли дата-центров строились на SimilarWeb, а аудитория Teams и Zoom цитировалась по форуму и Reuters. Сейчас это исправлено: доли считаются по ВВП (МВФ), аудитория — по отчётности Microsoft и Zoom.

3.2. Исходные числа аудитории

Число	Значение	Источник	Что сказать, если спросят
MAU — пользователей в месяц	320 млн	Microsoft, звонок с инвесторами по итогам квартала FY24 Q1 (октябрь 2023): у Teams «more than 320 million monthly active users» [1]	«Это масштаб лидера рынка — Teams. Более свежей публичной цифры MAU нет»
DAU — пользователей в день	115 млн	Блог Microsoft 365, октябрь 2020: у Teams 115 млн DAU [2]	«Это публичное число октября 2020 года, год подписываем. Берём его как консервативную оценку: DAU влияет только на лёгкие запросы — логин и список встреч, а тяжёлую нагрузку задают участия»
DMP — участий во встречах в день	300 млн	Проспект Zoom в SEC (январь 2021): в апреле 2020 платформа превысила 300 млн ежедневных участников встреч [3]	«Это не уникальные пользователи, а участия: один человек за день может побывать на трёх встречах и посчитается три раза. Поэтому участий больше, чем DAU»

Средняя встреча	10 участников, 54 минуты	Блог Zoom, данные за 15.11.2020–15.11.2021 [4]	«Средние значения Zoom за год. От них считается вся нагрузка на встречи»
Актуальность ниши	$4.87 млрд выручки Zoom за FY2026, 4 468 крупных клиентов	Пресс-релиз Zoom [9]	«Старые числа использования подписаны годом, а актуальность ниши подтверждаем финансами 2026 года»

Почему числа разных лет и разных компаний. Публично раскрыты только отдельные цифры: Microsoft даёт MAU и DAU, Zoom — участия и длительность встреч. Мы берём каждую там, где она есть, и подписываем год. На вопрос «не смешиваете ли вы разное?» ответ: «Каждая метрика взята из своего источника и используется только для своей цели: MAU — для объёма профилей, DAU — для действий пользователя, участия — для нагрузки на встречи».

3.3. Технические исходные числа

Число	Значение	Источник	Зачем нужно
Видео 720p в групповом звонке	отправка 2.6 Мбит/с, приём 1.8 Мбит/с	Требования Zoom к сети [5]	Трафик видео
Видео «high-quality» (ниже 720p)	1.0 / 0.6 Мбит/с	Там же [5]	Профиль, до которого снижаем качество при отказе ДЦ
Аудио	60–80 кбит/с, берём 0.08 Мбит/с	Там же [5]	Трафик аудио
Демонстрация экрана	50–150 кбит/с, берём 0.15 Мбит/с	Там же [5]	Трафик демонстрации, верхняя граница
Интервал heartbeat	25 секунд	Значение по умолчанию `pingInterval` в Socket.IO [8]	Число heartbeat-сообщений
Лимит сообщения чата	4 000 символов	Zoom Team Chat [6]	Максимальный размер сообщения
Квота записей	10 ГБ на платного пользователя	Zoom [7]	Сравнение с нашим объёмом записей
ВВП стран, 2025 год	194 экономики, 98 537.9 млрд $ без Китая	МВФ, World Economic Outlook, апрель 2026 [10]	Доли дата-центров
Население запада США	23.7%	Перепись США 2020 [18]	Деление США на два ДЦ
Выручка Zoom по регионам	Америка 72.1%, EMEA 15.8%, APAC 12.1%	Годовой отчёт Zoom 10-K за FY2026 [17]	Только для сверки долей ДЦ
Пинг Токио ↔ Вашингтон	≈ 153 мс	WonderNetwork [24]	Сценарий «США → Япония»

3.4. Бенчмарки — пределы одного сервера

Компонент	Предел	Источник	Как использован
NGINX, HTTPS-запросы по открытым соединениям	341 232 в секунду на 16 ядрах	Тест NGINX Ingress Controller [11]	Число узлов L7 по запросам
NGINX, новые TLS-	6 676 в секунду на 16 ядрах	Тест NGINX [12]	Число узлов L7 по

соединения			рукопожатиям
NGINX, сеть	8.8 Гбит/с	Тест NGINX Ingress [11]	Число узлов L7 по трафику
NGINX, память на WebSocket-соединение	меньше 20 КБ (50 000 соединений < 1 ГБ)	Тест NGINX WebSocket [21]	Число узлов L7 по памяти
TLS-буферы OpenSSL	до 34 КБ на соединение	Документация OpenSSL [22]	Худший случай памяти на соединение
SFU на ядро процессора	≈ 0.678 Гбит/с при 100% CPU — наша линейная экстраполяция замера	Тест Jitsi Videobridge: 550.4 Мбит/с при 20.3% CPU четырёхъядерного сервера [19]	Число медиасерверов
SFU, сверка	≈ 0.29 Гбит/с на ядро для видеопакетов, ≈ 0.08 для мелких	Тест LiveKit [20]	Показывает, что оценка Jitsi оптимистичная
Redis, один шард	72–180 тыс. запросов в секунду без пайплайнинга, берём 120 тыс.	Документация redis-benchmark [23]	Число узлов Redis
ScyllaDB, заполнение диска	не больше 50%: при compaction STCS в худшем случае нужна свободной половина диска	Документация ScyllaDB [26]	≤ 1 ТБ данных на узел с дисками 2 × 1 ТБ
IPVS, память на соединение	136 байт	Исходный код ядра Linux [25]	Размер таблицы соединений балансировщика L4

3.5. Цены

Что	Цена	Источник
Сервер EL11-SSD-10GE (6 ядер, 32 ГБ, 10 Гбит/с) — SFU, запись, L4	14 000 ₽/мес	Selectel [13]
Сервер CL34-NVMe (14 ядер, 64 ГБ, 2 × 1 ТБ NVMe) — сервисы, Redis, ScyllaDB, мониторинг	13 600 ₽/мес	Selectel [13]
Сервер BL18-NVMe (16 ядер, 128 ГБ) — PostgreSQL	34 900 ₽/мес	Selectel [13]
Облачный сервер L7: vCPU / ГБ RAM / ГБ SSD	735.19 / 267.34 / 10.48 ₽/мес	Selectel [14]
Managed Kubernetes с 3 master-нодами	17 085.59 ₽/мес	Selectel [15]
S3, ледяное хранение	0.89 ₽ за ГБ в месяц	Selectel [16]
Курс доллара	84.90 ₽	ЦБ РФ на 10.10.2026 [27]

Selectel — **ценовой ориентир**: его дата-центров в наших восьми городах нет, но публичный прайс даёт реальные цены на сопоставимое железо. Если спросят: «У Selectel нет ДЦ в Вирджинии» — ответ: «Да, это ориентир цены сопоставимого оборудования. Для оценки порядка стоимости этого достаточно».

3.6. Проектные допущения

Там, где публичных данных нет, работа принимает **явное допущение**. Правило: допущение подписано, используется одинаково во всей работе и его влияние понятно.

Допущение	Значение	Почему такое	Что поменяется, если оно неверно

Коэффициент пика K	1.5	Замечание ревьюера. Сервис глобальный: когда в Америке день, в Азии ночь, часовые пояса сглаживают общую кривую	Линейно меняются пиковые RPS, трафик, соединения и число серверов. Объём хранения не меняется
Сообщений чата на участие	5	Частоты нет ни у одного аналога, есть только лимит длины	Нагрузка на чат и его хранилище (45 ТБ)
Средний размер сообщения	1 КБ	Лимит — 4 000 символов (≈ 10 КБ), но большинство сообщений короткие	Хранилище чата
Открытий списка встреч	3 на DAU в сутки	Типичный рабочий день: утром, перед встречей, после	Нагрузка на `user_meetings`
Логинов	1 на DAU в сутки	Упрощённая модель: одна сессия в день	Нагрузка на сессии
Загрузок истории чата	1 на участие	При входе во встречу или переподключении	Нагрузка на чат
Просмотров записи	не больше 1 на запись	Верхняя оценка	Трафик скачивания записей
Часть записи при загрузке	8 МБ	Запись 0.761 ГБ — это 95 частей, S3 допускает до 10 000 частей	Число операций записи в S3
Хранение данных	30 дней	Продуктовое решение	Все объёмы хранения
Записывается каждая встреча	100%	Худший случай: реальная доля неизвестна	59% бюджета; это главный рычаг экономии
Материковый Китай	не обслуживаем	Нужен отдельный локальный контур: локализация данных, блокировки	Доли ДЦ
Загрузка CPU медиасервера	70%	Запас на пики пакетов и ключевые кадры видео	Число SFU
Ёмкость подов сервисов	1 000 / 1 500 / 5 000 запросов/с (Auth / встречи / чат)	Проектный норматив для C++-сервиса с точечными запросами	Число подов — это меньше 1% бюджета

Часть 4. Путь данных: откуда куда что идёт

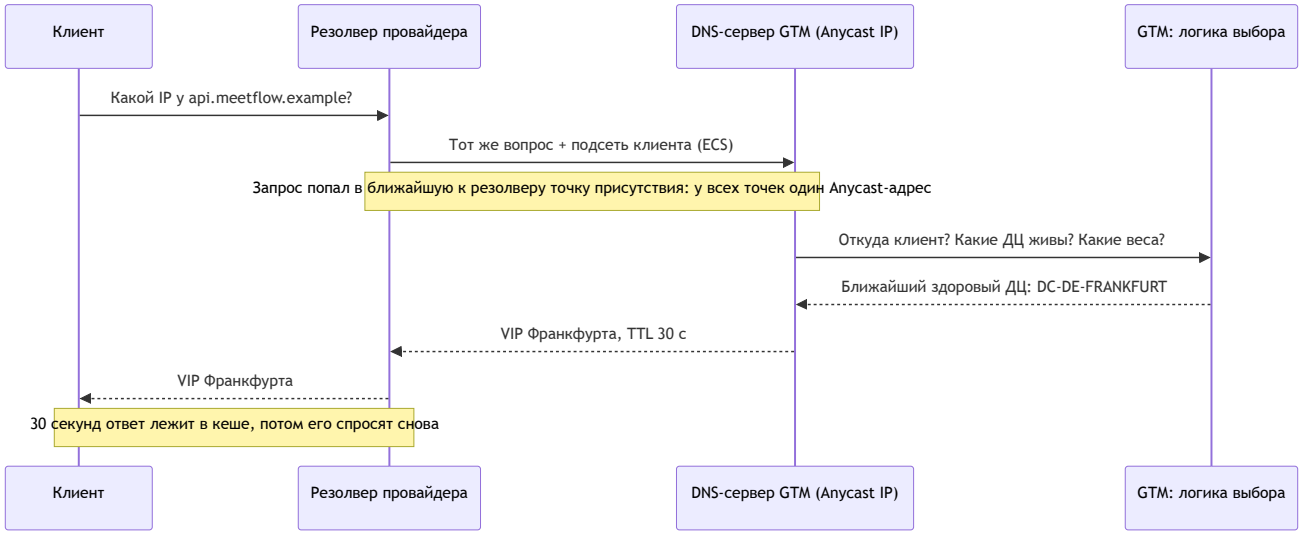
Эта часть — главная для понимания. Здесь каждое действие пользователя разобрано по шагам: через какие компоненты идёт запрос, что и куда записывается, сколько таких запросов в пик. Если вы понимаете эти 12 сценариев, вы понимаете всю работу.

Сначала короткая карта компонентов — кто есть кто:

Компонент	Что делает	Аналогия
GeoDNS / GTM	Отвечает клиенту, в какой ДЦ идти: выбирает ближайший здоровый	Справочная, которая говорит, в какой филиал ехать
L4-балансировщик (IPVS)	Принимает TCP-соединения на общий адрес ДЦ и раздаёт их по узлам NGINX, не заглядывая внутрь	Охранник на входе, который распределяет людей по окнам, не читая документы
NGINX L7	Расшифровывает HTTPS (TLS termination), смотрит путь запроса и отправляет его нужному сервису	Администратор, который читает заявление и решает, в какой отдел его нести
Auth API	Логин, профиль, сессии	Отдел пропусков
Control Plane	Создание встречи, вход, выход, список встреч	Отдел бронирования переговорных
WebSocket / Presence	Держит постоянное соединение с каждым участником: heartbeat, доставка событий и чата	Телефонная линия, которая всё время на связи
Chat	Принимает сообщения, выдаёт им номера, хранит историю	Секретарь, который ведёт протокол
SFU	Медиасервер: принимает видео каждого участника и пересылает остальным	Коммутатор, который соединяет видеопотоки
Recording workers	Записывают встречу в файл	Стенографист
PostgreSQL / Citus	Таблицы с транзакциями: пользователи, встречи, записи	Архив с договорами
ScyllaDB	Быстрая запись огромного числа строк: участники, чат, списки встреч	Журнал посещений
Redis	Данные в памяти: сессии, онлайн-состояние встреч	Доска с магнитами: быстро, но может стереться
S3	Объектное хранилище файлов: записи встреч	Склад

4.1. Как клиент находит дата-центр

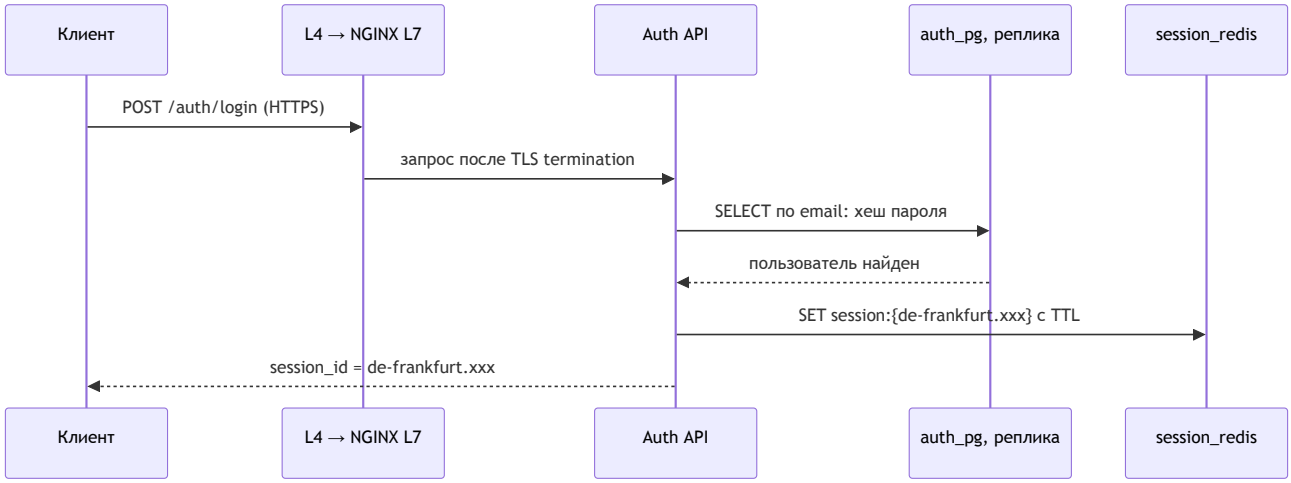
Клиент знает только имя `api.meetflow.example`. Чтобы получить IP-адрес, он делает DNS-запрос. Спрашивает он не наш DNS-сервер, а **рекурсивный резолвер** — DNS-сервер своего провайдера или публичный, например 8.8.8.8. Уже резолвер обращается к нашему авторитетному DNS-серверу (GTM).



Шаг	Что происходит	Термин
1	Клиент спрашивает резолвер, резолвер — наш DNS-сервер. Адрес DNS-сервера — **Anycast**: одинаковый у многих точек по всему миру, поэтому запрос сам попадает в ближайшую к резолверу	Anycast, BGP, PoP
2	**GTM** смотрит, откуда пришёл запрос: по IP резолвера или по подсети клиента, если резолвер её передал (ECS)	GeoDNS, ECS
3	GTM берёт ближайший ДЦ, который прошёл проверки здоровья и у которого есть свободная ёмкость	health check, веса
4	Резолвер возвращает клиенту **VIP** — общий адрес входа в ДЦ. Ответ хранится в кеше 30 секунд (**TTL**)	VIP, TTL

Что говорить: «ДЦ выбирает DNS. GeoDNS отдаёт каждому пользователю адрес ближайшего здорового ДЦ с учётом весов. Anycast используется только для самих DNS-серверов, чтобы DNS-ответ приходил из ближайшей точки».

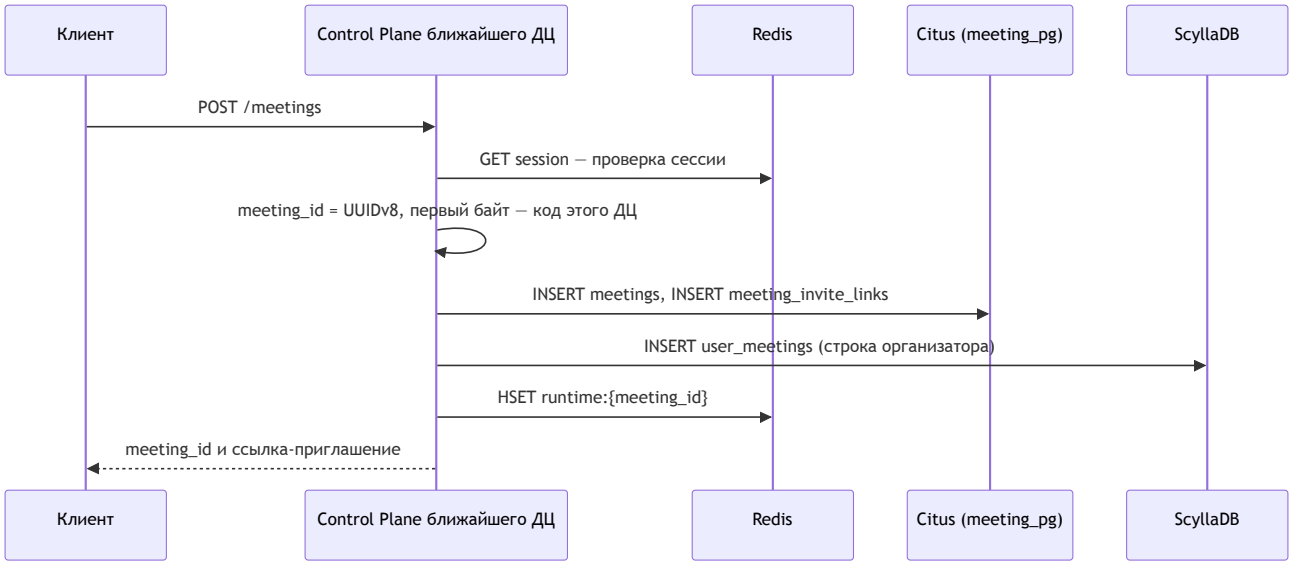
4.2. Логин



- Пароль проверяется по **локальной реплике** `auth_pg`: в каждом ДЦ есть копия таблицы пользователей только для чтения. Писать в основную базу для логина не нужно.
- Сессия создаётся в Redis этого ДЦ. В начале её идентификатора — **код ДЦ** (`de-frankfurt…`). По нему любой другой ДЦ поймёт, где сессия родилась.

- В пик **1 997 логинов в секунду**: 115 млн DAU × 1 логин / 86 400 с × 1.5.

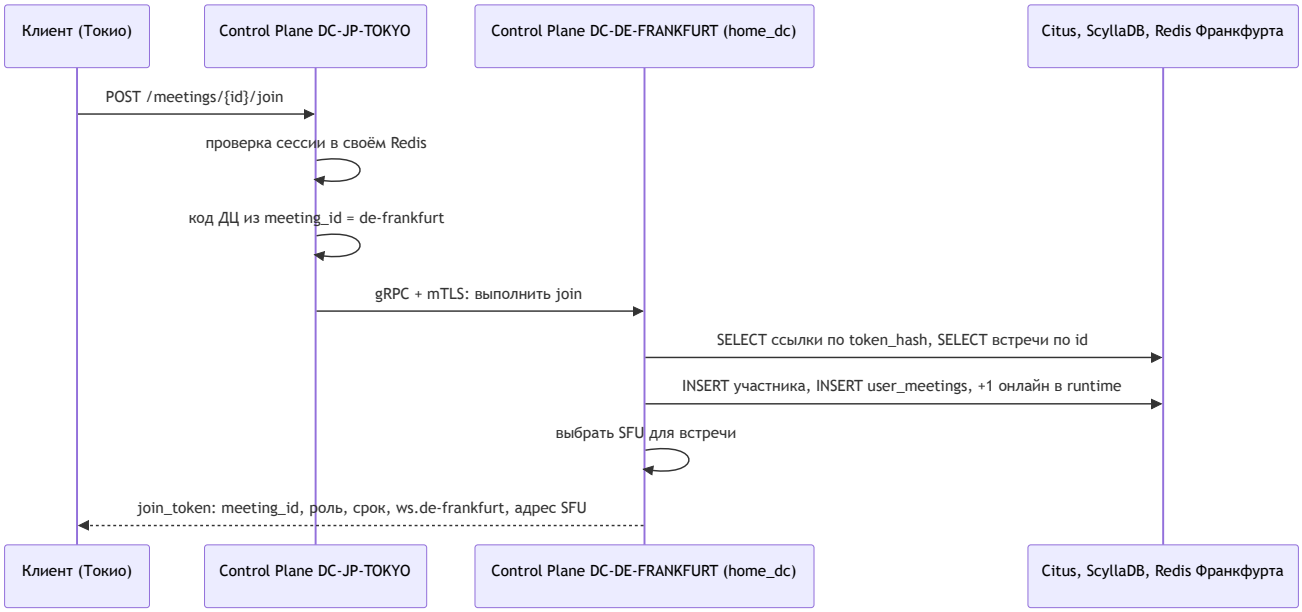
4.3. Создание встречи



- Встреча закрепляется за ДЦ, который обработал запрос. Это **`home_dc`** — «домашний» ДЦ встречи. Обычно это ближайший к организатору ДЦ.
- Код `home_dc` записан прямо в `meeting_id` (формат UUIDv8 позволяет занять первые биты своими данными) и в ссылку-приглашение. Поэтому любой ДЦ мира, получив `meeting_id`, сразу знает, где «живёт» встреча, без глобального справочника.
- Одна операция трогает 4 строки, в пик **521 создание в секунду** → 2 083 строк в секунду.

4.4. Вход во встречу (join)

Это самый «развилочный» сценарий: участник может войти из другой страны.



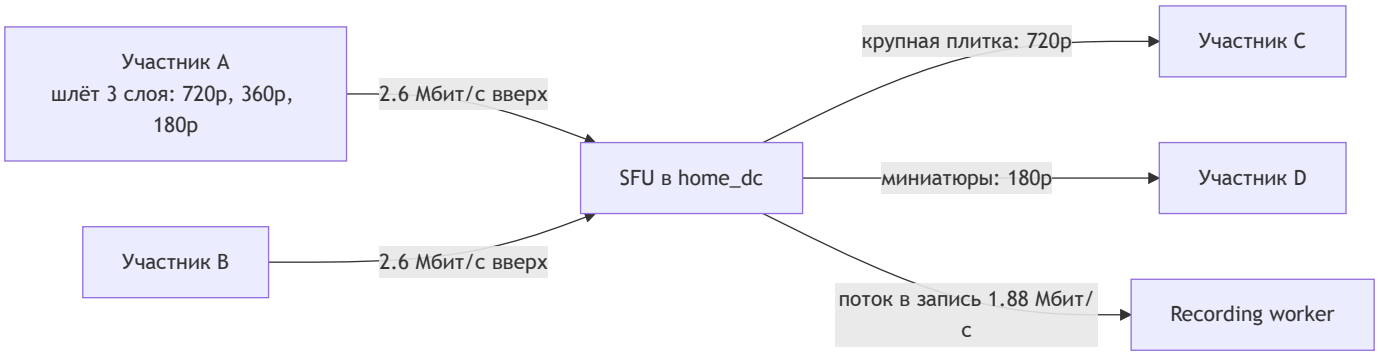
Шаг	Что происходит	Зачем
1	Запрос приходит в ближайший ДЦ (здесь — Токио)	Так решил DNS
2	Проверяется сессия в локальном Redis	Только авторизованные входят во встречи

3	По первому байту `meeting_id` определяется `home_dc`	Без обращения к справочнику
4	Если `home_dc` другой, запрос **проксируется** туда по защищённому каналу gRPC + mTLS	Все данные встречи — в одном ДЦ
5	В `home_dc`: проверка ссылки и статуса встречи (Citus), запись участника и строки его списка встреч (ScyllaDB), +1 к числу участников онлайн (Redis)	6 строк на вход
6	Control Plane выбирает SFU для встречи: все участники одной встречи — на одном SFU	Иначе участники не увидят друг друга
7	Клиент получает **`join_token`** — подписанный пропуск. Дальше WebSocket, чат и медиа проверяют подпись и не ходят за сессией	Меньше нагрузки на хранилище сессий

В пик **5 208 входов в секунду**: 300 млн участий / 86 400 × 1.5. Каждый вход трогает 6 строк → **31 250 строк в секунду**.

Что говорить: «Запрос приходит в ближайший ДЦ. По коду ДЦ в `meeting_id` он понимает, где живёт встреча, и проксирует `join` туда. Все данные встречи пишутся в её `home_dc`, а клиент получает подписанный `join_token` с адресом WebSocket и SFU этого ДЦ».

4.5. Видео и звук (медиа)

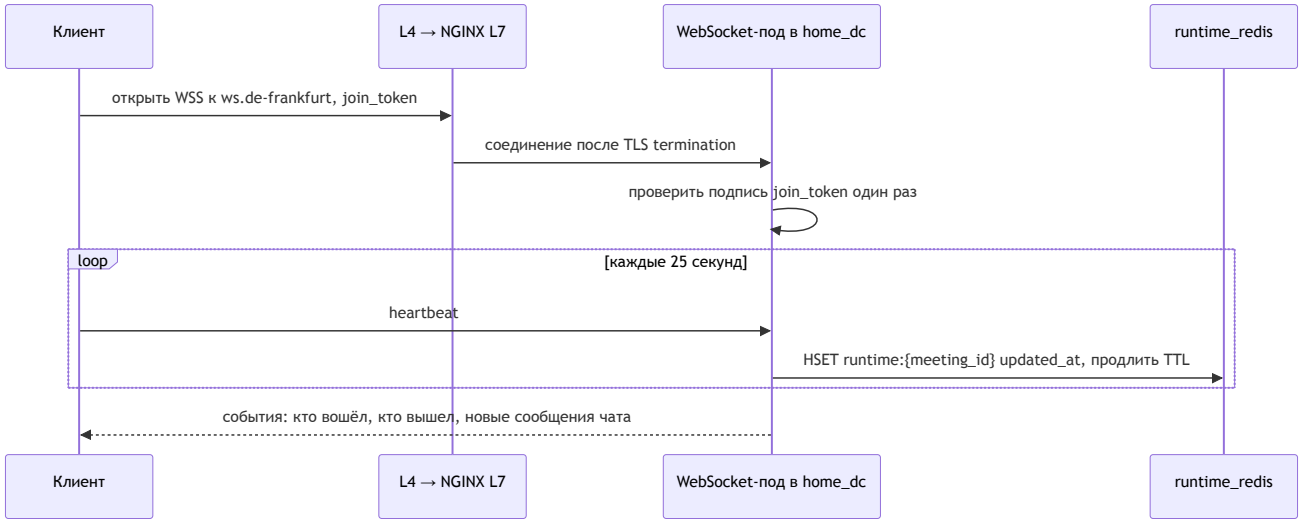


- Видео идёт **напрямую** на публичный адрес SFU, который клиент получил в `join_token`. Балансировщики L4 и L7 в этом не участвуют: им не нужно пропускать 78 Тбит/с.
- **SFU** (Selective Forwarding Unit) пересылает пакеты, а не перекодирует видео. Перекодирование — работа **MCU**, и это очень дорого по процессору.
- **Simulcast**: каждый клиент отправляет своё видео сразу в нескольких качествах. SFU каждому зрителю отдаёт подходящий слой: видео говорящего в крупной плитке — 720p, миниатюры — низким слоем.
- **ABR** (adaptive bitrate): SFU по отчётам RTCP видит потери, задержку и джиттер у каждого зрителя. При плохой сети он сразу переключает зрителя на слой ниже, при стабильной — плавно повышает. Звук важнее видео.
- **NAT и межсетевые экраны.** SFU — ICE-Lite с публичным адресом: соединение начинает клиент, поэтому для связи с SFU не нужны ни STUN, ни TURN. Если UDP закрыт, клиент подключается к тому же SFU по TCP. Не поддерживаем в MVP только сети, где прокси пропускает исключительно TLS на порт 443: там нужен TURN по TLS — это улучшение после MVP.
- На одного участника: 2.6 Мбит/с вверх + 1.8 вниз + 0.08 аудио + 0.15 демонстрация = 4.63 Мбит/с. В пик 16.875 млн участников → **78 131 Гбит/с**. Плюс поток в запись 3 172.5 Гбит/с → SFU обрабатывают **81 304 Гбит/с**.
- Один SFU-сервер на 6 ядер тянет **2.847 Гбит/с** (упирается в процессор) → **28 570 серверов**.

Что говорить: «Медиа — самый тяжёлый блок: 78 Тбит/с в пик. Поэтому SFU, а не MCU, simulcast и

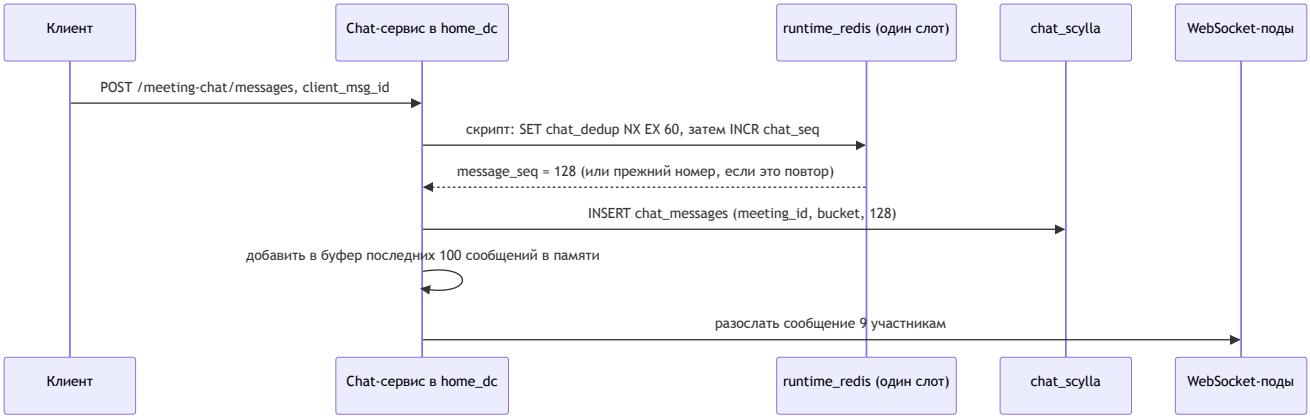
ABR. Видео идёт мимо балансировщиков, напрямую на адрес SFU из ответа `join`. SFU упирается в процессор: 2.85 Гбит/с на сервер по замеру Jitsi, отсюда 28.6 тысячи серверов».

4.6. Присутствие и heartbeat



- У каждого участника одно постоянное **WebSocket**-соединение (WSS — с шифрованием) с WebSocket-подом в `home_dc` встречи.
- **Heartbeat** — сообщение «я ещё здесь» раз в 25 секунд. По нему сервис понимает, кто онлайн. Если heartbeat-ы перестали приходить, у ключа встречи в Redis истекает **TTL**, и брошенная встреча удаляется сама.
- Числа: 16.2 млрд участие-минут × 60 / 25 = 38.88 млрд heartbeat-ов в сутки → **675 000 в секунду** в пик. Обратно сервер рассылает события: (чат + вход + выход) × 9 остальных участников = **328 125 сообщений в секунду**.
- **Одновременных соединений 16.9 млн.** Именно это число определяет размер NGINX L7 и WebSocket-подов: каждое соединение занимает память.

4.7. Чат

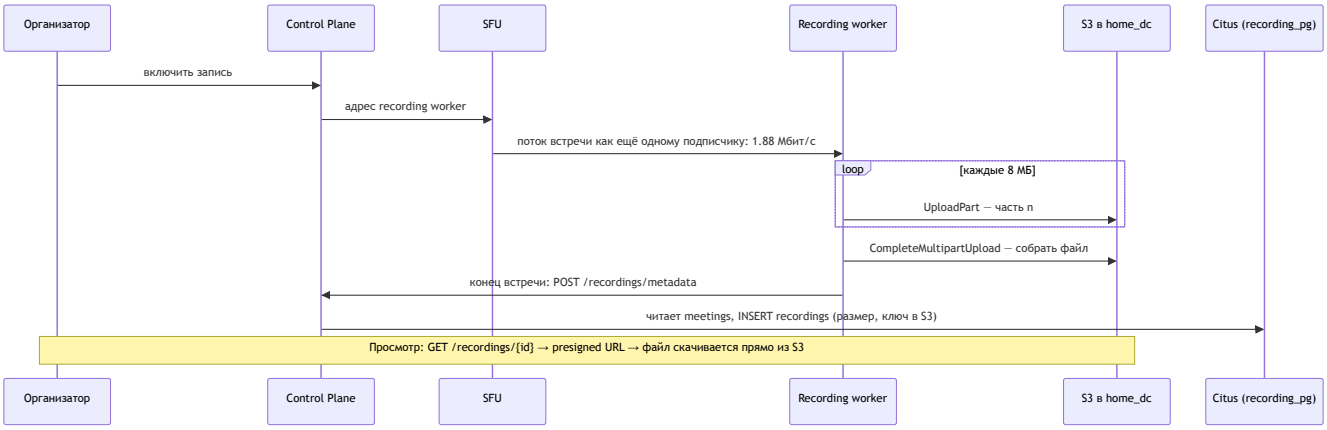


- **Порядок сообщений.** Номер `message_seq` выдаёт счётчик в Redis (`INCR`), один на встречу. Клиент читает чат «от последнего увиденного номера».
- **Идемпотентность.** Если сеть оборвалась и клиент повторил отправку, у повтора тот же `client_msg_id`. Скрипт в Redis проверяет его (`SET … NX` — «записать, только если ключа нет») и возвращает уже выданный номер вместо нового. Дубля не будет. Проверку пришлось сделать в Redis: в ScyllaDB нет `UNIQUE`, повторный `INSERT` с тем же ключом просто перезаписывает строку.
- Ключи встречи в Redis лежат в **одном слоте** благодаря hash tag `{meeting_id}`. Поэтому проверка

повтора и выдача номера — один атомарный скрипт.

- История чата при входе отдаётся из **буфера в памяти** Chat-сервиса. В ScyllaDB идут только промахи.
- Числа: 300 млн участий × 5 сообщений = 1.5 млрд сообщений в сутки → **26 042 в секунду** в пик. Хранилище — 45 ТБ за 30 дней.

4.8. Запись встречи

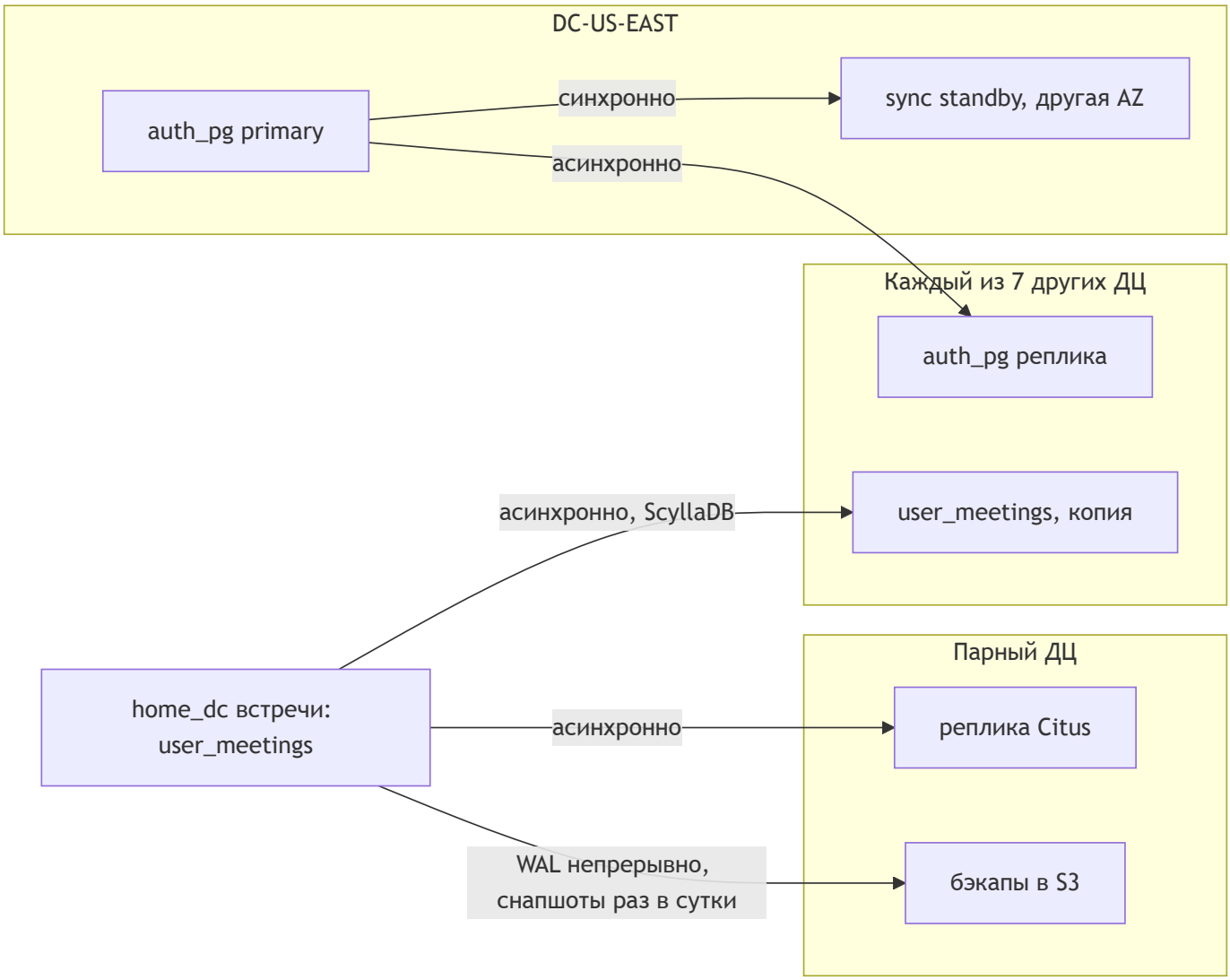


- Запись делает отдельный **recording worker**, а не клиент. SFU отправляет ему потоки встречи, как обычному участнику.
- Файл загружается в S3 частями по 8 МБ (**multipart upload**). Если сбой — воркер дозагружает только недостающие части. Одна запись ≈ 0.761 ГБ ≈ 95 частей.
- Метаданные записи пишутся в БД **один раз — в конце встречи**: `POST /recordings/metadata` добавляет строку в `recordings`. В пик это не больше 521 записи в секунду.
- Просмотр: сервис отдаёт **presigned URL** — временную подписанную ссылку. Файл скачивается из S3 напрямую, мимо наших серверов.
- Числа: 30 млн встреч × 0.761 ГБ = **22.8 ПБ в сутки**, **685 ПБ за 30 дней** — если записывать каждую встречу. В пик поток в запись 3 172.5 Гбит/с → **1 127 воркеров** и 49 570 загрузок частей в секунду.

4.9. Список встреч пользователя

- Запрос `GET /users/{id}/meetings?limit=100` идёт в ближайший ДЦ и читает таблицу **`user_meetings`** из локальной копии ScyllaDB.
- `user_meetings` — **read model**: таблица, заточенная под один вопрос «покажи 100 последних встреч пользователя». Строки в неё пишет `home_dc` встречи при создании, входе и выходе. ScyllaDB копирует таблицу **во все ДЦ** (RF=3 в каждом). Поэтому список доступен локально в любой стране, без JOIN и без обхода всех шардов.
- Копия в другом ДЦ может немного отставать — это **eventual consistency**. Для списка встреч это нормально.
- Числа: 3 открытия на DAU в день → **5 990 запросов в секунду** в пик по 100 строк → 598 958 строк в секунду.

4.10. Что копируется между дата-центрами



Данные	Куда копируются	Как
`users`	Во все ДЦ	Primary в `DC-US-EAST` пишет синхронно в standby в соседней зоне и асинхронно — напрямую в реплики 7 остальных ДЦ
`user_meetings`	Во все ДЦ	ScyllaDB сама реплицирует keyspace: RF=3 в каждом ДЦ
Встречи, ссылки, метаданные записей (Citus)	В парный ДЦ	Асинхронная реплика + архив WAL в S3 парного ДЦ
Участники, чат (ScyllaDB `home_dc`)	Только бэкап	Снапшоты раз в сутки в S3 парного ДЦ
Сессии, онлайн-состояние (Redis)	Никуда	Данные эфемерные: потеряется — восстановится
Файлы записей (S3)	Никуда	≥ 3 копии внутри ДЦ в 3 зонах; копия 685 ПБ в другом регионе удвоила бы главную статью расходов

4.11. Что происходит при поломках

Что сломалось	Что видит пользователь	Что делает система
Под сервиса (Auth, Control Plane, Chat,	Почти ничего: повтор запроса или переподключение	Kubernetes пересоздаёт под; запрос уходит в соседний. Сервисы **stateless** — не хранят состояние у себя

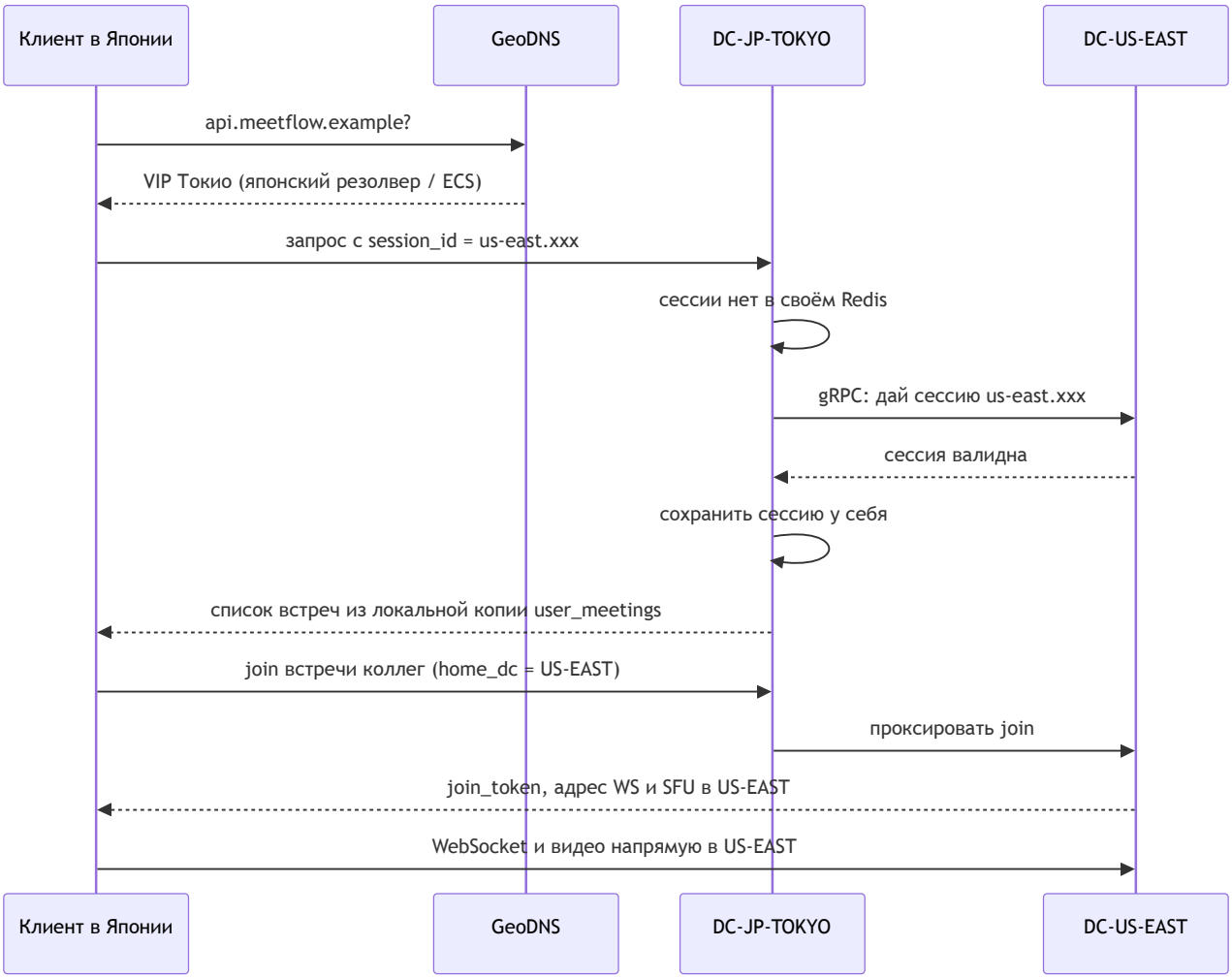
WebSocket)		
Узел NGINX L7	Переподключение WebSocket	keepalived исключает узел; клиенты открывают соединение на живой узел (резерв N+1)
Директор L4	HTTPS- и WSS-соединения рвутся, клиенты переподключаются	keepalived за секунды переносит адрес VIP на резервный директор по протоколу VRRP
Сервер SFU	Короткое замирание видео и звука	Control plane назначает встрече новый SFU, клиент получает адрес по WebSocket и заново устанавливает медиасоединение (ICE + DTLS)
Узел Redis	`runtime_redis`: часть онлайн-состояния пропадает до следующих heartbeat-ов. `session_redis`: часть свежих сессий может потеряться — эти пользователи войдут заново	Реплика становится master; онлайн-состояние восстанавливают heartbeat-ы
Узел ScyllaDB	Ничего	RF=3: данные есть ещё на двух узлах, кворум жив
Основной узел PostgreSQL	Ничего	Patroni автоматически делает синхронную standby основной — без потери данных
Целый ДЦ	Звонки этого ДЦ обрываются, переподключение за минуту	См. ниже

Отказ целого ДЦ. Худший случай — самый большой ДЦ, Франкфурт (35.69% нагрузки):

1. GTM видит, что ДЦ не отвечает на проверки, и убирает его из DNS-ответов: 3 проверки по 10 с + TTL 30 с ≈ **1 минута**.
2. Новые подключения уходят в Лондон, затем в US-EAST и другие ДЦ — по свободной ёмкости.
3. Клиенты оборванных встреч переподключаются **со случайной задержкой в пределах минуты**, чтобы не устроить шторм. Без задержки 6 млн клиентов открыли бы новые TLS-соединения разом; 21 оставшийся узел NGINX справился бы за ≈ 43 секунды.
4. `join_token` подписан, его проверит любой ДЦ, поэтому комната пересоздаётся в резервном ДЦ без обращения к упавшей базе.
5. Дежурный по инструкции повышает реплику Citus в парном ДЦ до основной: запланированные встречи снова доступны через минуты.
6. **Хватит ли ёмкости?** Двойной комплект серверов не держим. На время аварии все SFU снижают видео до профиля «high-quality» (1.0/0.6 Мбит/с вместо 2.6/1.8). Нагрузка на участника падает до 41.9% от обычной — с учётом потока в запись. Оставшиеся ДЦ имеют 64.31% штатной ёмкости. 41.9% < 64.31% — помещается, даже если пики всех ДЦ совпали.

4.12. Пользователь залогинился в США и улетел в Японию

Это вопрос Быкова с прошлой защиты. Ответ — сценарий по шагам:



1. **В США** пользователь залогинился, сессия `us-east.xxx` лежит в Redis `DC-US-EAST`.
2. **В Японии** DNS видит японский адрес и отправляет его в Токио. ДЦ выбирается по тому, где пользователь сейчас, а не где он логинился.
3. В Токио этой сессии нет. По префиксу `us-east` Токио **один раз** забирает её из США (+153 мс) и сохраняет у себя. **Перелогиниваться не нужно.**
4. Профиль читается из локальной реплики. Если пользователь меняет профиль, запись уходит в основную базу в США.
5. Список встреч читается из локальной копии: `user_meetings` есть во всех ДЦ.
6. Встреча американских коллег по-прежнему живёт в США. Токио проксирует `join` туда, а видео идёт на SFU в США. Сеть в одну сторону ≈ 77 мс. Норма разговора по ITU-T G.114 — до 150 мс «от рта до уха», и в неё входят ещё кодек и путь до собеседника. **Разговор возможен, но близко к границе нормы.** Улучшение после MVP — каскад SFU: клиент подключается к SFU в Токио, а тот пересылает поток в SFU США по магистрали.
7. Если пользователь сам создаст встречу в Японии, её `home_dc` будет Токио.
8. Если он остался жить в Японии, **ничего мигрировать не нужно**: данные пользователя есть во всех ДЦ, а встречи привязаны к своим ДЦ, а не к людям.

Что говорить (коротко): «DNS отправит его в Токио. Сессия переедет один раз по префиксу `session_id`, перелогиниваться не нужно. Профиль и список встреч читаются локально. Старые встречи живут в своём ДЦ в США — `join` проксируется туда. Новые встречи будут в Токио. Мигрировать ничего не надо, потому что в системе нет домашнего ДЦ пользователя — есть домашний ДЦ встречи».

Часть 5. Разбор работы по разделам

Каждый раздел разобран по одной схеме: **что требует методичка** → **что сделано** → **как посчитано** → **почему так** → **что говорить** → **какие вопросы зададут**. Номера пунктов (2.1, 3.4…) — это номера в README.

Раздел 1. Тема и целевая аудитория

Что требует методичка. Тип сервиса и реальные аналоги, аудитория не меньше 1 млн в день, её размер и местоположение, 3–7 ключевых функций MVP, ключевые продуктовые решения.

Что сделано.

- **1.1–1.2.** Сервис — Zoom-подобные видеовстречи. Аналоги: Zoom, Microsoft Teams, Google Meet. Масштаб: 320 млн MAU, 115 млн DAU, 300 млн участий в день.
- **1.3.** Шесть функций MVP: создание встречи, вход по ссылке, групповое видео 720p, демонстрация экрана, чат, облачная запись. Для каждой указано, какая метрика из раздела 2 её описывает.
- **1.4.** Ключевые продуктовые решения: считаем от групповой встречи; базовое качество — 720p; запись в облако с квотой; частота чата — явное допущение; данные хранятся 30 дней; записывается каждая встреча (худший случай); материковый Китай не обслуживаем.
- **1.5.** Аудитория — весь мир кроме материкового Китая, 8 дата-центров.

Почему так.

- **MVP** — минимальный набор функций, без которого продукт не имеет смысла. Методичка просит урезать функционал, чтобы проектировать ядро системы, а не вспомогательные службы.
- **Ключевые продуктовые решения** меняют архитектуру. «Записываем каждую встречу» даёт 685 ПБ и 59% бюджета. «Храним 30 дней» ограничивает все объёмы. «Без Китая» меняет доли ДЦ.
- **Китай исключён**, потому что для него нужен отдельный локальный контур: требования к локализации данных и блокировки трафика. Это решение продукта, его можно пересмотреть.

Что говорить. «MeetFlow — сервис групповых видеовстреч, как Zoom. Масштаб взят у лидеров рынка: 320 миллионов пользователей в месяц и 115 миллионов в день у Teams, 300 миллионов участий во встречах в день у Zoom. В MVP шесть функций: создание встречи, вход по ссылке, видео 720p, демонстрация экрана, чат и облачная запись».

Вопросы.

- *Почему аудитория Teams, а встречи Zoom?* — Каждая компания раскрывает свои числа: Microsoft — пользователей, Zoom — участия и длительность встреч. Берём каждое число там, где оно опубликовано, и подписываем год.
- *Почему числа 2020 года?* — MAU свежий, за 2023 год. Дневные числа — DAU Teams и участия Zoom — публичные значения 2020 года; берём их как консервативную оценку и подписываем год. Если спросят про 145 млн DAU, которые Microsoft называла в апреле 2021 года: DAU у нас влияет только на лёгкие запросы. С 145 млн логинов станет ≈ 2 517 в секунду вместо 1 997, а число серверов почти не изменится — тяжёлую нагрузку задают 300 млн участий. Актуальность ниши подтверждаем финансами Zoom за 2026 год: $4.87 млрд выручки.
- *Чем DMP отличается от DAU?* — DAU — уникальные люди за день, DMP — участия во встречах. Один человек на трёх встречах — это 1 DAU и 3 участия.

Раздел 2. Расчёт нагрузки

Что требует методичка. Пересчитать аудиторию в продуктовые и технические метрики. Продуктовые — MAU, DAU, средний объём хранилища пользователя по типам (в штуках и ГБ), действия пользователя в день по типам, сводной таблицей. Технические — объём хранения по типам (в штуках и ТБ), трафик по

типам (пиковый в Гбит/с и суточный в ГБ), RPS по методам (средний и пиковый, сводной таблицей). SimilarWeb как первоисточник запрещён.

2.1. Исходные данные

Все входные числа собраны в одну таблицу с источниками (см. часть 3). Обратите внимание на коэффициент пика **K = 1.5**: он умножает все пиковые значения.

2.2. Продуктовые метрики — вывод по шагам

Этот вывод нужно уметь повторить устно:

```
Шаг 1. Встреч в сутки        = 300 млн участий / 10 участников         = 30 млн встреч
Шаг 2. Участие-минут в сутки = 300 млн × 54 мин                        = 16.2 млрд
Шаг 3. Онлайн в среднем      = 16.2 млрд / 1 440 минут в сутках         = 11.25 млн человек
Шаг 4. Онлайн в пик          = 11.25 млн × K 1.5                        = 16.875 млн
                               = одновременных WebSocket-соединений
Шаг 5. Встреч одновременно   = 16.875 млн / 10                          = 1.6875 млн
```

Почему шаг 3 работает: если за сутки набирается 16.2 млрд «человеко-минут» во встречах, а в сутках 1 440 минут, то в каждую минуту в среднем во встречах сидят 16.2 млрд / 1 440 = 11.25 млн человек. Это **закон Литтла**: среднее число объектов в системе = поток × время пребывания.

Проверка по аналогу. Zoom раскрывал годовой темп более 3.5 трлн минут встреч на конец октября 2020 года, это ≈ 9.6 млрд минут в сутки. У нас 16.2 млрд: мы считаем, что каждый участник сидит все 54 минуты. **Ошибка в безопасную сторону**: онлайн, трафик и число серверов завышены, а не занижены.

Сводная таблица (требование методички). Работа даёт:

- **хранилище на пользователя** (на MAU за 30 дней): профиль — 256 Б, 2.81 созданной встречи — 720 Б, 2.81 ссылки-приглашения — 360 Б, 28.1 участия — 3.6 КБ, 28.1 строки списка встреч — 3.6 КБ, 140.6 сообщения чата — 140.6 КБ, итого без записей ≈ 149 КБ. Записи — 2.14 ГБ в худшем случае при квоте аналога 10 ГБ;
- **действия на DAU в сутки**: логин — 1, создание встречи — 0.26, вход — 2.61, выход — 2.61, сообщений в чат — 13.04, загрузка истории чата — 2.61, открытие списка встреч — 3, heartbeat — 338 (автоматически).

Как получено «2.81 встречи на пользователя»: 900 млн встреч за 30 дней / 320 млн MAU = 2.81.

2.3. Хранение

Данные	Штук	Объём	Как посчитано
Профили	320 млн	81.92 ГБ	320 млн × 256 Б
Метаданные встреч, 30 дней	900 млн	230.4 ГБ	30 млн × 30 × 256 Б
Ссылки-приглашения	900 млн	115.2 ГБ	30 млн × 30 × 128 Б
Участники встреч	9 млрд	1.152 ТБ	300 млн × 30 × 128 Б
Списки встреч	9 млрд	1.152 ТБ	так же
Сессии	115 млн	14.72 ГБ	115 млн × 128 Б
Чат	45 млрд	45 ТБ	1.5 млрд × 30 × 1 КБ
Итого без записей		**47.75 ТБ**	
Одна запись		0.761 ГБ	(1.8 + 0.08) Мбит/с × 54 мин × 60 с / 8 / 1000
Записи за 30 дней	900 млн	**685.26 ПБ**	30 млн × 0.761 ГБ × 30

Почему запись = 1.88 Мбит/с × 54 минуты. Запись — это то, что видит участник: видео на приём (1.8 Мбит/с) и звук (0.08). Делим на 8, чтобы перевести биты в байты.

Объём хранения не зависит от K. Пик меняет скорость, а не суточный объём: за пиковый час пишется больше, за ночь — меньше, а за сутки — столько же.

2.4. Трафик и RPS

Трафик. Средний трафик = онлайн в среднем × битрейт; пиковый = средний × 1.5.

Тип	Средний	Пиковый
Видео вверх: 11.25 млн × 2.6 Мбит/с	29 250 Гбит/с	43 875 Гбит/с
Видео вниз: × 1.8	20 250	30 375
Аудио: × 0.08	900	1 350
Демонстрация экрана: × 0.15	1 687.5	2 531.25
Медиа итого	**52 087.5**	**78 131.25**
Поток в запись (внутри ДЦ): 1.125 млн встреч × 1.88 Мбит/с	2 115	3 172.5

> Единицы: **бит ≠ байт**. Мбит/с (мегабиты) делим на 8, чтобы получить МБ/с (мегабайты). Сетевые карты меряют в битах (10 Гбит/с), диски — в байтах.

Почему вверх больше, чем вниз. Из-за simulcast: каждый отправляет своё видео сразу в нескольких качествах, а получает от каждого собеседника только одно, обычно маленькое.

RPS. Формула одна для всех методов:

```
средний RPS = событий в сутки / 86 400 секунд
пиковый RPS = средний × 1.5
```

Метод	События в сутки	Средний	Пиковый
Логин	115 млн	1 331	1 997
Создание встречи	30 млн	347	521
Вход / выход	по 300 млн	3 472	5 208
Heartbeat	38.88 млрд	450 000	**675 000**
Сообщение в чат	1.5 млрд	17 361	26 042
История чата	300 млн	3 472	5 208
Список встреч	345 млн	3 993	5 990
Метаданные записи / открытие записи	до 30 млн	до 347	до 521
Рассылка событий по WebSocket	(чат + вход + выход) × 9	218 750	328 125

Главный вывод раздела. Основная нагрузка — не API-запросы, а медиа (78 Тбит/с) и присутствие (675 тыс. heartbeat в секунду и 16.9 млн соединений). API — тысячи запросов в секунду.

Что говорить. «300 миллионов участий при встрече в 10 человек — это 30 миллионов встреч в сутки. Умножаем участия на 54 минуты и делим на минуты в сутках — получаем 11.25 миллиона человек онлайн в среднем, в пик с коэффициентом 1.5 — 16.9 миллиона. Умножаем на битрейт Zoom — 78 терабит в секунду медиа в пик. Heartbeat раз в 25 секунд даёт 675 тысяч сообщений в секунду. Записи всех встреч за 30 дней — 685 петабайт».

Вопросы.

- *Почему K = 1.5?* — Это отношение пикового часа к среднему. Сервис глобальный: когда в Америке рабочий день, в Азии ночь, и общая кривая сглажена. Значение задал ревьюер; в прошлой версии было 3, и все пики были вдвое больше.

- *Почему пик не влияет на хранилище?* — Пик — это скорость, а не объём за сутки.
- *Почему heartbeat раз в 25 секунд?* — Это значение по умолчанию в Socket.IO, популярной библиотеке WebSocket. Чаще — больше нагрузки, реже — дольше замечаем, что участник пропал.
- *Откуда 5 сообщений чата?* — Это явное допущение: частоту чата никто не публикует. Она влияет только на нагрузку и хранилище чата, это меньше 1% бюджета.
- *Почему каждая встреча записывается?* — Худший случай: реальная доля записываемых встреч неизвестна. Из-за этого записи — 59% бюджета, и ретеншн записей — главный рычаг экономии.

Раздел 3. Глобальная балансировка

Что требует методичка. Разбиение по доменам; обоснование расположения ДЦ через влияние на продуктовые метрики; распределение запросов по ДЦ; схема DNS-балансировки; схема Anycast-балансировки; механизм регулировки трафика между ДЦ.

3.1. Домены

Сервис разбит на домены: Auth/Users, Meeting Control Plane, Media/SFU, Presence, Chat, Recording, список встреч. Для каждого указано, **куда идёт запрос** (ближайший ДЦ или `home_dc` встречи) и **где точка записи**. Главное правило: API обслуживает ближайший ДЦ, всё, что относится к встрече, — `home_dc` встречи.

3.2. Где стоят 8 дата-центров и почему такие доли

Почему не один ДЦ.

1. **Задержка.** Свет в оптоволокне не мгновенный: пинг Токио ↔ Вашингтон ≈ 153 мс. Для видеозвонка сервер должен быть близко.
2. **Отказоустойчивость.** Один ДЦ — одна точка отказа.
3. **Законы о данных.** Например, отдельный ДЦ для Великобритании после Brexit.

Как посчитаны доли — три шага.

1. **Доля страны = её доля в мировом ВВП** (МВФ, 2025 год). Логика: видеовстречи — рабочий инструмент, поэтому их объём следует за экономикой страны, а не за численностью населения. Материковый Китай исключён. Остаётся 194 экономики на 98 537.9 млрд $.
2. **Каждую страну относим к ближайшему по задержке ДЦ.**
3. **США делим на восток и запад по населению** (перепись 2020: запад — 23.7%). Внутри одной страны деловая активность на человека примерно одинакова.

ДЦ	Город	Кого обслуживает	Доля
`DC-DE-FRANKFURT`	Франкфурт	Европа без Великобритании, СНГ, Ближний Восток, Африка	**35.69%**
`DC-US-EAST`	Ашберн, Вирджиния	Восток и центр США, Канада, Мексика, Центральная Америка, Карибы	**28.84%**
`DC-US-WEST`	Сан-Хосе	Запад США	7.40%
`DC-JP-TOKYO`	Токио	Япония, Корея, Тайвань, Монголия	7.36%
`DC-SG-SINGAPORE`	Сингапур	Юго-Восточная Азия, Австралия, Океания, Гонконг	6.97%
`DC-IN-MUMBAI`	Мумбаи	Индия и Южная Азия	5.03%
`DC-BR-SAOPAULO`	Сан-Паулу	Южная Америка	4.65%
`DC-UK-LONDON`	Лондон	Великобритания	4.06%

Сверка. По макрорегионам получается Америка 40.89%, EMEA 39.75%, APAC 19.36%. У Zoom в

выручке Америка — 72.1%. Расхождение объясняется тем, что Zoom исторически сильнее всего в США. Наш масштаб — масштаб лидера рынка (Teams), поэтому берём нейтральную оценку по экономике.

Почему не SimilarWeb. Методичка прямо запрещает его как первоисточник из-за погрешности. В прошлой версии доли были на SimilarWeb, и 42.5% нагрузки попадало в абстрактный «прочий пул» без места и серверов. Ревьюеру это не понравилось.

3.3. Нагрузка по ДЦ

Любая метрика ДЦ = глобальная метрика × доля ДЦ. Пример: heartbeat во Франкфурте = 675 000 × 35.69% = 240 908 в секунду. Предполагаем, что встречи распределены по `home_dc` так же, как пользователи: организатор и большинство участников обычно из одного региона.

3.4. Точки записи — ответ на замечание ревьюера

Правило: **у каждого объекта одна точка записи** (primary), читать по возможности из ближайшей копии. Тогда не бывает двух одновременных записей одного объекта в разных ДЦ и не нужен сложный multi-master.

Данные	Где пишутся	Где читаются
`users`	Один primary на весь мир — `DC-US-EAST`	Локальная реплика в каждом ДЦ
Сессии	ДЦ, где был логин	Там же; другой ДЦ переносит сессию по префиксу
Встречи, ссылки, участники, чат, онлайн-состояние, записи	`home_dc` встречи	`home_dc`
Список встреч `user_meetings`	`home_dc` встречи	Локальная копия в любом ДЦ

Почему `users` — один primary. Нужна глобальная уникальность email: два человека в разных ДЦ не должны зарегистрировать один адрес. Записей мало — даже если каждый DAU раз в сутки меняет профиль, это не больше 1 997 записей в секунду, и один сервер справляется.

Если запись в другом ДЦ. Запрос проксируется туда по gRPC с mTLS. Это плюс сетевая задержка, но только для редких записей. После своей записи пользователь пару секунд читает из primary (**read-your-writes**), пока реплика не догонит.

Отказ точки записи. Внутри ДЦ Patroni автоматически переключает primary на синхронную standby в соседней зоне — без потери данных. При отказе всего `DC-US-EAST` реплику в `DC-US-WEST` повышает **дежурный по инструкции**, а не автомат: второй ДЦ не может сам отличить отказ первого от обрыва связи между ними. Логин всё это время работает на локальных репликах.

3.5–3.6. DNS или Anycast — ответ на замечание ревьюера

Ответ: ДЦ выбирает и переключает DNS. GeoDNS/GTM — managed-сервис, который отвечает каждому пользователю адресом ближайшего здорового ДЦ с учётом весов.

Имя	Как выбирается
`api.meetflow.example`	GeoDNS: гео + задержка + здоровье + веса, TTL 30 с
`ws.<dc>.meetflow.example`	Фиксированное имя ДЦ встречи, приходит в ответе `join`
Адрес SFU	Не DNS: приходит в ответе `join`
`rec.<dc>.meetflow.example`	Скачивание записей из ДЦ встречи
`static.meetflow.example`	CDN провайдера

Где Anycast и почему только там. Anycast — один IP, который анонсируется по BGP из многих точек;

пакет приходит в ближайшую по маршрутам. Он хорош для **коротких запросов без состояния** — DNS и статика. Для API, WebSocket и медиа он не подходит:

1. BGP выбирает путь по своим правилам (AS-path), а не по загрузке ДЦ. Нельзя задать веса или вывести ДЦ из работы.
2. Если маршрут перестроится посреди звонка, пакеты долгого соединения уйдут в другой ДЦ, где о нём ничего не знают, и соединение порвётся.

Время переключения. Проверка здоровья раз в 10 секунд (минимум на тарифе Enterprise у Cloudflare), 3 неудачи подряд — ДЦ выведен (≈ 30 с), плюс TTL 30 с. Итого ≈ **1 минута**. На случай резолверов, которые кешируют дольше TTL, у приложения есть свой запасной список адресов.

3.7. Регулировка трафика

Вес ДЦ в GTM = совпадение по географии × здоровье × вес ёмкости × ручной вес. Перегруз — снижаем вес ёмкости. Плановые работы — ручной вес 0 (**drain**): новые встречи не создаются, текущие доживают. Отказ — здоровье 0. У каждого ДЦ есть порядок резервных ДЦ, но трафик упавшего ДЦ раздаётся по нескольким ДЦ пропорционально свободной ёмкости: Лондон (4%) не вместит Франкфурт (36%).

3.8. Переезд пользователя — см. часть 4.12.

Что говорить по разделу 3. «Восемь ДЦ, доли — по ВВП стран из базы МВФ без Китая, США делим по переписи. Самые крупные — Франкфурт, 36%, и восток США, 29%. ДЦ выбирает DNS: GeoDNS с проверками здоровья и весами, переключение за минуту. Anycast только у DNS-серверов, потому что BGP не знает о загрузке ДЦ и при смене маршрута рвёт долгие соединения. Точка записи у каждого объекта одна: пользователи — в Вирджинии с репликами везде, встреча — в своём домашнем ДЦ, код которого зашит в идентификатор встречи».

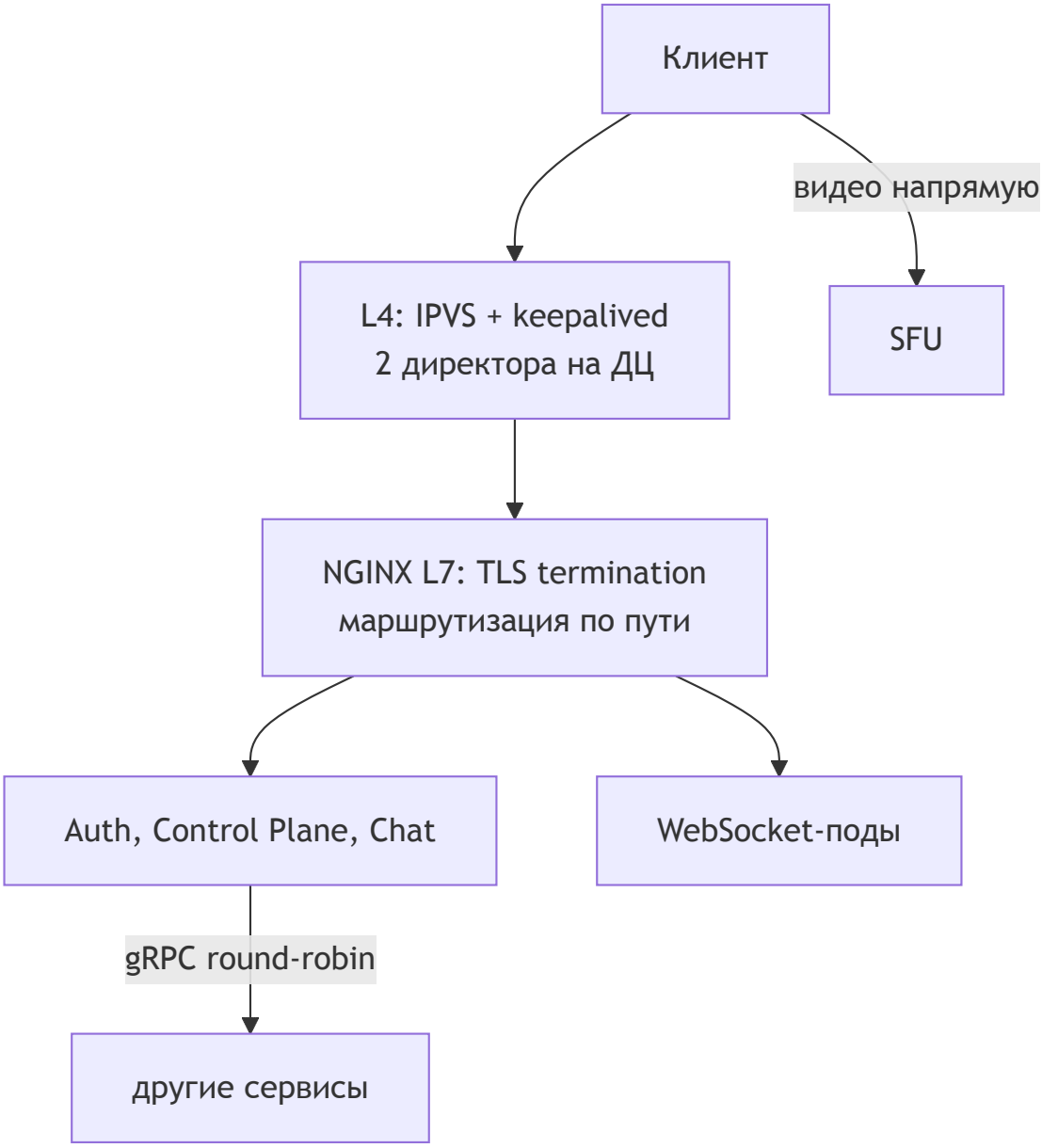
Вопросы.

- *Почему по ВВП, а не по населению?* — Видеовстречи — рабочий инструмент. По населению Индия и Африка получили бы больше всего нагрузки, что не соответствует реальному использованию.
- *А если реальные доли другие?* — Меняются только размеры пулов в ДЦ, не архитектура. GTM перераспределит подключения весами, а после запуска доли уточняются по GeoIP-аналитике.
- *Зачем Лондон, если Франкфурт рядом?* — Отдельная юрисдикция после Brexit и пара для резерва Франкфурта.
- *Почему у Франкфурта 36%?* — Он обслуживает всю Европу без Великобритании, СНГ, Ближний Восток и Африку — на эти страны приходится 35.69% ВВП 194 экономик без Китая (35 168.5 из 98 537.9 млрд $).
- *Что, если упадёт Франкфурт?* — См. часть 4.11: DNS уберёт его за минуту, нагрузка уйдёт в Лондон и другие ДЦ, на время аварии видео снижается до «high-quality», и ёмкости хватает.

Раздел 4. Локальная балансировка

Что требует методичка. Схема балансировки со всеми уровнями и механизмами резервирования, формула резервирования (N×2 или N+1), расчёт числа балансировщиков; ограничители — SSL termination и пропускная способность сети.

4.1–4.2. Уровни



Уровень	Что делает	Почему такой
L4 — IPVS + keepalived	Принимает TCP-соединения на общий адрес ДЦ (VIP) и раздаёт по узлам NGINX, не расшифровывая	Облачный балансировщик Selectel тестирован на 10 000 соединений и до 3–5 Гбит/с, а у нас в одном ДЦ до 6 млн соединений. Поэтому свой L4 на ядре Linux
L7 — NGINX	Расшифровывает HTTPS, смотрит имя хоста и путь (`api.meetflow.example` + `/meetings/…`, `ws.<dc>.meetflow.example`) и отправляет в нужный сервис	Нужно читать содержимое запроса
Медиа	Идёт мимо L4 и L7 напрямую в SFU	78 Тбит/с не нужно пропускать через балансировщики; встреча живёт на одном SFU
Между сервисами	gRPC, клиент сам раздаёт вызовы по подам (round-robin)	gRPC держит одно долгое соединение: L4 отправил бы все вызовы в один под

Direct Routing — режим IPVS, в котором ответ идёт от NGINX к клиенту напрямую, мимо директора. Поэтому директор видит только входящий трафик и не упирается в сеть. **keepalived** проверяет здоровье узлов NGINX и при отказе директора переносит адрес VIP на резервный по протоколу **VRRP**.

4.4. Ограничители NGINX

Ограничитель	Значение на узел	Что значит
HTTPS-запросы по открытым соединениям	341 232 RPS	Сколько запросов в секунду проходит через уже открытые соединения
Новые TLS-соединения	6 676 в секунду	Сколько **рукопожатий** (handshake) в секунду: установка шифрования — самая дорогая операция TLS
Сеть	8.8 Гбит/с	Пропускная способность
Память	54 КБ на соединение	20 КБ на проксирование WebSocket + до 34 КБ буферов TLS, худший случай

Значения взяты для 16 ядер, а не для 24: в тестах NGINX «CPU» — это физические ядра, а облачный узел на 24 vCPU — это примерно 12 физических ядер с Hyper-Threading.

Ошибка прошлой версии. Раньше весь поток запросов, включая heartbeat, делился на скорость **новых** TLS-рукопожатий. Но heartbeat идёт по уже открытому WebSocket-соединению — рукопожатия у него нет. Теперь у каждого ограничителя своя метрика.

4.5. Расчёт

```
Запросов в секунду  = REST 51 215 + heartbeat 675 000 + рассылка 328 125 = 1 054 340
Новых TLS в секунду = 56 424   (худший случай: каждый REST-запрос без keep-alive + новое WSS на
каждый join)
Трафик              = 1 054 340 × 1 КБ × 8 ≈ 8.43 Гбит/с
Соединений          = 16 875 000

Узлов по памяти в ДЦ = соединений_ДЦ × 54 КБ / 56 ГБ
Узел: 24 vCPU / 64 ГБ, из них 56 ГБ под соединения ≈ 1.04 млн соединений
```

Пример для Франкфурта: 6.02 млн соединений × 54 КБ = 325 ГБ / 56 ГБ = 5.8 → **6 узлов** по памяти. По запросам — 2, по TLS — 4, по сети — 1. Берём максимум (6) и добавляем 1 в резерв: **7 узлов**.

Итого **28 узлов NGINX** в режиме N+1 и 40 в режиме N×2. **Главный ограничитель — память под одновременные соединения**, потому что WebSocket держит соединение весь звонок.

4.6. L4

Директор IPVS хранит таблицу соединений: 136 байт на соединение. Во Франкфурте 6.02 млн соединений → 0.82 ГБ при 32 ГБ памяти; входящий трафик ≤ 3 Гбит/с при карте 10 Гбит/с. Один директор справляется, второй — резерв. Итого **16 директоров**, по два на ДЦ.

Резервирование. N+1 — нужно N узлов, ставим на один больше: переживаем отказ одного узла. **N×2** — двойной комплект: переживаем потерю целой зоны доступности.

Что говорить. «На входе ДЦ — свой L4 на IPVS с keepalived, два директора на ДЦ. Облачный балансировщик провайдера рассчитан на тысячи соединений, а у нас миллионы. Дальше NGINX: расшифровывает TLS и маршрутизирует по пути. NGINX упирается не в запросы, а в память под 16.9 миллиона одновременных WebSocket-соединений: 54 КБ на соединение, отсюда 28 узлов с резервом N+1. Видео идёт мимо балансировщиков — напрямую на SFU».

Вопросы.

- *Почему медиа мимо L7?* — SRTP уже зашифрован, а расшифровывать 78 Тбит/с на балансировщике дорого и бессмысленно. И встреча должна оставаться на одном SFU.
- *Как клиент находит SFU без балансировщика?* — Control Plane при `join` выбирает SFU для встречи и отдаёт его адрес. Балансирует именно Control Plane.

- *Что такое TLS termination?* — Расшифровка HTTPS на балансировщике. Дальше внутри ДЦ запрос идёт к сервису уже по внутренней сети.
- *Почему 54 КБ на соединение?* — 20 КБ — замер NGINX для WebSocket, 34 КБ — буферы OpenSSL, если они не освобождены. Это худший случай, реальные цифры меньше.

Раздел 5. Логическая схема БД

Что требует методичка. Картинка таблиц, полей и связей **без привязки к СУБД**; таблица с описанием; **все** данные, включая файлы, кеши и буферы; размеры (строки, байты) и нагрузка (QPS) на чтение и запись; требования к консистентности; особенности распределения нагрузки по ключам.

Что такое логическая схема. Это ответ на вопрос «какие данные у нас есть и как они связаны», пока без ответа «в какой базе они лежат». Физическая схема (раздел 6) отвечает на второй вопрос.

5.1. Таблицы

Таблица	Что хранит	Ключ	Тип данных
`users`	Аккаунт пользователя	`id`	Постоянные
`user_sessions`	Сессии после логина	`session_id`	Состояние сессии, с TTL
`meetings`	Карточка встречи: владелец, тема, время, статус, `home_dc`	`id`	Постоянные
`meeting_participants`	Кто во встрече, роль, когда вошёл и вышел	`meeting_id + user_id`	Постоянные
`meeting_invite_links`	Ссылки-приглашения (токены)	`id`	Постоянные
`user_meetings`	Готовый список встреч пользователя	`user_id + meeting_id`	**Read model**
`chat_messages`	Сообщения чата	`id`	Постоянные, только добавление
`recordings`	Метаданные записи и ссылка на файл	`id`	Метаданные файла
`recording_upload_buffer`	Части файла записи до сборки	`chunk_id`	Буфер

Кеши, счётчики и буферы — их тоже требует методичка:

Сущность	Что хранит
`meetings_runtime`	Онлайн-состояние идущей встречи: статус, сколько участников онлайн, какой SFU, когда был последний heartbeat
`chat_seq`	Последний выданный номер сообщения в чате встречи
`chat_dedup`	Уже принятые `client_msg_id` за последние 60 секунд — защита от дублей
`active_invite_tokens`	Кеш токенов приглашений за сутки
`chat_last100`	Последние 100 сообщений идущей встречи в памяти

Связи. Пользователь создаёт встречи (1 ко многим), участвует во встречах (многие ко многим через `meeting_participants`), пишет сообщения. У встречи много участников, сообщений, записей и ссылок.

5.2. Размеры и нагрузка

Таблица	Строк	Объём	Чтений/с в пик	Записей/с в пик
`users`	320 млн	81.92 ГБ	7 726	≤ 1 997
`user_sessions`	115 млн	14.72 ГБ	12 240	1 997

`meetings`	900 млн за 30 дней	230.4 ГБ	10 938	521
`meeting_participants`	9 млрд	1.152 ТБ	5 208	10 417
`meeting_invite_links`	900 млн	115.2 ГБ	5 208	521
`user_meetings`	9 млрд	1.152 ТБ, копия в каждом ДЦ	5 990 запросов по 100 строк	10 938
`chat_messages`	45 млрд	45 ТБ	≤ 5 208 запросов, только промахи буфера	26 042
`recordings`	900 млн	230.4 ГБ + 685.26 ПБ файлов	≤ 521	521
`meetings_runtime`	1.69 млн в пик	0.864 ГБ	—	685 938

Как читать строку: «`meeting_participants` пишется 10 417 раз в секунду в пик — это вход + выход (по 5 208), а читается 5 208 раз — при входе проверяем, не в комнате ли уже участник».

5.3. Консистентность

Консистентность — насколько свежие данные видит читатель.

- **Strong (строгая)** — всегда последняя версия. Нужна, где ошибка недопустима: `users` (нельзя зарегистрировать два аккаунта на один email), `meetings` (нельзя впустить во встречу, которую уже завершили), `meeting_participants` (нельзя добавить участника дважды), ссылки-приглашения (токен либо действует, либо отозван).
- **Eventual (в конечном счёте)** — данные доедут, но с задержкой. Подходит для `user_meetings` (новая встреча появится в списке с небольшой задержкой) и `meetings_runtime` (потерянное состояние восстановят heartbeat-ы).
- **Strong внутри встречи** — чат: порядок сообщений задаёт один счётчик.

5.4. Распределение по ключам

Ключ	Риск горячего ключа	Что делаем
`user_id`	Низкий: пользователи примерно одинаковы	Данные пользователя отдельно от встреч
`meeting_id`	**Высокий** на больших встречах	Встреча закреплена за `home_dc`; участников большой встречи разбиваем по **bucket**
`session_id`, `token`	Низкий	Случайные идентификаторы распределяются равномерно

Горячий ключ (hot key) — ключ, на который приходится непропорционально много нагрузки. Встреча на 1 000 человек даёт в 100 раз больше записей, чем средняя.

Что говорить. «Девять основных таблиц и пять производных: кеши, счётчики, буферы. Для каждой посчитаны строки, объём и нагрузка в пик. Строгая консистентность — там, где ошибка недопустима: пользователи, встречи, участники, ссылки. Eventual — для списка встреч и онлайн-состояния. Горячий ключ — большая встреча, поэтому участников разбиваем по bucket».

Вопросы.

- *Зачем `user_meetings`, если есть `meeting_participants`?* — `meeting_participants` разложена по встречам. Чтобы собрать список встреч одного пользователя, пришлось бы обойти все шарды. `user_meetings` разложена по пользователю: список читается одним запросом.
- *Почему у `users` записей «≤ 1 997»?* — Регистрации и правки профиля редки. Верхняя оценка: каждый DAU раз в сутки, в пик — 1 997 в секунду.

Раздел 6. Физическая схема БД

Что требует методичка. Картинка с привязкой к СУБД, индексами и шардированием; индексы; денормализация; выбор СУБД по таблицам; шардирование и резервирование по таблицам; клиентские библиотеки; балансировка запросов и мультиплексирование подключений; схема резервного копирования.

6.2–6.3. Какая СУБД для чего

Хранилище	Таблицы	Почему именно оно
PostgreSQL (`auth_pg`)	`users`	Нужны транзакции и уникальный email. 82 ГБ помещаются на один сервер
PostgreSQL + Citus (`meeting_pg`, `recording_pg`)	`meetings`, `meeting_invite_links`, `recordings`	SQL и транзакции для метаданных + шардирование по `meeting_id`, когда данных станет много
ScyllaDB (`meeting_scylla`, `chat_scylla`)	Участники, списки встреч, чат	Очень много записей, чтение по ключу, JOIN не нужен; умеет копировать данные между ДЦ
Redis Cluster (`session_redis`, `runtime_redis`)	Сессии, онлайн-состояние, счётчики	Всё в памяти: сотни тысяч операций в секунду, ключи сами удаляются по TTL
S3	Файлы записей	Огромные неизменяемые файлы, дёшево за гигабайт

Что такое Citus. Расширение PostgreSQL для шардирования: таблица режется на части по **колонке распределения** (у нас `meeting_id`), части лежат на разных узлах, а координатор сам направляет запрос в нужную часть. Сейчас в каждом ДЦ данных немного (самый большой — Франкфурт, ≈ 206 ГБ и ≈ 5 950 чтений в секунду), поэтому Citus работает в режиме **одного узла**: координатор сам хранит шарды. Колонка распределения задана сразу, и при росте добавляются узлы без переделки приложения.

Что такое ScyllaDB. Распределённая NoSQL-база, совместимая с Cassandra. Сильна в записи: данные сначала пишутся в память и журнал, потом сбрасываются на диск большими файлами. Нет JOIN и нет транзакций сразу над несколькими ключами раздела (partition key) — есть только «лёгкие» транзакции LWT вида `IF NOT EXISTS` внутри одного раздела. Зато линейно масштабируется добавлением узлов.

6.4. Индексы и ограничения конкретных СУБД

Индекс — «оглавление» таблицы: по нему база находит строку, не просматривая всю таблицу.

Три технические тонкости, которые исправлены во втором проходе. Их стоит знать: это проверка понимания:

1. **Citus.** Первичный ключ и UNIQUE в распределённой таблице обязаны включать колонку распределения. Поэтому у `recordings` ключ `PK(meeting_id, id)`, у ссылок — `PK(token_hash, id)` и `UNIQUE(token_hash, token)`. Так как `token_hash` вычисляется из `token`, это та же уникальность токена.
2. **ScyllaDB.** В ней нет UNIQUE: повторный `INSERT` с тем же ключом просто перезаписывает строку. Поэтому защиту от дублей сообщений сделали в Redis через `SET … NX` (п. 4.7 этого документа).
3. **Redis.** Вторичных индексов нет, доступ только по ключу. Поэтому брошенные встречи не ищем «по времени последнего heartbeat», а ставим ключу TTL, который продлевает каждый heartbeat.

6.5. Шардирование и резервирование

Шардирование — деление данных на части (шарды) по разным серверам по **ключу шардирования**. Строки одной встречи лежат вместе, потому что ключ — `meeting_id`.

Двухуровневое шардирование в работе:

1. **По ДЦ.** Всё, что относится к встрече, лежит в её `home_dc`.
2. **Внутри ДЦ.** По ключу: `meeting_id`, `user_id`, `token_hash`, `session_id`.

Репликация масштабирует чтение, шардирование — запись. Реплики могут отвечать на чтения, но каждая запись копируется на все реплики. В PostgreSQL она проходит через один основной узел. В ScyllaDB основного узла нет, но запись всё равно ложится на все RF копий. Поэтому копии не добавляют ёмкости на запись. Чтобы писать больше, данные делят на шарды.

Хранилище	Резервирование
PostgreSQL `auth_pg`	Синхронная standby в другой зоне + асинхронная реплика в каждом ДЦ
Citus	Синхронная standby в другой зоне + асинхронная реплика в парном ДЦ
ScyllaDB	RF=3: три копии в трёх зонах ДЦ; `user_meetings` — RF=3 в каждом ДЦ
Redis	У каждого master есть replica на другом сервере
S3	Не меньше 3 копий в трёх зонах

RF = 3 (replication factor) — каждая строка хранится в трёх копиях. **Кворум** — большинство копий, 2 из 3: пока живо большинство, данные можно читать и писать. **LOCAL_QUORUM** — кворум внутри своего ДЦ, без ожидания других континентов.

6.6–6.7. Нагрузка на строки и запросы

Один запрос API трогает несколько таблиц. Например, `join` — 6 строк: сессия, ссылка, встреча, участник, список встреч, онлайн-счётчик. Поэтому 5 208 входов в секунду дают 31 250 обращений к строкам. Самое «широкое» чтение — список встреч: 5 990 запросов × 100 строк = 598 958 строк в секунду из ScyllaDB.

6.8. Кеш

Кеш	Объём	Операций в секунду
`meetings_runtime`	0.864 ГБ	685 938
`chat_seq` + `chat_dedup`	0.16 ГБ	52 083
`user_sessions`	14.72 ГБ	14 236
`chat_last100` (в памяти Chat-сервиса)	168.75 ГБ	до 520 833 строк чтения

Вывод: `meetings_runtime` весит меньше гигабайта, но получает почти 700 тысяч операций в секунду. Это классический случай для Redis: мало данных, очень много операций, данные эфемерные.

6.9. Денормализация

Денормализация — сознательное дублирование данных ради скорости чтения. Например, тема, время и статус встречи хранятся и в `meetings`, и в `user_meetings`: список встреч читается одним запросом, без **JOIN** (объединения таблиц). На больших нагрузках JOIN дорогой, а между разными СУБД он вообще невозможен.

6.10–6.11. Библиотеки и подключения

- C++-клиенты: `libpqxx` для PostgreSQL и Citus, `redis-plus-plus` и `hiredis` для Redis, **ScyllaDB CPP-RS Driver** (прежний C/C++ драйвер больше не развивается), AWS SDK for C++ для S3.
- **PgBouncer** — пул соединений к PostgreSQL в режиме transaction pooling: соединение с базой выдаётся клиенту только на время транзакции. Тысячи соединений подов сводятся к десяткам соединений с базой.
- **Redis Cluster**: клиент сам вычисляет **слот** ключа (их 16 384) и идёт сразу в нужный master.
- **ScyllaDB**: драйвер отправляет запрос прямо на узел с данными и даже на нужное ядро (**shard-**

aware), а DC-aware политика держит запросы в своём ДЦ.

6.12. Резервное копирование

Хранилище	Как	Сколько можно потерять при гибели ДЦ (RPO)
PostgreSQL, Citus	Непрерывный архив WAL (журнала изменений): сегмент уходит в архив не реже раза в минуту (`archive_timeout` = 60 с) + базовая копия раз в сутки, хранение 14 дней, в S3 парного ДЦ	≈ 1 минута по архиву; раньше срабатывает асинхронная реплика
ScyllaDB	Снапшоты раз в сутки через ScyllaDB Manager в S3 парного ДЦ, хранение 7 дней	До 24 часов истории чата и участников — осознанный компромисс
Redis	Не копируем	Сессии — повторный логин; онлайн-состояние восстанавливают heartbeat-ы
S3, записи	Отдельной копии нет, 3 копии внутри ДЦ	Записи ДЦ теряются: копия 685 ПБ удвоила бы главную статью расходов

WAL (write-ahead log) — журнал PostgreSQL, куда каждое изменение пишется до того, как попадёт в таблицу. По архиву WAL базу можно восстановить на любой момент времени — это **PITR** (point-in-time recovery).

Что говорить. «Каждому паттерну доступа — своя СУБД. PostgreSQL — пользователи: транзакции и уникальный email. Citus — метаданные встреч и записей с шардированием по `meeting_id`. ScyllaDB — то, что много пишется: участники, чат, списки встреч. Redis — горячее состояние и сессии. S3 — записи. JOIN в горячем пути нет: вместо него денормализованные проекции вроде `user_meetings`. Бэкапы: непрерывный архив WAL с отставанием не больше минуты и снапшоты ScyllaDB раз в сутки — в S3 парного ДЦ».

Вопросы.

- *Почему не одна PostgreSQL на всё?* — 45 ТБ чата и десятки тысяч записей в секунду без транзакций — это работа для ScyllaDB. 700 тысяч операций в секунду над эфемерными данными — для Redis.
- *Зачем Citus, если данные помещаются на один сервер?* — Сейчас помещаются, поэтому Citus в режиме одного узла. Он нужен для роста: колонка распределения задана сразу, и узлы добавляются без переделки приложения.
- *Как не создать дубль сообщения при повторе запроса?* — Redis-скрипт: `SET chat_dedup NX`. Повтор с тем же `client_msg_id` получает уже выданный номер.
- *Что такое PgBouncer и зачем он?* — Пул соединений. У PostgreSQL каждое соединение — отдельный процесс, тысячи соединений от подов база не выдержит.

Раздел 7. Алгоритмы

Что требует методичка. Раздел необязательный. Если он есть, для каждого алгоритма: задача, входные данные, шаги, структуры данных, альтернативы, обоснование, влияние на нагрузку и схему БД.

7.2. Adaptive Bitrate (ABR)

Задача. Подобрать качество видео под текущую сеть каждого участника, чтобы звонок не рвался.

Шаги.

1. SFU получает от клиента **RTCP-отчёты** — служебные сообщения внутри самой медиасессии с потерями, джиттером и временем доставки пакетов.

2. Если сеть плохая — профиль снижается **сразу**.
3. Если сеть стабильна — профиль повышается **на одну ступень**: понижаем быстро, повышаем плавно, чтобы качество не «прыгало».
4. Если страдает звук — первым снижается видео: звук важнее.

Профили: 720p → 360p → 180p → только звук. Верхний профиль — 720p, потому что на нём построен расчёт трафика. 1080p в MVP нет: он требует 3.8/3.0 Мбит/с и увеличил бы трафик и число SFU.

Почему server-guided ABR. SFU видит всех подписчиков и управляет исходящим трафиком. Альтернативы: всегда 720p — звонок рвётся на плохой сети; решает только клиент — SFU хуже управляет трафиком.

Влияние на нагрузку. Статистика идёт внутри медиасессии (RTCP), отдельных запросов к API нет, поэтому в расчёт RPS она не входит.

7.3. Выбор слоя SFU (simulcast)

Задача. Не перекодировать видео, а выбирать готовый слой.

Шаги. Клиент публикует несколько слоёв видео. SFU помнит доступные слои каждого отправителя. Для каждого зрителя считается нужный профиль: видео активного говорящего в крупной плитке — слоем выше, миниатюры — ниже. При ухудшении сети зритель переключается на слой ниже.

Альтернативы.

Вариант	Как работает	Минус
Single stream	Каждый отправляет одно качество, всем отдаётся оно же	Зрителю со слабой сетью оно не по силам, а снизить его — значит ухудшить картинку всем
MCU transcoding	Сервер декодирует все видео, склеивает и кодирует заново	Огромный расход CPU — для highload не подходит
SFU + simulcast/SVC	Сервер пересылает готовые слои, ничего не перекодируя	Больше исходящего трафика от каждого участника; CPU сервера уходит только на пересылку пакетов — **выбрано**

Ещё бывает **P2P** (mesh): каждый шлёт видео каждому напрямую, без сервера. На 10 участниках это 9 исходящих потоков с каждого клиента, поэтому для групповых встреч не подходит.

Что говорить. «Два алгоритма прямо снижают 78 терабит. ABR: SFU по RTCP-отчётам видит потери и задержку и снижает качество сразу, а повышает плавно, звук важнее видео. Выбор слоя: клиент шлёт несколько качеств, SFU каждому отдаёт нужное без перекодирования — видео говорящего в 720p, миниатюры низким слоем».

Раздел 8. Технологии

Что требует методичка. Сводная таблица: технология, область применения, мотивация.

Группа	Технологии	Почему
Язык	C++23	Низкие задержки, контроль памяти — важно для медиа и control plane
Сеть и API	Boost.Asio / Boost.Beast, gRPC + Protobuf, OpenSSL	Асинхронная сеть в C++, строгие контракты между сервисами, шифрование
Медиа	WebRTC, SFU (как mediasoup)	Стандарт браузера для звонков, пересылка без перекодирования
Балансировка	GeoDNS/GTM (managed), IPVS + keepalived, NGINX	Выбор ДЦ, L4 в ядре Linux, L7 с TLS termination
Инфраструктура	Kubernetes, Docker, Helm, CMake, Conan 2	Запуск и автоперезапуск сервисов, сборка C++
Данные	PostgreSQL, Citus, PgBouncer, Patroni, Redis	См. раздел 6

	Cluster, ScyllaDB, S3	
Наблюдаемость	Prometheus, Grafana, OpenTelemetry, Loki, Jaeger	Метрики, дашборды, сбор телеметрии, логи, трассировки

Что говорить. «Основной язык — C++23: медиа и control plane чувствительны к задержкам. Между сервисами — gRPC. Всё в Kubernetes. Данные — PostgreSQL с Citus, ScyllaDB, Redis и S3. Наблюдаемость — Prometheus, Grafana, Loki и Jaeger через OpenTelemetry».

Раздел 9. Надёжность

Что требует методичка. Сводная таблица по всем компонентам со способами резервирования.

Три уровня отказа и как работа их переживает:

Уровень	Пример	Как переживаем
Сервер или под	Упал сервер SFU, под Chat, узел Redis	**N+1**: запасной узел в каждом ДЦ. Kubernetes пересоздаёт поды. Копии данных: ScyllaDB — 3 (RF=3), Redis — master и реплика, PostgreSQL — синхронная standby
Зона доступности (AZ)	Пропало питание в одном здании ДЦ	Серверы и копии данных разнесены по 3 зонам. Синхронная standby PostgreSQL — в другой зоне, переключение без потери данных. L7 в расчёте — N+1 (28 узлов); пережить потерю целой зоны без перегрузки позволяет N×2 (40 узлов) — вариант жёсткого резервирования, в стоимость не заложен
Целый ДЦ	Регион недоступен	DNS уводит пользователей за минуту, видео временно понижается до «high-quality», дежурный повышает реплики в парном ДЦ

Ключевые термины.

- **Failover** — переключение на резерв при отказе. Автоматическое — внутри ДЦ (Patroni, Redis, ScyllaDB). Ручное по инструкции (**runbook**) — между ДЦ: второй ДЦ не может сам отличить падение первого от обрыва связи между ними, а автоматическое переключение в такой ситуации даёт **split brain** — две основные базы одновременно.
- **RPO** (recovery point objective) — сколько данных можно потерять. Для асинхронной реплики — секунды отставания.
- **RTO** (recovery time objective) — за сколько восстанавливаемся. Переключение ДЦ — минуты.
- **Stateless-сервис** — не хранит состояние у себя: любой экземпляр обработает любой запрос. Все API-сервисы stateless, состояние — в Redis и базах.
- **Graceful degradation** — работать хуже, но работать: снизить качество видео вместо отключения звонков.

Отказ самого большого ДЦ (Франкфурт, 35.69%) — расчёт ёмкости.

```
трафик участника, 720p            = 2.6 + 1.8 + 0.08 + 0.15 = 4.63 Мбит/с
трафик участника, high-quality    = 1.0 + 0.6 + 0.08 + 0.15 = 1.83 Мбит/с  → 39.5% от 720p
с потоком в запись (0.188 Мбит/с на участника): 2.018 / 4.818             → 41.9%
ёмкость SFU без Франкфурта        = 100% − 35.69% = 64.31%
41.9% < 64.31%                                                             → помещается
```

Двойной комплект серверов (холодный резерв) не держим: это ещё ≈ 400 млн ₽ в месяц на одни SFU.

Шторм переподключений. 6 млн клиентов Франкфурта одновременно открывают новые TLS-соединения. 21 оставшийся узел NGINX × 6 676 рукопожатий в секунду ≈ 140 тыс. в секунду → все переподключатся за ≈ 43 с. Поэтому клиент переподключается со **случайной задержкой** в пределах минуты.

Что говорить. «Три уровня. Отказ сервера — N+1 и три копии данных. Отказ зоны — серверы и копии разнесены по трём зонам, синхронная standby в другой зоне. Отказ целого ДЦ — DNS уводит пользователей за минуту, а чтобы хватило ёмкости, на время аварии видео понижается до high-quality: нагрузка падает до 42%, а у оставшихся ДЦ 64% ёмкости. Двойной комплект серверов не держим — это плюс 400 миллионов рублей в месяц».

Вопросы.

- *Почему между ДЦ переключение ручное?* — Документация Patroni: standby-кластер во втором ДЦ повышают вручную. Автомат не отличит падение ДЦ от обрыва связи и может сделать две основные базы — split brain.
- *Что теряется при гибели ДЦ?* — Записи встреч этого ДЦ и до суток истории чата. Это осознанный компромисс ради денег; метаданные встреч сохраняются через реплику в парном ДЦ.
- *Почему пики разных ДЦ не совпадают?* — Часовые пояса: когда во Франкфурте пик, в Токио вечер, в Калифорнии раннее утро. Но расчёт ёмкости проверен даже для худшего случая — совпадения пиков.

Раздел 10. Схема проекта

Что требует методичка. Картинка взаимодействия сервисов, пояснения, потоки данных, внутренняя и внешняя балансировка.

Схема — в части 2.3 этого документа и в приложении А PDF-версии. Её можно открыть с масштабированием в diagrams.net по ссылке из README (после слияния ветки в `main`).

№	Поток	Путь
1	Авторизация	Клиент → L4 → NGINX L7 → Auth API в ближайшем ДЦ → реплика `auth_pg` + `session_redis`
2	Создание и вход	Клиент → L4 → NGINX L7 → Control Plane → PgBouncer → Citus, ScyllaDB, Redis; `join` выполняется в `home_dc`
3	Присутствие	Клиент → L4 → NGINX L7 → WebSocket-под в `home_dc` → `runtime_redis`
4	Чат	Клиент → L4 → NGINX L7 → Chat в `home_dc` → Redis (номер, идемпотентность) + `chat_scylla`; доставка — через WebSocket
5	Медиа	Клиент → SFU в `home_dc`, напрямую по адресу из `join`
6	Запись	SFU → воркер записи → S3; скачивание — клиент ← S3 по presigned URL
7	Межрегиональный прокси	Control Plane ближайшего ДЦ → gRPC/mTLS → Control Plane `home_dc`
8	Репликация	`auth_pg` → все ДЦ; `user_meetings` → все ДЦ; Citus → парный ДЦ

Внешняя и внутренняя балансировка: глобально — GeoDNS/GTM (Anycast только у DNS-серверов); в ДЦ — L4 (IPVS) → NGINX L7, медиа — напрямую в SFU, его выбирает Control Plane; между сервисами — gRPC round-robin; между ДЦ — gRPC с mTLS.

Что говорить. «Клиент через DNS попадает в ближайший ДЦ, дальше L4 и NGINX, оттуда в сервисы. Зелёное — сеть, синее — сервисы на C++, оранжевое — хранилища, фиолетовое — Redis. Медиа идёт отдельно, напрямую в SFU. Подробно показан один ДЦ, остальные семь такие же; в нарисованном стоит ещё основная база пользователей».

Раздел 11. Серверы и стоимость

Что требует методичка. Таблица с конфигурациями, количеством серверов и сервисами на них; для Kubernetes — контейнеры с выделенными ресурсами.

Общее правило. Для каждого компонента в каждом ДЦ:

```
серверов = ceil(нагрузка_ДЦ / предел_одного_сервера) + 1
```

Здесь `ceil` — округление вверх (нельзя купить 2.3 сервера), а `+1` — запасной сервер, резерв N+1. Для баз добавляется минимальный размер кластера: Redis Cluster — 3 master, ScyllaDB с RF=3 — 3 узла. Отдельный `+1` для ScyllaDB не нужен: при RF=3 данные каждого узла есть ещё на двух.

Почему считаем по каждому ДЦ. Данные встречи живут в её `home_dc`, поэтому в каждом ДЦ свой набор кластеров. Сумма по ДЦ больше, чем «один расчёт на весь мир»: в каждом ДЦ своё округление вверх и свой запасной сервер.

Как получено число SFU — по шагам

```
1. Замер Jitsi: 550.4 Мбит/с при 20.3% загрузки 4-ядерного процессора
2. На ядро при 100% загрузки: 550.4 / 0.203 / 4 ≈ 678 Мбит/с ≈ 0.678 Гбит/с   (допущение: CPU
растёт почти линейно)
3. Грузим CPU на 70% — запас на пики: 0.678 × 0.7 ≈ 0.4745 Гбит/с на ядро
4. Сервер EL11 — 6 ядер: 6 × 0.4745 ≈ 2.847 Гбит/с  (сетевая карта 10 Гбит/с — не ограничитель)
5. Трафик SFU в пик: 78 131.25 медиа + 3 172.5 поток в запись = 81 303.75 Гбит/с
6. По каждому ДЦ: ceil(Гбит/с_ДЦ / 2.847) + 1  →  всего 28 570 серверов
```

Сверка с LiveKit. На видеопакетах LiveKit даёт ≈ 0.29 Гбит/с на ядро, на мелких пакетах — ≈ 0.08. SFU нагружают **пакеты в секунду**, а не биты: мелкие пакеты дороже. Поэтому оценка Jitsi **оптимистичная**: если реальные пакеты окажутся мельче, SFU-серверов понадобится больше. Это честно отмечено в работе.

Прошлая ошибка. Раньше на сервер закладывали 10 Гбит/с — по сетевой карте, без проверки процессора. Это в 2.5–6 раз больше, чем дают замеры Jitsi и LiveKit в пересчёте на 6 ядер.

Сколько всего серверов

Компонент	Серверов	Чем ограничен
SFU	28 570	CPU: 2.847 Гбит/с на сервер
Воркеры записи	1 127	CPU: так же, по аналогии
ScyllaDB чат / встречи	138 / 34	Диск: ≤ 1 ТБ данных на сервер
Узлы Kubernetes под сервисы	59	CPU подов: 572 ядра
NGINX L7	28	Память под соединения
PostgreSQL + Citus / `auth_pg`	24 / 9	Минимальный кластер: основной узел + standby + реплики
Redis	24	Операции в секунду, минимум 3 на ДЦ
Мониторинг	18	2 на ДЦ + 2 центральных
L4 (IPVS)	16	Резерв: 2 на ДЦ
Итого	**30 047**	

Поды Kubernetes. WebSocket-поды считаются по **соединениям**: 300 тысяч соединений на под (6 ГБ / 20 КБ) → 69 подов. Auth — 1 000 запросов в секунду на под → 20 подов. Control Plane — 1 500 → 23 пода. Chat — 5 000 → 19 подов. PgBouncer — 3 пода на ДЦ → 24.

Сколько это стоит

Статья	₽ в месяц	Доля
Хранение записей, S3: 685 260 000 ГБ × 0.89 ₽	609.88 млн	59.1%
SFU: 28 570 × 14 000 ₽	399.98 млн	38.8%
Воркеры записи	15.78 млн	1.5%

Всё остальное: балансировщики, сервисы, базы, мониторинг	6.23 млн	0.6%
Итого: серверы и хранение	**1 031.87 млн ₽ ≈ $12.15 млн**	

Внешний трафик — отдельно. Через порты SFU за месяц проходит ≈ 16.9 ЭБ. По публичному тарифу Selectel трафик сверх лимита стоил бы ещё ≈ 930 млн ₽ в месяц. Это **верхняя оценка**: на таком объёме трафик покупают по отдельному договору (транзит и пиринг), и выходит дешевле. Поэтому в итог он не включён и показан отдельно.

Изменения относительно версии, которую смотрел ревьюер (800 млн ₽): K = 1.5 вместе с новыми долями ДЦ — минус ≈ 120 млн ₽; проверка SFU по процессору — плюс ≈ 297 млн; рост цен Selectel с июля 2026 — плюс ≈ 55 млн.

Как удешевить. Ограничить хранение записей: 30 дней → 7 дней уменьшает главную статью в 4 раза. Записывать не каждую встречу. Агрессивнее понижать качество (ABR). Взять более эффективный видеокодек. Оптимизировать «всё остальное» бессмысленно — это 0.6%.

Что говорить. «Серверы считаются по каждому ДЦ: нагрузка делить на предел сервера, вверх, плюс один в резерв. Главное — SFU: в тесте Jitsi сервер прокачал 550 мегабит при 20% загрузки четырёх ядер; отсюда 0.68 гигабита на ядро, и шестиядерный сервер при загрузке 70% тянет 2.85 гигабита. На 81 терабит это 28.6 тысячи серверов. Всего 30 тысяч серверов, около миллиарда рублей в месяц: 59% — хранение записей, 39% — SFU, полтора процента — воркеры записи, всё остальное — меньше процента. Стоимость определяют те же блоки, что и нагрузку».

Вопросы.

- *Почему серверы Selectel, если его ДЦ нет в этих городах?* — Это ценовой ориентир для сопоставимого железа; для оценки порядка стоимости достаточно.
- *Почему именно EL11 для SFU?* — Это сервер из публичного прайса с сетевой картой 10 Гбит/с. SFU упирается в процессор, поэтому считаем по ядрам: 14 000 ₽ за 2.847 Гбит/с — около 4 900 ₽ за гигабит в секунду. Сетевая карта при этом загружена меньше чем на треть.
- *Почему трафик не в итоге?* — Публичный тариф на таком объёме неприменим, трафик покупают по договору. Показываем верхнюю оценку отдельно, чтобы было честно.
- *Что будет, если K = 3?* — Пиковые значения вырастут ровно вдвое, а число серверов SFU, NGINX и подов — примерно вдвое: мешают округление вверх и минимальные размеры кластеров. Хранение не изменится. Скрипт пересчитает всё за секунду.

Часть 6. Замечания ревьюера: что спросили и как отвечать

На прошлой защите ревьюер сделал шесть замечаний. Все закрыты. Если преподаватель спросит «что изменилось после ревью?», отвечайте по этой таблице.

Замечание	Что было не так	Что сделано	Где в README
K = 1.5	Пик считался с K = 3, все пиковые значения были вдвое больше	K = 1.5, всё пересчитано одним скриптом	2.1, 11
other_pool не понравился	42.5% нагрузки уходило в абстрактный «прочий пул»: без города, без состава стран, без серверов. Доли брались из SimilarWeb, который методичка запрещает	8 конкретных ДЦ. Доли — по ВВП стран из базы МВФ без Китая, США делятся по переписи	3.2
Зашарить схемы	Общую схему нельзя было открыть и увеличить, исходника не было	Исходник `.drawio` в репозитории, ссылки на просмотр в diagrams.net, таблица всех схем	10.1, 10.5
Где точка записи	Не было сказано, куда идут записи; для `users` стояло «любой здоровый ДЦ» — тогда нельзя гарантировать уникальность email	Отдельный подраздел: одна точка записи на каждый объект	3.4
Логин в США, потом Япония (вопрос Быкова)	Сценария не было	Пошаговый сценарий с диаграммой	3.8
Anycast или DNS	Для API были заявлены сразу оба механизма — противоречие	Однозначно DNS; Anycast — только у DNS-серверов; медиа переключает приложение	3.5–3.7

Короткие ответы для устной защиты.

1. **K.** «Коэффициент пика — отношение пикового часа к среднему за сутки. Для глобального сервиса кривая сглажена часовыми поясами, поэтому 1.5. От K линейно зависят пиковые нагрузки, но не объём хранения».
2. **Other pool.** «Абстрактного пула больше нет: каждая страна отнесена к ближайшему по задержке ДЦ, доля страны — её доля в ВВП по данным МВФ. Получилось 8 ДЦ в конкретных городах, у каждого есть резервный».
3. **Схемы.** «Исходник общей схемы лежит в репозитории, по ссылке она открывается в diagrams.net с масштабированием. Схемы БД — в dbdiagram без логина. Все ссылки собраны в одной таблице».
4. **Точка записи.** «У каждого объекта одна точка записи. Пользователи — одна основная база в Вирджинии, реплики во всех ДЦ. Встреча — в своём домашнем ДЦ, код которого зашит в идентификатор встречи. Список встреч пишется в домашнем ДЦ встречи и асинхронно копируется во все ДЦ».
5. **Япония.** «DNS отправит пользователя в Токио. Сессия один раз переедет по префиксу, перелогиниваться не нужно. Профиль и список встреч читаются локально, старые встречи живут в США — `join` проксируется туда. Мигрировать ничего не нужно».
6. **DNS или Anycast.** «DNS. GTM выбирает ДЦ с учётом здоровья и весов, переключение — около минуты. Anycast только у DNS-серверов: BGP не знает о загрузке ДЦ и при смене маршрута рвёт долгие соединения».

Что ещё найдено и исправлено сверх замечаний (на случай вопроса «а что ещё меняли?»):

- NGINX считался не той метрикой: весь поток делился на скорость новых TLS-рукопожатий, а память под миллионы WebSocket-соединений не учитывалась;

- SFU не был проверен по процессору: 10 Гбит/с на шестиядерный сервер — в 2.5–6 раз больше, чем дают замеры Jitsi и LiveKit в пересчёте на 6 ядер;
- серверы баз данных считались одной суммой на весь мир, хотя данные встречи живут в её ДЦ;
- облачный L4 провайдера не выдержал бы миллионы соединений — заменён своим на IPVS;
- в схеме БД были конструкции, которые не работают в этих СУБД: UNIQUE без колонки распределения в Citus, UNIQUE в ScyllaDB, вторичные индексы в Redis;
- добавлены метрики на пользователя, объёмы в штуках, кеши и буферы, балансировка запросов к БД и резервное копирование — их требует методичка;
- источники заменены первоисточниками, цены и курс обновлены.

Полный список — в ревью.md.

Часть 7. Словарь терминов

7.1. Нагрузка и расчёты

Термин	Что значит
MAU (monthly active users)	Уникальные пользователи за месяц. У нас 320 млн
DAU (daily active users)	Уникальные пользователи за день. У нас 115 млн
DMP (daily meeting participants)	Участия во встречах за день. Один человек на трёх встречах — 3 участия. У нас 300 млн
MVP	Минимальный набор функций, без которого продукт не имеет смысла
RPS (requests per second)	Запросов в секунду к сервису
QPS (queries per second)	Запросов в секунду к базе данных. Один запрос API может дать несколько запросов к БД
Пиковый коэффициент K	Во сколько раз пиковый час нагруженнее среднего за сутки. У нас 1.5
Средняя / пиковая нагрузка	Среднее за сутки / в самый нагруженный час. Серверы считаем по пиковой
Bandwidth, трафик	Объём данных в секунду по сети
бит / байт	1 байт = 8 бит. Сеть меряют в битах (Мбит/с, Гбит/с), диски — в байтах (МБ, ГБ)
Мбит/с, Гбит/с, Тбит/с	Мегабиты, гигабиты, терабиты в секунду; каждая следующая единица в 1 000 раз больше
ГБ, ТБ, ПБ, ЭБ	Гига-, тера-, пета-, эксабайт; каждая следующая в 1 000 раз больше
Participant-minutes	Сумма минут, проведённых всеми участниками во встречах
Закон Литтла	Среднее число объектов в системе = поток × время пребывания. Отсюда «онлайн в среднем»
Heartbeat	Сообщение «я ещё здесь», у нас раз в 25 секунд
Presence	Знание о том, кто сейчас онлайн во встрече
Fan-out	Рассылка одного события многим: сообщение чата уходит 9 участникам
Worst case, верхняя оценка	Расчёт для худшего случая: например, «записывается каждая встреча»
Допущение	Значение, которое мы приняли сами, потому что публичных данных нет
ceil	Округление вверх: 2.3 сервера → 3
Ограничитель	Ресурс, в который упирается сервер первым: CPU, память, сеть или диск
Горячий ключ (hot key)	Ключ с непропорционально большой нагрузкой, например огромная встреча

7.2. Сеть и балансировка

Термин	Что значит
ДЦ (дата-центр), регион	Площадка с серверами. В работе ДЦ — регион провайдера из 3 зон доступности
AZ (availability zone)	Зона доступности: отдельное здание внутри региона со своим питанием и сетью
RTT (round-trip time)	Время «туда и обратно» для пакета. Токио ↔ Вашингтон ≈ 153 мс
Задержка (latency)	Время доставки данных. Для разговора важно не больше 150 мс в одну сторону «от рта до уха» (ITU-T G.114)
DNS	Система, которая превращает имя (`api.meetflow.example`) в IP-адрес
GeoDNS	DNS, который отвечает разным пользователям по-разному в зависимости от того, где они

GTM (global traffic manager)	Сервис глобальной балансировки поверх DNS: учитывает здоровье ДЦ, задержку и веса
TTL в DNS	Сколько секунд клиент может помнить DNS-ответ. У нас 30 с
ECS (EDNS Client Subnet)	Подсказка DNS-серверу о подсети клиента, чтобы точнее определить его местоположение
Health check	Регулярная проверка «жив ли сервис»
Веса, drain	Веса задают, какую долю трафика получает ДЦ. Drain — плавный вывод из работы: новых не пускаем, старые доживают
Anycast	Один IP-адрес, объявленный из многих точек мира; пакет приходит в ближайшую по маршрутам
BGP	Протокол, которым сети интернета договариваются о маршрутах
AS-path	Цепочка сетей (автономных систем) на пути пакета; по ней BGP выбирает маршрут
PoP (point of presence)	Точка присутствия сети провайдера
VIP (virtual IP)	Общий адрес входа в ДЦ, за которым стоят несколько серверов
L4 / L7	Уровни модели OSI: L4 — транспортный (адреса и порты), L7 — прикладной (содержимое HTTP)
L4-балансировщик	Раздаёт соединения по серверам, не заглядывая внутрь
L7-балансировщик	Читает запрос (путь, заголовки) и решает, в какой сервис его отправить
IPVS	Балансировщик L4 в ядре Linux
Direct Routing	Режим IPVS: ответ идёт от сервера к клиенту напрямую, мимо балансировщика
keepalived	Программа, которая проверяет здоровье серверов и переносит VIP на резервный балансировщик
VRRP	Протокол, по которому резервный балансировщик забирает адрес VIP у упавшего
NGINX	Веб-сервер и L7-балансировщик
TLS / SSL	Шифрование соединения. HTTPS = HTTP + TLS. SSL — старое название
TLS handshake	«Рукопожатие» при открытии защищённого соединения: самая дорогая по CPU часть TLS
TLS termination	Расшифровка TLS на балансировщике
CPS / TPS	Новых соединений в секунду / новых TLS-соединений в секунду
Keep-alive	Повторное использование открытого соединения для многих запросов
TCP / UDP	TCP — надёжная доставка с подтверждениями; UDP — быстрая без гарантий, нужна для живого видео
WebSocket, WSS	Постоянное двустороннее соединение браузера с сервером; WSS — с шифрованием
gRPC	Протокол вызовов между сервисами поверх HTTP/2
mTLS	TLS, в котором и клиент, и сервер предъявляют сертификаты
round-robin	Раздача запросов по кругу: первому, второму, третьему, снова первому
CDN	Сеть серверов по миру, которая раздаёт статику из ближайшей точки
Presigned URL	Временная подписанная ссылка на файл в S3: скачать можно без нашего сервера

7.3. Медиа

Термин	Что значит
WebRTC	Стандарт браузера для звонков в реальном времени
RTP / SRTP	Протокол передачи аудио и видео поверх UDP / то же с шифрованием

RTCP	Служебные сообщения внутри медиасессии: потери, джиттер, время доставки
SFU (selective forwarding unit)	Медиасервер, который пересылает видеопотоки, не перекодируя
MCU (multipoint control unit)	Медиасервер, который склеивает все потоки в одну картинку: дорого по CPU
P2P	Каждый с каждым напрямую, без сервера
Simulcast	Отправка видео сразу в нескольких качествах (слоях)
SVC	То же одним потоком со слоями внутри
ABR (adaptive bitrate)	Подстройка качества видео под текущую сеть
Профиль, слой	Конкретное качество видео: 720p, 360p, 180p
Битрейт	Сколько бит в секунду занимает поток
Jitter	Неравномерность задержки пакетов
Packet loss	Доля потерянных пакетов
Кодек	Алгоритм сжатия видео или звука
NAT	Механизм, при котором много устройств выходят в интернет через один адрес
ICE	Механизм WebRTC для поиска рабочего сетевого пути между клиентом и сервером
ICE-Lite	Упрощённый ICE на сервере с публичным адресом: сервер отвечает на проверки клиента
ICE restart	Повторный поиск сетевого пути внутри того же медиасоединения — например, когда телефон перешёл с Wi-Fi на мобильную сеть. Для перехода на другой SFU его мало: нужно новое медиасоединение
DTLS	Рукопожатие, которым клиент и SFU договариваются о ключах шифрования медиа (DTLS-SRTP). С новым SFU оно проходит заново
STUN / TURN	STUN помогает клиенту узнать свой внешний адрес; TURN — релей, который пересылает медиа, если напрямую нельзя
Каскад SFU	Цепочка SFU в разных ДЦ: клиент подключается к ближайшему, тот пересылает потоки в SFU встречи
Active speaker	Участник, который сейчас говорит; его видео остальным отдают слоем выше

7.4. Данные

Термин	Что значит
Primary / replica	Основная копия, принимает записи / копия для чтения
Точка записи	Место, где находится primary для данного объекта
Синхронная репликация	Запись подтверждается, только когда её сохранила и реплика: надёжно, медленнее
Асинхронная репликация	Запись подтверждается сразу, реплика догоняет позже: быстро, но при аварии можно потерять последние секунды
Standby	Резервная копия, готовая стать основной
Read-your-writes	Пользователь сразу видит свои изменения: после записи пару секунд читает из primary
Strong consistency	Читатель всегда видит последнюю версию

Eventual consistency	Читатель увидит последнюю версию с небольшой задержкой
Multi-master	Запись разрешена в нескольких местах сразу; требует разрешения конфликтов. В работе не используется
Шардирование, шард	Деление данных на части по разным серверам / одна часть
Ключ шардирования, колонка распределения	Поле, по которому решается, в какой шард идёт строка (`meeting_id`)
Bucket	Дополнительная часть ключа, чтобы разбить большую встречу на части
Репликация	Хранение копий данных на нескольких серверах
RF (replication factor)	Сколько копий хранится. У нас RF = 3
Кворум	Большинство копий: 2 из 3
LOCAL_QUORUM	Кворум внутри своего ДЦ, без ожидания других континентов
NetworkTopologyStrategy	Настройка ScyllaDB: сколько копий хранить в каждом ДЦ
Keyspace	В ScyllaDB — аналог базы данных со своими настройками репликации
Индекс	«Оглавление» таблицы для быстрого поиска
PK / UNIQUE	Первичный ключ / ограничение уникальности
JOIN	Объединение строк нескольких таблиц в одном запросе
Денормализация	Сознательное дублирование данных ради быстрого чтения
Read model	Таблица, заточенная под один запрос на чтение (`user_meetings`)
Кеш	Копия горячих данных в памяти
TTL в Redis	Время жизни ключа: потом ключ удаляется сам
Идемпотентность	Повтор операции не меняет результат: повторная отправка сообщения не создаёт дубль
SET NX	Команда Redis «записать, только если ключа ещё нет»
Hash tag, слот	В Redis Cluster ключи делятся на 16 384 слота; часть ключа в фигурных скобках кладёт связанные ключи в один слот
PostgreSQL	Реляционная база данных с транзакциями
Citus	Расширение PostgreSQL для шардирования
Co-location	В Citus: шарды связанных таблиц с одинаковым ключом лежат на одном узле
ScyllaDB	Распределённая NoSQL-база, совместимая с Cassandra, сильна в записи
CQL	Язык запросов ScyllaDB и Cassandra, похож на SQL
Compaction, STCS	В ScyllaDB — слияние файлов данных на диске; стратегия STCS в худшем случае требует половину диска свободной
Redis, Redis Cluster	База данных в памяти / её вариант с шардами и репликами
PgBouncer	Пул соединений к PostgreSQL
Transaction pooling	Режим PgBouncer: соединение с базой выдаётся клиенту только на время транзакции
S3, объектное хранилище	Хранилище файлов по ключу, «бесконечный диск» с оплатой за гигабайт
Multipart upload	Загрузка большого файла частями, с возможностью дозагрузить упавшую часть
Lifecycle	Правила S3: что удалять или переносить в более дешёвый класс хранения со временем
Ледяное хранение	Самый дешёвый класс хранения S3 у Selectel — 0.89 ₽ за ГБ в месяц. Хранить дёшево, а запросы к файлам дороже, чем в стандартном классе; подходит для данных, которые читают редко
WAL (write-ahead log)	Журнал PostgreSQL: изменение сначала пишется в журнал, потом в таблицу
PITR	Восстановление базы на любой момент времени по архиву WAL

Снапшот	Мгновенный снимок данных узла для бэкапа
UUID, UUIDv8	128-битный идентификатор; в версии 8 часть битов можно занять своими данными — у нас кодом ДЦ
DBML	Язык описания схем БД для dbdiagram.io

7.5. Инфраструктура и надёжность

Термин	Что значит
Kubernetes	Система, которая запускает контейнеры на кластере серверов и сама их перезапускает
Под (pod) / узел (node)	Запущенный экземпляр сервиса / сервер кластера
Контейнер, Docker	Упаковка программы со всеми зависимостями
Helm	Шаблоны для развёртывания сервисов в Kubernetes
Stateless	Сервис не хранит состояние у себя: любой экземпляр обработает любой запрос
N+1 / N×2	Один запасной узел / двойной комплект
HA (high availability)	Высокая доступность: работа без простоя при отказе части системы
Failover	Переключение на резерв при отказе
Runbook	Пошаговая инструкция дежурному на случай аварии
Split brain	Две части системы одновременно считают себя главными — например, две основные базы
Patroni	Инструмент автоматического failover для PostgreSQL
RPO / RTO	Сколько данных можно потерять / за сколько восстанавливаемся
Graceful degradation	Работать хуже, но не падать
Шторм переподключений	Миллионы клиентов одновременно переподключаются после аварии; лечится случайной задержкой
Observability, наблюдаемость	Метрики, логи, трассировки
Prometheus / Grafana / Loki / Jaeger / OpenTelemetry	Метрики / дашборды / логи / трассировки запросов / единый сбор телеметрии
Managed-сервис	Сервис, который обслуживает провайдер: DNS, Kubernetes, S3
vCPU, Hyper-Threading	Виртуальное ядро; Hyper-Threading делает из одного физического ядра два логических
10GE	Сетевая карта 10 Гбит/с
Транзит, пиринг	Транзит — доступ ко всему интернету, купленный у магистрального оператора по договору; на больших объёмах дешевле публичного тарифа. Пиринг — прямой обмен трафиком с другой сетью, часто без оплаты

7.6. Источники и экономика

Термин	Что значит
SEC	Комиссия по ценным бумагам США; туда публичные компании подают отчётность
10-K	Годовой отчёт публичной компании в SEC
Проспект (424B5)	Документ для инвесторов при выпуске акций; у Zoom там число участников встреч
Earnings call	Звонок компании с инвесторами по итогам квартала; у Microsoft там 320 млн MAU
FY (fiscal year)	Финансовый год компании; у Zoom FY2026 закончился 31 января 2026
Run rate	Годовой темп: показатель за период, пересчитанный на год

ВВП	Стоимость всего, что произвела экономика страны за год
IMF WEO	База МВФ World Economic Outlook с ВВП всех стран
ARPU	Средняя выручка на пользователя
НДС	Налог; цены Selectel указаны с НДС 22%
Egress	Исходящий трафик, за который берут плату

Часть 8. Вопросы и ответы

Вопросы, которые вероятнее всего зададут, сгруппированы по темам. Отвечайте коротко: сначала вывод, потом одно-два обоснования, потом число.

8.1. Нагрузка

1. **Откуда 30 млн встреч?** — 300 млн участий в день у Zoom делим на 10 участников средней встречи.
2. **Откуда 16.9 млн онлайн в пик?** — 300 млн участий × 54 минуты = 16.2 млрд участие-минут; делим на 1 440 минут в сутках — 11.25 млн в среднем; × 1.5 в пик.
3. **Откуда 78 Тбит/с?** — 16.875 млн участников в пик × 4.63 Мбит/с на участника по требованиям Zoom к 720p.
4. **Почему вверх больше, чем вниз?** — Simulcast: каждый отправляет несколько качеств, а получает от каждого собеседника одно.
5. **Какой запрос самый частый?** — Heartbeat: 675 тысяч в секунду, в 130 раз чаще входа во встречу.
6. **Не завышены ли числа?** — Завышены, и это осознанно: мы считаем, что каждый участник сидит все 54 минуты. По данным Zoom реальный объём минут ≈ 9.6 млрд в сутки против наших 16.2 млрд. Ошибка в безопасную сторону.
7. **Почему K = 1.5?** — Глобальный сервис: часовые пояса сглаживают общую кривую. Значение задал ревьюер.

8.2. Дата-центры и балансировка

8. **Почему 8 ДЦ и такие доли?** — Доля страны — её доля в ВВП (МВФ), каждая страна отнесена к ближайшему ДЦ, США делятся по переписи. Китай не обслуживаем.
9. **Почему не SimilarWeb?** — Методичка запрещает его как первоисточник из-за погрешности.
10. **Почему у Zoom Америка — 72% выручки, а у вас 41% нагрузки?** — Zoom исторически сильнее в США, а наш масштаб — масштаб лидера рынка. Мы берём нейтральную оценку по экономике. Если доля Америки выше, меняются только размеры пулов, а не архитектура.
11. **Как пользователь попадает в ДЦ?** — Через DNS: GeoDNS отвечает адресом ближайшего здорового ДЦ.
12. **Anycast или DNS?** — DNS. Anycast — только у DNS-серверов.
13. **Почему не Anycast для API?** — BGP не знает о загрузке ДЦ: нельзя задать веса и drain. Смена маршрута рвёт долгие соединения.
14. **За сколько переключается ДЦ?** — Около минуты: 3 проверки по 10 с + TTL 30 с.
15. **А если резолвер игнорирует TTL?** — У клиента свой fallback: при ошибке он заново резолвит имя, затем идёт на резервный адрес из конфигурации.
16. **Что такое `home_dc`?** — ДЦ, за которым закреплена встреча: тот, что обработал её создание. Код зашит в `meeting_id`.
17. **Зачем закреплять встречу за ДЦ?** — Все участники должны быть на одном SFU, а данные встречи — в одном месте со строгой консистентностью.
18. **Участники из разных стран в одной встрече?** — Все подключаются к SFU в `home_dc`. Улучшение после MVP — каскад SFU.

8.3. Точки записи и консистентность

19. **Где точка записи пользователей?** — Один primary в `DC-US-EAST`, реплики во всех ДЦ.
20. **Не узкое место?** — Нет: записей не больше 1 997 в секунду даже в худшем случае, чтения идут из локальных реплик.
21. **Что, если реплика отстаёт?** — После своей записи пользователь пару секунд читает из primary

(read-your-writes).

22. **Что при отказе `DC-US-EAST`?** — Логин работает на репликах. Регистрация и правка профиля ждут, пока дежурный повысит реплику в `DC-US-WEST` — минуты.
23. **Почему переключение между ДЦ ручное?** — Иначе возможен split brain: автомат не отличит падение ДЦ от обрыва связи.
24. **Пользователь улетел из США в Японию?** — DNS отправит в Токио, сессия переедет по префиксу, перелогиниваться не нужно, старые встречи — в США, новые — в Токио, мигрировать ничего не надо.

8.4. Медиа

25. **Почему SFU, а не MCU?** — MCU перекодирует видео — огромный CPU. SFU только пересылает пакеты.
26. **Почему медиа мимо балансировщиков?** — 78 Тбит/с не нужно пропускать через L4 и L7, SRTP уже зашифрован, встреча должна жить на одном SFU.
27. **Как клиент за NAT подключается к SFU?** — У SFU публичный адрес, он работает как ICE-Lite: соединение начинает клиент, поэтому для связи с SFU STUN и TURN не нужны. Если UDP закрыт — подключение к тому же SFU по TCP. Не поддерживаем в MVP только сети, где прокси пропускает исключительно TLS на порт 443: для них нужен TURN по TLS, это после MVP.
28. **Что при отказе SFU?** — Control plane назначает встрече новый SFU и присылает клиентам его адрес по WebSocket. Клиенты заново устанавливают медиасоединение — новое ICE-согласование и DTLS-рукопожатие — и заново публикуют свои потоки. ICE restart тут не подходит: он ищет новый путь к тому же серверу.
29. **Почему 2.85 Гбит/с на сервер?** — Замер Jitsi: 550.4 Мбит/с при 20.3% загрузки четырёх ядер. CPU растёт почти линейно с битрейтом, поэтому экстраполируем — это наше допущение: ≈ 0.678 Гбит/с на ядро при 100%. Грузим на 70% → 0.4745 на ядро, × 6 ядер = 2.847 Гбит/с. Сетевая карта 10 Гбит/с не ограничитель — упираемся в CPU.
30. **Не оптимистично ли?** — Оптимистично: на мелких пакетах LiveKit даёт меньше. Это отмечено в работе; если пакеты окажутся мельче, серверов понадобится больше.
31. **Почему верхний профиль 720p?** — На нём построен расчёт трафика. 1080p требует 3.8/3.0 Мбит/с и увеличил бы трафик и число SFU.

8.5. Балансировщики

32. **Почему NGINX упирается в память?** — WebSocket держит соединение весь звонок: 16.9 млн соединений по 54 КБ.
33. **Почему свой L4, а не облачный?** — Облачный балансировщик Selectel тестирован на 10 тысяч соединений и до 3–5 Гбит/с, а у нас до 6 млн соединений в одном ДЦ.
34. **Как L4 переживает отказ?** — Два директора на ДЦ: keepalived переносит VIP на резервный по VRRP.
35. **Почему значения NGINX для 16 ядер, а сервер 24 vCPU?** — В тесте «CPU» — физические ядра; 24 vCPU — это примерно 12 физических ядер с Hyper-Threading, ближайшая точка теста — 16.

8.6. Базы данных

36. **Почему разные СУБД?** — Каждому паттерну доступа своя: транзакции — PostgreSQL, много записей — ScyllaDB, горячие данные — Redis, файлы — S3.
37. **Зачем Citus?** — Шардирование PostgreSQL по `meeting_id` на вырост. Сейчас в каждом ДЦ Citus работает в режиме single-node: координатор сам хранит шарды. Рядом синхронная standby, плюс реплика в парном ДЦ — всего 24 сервера. При росте добавляются worker-узлы без переделки приложения.

38. **Почему нет JOIN?** — На больших нагрузках JOIN дорогой, а между разными СУБД невозможен. Вместо него — денормализованные проекции.
39. **Как избежать дублей сообщений?** — `client_msg_id` проверяется в Redis через `SET NX`; повтор получает тот же номер.
40. **Почему не UNIQUE в ScyllaDB?** — Его там нет: `INSERT` с тем же ключом перезаписывает строку.
41. **Что такое RF=3 и кворум?** — Три копии; пока живы две из трёх, можно читать и писать.
42. **Как бэкапите?** — PostgreSQL — непрерывный архив WAL (отстаёт не больше чем на минуту) и базовая копия раз в сутки; ScyllaDB — снапшоты раз в сутки; всё в S3 парного ДЦ. Redis не бэкапим — данные эфемерные.

8.7. Деньги

43. **Сколько стоит?** — ≈ 1 032 млн ₽ в месяц, ≈ $12.15 млн: 59% — хранение записей, 39% — SFU.
44. **Почему записи — главный расход?** — 30 млн встреч × 0.761 ГБ × 30 дней = 685 ПБ. Даже в самом дешёвом, ледяном классе S3 по 0.89 ₽ за ГБ это 610 млн ₽ в месяц — 59% бюджета.
45. **Как удешевить?** — Ретеншн и квоты на записи — главный рычаг; записывать не каждую встречу; ABR; более эффективный кодек.
46. **Почему трафик не включён в итог?** — По публичному тарифу это ещё ≈ 930 млн ₽, но на таком объёме трафик покупают по договору. Показываем отдельно как верхнюю оценку.
47. **Почему цены Selectel?** — Реальный публичный прайс сопоставимого железа как ориентир цены.

Часть 9. Шпаргалка и речь на 5 минут

9.1. Числа на одну страницу

Что	Число
MAU / DAU / участий в день	320 млн / 115 млн / 300 млн
Средняя встреча	10 человек, 54 мин
K	1.5
Встреч в сутки / одновременно в пик	30 млн / 1.69 млн
Онлайн в среднем / в пик (= WebSocket-соединений)	11.25 млн / 16.9 млн
Медиа в пик	78 Тбит/с (+3.2 Тбит/с поток в запись)
Heartbeat в пик	675 тыс./с
Вход во встречу в пик	5 208/с, 31 250 строк/с
Чат в пик	26 042 сообщений/с
Записи	0.761 ГБ на встречу, 22.8 ПБ в сутки, 685 ПБ за 30 дней
Хранение без записей	47.75 ТБ
ДЦ	8: Франкфурт 35.69%, Вирджиния 28.84%, Сан-Хосе 7.40%, Токио 7.36%, Сингапур 6.97%, Мумбаи 5.03%, Сан-Паулу 4.65%, Лондон 4.06%
Переключение ДЦ	≈ 1 минута
Токио ↔ Вашингтон	≈ 153 мс туда-обратно
NGINX L7	28 узлов (N+1), 40 (N×2); ограничитель — память
L4	16 директоров IPVS, по 2 на ДЦ
SFU	2.847 Гбит/с на сервер, 28 570 серверов
Серверов всего	30 047
Стоимость	≈ 1 032 млн ₽/мес ≈ $12.15 млн; S3 59%, SFU 39%
Внешний трафик (верхняя оценка, не в итоге)	≈ 930 млн ₽/мес

9.2. Речь на 5 минут

По методичке на презентацию 5 минут и 5–15 слайдов. Ниже текст по слайдам, ≈ 30 секунд на слайд. Числа в нём — текущие; в старых слайдах `MeetFlow_10min.pptx` и `MeetFlow_presentation.pptx` числа устарели (K = 3, 5 ДЦ, 156 Тбит/с), их нужно сверить с этой шпаргалкой.

1. **Титул.** «MeetFlow — сервис групповых видеовстреч, как Zoom. Масштаб лидеров рынка: 320 миллионов пользователей в месяц, 115 миллионов в день, 300 миллионов участий во встречах в сутки».
2. **Функции и допущения.** «Шесть функций MVP: создание встречи, вход по ссылке, видео 720p, демонстрация экрана, чат, запись. Средняя встреча — 10 человек на 54 минуты, значит 30 миллионов встреч в сутки. Коэффициент пика — 1.5».
3. **Нагрузка.** «Главная нагрузка — не API. В пик онлайн 16.9 миллиона человек: это 78 терабит в секунду медиа, 675 тысяч heartbeat в секунду и 16.9 миллиона открытых соединений. Записи за 30 дней — 685 петабайт».
4. **Глобальная балансировка.** «Восемь ДЦ, доли по ВВП стран из базы МВФ. ДЦ выбирает DNS — GeoDNS с проверками здоровья и весами, переключение за минуту. Anycast только у DNS-серверов:

BGP не знает о загрузке ДЦ и рвёт долгие соединения».

5. **Точки записи.** «У каждого объекта одна точка записи. Пользователи — основная база в Вирджинии, реплики везде. Встреча — в своём домашнем ДЦ, код которого зашит в её идентификатор. Пользователь, улетевший в Японию, работает без перелогина и без миграции данных».
6. **Control plane ≠ media plane.** «Управляющие запросы идут через L4 на IPVS и NGINX. NGINX упирается в память под соединения — 28 узлов. Видео идёт мимо балансировщиков, напрямую в SFU».
7. **Медиа.** «SFU, а не MCU: пересылаем, а не перекодируем. Simulcast и адаптивный битрейт. Сервер SFU упирается в процессор — 2.85 гигабита, отсюда 28.6 тысячи серверов».
8. **Данные.** «Каждому паттерну — своя СУБД: PostgreSQL для пользователей, Citus для метаданных встреч, ScyllaDB для участников и чата, Redis для онлайн-состояния, S3 для записей. JOIN в горячем пути нет».
9. **Надёжность.** «N+1 на серверах, три копии данных в трёх зонах. При отказе целого ДЦ DNS уводит пользователей, а видео временно понижается — нагрузка падает до 42% при оставшихся 64% ёмкости».
10. **Стоимость и вывод.** «30 тысяч серверов, около миллиарда рублей в месяц: 59% — хранение записей, 39% — SFU. Стоимость определяют те же блоки, что и нагрузку: медиа и записи. Спасибо, готов к вопросам».

9.3. Как отвечать, если не знаете ответ

- Вернитесь к цепочке «аудитория → нагрузка → пик → компоненты → предел узла → серверы → деньги» и найдите звено, о котором вопрос.
- Скажите, от какого допущения это зависит и в какую сторону изменится результат.
- Честное «это допущение, вот чем оно ограничено» лучше уверенной ошибки.

Источники

Нумерация совпадает со списком источников в README.

1. Microsoft Investor Relations — Earnings Conference Call FY24 Q1 (транскрипт: «more than 320 million monthly active users» у Teams) ↩
2. Microsoft 365 Blog — Microsoft Teams reaches 115 million DAU ↩
3. Zoom Video Communications — Prospectus Supplement (Form 424B5), SEC, 13.01.2021: «In April 2020, our platform surpassed more than 300 million daily meeting participants»; run rate более 3.5 трлн минут встреч в год на 31.10.2020 ↩
4. Zoom Blog — Here's How You Zoomed Over the Past Year ↩
5. Zoom Support — Zoom system requirements: Zoom Web App ↩↩↩↩
6. Zoom Support — Zoom Chat specifications and limitations ↩
7. Zoom Support — Cloud recording storage capacity ↩
8. Socket.IO Documentation — Server options ↩
9. Zoom Investor Relations — Zoom Communications Reports Fourth Quarter and Fiscal Year 2026 Financial Results ↩
10. IMF — World Economic Outlook (April 2026): GDP, current prices (NGDPD), млрд долларов США, данные 2025 года ↩
11. NGINX Community Blog — Testing the Performance of NGINX Ingress Controller for Kubernetes ↩↩
12. NGINX Community Blog — Testing the Performance of NGINX and NGINX Plus Web Servers ↩
13. Selectel — Тарифы и цены на услуги: выделенные серверы ↩↩↩
14. Selectel — Тарифы и цены на услуги: облачные серверы и балансировщики ↩
15. Selectel — Тарифы и цены на услуги: Managed Kubernetes ↩
16. Selectel — Тарифы и цены на услуги: S3 ↩
17. Zoom Communications — Form 10-K за FY2026 (выручка по регионам: Americas 72.1%, APAC 12.1%, EMEA 15.8%) ↩
18. U.S. Census Bureau — Age and Sex Composition: 2020 (регион West — 78.6 млн, 23.7% населения) ↩
19. Jitsi — Jitsi Videobridge Performance Evaluation ↩
20. LiveKit Docs — Benchmarks ↩
21. NGINX (F5) Blog — NGINX WebSocket Performance ↩
22. OpenSSL Documentation — SSL_CTX_set_mode, SSL_MODE_RELEASE_BUFFERS ↩
23. Redis Docs — Redis benchmark ↩
24. WonderNetwork — Ping time between Tokyo and Washington ↩
25. Linux kernel — net/netfilter/ipvs/Kconfig: запись в таблице соединений IPVS занимает 128 B, элемент хеш-таблицы — 8 B ↩
26. ScyllaDB Documentation — Compaction Strategies: STCS в худшем случае требует свободной половины диска ↩
27. Банк России — Официальные курсы валют на 10.10.2026: 84.9048 ₽ за $1 ↩

Приложение A. Полный текст работы (README.md)

Ветка `review-fixes`, коммит `226cba1`. Все таблицы, схемы и источники — как в репозитории.

Проектирование высоконагруженного Zoom-подобного сервиса видеовстреч

Изменения после ревью

Замечание	Что сделано	Где
`K = 1.5`	`MF_K_PEAK = 1.5` вместо 3; все пиковые значения, балансировщики, серверы и стоимость пересчитаны одним скриптом `meetflow_calc.py`	2.1, 11
`other_pool` не понравился	Абстрактного `OTHER_REGIONAL_POOL` больше нет: каждая группа стран привязана к одному из 8 ДЦ с городами и парами для резерва. Доли ДЦ считаются по ВВП стран (IMF WEO) без материкового Китая, США делятся по переписи. SimilarWeb, который методичка запрещает как первоисточник, не используется	3.2
Зашарить схемы	Исходник общей схемы в репозитории, ссылки на просмотр в diagrams.net; все схемы со ссылками в одной таблице	10.1, 10.5
Где в глобальной балансировке точка записи	Отдельный подраздел: `users` — один primary в `DC-US-EAST` + реплики; данные встречи — `home_dc`; `user_meetings` — мультиДЦ-репликация	3.4
Пользователь залогинился в США и поехал в Японию	Пошаговый сценарий с sequence-диаграммой: без повторного логина и без миграции данных	3.8
Механизм переключения: Anycast или DNS	**DNS (GeoDNS/GTM)**; Anycast — только для авторитетных DNS-серверов; медиа переключает приложение (новый SFU, новое медиасоединение)	3.5 – 3.7

Попутно исправлено:

- расчёт L7: теперь учтена память под одновременные WSS-соединения;
- ёмкость SFU проверена по CPU;
- L4 — свой (IPVS + keepalived): облачный балансировщик провайдера не рассчитан на миллионы соединений;
- медиа идёт в SFU напрямую, мимо L4;
- серверы данных посчитаны по каждому ДЦ;

- цены обновлены на 11.10.2026, отдельно оценён внешний трафик;
- добавлены сводные продуктовые метрики, схема резервного копирования и балансировка запросов к БД;
- несостыковки между разделами устранены.

Содержание

- Основная часть
- 1. Тема и целевая аудитория
  - 1.1. Тип сервиса
  - 1.2. Рыночная ниша и реальные аналоги
  - 1.3. Функционал MVP
  - 1.4. Ключевые продуктовые решения
  - 1.5. Целевая аудитория
- 2. Расчёт нагрузки
  - 2.1. Исходные метрики и методика расчёта
  - 2.2. Продуктовая оценка видеовстреч
  - 2.3. Хранилище пользователя и техническое хранение
  - 2.4. Сетевой трафик и RPS
- 3. Глобальная балансировка нагрузки
  - 3.1. Функциональное разбиение по доменам
  - 3.2. Выбор и расположение дата-центров
  - 3.3. Распределение нагрузки по дата-центрам
  - 3.4. Точки записи данных
  - 3.5. Схема DNS-балансировки — основной механизм переключения
  - 3.6. Схема Anycast-балансировки
  - 3.7. Регулировка трафика и переключение между дата-центрами
  - 3.8. Сценарий: вход в США, затем поездка в Японию
  - 3.9. Вывод по глобальной балансировке
- 4. Локальная балансировка нагрузки
  - 4.1. Границы локальной балансировки
  - 4.2. Схема локальной и межсервисной балансировки
  - 4.3. Механизмы резервирования
  - 4.4. Ограничители для расчёта

    - 4.5. Расчёт количества L7-балансировщиков
    - 4.6. Итоговое количество балансировщиков
- 5. Логическая схема БД
    - 5.1. Список логических таблиц
    - 5.2. Размеры данных и QPS
    - 5.3. Требования к консистентности
    - 5.4. Распределение нагрузки по ключам
- 6. Физическая схема БД
    - 6.1. Физическая схема БД
    - 6.2. Где хранятся данные
    - 6.3. Выбор СУБД по таблицам
    - 6.4. Индексы
    - 6.5. Шардирование и размещение по дата-центрам
    - 6.6. Нагрузка на строки
    - 6.7. API -> запрос -> БД
    - 6.8. Кеш
    - 6.9. Денормализация и отказ от JOIN
    - 6.10. Клиентские библиотеки
    - 6.11. Балансировка запросов и мультиплексирование подключений
    - 6.12. Резервное копирование
- 7. Алгоритмы
    - 7.1. Список алгоритмов
    - 7.2. Adaptive Bitrate
    - 7.3. Выбор SFU-слоя видео
- 8. Технологии
    - 8.1. Языки
    - 8.2. Сводная таблица технологий
- 9. Обеспечение надёжности
    - 9.1. Резервирование компонентов
    - 9.2. Отказ целого дата-центра
- 10. Схема проекта

- 10.1. Общая схема взаимодействия
- 10.2. Как читать схему
- 10.3. Потоки данных
- 10.4. Балансировка: внешняя и внутренняя
- 10.5. Исходники и ссылки на схемы

- 11. Список серверов
  - 11.1. Начальные нормативы для расчёта ресурсов
  - 11.2. Расчёт ресурсов по сервисам
  - 11.3. Итоговая таблица серверов
  - 11.4. Оценка стоимости инфраструктуры

- Список источников

Основная часть

1. Тема и целевая аудитория

1.1. Тип сервиса

Параметр	Значение
Учебное название	**MeetFlow**
Тип сервиса	Zoom-подобный сервис видеовстреч
Основной сценарий	Групповая онлайн-встреча с аудио, видео, демонстрацией экрана, чатом и записью
Целевые регионы	Глобально, кроме материкового Китая: Америка, EMEA, Азиатско-Тихоокеанский регион; крупнейшие экономики среди обслуживаемых стран — США, Германия, Япония, Великобритания, Индия [11]
Реальные аналоги	Zoom, Microsoft Teams, Google Meet
Масштаб для расчёта	`MF_MAU = 320 млн MAU`, `MF_DAU = 115 млн DAU`, `MF_DMP = 300 млн участий во встречах в день` [1,2,3]

1.2. Рыночная ниша и реальные аналоги

Аналог	Метрика	Что подтверждает
Microsoft Teams	320 млн monthly active users — Microsoft на звонке по итогам FY24 Q1, октябрь 2023 [1]	Массовый размер аудитории коммуникационного сервиса
Microsoft Teams	115 млн daily active users, октябрь 2020 [2]	Нижнюю публичную границу daily-аудитории для расчёта
Zoom	Более 300 млн daily meeting participants в апреле 2020 — из проспекта Zoom, поданного в SEC; это участники встреч, а не unique users [3]	Массовую ежедневную нагрузку именно на видеовстречи
Zoom	Средняя встреча — 54 минуты и 10 участников, данные за 15.11.2020–15.11.2021 [4]	Базу для продуктовой оценки видеозвонков
Zoom	$4.8688 млрд выручки за FY2026 и 4 468 клиентов с TTM-выручкой больше $100 000 [10]	Актуальность ниши на 2026 год

1.3. Функционал MVP

№	Функция MVP	Кратко	Метрика для расчёта
1	Создание встречи	Пользователь создаёт мгновенную или запланированную встречу	`MF_MEETINGS_DAY`, `POST /meetings`
2	Подключение к встрече	Участник входит по ссылке или приглашению	`MF_DMP`, `POST /meetings/{id}/join`
3	Групповая аудио- и видеосвязь	Участники общаются в 720p group video	`MF_VIDEO_UP_MBPS`, `MF_VIDEO_DOWN_MBPS`, `MF_AUDIO_MBPS`
4	Демонстрация экрана	Участник показывает экран	`MF_SCREEN_MBPS_MAX`
5	Чат во время встречи	Участники отправляют текстовые сообщения	`MF_CHAT_MSG_LIMIT`, `MF_CHAT_PER_JOIN`

6	Облачная запись	Организатор включает запись встречи	`MF_RECORD_QUOTA_GB`, `MF_RECORDING_SIZE_GB`

1.4. Ключевые продуктовые решения

Решение	Значение для MVP	Что считается дальше
Расчёт ведётся от групповой встречи	Средняя встреча: `MF_MEET_SIZE = 10` участников	Количество встреч в сутки
Базовый профиль качества	720p group video по официальным требованиям Zoom [5]	Видео-трафик upload/download
Запись встречи включается в облако	Cloud recording сохраняет видео, аудио и текст чата [7]	Хранилище записей
Хранилище записей ограничено квотой	Для Pro/Business у Zoom — 10 GB на licensed user [8]	Верхняя граница object storage
Частота чата — явное допущение	Есть лимит сообщения, но нет публичной частоты сообщений на пользователя [6]; принимаем 5 сообщений на одно участие	Нагрузка и хранилище чата (2.2–2.4), уточняется после запуска
Данные встреч хранятся 30 дней	Метаданные, участники, чат и записи старше 30 дней удаляются	Объём хранения (2.3)
Худший случай для записей	Считаем, что записывается каждая встреча; реальная доля записываемых встреч публично неизвестна	Объём object storage и его стоимость (2.3, 11.4)
Материковый Китай не обслуживаем	Для Китая нужен отдельный локальный контур: требования к локализации данных и блокировки трафика. В MVP его нет	Доли ДЦ (3.2)

1.5. Целевая аудитория

Параметр	Значение
География	Глобальный рынок без материкового Китая; Zoom использовался почти в 200 странах и территориях [4]; обслуживаем из 8 ДЦ (п. 3.2)
Месячная аудитория	`MF_MAU = 320 000 000` пользователей/месяц
Дневная аудитория	`MF_DAU = 115 000 000` пользователей/день
Участий во встречах в день	`MF_DMP = 300 000 000` участий/день
Средний размер встречи	`MF_MEET_SIZE = 10` участников
Средняя длительность встречи	`MF_MEET_LEN_MIN = 54` минуты

2. Расчёт нагрузки

2.1. Исходные метрики и методика расчёта

Название	Описание (бывшее значение)	Источник / расчёт
`MF_MAU`	320 000 000 monthly active users; публичное значение Microsoft Teams, FY24 Q1 / 2023	Берётся как целевой месячный масштаб аналога; без пересчёта на 2026, потому что более новой публичной MAU-метрики Teams в открытых источниках не найдено [1]
`MF_DAU`	115 000 000 daily active users; публичное значение Microsoft Teams, октябрь 2020	Берётся как консервативная daily-аудитория аналога; это не статистика Zoom и не результат экстраполяции [2]
`MF_DMP`	300 000 000 daily meeting participants; публичное значение Zoom, апрель 2020	Используется для нагрузки на видеовстречи; это участники встреч, один пользователь может считаться несколько раз [3]
`MF_MEET_SIZE`	10 участников в средней Zoom-встрече; данные за 15.11.2020–15.11.2021	Используется для расчёта `MF_MEETINGS_DAY = MF_DMP / MF_MEET_SIZE` [4]
`MF_MEET_LEN_MIN`	54 минуты — средняя длительность Zoom-встречи; данные за 15.11.2020–15.11.2021	Используется для расчёта participant-minutes [4]
`MF_VIDEO_UP_MBPS`	2.6 Mbps на участника для 720p group video upload	Официальные требования Zoom к bandwidth [5]
`MF_VIDEO_DOWN_MBPS`	1.8 Mbps на участника для 720p group video download	Официальные требования Zoom к bandwidth [5]
`MF_VIDEO_HQ_MBPS`	1.0 / 0.6 Mbps up/down для group «high-quality video» (ниже 720p)	Требования Zoom [5]; профиль деградации при отказе ДЦ (п. 9.2)
`MF_AUDIO_MBPS`	0.08 Mbps на участника для VoIP audio	Верхняя граница диапазона 60–80 kbps из требований Zoom [5]
`MF_SCREEN_MBPS_MAX`	0.15 Mbps на участника для screen sharing upper-bound	Верхняя граница диапазона 50–150 kbps для демонстрации экрана с миниатюрой видео (без 1080p) из требований Zoom [5]
`MF_CHAT_MSG_LIMIT`	4 000 символов на одно chat-сообщение	Лимит сообщения в Zoom Team Chat [6]; для чата встречи берём его как ориентир
`MF_CHAT_PER_JOIN`	5 сообщений на одно участие во встрече	**Проектное допущение**: публичной частоты нет [6]. Раньше стояло только в п. 6.6 — теперь задано здесь и используется везде
`MF_CHAT_MSG_SIZE`	1 KB в среднем на сообщение	Проектное допущение для хранилища чата

	(максимум ≈ 10 KB при 4 000 символов)	
`MF_LIST_PER_DAU`	3 открытия списка встреч на DAU в сутки	Проектное допущение. Раньше было неявно зашито в число 11 979 в п. 6.6
`MF_LOGIN_PER_DAU`	1 создание сессии (логин) на DAU в сутки	Проектное допущение. Раньше было неявно зашито в число 3 993 в п. 5.2
`MF_CHAT_HISTORY_PER_JOIN`	1 загрузка истории чата на одно участие: при входе во встречу и после переподключения	Проектное допущение
`MF_REC_VIEWS_PER_REC`	не больше 1 открытия записи на одну запись	Проектное допущение, верхняя оценка: публичной частоты просмотров нет
`MF_REC_PART_MB`	8 MB — размер части при multipart upload записи в S3	Проектное допущение: запись в 0.761 GB — это ≈ 95 частей, а S3 допускает до 10 000 частей на объект [46]
`MF_RECORD_QUOTA_GB`	10 GB cloud recording storage на licensed user	Квота Zoom для Pro/Business [8]
`MF_HEARTBEAT_SEC`	25 секунд между heartbeat-событиями	Дефолтный `pingInterval = 25000 ms` в Socket.IO [9]
`MF_K_PEAK`	**1.5** (в прошлой версии — 3)	Коэффициент суточного пика: пиковый час / среднее за сутки. Применяется одинаково к RPS, bandwidth и числу одновременных соединений. Сервис глобальный: когда в Америке рабочий день, в Азии ночь, и часовые пояса сглаживают общую суточную кривую. Значение 1.5 — по замечанию ревьюера; 3 удваивало пиковые значения

Текущая актуальность ниши проверяется не устаревшими usage-числами, а финансовой метрикой: Zoom в FY2026 показал $4.8688 млрд выручки и 4 468 крупных клиентов с TTM-выручкой больше $100 000 [10]. Старые usage-метрики используются только там, где они прямо подписаны по году и где показана формула расчёта.

Все числа разделов 2–6, 9 и 11 считаются одним скриптом `resource/calc/meetflow_calc.py`: при изменении допущения достаточно поменять константу и перезапустить. Данные о ВВП для долей ДЦ (п. 3.2) лежат рядом в `imf_weo_2026apr_gdp_2025.json`.

2.2. Продуктовая оценка видеовстреч

Название	Расчёт	Значение
`MF_MEETINGS_DAY`	`MF_DMP / MF_MEET_SIZE = 300 000 000 / 10`	**30 000 000 встреч/сутки**
`MF_JOINS_PER_DAU_DAY`	`MF_DMP / MF_DAU = 300 000 000 / 115 000 000`	**2.61 подключения/DAU/день**
`MF_CREATED_MEETINGS_PER_DAU_DAY`	`MF_MEETINGS_DAY / MF_DAU = 30 000 000 / 115 000 000`	**0.26 созданных встреч/DAU/день**
`MF_PARTICIPANT_MIN_DAY`	`MF_DMP * MF_MEET_LEN_MIN = 300 000 000 * 54`	**16 200 000 000 participant-minutes/сутки**
`MF_AVG_ONLINE_PARTICIPANTS`	`MF_PARTICIPANT_MIN_DAY / 1440`	**11 250 000 участников online в среднем**
`MF_PEAK_ONLINE_PARTICIPANTS`	`MF_AVG_ONLINE_PARTICIPANTS * MF_K_PEAK = 11 250 000 * 1.5`	**16 875 000 участников online в пик**

`MF_PEAK_WSS_CONNECTIONS`	`= MF_PEAK_ONLINE_PARTICIPANTS`, одно WSS-соединение на участника	**16 875 000 одновременных соединений**
`MF_PEAK_ACTIVE_MEETINGS`	`MF_PEAK_ONLINE_PARTICIPANTS / MF_MEET_SIZE`	**1 687 500 встреч одновременно**
`MF_HEARTBEAT_EVENTS_DAY`	`MF_PARTICIPANT_MIN_DAY * 60 / MF_HEARTBEAT_SEC`	**38 880 000 000 heartbeat-событий/сутки**
`MF_CHAT_MESSAGES_DAY`	`MF_DMP * MF_CHAT_PER_JOIN = 300 000 000 * 5`	**1 500 000 000 сообщений/сутки**
`MF_MEETING_LIST_VIEWS_DAY`	`MF_DAU * MF_LIST_PER_DAU = 115 000 000 * 3`	**345 000 000 открытий списка/сутки**
`MF_LOGINS_DAY`	`MF_DAU * MF_LOGIN_PER_DAU`	**115 000 000 логинов/сутки**

Проверка по аналогу: на 31.10.2020 Zoom раскрывал годовой run rate более 3.5 трлн минут встреч [3], то есть ≈ 9.6 млрд минут в сутки. Наши 16.2 млрд — верхняя граница: мы считаем, что каждый участник проводит во встрече все 54 минуты. Ошибка в безопасную сторону: онлайн, трафик и число SFU завышены, а не занижены.

Сводная таблица продуктовых метрик

Метрика	Значение
Месячная аудитория (MAU)	**320 млн** [1]
Дневная аудитория (DAU)	**115 млн** [2]

Средний размер хранилища пользователя — на одного MAU, данные за 30 дней хранения:

Тип данных	Штук на пользователя	Объём на пользователя	Расчёт
Профиль	1	256 B	—
Созданные встречи (метаданные)	2.81	720 B	900 млн встреч за 30 дней / 320 млн MAU
Ссылки-приглашения	2.81	360 B	одна ссылка на встречу, 128 B
Участия во встречах	28.1	3.6 KB	9 млрд участий за 30 дней / 320 млн MAU, 128 B
Строки списка встреч	28.1	3.6 KB	одна строка на участие, 128 B
Сообщения чата	140.6	140.6 KB	45 млрд сообщений / 320 млн MAU, 1 KB
Итого без записей	—	**≈ 149 KB**	
Записи встреч, worst-case	2.81	**2.14 GB**	2.81 записи × 0.761 GB; у аналога квота — 10 GB [8]

Среднее количество действий пользователя в сутки — на одного DAU:

Действие	На DAU в сутки	Расчёт
Логин	1.00	`MF_LOGIN_PER_DAU`
Создание встречи	0.26	30 млн / 115 млн

Вход во встречу (`join`)	2.61	300 млн / 115 млн
Выход из встречи (`leave`)	2.61	одно на каждый вход
Сообщение в чат встречи	13.04	1.5 млрд / 115 млн
Загрузка истории чата	2.61	одна на каждый вход
Открытие списка встреч	3.00	`MF_LIST_PER_DAU`
Метаданные записи	≤ 0.26	не больше одной записи на встречу
Открытие записи	≤ 0.26	`MF_REC_VIEWS_PER_REC`
Heartbeat, автоматически	338.09	38.88 млрд / 115 млн

2.3. Хранилище пользователя и техническое хранение

Блок данных	Название	Расчёт	Штук	Общий объём
Профили пользователей	`MF_USERS_STORAGE_GB`	`MF_MAU * 256 B`	320 млн	**81.92 GB**
Метаданные встреч, 30 дней	`MF_MEETINGS_META_STORAGE_GB`	`MF_MEETINGS_DAY * 30 * 256 B`	900 млн	**230.4 GB**
Ссылки-приглашения, 30 дней	`MF_INVITES_STORAGE_GB`	`MF_MEETINGS_DAY * 30 * 128 B`	900 млн	**115.2 GB**
Участники встреч, 30 дней	`MF_PARTICIPANTS_STORAGE_TB`	`MF_DMP * 30 * 128 B`	9 млрд	**1.152 TB**
Списки встреч пользователей, 30 дней	`MF_USER_MEETINGS_STORAGE_TB`	`MF_DMP * 30 * 128 B`	9 млрд	**1.152 TB**
Активные сессии	`MF_SESSIONS_STORAGE_GB`	`MF_DAU * 128 B`	115 млн	**14.72 GB**
Чат встречи, 30 дней	`MF_CHAT_STORAGE_TB`	`MF_CHAT_MESSAGES_DAY * 30 * MF_CHAT_MSG_SIZE`	45 млрд	**45 TB**
Суммарное техническое хранение без записей	`MF_TECH_STORAGE_TOTAL_TB`	сумма строк выше, raw без репликации	—	**47.75 TB**
Квота облачных записей	`MF_RECORDING_QUOTA_EB`	`MF_MAU * MF_RECORD_QUOTA_GB`	—	**3.2 EB квоты**
Размер одной записи	`MF_RECORDING_SIZE_GB`	`(MF_VIDEO_DOWN_MBPS + MF_AUDIO_MBPS) * MF_MEET_LEN_MIN * 60 / 8 / 1000`	—	**0.761 GB**
Worst-case рост записей	`MF_RECORDING_WORST_PB_DAY`	`MF_MEETINGS_DAY * MF_RECORDING_SIZE_GB`	30 млн в сутки	**22.842 PB/сутки**
Записи за 30 дней, worst-case	`MF_RECORDING_WORST_PB_30D`	`MF_RECORDING_WORST_PB_DAY * 30`	900 млн	**685.26 PB за 30 дней**
Суммарное хранение, 30 дней	`MF_STORAGE_TOTAL_30D_PB`	`MF_RECORDING_WORST_PB_30D + MF_TECH_STORAGE_TOTAL_TB`	—	**≈ 685.31 PB**

В итоговый суммарный объём не включается `MF_RECORDING_QUOTA_EB`: это продуктовая квота, а не фактически занятое хранилище. Для sizing object storage используется worst-case 30-дневное хранение записей: `MF_RECORDING_WORST_PB_30D = 685.26 PB` (каждая встреча записывается). Объёмы хранения не зависят от `MF_K_PEAK`: пик меняет скорость, а не суточный объём.

2.4. Сетевой трафик и RPS

Сетевой трафик

Тип трафика	Название	Формула суточного объёма	Суточный объём	Средний bandwidth	Пиковый bandwidth
Video upload	`MF_VIDEO_UP_GB_DAY`	`MF_PARTICIPANT_MIN_DAY * 60 * MF_VIDEO_UP_MBPS / 8 / 1000`	**315 900 000 GB/сутки**	**29 250 Gbps**	**43 875 Gbps**
Video download	`MF_VIDEO_DOWN_GB_DAY`	`MF_PARTICIPANT_MIN_DAY * 60 * MF_VIDEO_DOWN_MBPS / 8 / 1000`	**218 700 000 GB/сутки**	**20 250 Gbps**	**30 375 Gbps**
Audio VoIP	`MF_AUDIO_GB_DAY`	`MF_PARTICIPANT_MIN_DAY * 60 * MF_AUDIO_MBPS / 8 / 1000`	**9 720 000 GB/сутки**	**900 Gbps**	**1 350 Gbps**
Screen sharing upper-bound	`MF_SCREEN_GB_DAY_MAX`	`MF_PARTICIPANT_MIN_DAY * 60 * MF_SCREEN_MBPS_MAX / 8 / 1000`	**18 225 000 GB/сутки**	**1 687.5 Gbps**	**2 531.25 Gbps**
Media клиент ↔ SFU, итого	`MF_MEDIA_GB_DAY`	сумма строк выше	**562 545 000 GB/сутки**	**52 087.5 Gbps**	**78 131.25 Gbps**
Поток в запись SFU → recording workers, worst-case	`MF_REC_STREAM_GBPS`	`active_meetings * (MF_VIDEO_DOWN_MBPS + MF_AUDIO_MBPS)`	**22 842 000 GB/сутки** (= объём записей)	**2 115 Gbps**	**3 172.5 Gbps**

Расчёт bandwidth в таблице:

```
avg_bandwidth_Gbps  = MF_AVG_ONLINE_PARTICIPANTS * bitrate_Mbps / 1000
peak_bandwidth_Gbps = avg_bandwidth_Gbps * MF_K_PEAK
rec_stream_Gbps     = (MF_AVG_ONLINE_PARTICIPANTS / MF_MEET_SIZE) * (1.8 + 0.08) / 1000 * K
```

Поток в запись — внутренний трафик ДЦ: SFU отправляет его recording workers как ещё одному подписчику встречи. В интернет он не уходит, но нагружает SFU и воркеры записи, поэтому учитывается в разделе 11.

RPS по основным методам

Метод / событие	Название	Событий в сутки	Средний RPS	Пиковый RPS
`POST /auth/login`	`MF_LOGIN_RPS`	`MF_LOGINS_DAY` = 115 000 000	**1 331**	**1 997**
`POST /meetings`	`MF_CREATE_MEETING_RPS`	`MF_MEETINGS_DAY` = 30 000 000	**347**	**521**
`POST /meetings/{id}/join`	`MF_JOIN_RPS`	`MF_DMP` = 300 000 000	**3 472**	**5 208**
`POST /meetings/{id}/leave`	`MF_LEAVE_RPS`	`MF_DMP` = 300 000 000	**3 472**	**5 208**
`WS /presence/heartbeat`	`MF_HEARTBEAT_RPS`	`MF_HEARTBEAT_EVENTS_DAY` = 38 880 000 000	**450 000**	**675 000**
`POST /meeting-chat/messages`	`MF_CHAT_RPS`	`MF_CHAT_MESSAGES_DAY` = 1 500 000 000	**17 361**	**26 042**
`GET /meeting-chat/messages?after_seq=N`	`MF_CHAT_HISTORY_RPS`	`MF_DMP * MF_CHAT_HISTORY_PER_JOIN` = 300 000 000	**3 472**	**5 208**

`GET /users/{id}/meetings?limit=100`	`MF_LIST_RPS`	`MF_MEETING_LIST_VIEWS_DAY` = 345 000 000	**3 993**	**5 990**
`POST /recordings/metadata`	`MF_RECORDING_META_RPS`	до `MF_MEETINGS_DAY` = 30 000 000	**до 347**	**до 521**
`GET /recordings/{id}` — ссылка на скачивание записи	`MF_RECORDING_VIEW_RPS`	до `MF_MEETINGS_DAY * MF_REC_VIEWS_PER_REC` = 30 000 000	**до 347**	**до 521**
WS push: доставка чата и событий join/leave остальным участникам	`MF_WS_PUSH_RPS`	`(chat + join + leave) * (MF_MEET_SIZE − 1)`	**218 750**	**328 125**

Формула для RPS:

```
avg_RPS  = events_per_day / 86400
peak_RPS = avg_RPS * MF_K_PEAK
```

`GET /recordings/{id}` возвращает только presigned URL. Сам файл клиент скачивает напрямую из object storage `home_dc` (`rec.<dc>`, п. 3.5), мимо L7 и SFU. Частота просмотров публично неизвестна, поэтому трафик скачивания в таблицу выше не входит; тариф на исходящий трафик S3 указан в п. 11.4.

3. Глобальная балансировка нагрузки

Глобальная балансировка отвечает на четыре вопроса. Короткие ответы:

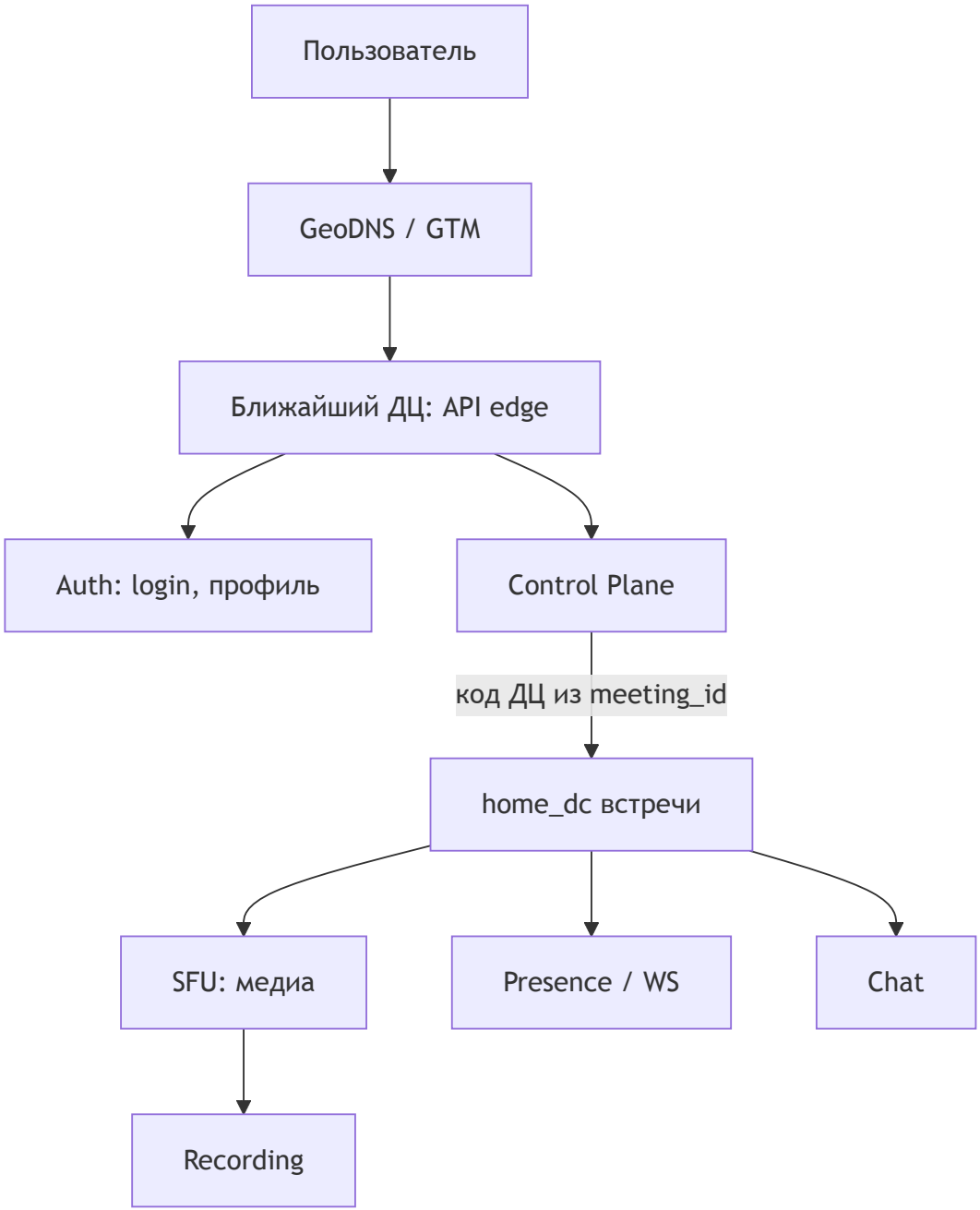
Вопрос	Решение в MeetFlow	Где подробно
Где стоят дата-центры	8 ДЦ: `DC-US-EAST`, `DC-US-WEST`, `DC-BR-SAOPAULO`, `DC-UK-LONDON`, `DC-DE-FRANKFURT`, `DC-IN-MUMBAI`, `DC-SG-SINGAPORE`, `DC-JP-TOKYO`. Абстрактного пула `OTHER` нет: каждая группа стран привязана к конкретному ДЦ	3.2
Как пользователь попадает в ДЦ и как ДЦ переключаются	**DNS**: GeoDNS/GTM выбирает ДЦ по гео, latency, health и весам. **Anycast** — только для доставки DNS-запросов к авторитетным NS-серверам. Медиа переключается на уровне приложения: control plane выдаёт новый SFU	3.5–3.7
Где точка записи	`users` — один primary в `DC-US-EAST`; всё, что относится к встрече, — в `home_dc` встречи; список встреч пользователя — локально с асинхронной репликацией во все ДЦ	3.4
Что при отказе ДЦ или переезде пользователя	ДЦ убирается из DNS, новые встречи уходят в резервные ДЦ. Переезд не требует ни миграции данных, ни повторного логина	3.7, 3.8, 9.2

«ДЦ» в работе — это регион провайдера из 3 зон доступности (AZ). Отказ одной AZ закрывается резервированием внутри ДЦ (разделы 4 и 9), отказ всего ДЦ — глобальной балансировкой.

3.1. Функциональное разбиение по доменам

Домен	Основная нагрузка	Куда направляется запрос	Точка записи
`Auth / Users`	логин, профиль	ближайший ДЦ по GeoDNS	`users` — primary `auth_pg` в `DC-US-EAST`; `user_sessions` — ДЦ, где выполнен логин
`Meeting Control Plane`	`POST /meetings`, `join`, `leave`, список встреч	ближайший ДЦ; `join` и `leave` проксируются в `home_dc` встречи	`meetings`, `meeting_invite_links` — `home_dc`
`Media Plane / SFU`	видео, аудио, screen sharing	только SFU в `home_dc` встречи, адрес приходит в ответе `join`	не хранится, это поток
`Presence / Heartbeat`	`WS /presence/heartbeat`	WS-endpoint `home_dc` встречи	`meetings_runtime` — Redis `home_dc`
`Meeting Chat`	сообщения внутри встречи	`home_dc` встречи	`chat_messages` — ScyllaDB `home_dc`
`Recording`	запись встречи	SFU → recording workers в `home_dc`	файлы — S3 `home_dc`, метаданные — `recording_pg` `home_dc`
`User meetings list`	top-100 встреч пользователя	ближайший ДЦ, чтение локальной копии	`user_meetings` пишется в `home_dc` встречи, ScyllaDB реплицирует его во все ДЦ

`home_dc` — ДЦ, за которым закреплена встреча. Control plane выбирает его при `POST /meetings`: это ДЦ, который обработал запрос, то есть ближайший к организатору. Код `home_dc` записан в `meeting_id` (UUIDv8, первый байт — код ДЦ [85]) и в invite-токен. Поэтому любой ДЦ понимает, куда направить `join`, без глобального справочника встреч.


Пользователь
GeoDNS / GTM
Ближайший ДЦ: API edge
Auth: login, профиль
Control Plane
код ДЦ из meeting_id
home_dc встречи
SFU: медиа
Presence / WS
Chat
Recording

3.2. Выбор и расположение дата-центров

Откуда доли ДЦ. Первоисточники не раскрывают, сколько пользователей видеосервисов в каждой стране: Zoom и Microsoft публикуют только глобальные числа, а данные SimilarWeb методичка запрещает использовать как первоисточник. Поэтому нагрузку распределяем по официальной статистике, в три шага.

Шаг 1. Доля страны — её доля в мировом ВВП. Источник — IMF World Economic Outlook (апрель 2026): ВВП 2025 года в текущих долларах [11]. Видеовстречи — прежде всего рабочий инструмент: средняя встреча длится 54 минуты и собирает 10 участников [,4]. Поэтому объём встреч следует за экономикой страны, а не за численностью населения. Материковый Китай не обслуживаем (п. 1.4), его ВВП не учитываем. Остаётся 194 экономики на 98 537.9 млрд $.

Шаг 2. Каждую страну относим к ближайшему по задержке ДЦ. Полный список стран по группам — в скрипте расчёта `meetflow_calc.py`, значения ВВП — в `imf_weo_2026apr_gdp_2025.json`.

Шаг 3. США делим на восток и запад по населению. На регион West по переписи 2020 приходится 23.7% населения [73]. Внутри одной страны деловая активность на человека примерно одинакова, поэтому здесь берём население, а не ВВП. Западному побережью нужен свой ДЦ: путь до Вирджинии пересекает весь континент.

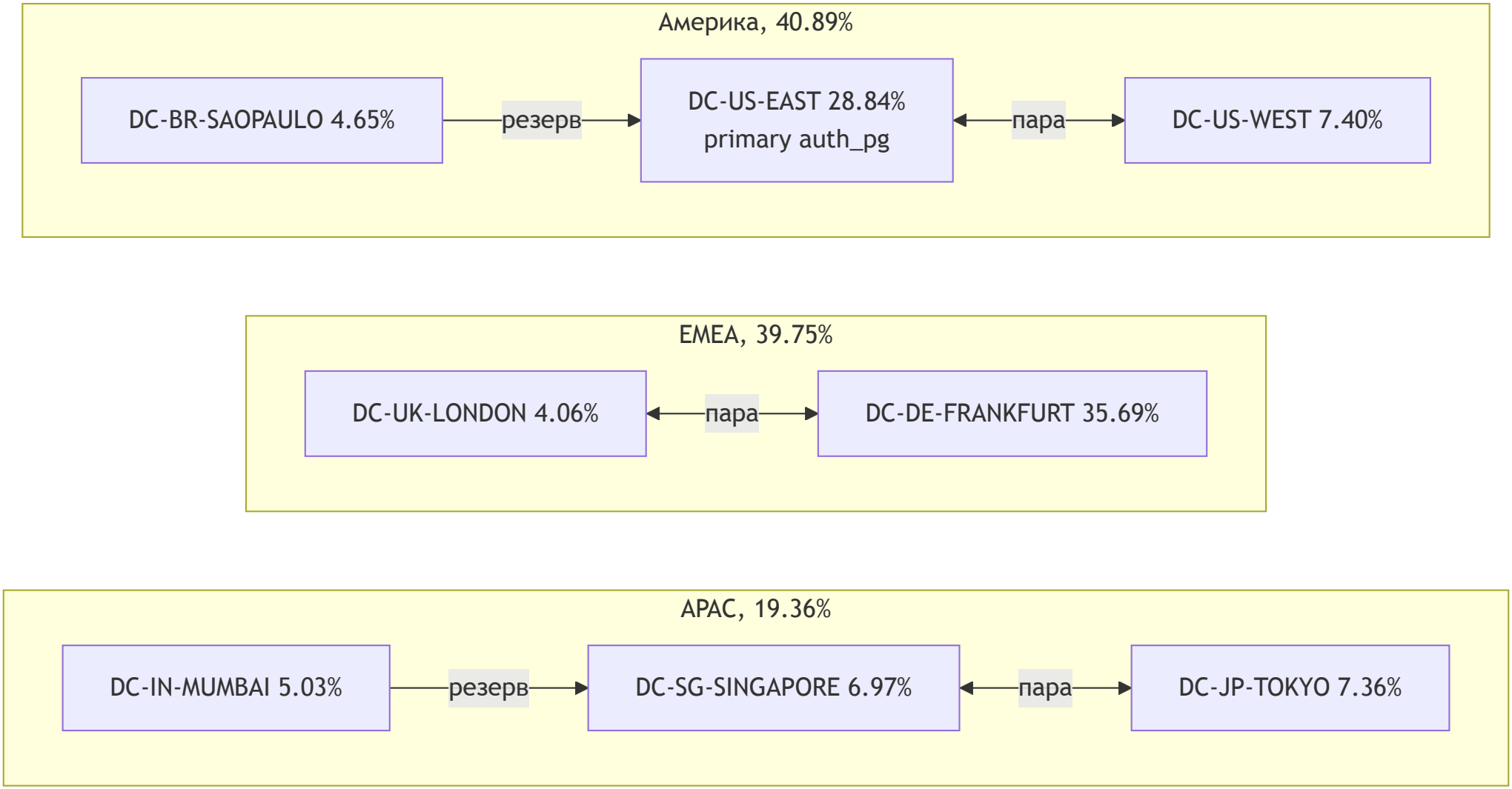
ДЦ	Город	Какие страны обслуживает	ВВП 2025, млрд $	Доля нагрузки
`DC-US-EAST`	Ашберн, Вирджиния	США: Northeast, Midwest, South (76.3% населения); Канада; Мексика, Центральная Америка, Карибы	23 475.3 + 2 319.9 + 2 622.8 = **28 418.0**	**28.84%**
`DC-US-WEST`	Сан-Хосе, Калифорния	США: West (23.7% населения)	**7 291.8**	**7.40%**
`DC-BR-SAOPAULO`	Сан-Паулу	Южная Америка	**4 576.1**	**4.65%**
`DC-UK-LONDON`	Лондон	Великобритания	**4 003.0**	**4.06%**
`DC-DE-FRANKFURT`	Франкфурт	Европа без Великобритании, СНГ, Ближний Восток, Африка	**35 168.5**	**35.69%**
`DC-IN-MUMBAI`	Мумбаи	Индия, Пакистан, Бангладеш, Шри-Ланка, Непал, Бутан, Мальдивы, Афганистан	**4 956.6**	**5.03%**
`DC-SG-SINGAPORE`	Сингапур	Юго-Восточная Азия, Австралия, Новая Зеландия, Океания, Гонконг, Макао	**6 870.9**	**6.97%**
`DC-JP-TOKYO`	Токио	Япония, Республика Корея, Тайвань, Монголия	**7 253.0**	**7.36%**
Итого		194 экономики без материкового Китая	**98 537.9**	**100%**

Доля = ВВП группы / 98 537.9. Доли округлены до 0.01 п. п. так, чтобы сумма была ровно 100%: у Южной Америки 4.644% → 4.65%. Именно эти доли используются во всех расчётах по ДЦ.

Проверка по аналогу. По макрорегионам получается: Америка — 40.89%, EMEA — 39.75%, APAC — 19.36%. У Zoom доля Америки в выручке выше — 72.1% (EMEA — 15.8%, APAC — 12.1%) [72]. Zoom исторически сильнее всего в США. Наш масштаб — масштаб лидера рынка (320 млн MAU у Teams [,1]), поэтому берём нейтральную оценку по экономике, а не по выручке одного Zoom. Если реальная доля Америки окажется выше, меняются только размеры пулов в ДЦ, а не архитектура: GTM перераспределит новые подключения весами (п. 3.7), а запас на перекос даёт резерв из п. 9.2.

Границы групп условные: GeoDNS выбирает ДЦ по фактической задержке. Например, западная Канада может уйти в `DC-US-WEST`, а страны Персидского залива — в `DC-IN-MUMBAI`. Доли сдвинутся на единицы процентов, это компенсируют веса GTM (п. 3.7) и резерв `N+1`. Точные доли уточняются по GeoIP-аналитике после запуска.

ДЦ	Доля нагрузки	Пара для резерва	На какие продуктовые метрики влияет
`DC-US-EAST`	28.84%	`DC-US-WEST`	join latency и RTT медиа для востока США, Канады, Мексики
`DC-US-WEST`	7.40%	`DC-US-EAST`	RTT медиа для западного побережья США
`DC-BR-SAOPAULO`	4.65%	`DC-US-EAST`	без своего ДЦ медиа Южной Америки шло бы через подводные кабели в США
`DC-UK-LONDON`	4.06%	`DC-DE-FRANKFURT`	latency для UK, отдельная юрисдикция после Brexit
`DC-DE-FRANKFURT`	35.69%	`DC-UK-LONDON`	latency для Европы, СНГ, Ближнего Востока и Африки; самый большой ДЦ, его отказ разобран в п. 9.2
`DC-IN-MUMBAI`	5.03%	`DC-SG-SINGAPORE`	latency для Индии и Южной Азии, стабильность screen sharing
`DC-SG-SINGAPORE`	6.97%	`DC-JP-TOKYO`	latency для Юго-Восточной Азии и Океании
`DC-JP-TOKYO`	7.36%	`DC-SG-SINGAPORE`	RTT медиа и packet loss для Японии, Кореи и Тайваня



Zoom как реальный аналог позволяет платным аккаунтам выбирать ДЦ для real-time video/audio/shared content [13]. Поэтому в MeetFlow **control plane** и **media plane** разделены: API обслуживает ближайший ДЦ, а медиа всегда идёт в `home_dc` встречи.

3.3. Распределение нагрузки по дата-центрам

Формула:

```
dc_metric = global_metric * dc_share
```

Предполагается, что встречи распределены по `home_dc` так же, как пользователи по ДЦ: организатор и большинство участников обычно из одного региона. В строке «Итого» — глобальное значение; сумма округлённых ячеек может отличаться от него на единицу последнего знака.

Продуктовая нагрузка

ДЦ	Доля	DAU	Участий/сутки	Встреч/сутки	Peak online = WSS-соединений
`DC-US-EAST`	28.84%	**33 166 000**	**86 520 000**	**8 652 000**	**4 866 750**
`DC-US-WEST`	7.40%	**8 510 000**	**22 200 000**	**2 220 000**	**1 248 750**
`DC-BR-SAOPAULO`	4.65%	**5 347 500**	**13 950 000**	**1 395 000**	**784 688**
`DC-UK-LONDON`	4.06%	**4 669 000**	**12 180 000**	**1 218 000**	**685 125**
`DC-DE-FRANKFURT`	35.69%	**41 043 500**	**107 070 000**	**10 707 000**	**6 022 688**
`DC-IN-MUMBAI`	5.03%	**5 784 500**	**15 090 000**	**1 509 000**	**848 813**
`DC-SG-SINGAPORE`	6.97%	**8 015 500**	**20 910 000**	**2 091 000**	**1 176 188**
`DC-JP-TOKYO`	7.36%	**8 464 000**	**22 080 000**	**2 208 000**	**1 242 000**
Итого	**100%**	**115 000 000**	**300 000 000**	**30 000 000**	**16 875 000**

Пиковый RPS

ДЦ	`login`	`create`	`join`	`leave`	`heartbeat`	`chat`	`chat history`	`list`	`recording meta`	`recording view`
`DC-US-EAST`	**576**	**150**	**1 502**	**1 502**	**194 670**	**7 510**	**1 502**	**1 727**	**150**	**150**
`DC-US-WEST`	**148**	**39**	**385**	**385**	**49 950**	**1 927**	**385**	**443**	**39**	**39**
`DC-BR-SAOPAULO`	**93**	**24**	**242**	**242**	**31 388**	**1 211**	**242**	**279**	**24**	**24**
`DC-UK-LONDON`	**81**	**21**	**211**	**211**	**27 405**	**1 057**	**211**	**243**	**21**	**21**
`DC-DE-FRANKFURT`	**713**	**186**	**1 859**	**1 859**	**240 908**	**9 294**	**1 859**	**2 138**	**186**	**186**
`DC-IN-MUMBAI`	**100**	**26**	**262**	**262**	**33 952**	**1 310**	**262**	**301**	**26**	**26**
`DC-SG-SINGAPORE`	**139**	**36**	**363**	**363**	**47 048**	**1 815**	**363**	**417**	**36**	**36**
`DC-JP-TOKYO`	**147**	**38**	**383**	**383**	**49 680**	**1 917**	**383**	**441**	**38**	**38**
Итого	**1 997**	**521**	**5 208**	**5 208**	**675 000**	**26 042**	**5 208**	**5 990**	**521**	**521**

Пиковый media bandwidth

ДЦ	Video upload	Video download	Audio	Screen sharing	Media суммарно	SFU → запись
`DC-US-EAST`	**12 653.5 Gbps**	**8 760.1 Gbps**	**389.3 Gbps**	**730.0 Gbps**	**22 533.1 Gbps**	914.9 Gbps

`DC-US-WEST`	**3 246.8 Gbps**	**2 247.8 Gbps**	**99.9 Gbps**	**187.3 Gbps**	**5 781.7 Gbps**	234.8 Gbps
`DC-BR-SAOPAULO`	**2 040.2 Gbps**	**1 412.4 Gbps**	**62.8 Gbps**	**117.7 Gbps**	**3 633.1 Gbps**	147.5 Gbps
`DC-UK-LONDON`	**1 781.3 Gbps**	**1 233.2 Gbps**	**54.8 Gbps**	**102.8 Gbps**	**3 172.1 Gbps**	128.8 Gbps
`DC-DE-FRANKFURT`	**15 659.0 Gbps**	**10 840.8 Gbps**	**481.8 Gbps**	**903.4 Gbps**	**27 885.0 Gbps**	1 132.3 Gbps
`DC-IN-MUMBAI`	**2 206.9 Gbps**	**1 527.9 Gbps**	**67.9 Gbps**	**127.3 Gbps**	**3 930.0 Gbps**	159.6 Gbps
`DC-SG-SINGAPORE`	**3 058.1 Gbps**	**2 117.1 Gbps**	**94.1 Gbps**	**176.4 Gbps**	**5 445.7 Gbps**	221.1 Gbps
`DC-JP-TOKYO`	**3 229.2 Gbps**	**2 235.6 Gbps**	**99.4 Gbps**	**186.3 Gbps**	**5 750.5 Gbps**	233.5 Gbps
Итого	**43 875.00 Gbps**	**30 375.00 Gbps**	**1 350.00 Gbps**	**2 531.25 Gbps**	**78 131.25 Gbps**	**3 172.5 Gbps**

Записи встреч

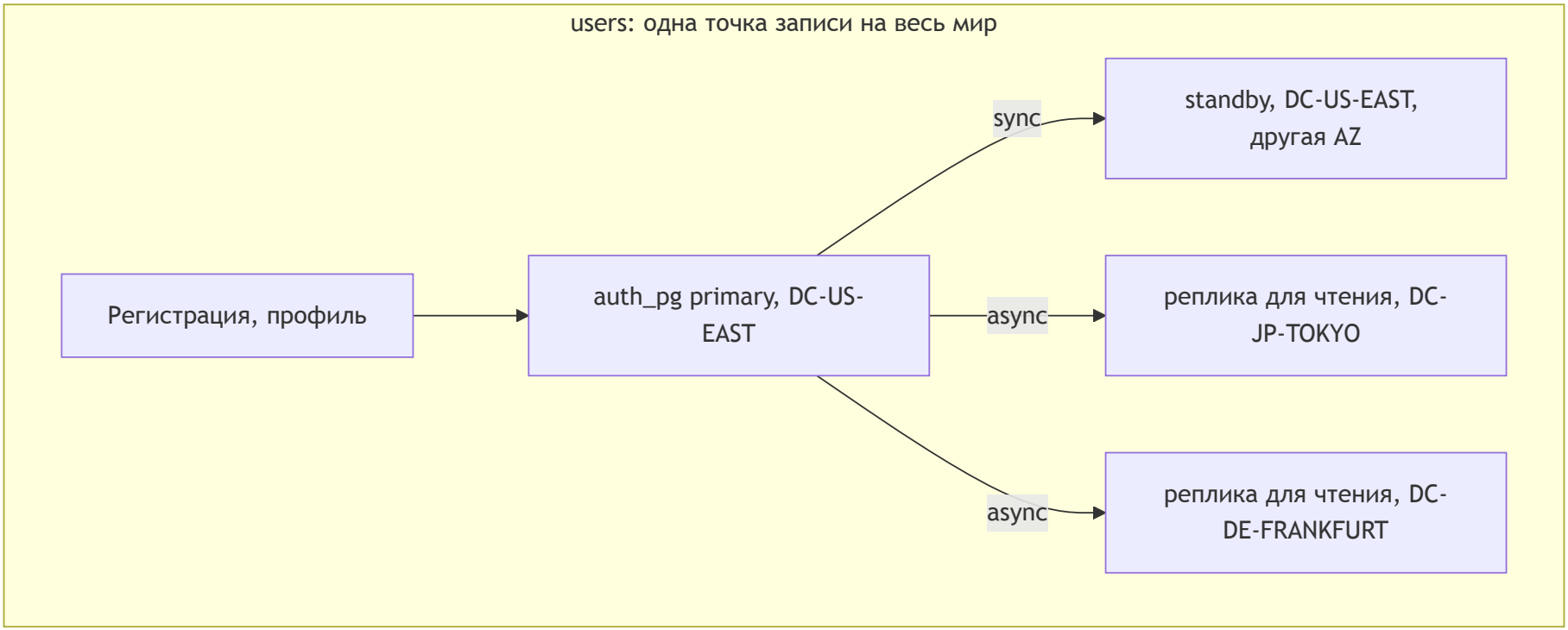
ДЦ	Worst-case запись/сутки	Worst-case запись за 30 дней
`DC-US-EAST`	**6.588 PB/сутки**	**197.63 PB**
`DC-US-WEST`	**1.690 PB/сутки**	**50.71 PB**
`DC-BR-SAOPAULO`	**1.062 PB/сутки**	**31.86 PB**
`DC-UK-LONDON`	**0.927 PB/сутки**	**27.82 PB**
`DC-DE-FRANKFURT`	**8.152 PB/сутки**	**244.57 PB**
`DC-IN-MUMBAI`	**1.149 PB/сутки**	**34.47 PB**
`DC-SG-SINGAPORE`	**1.592 PB/сутки**	**47.76 PB**
`DC-JP-TOKYO`	**1.681 PB/сутки**	**50.44 PB**
Итого	**22.842 PB/сутки**	**685.26 PB**

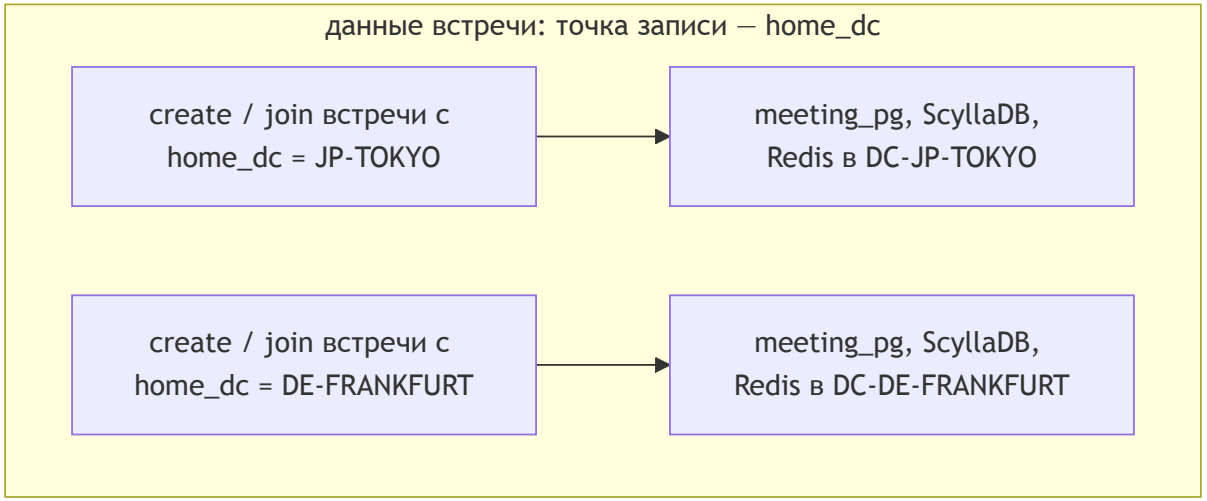
3.4. Точки записи данных

Правило: **у каждого объекта одна точка записи** (primary), а чтение по возможности идёт из локальной копии в ближайшем ДЦ. Тогда не бывает двух одновременных записей одного объекта в разных ДЦ, и не нужен сложный multi-master с разрешением конфликтов.

Данные	Точка записи	Где читаются	Репликация между ДЦ	Почему так
`users` (`auth_pg`)	**один primary на весь мир — `DC-US-EAST`**, синхронная standby в другой AZ того же ДЦ	локальная асинхронная реплика в каждом ДЦ	streaming replication: primary → 7 реплик [27,89]	Нужна глобальная уникальность `email`. Записи редкие: даже если каждый DAU раз в сутки меняет профиль, в пик это ≤ 1 997 записей/с — одного primary хватает
`user_sessions` (`session_redis`)	ДЦ, где выполнен логин; код ДЦ — префикс `session_id` (`us-east.<128 бит>`)	там же; другой ДЦ один раз забирает сессию по префиксу	нет, перенос по запросу	Сессия нужна там, где пользователь сейчас; копии во всех ДЦ не нужны

		(п. 3.8)		
`meetings`, `meeting_invite_links` (`meeting_pg`)	`home_dc` встречи	`home_dc`	асинхронная реплика в парный ДЦ на случай отказа	`join` должен видеть актуальный статус (strong, п. 5.3)
`meeting_participants` (`meeting_scylla`)	`home_dc`	`home_dc`	нет: у каждого ДЦ свой keyspace, он реплицируется только внутри этого ДЦ, RF=3 по 3 AZ; снапшоты — в S3 парного ДЦ	Нужны во время встречи и для истории
`meetings_runtime` (`runtime_redis`)	`home_dc`	`home_dc`	нет	Эфемерное состояние, клиенты восстанавливают его при переподключении
`chat_messages` (`chat_scylla`)	`home_dc`	`home_dc`	нет; снапшоты — в S3 парного ДЦ	Порядок задаёт `chat_seq` в одном Redis `home_dc`
`recordings` и файлы (`recording_pg`, S3)	`home_dc`	`home_dc` по presigned URL	метаданные — асинхронная реплика в парный ДЦ; файлы — ≥ 3 копии в 3 AZ `home_dc`	Копия 685 PB в другой регион удвоила бы самую дорогую статью расходов
`user_meetings` (`meeting_scylla`)	`home_dc` встречи, запись с `LOCAL_QUORUM`	ближайший к пользователю ДЦ, локально	keyspace с RF=3 **в каждом** ДЦ (NetworkTopologyStrategy) [80,81]	Read model с eventual consistency (п. 5.3): список встреч доступен локально в любой стране


users: одна точка записи на весь мир
Регистрация, профиль
auth_pg primary, DC-US-EAST
sync
standby, DC-US-EAST, другая AZ
async
реплика для чтения, DC-JP-TOKYO
async
реплика для чтения, DC-DE-FRANKFURT


данные встречи: точка записи — home_dc
create / join встречи с home_dc = JP-TOKYO
meeting_pg, ScyllaDB, Redis в DC-JP-TOKYO
create / join встречи с home_dc = DE-FRANKFURT
meeting_pg, ScyllaDB, Redis в DC-DE-FRANKFURT

Как запрос на запись попадает в точку записи:

1. Запрос приходит в ближайший ДЦ (GeoDNS, п. 3.5).
2. Сервис определяет точку записи: для `users` это всегда `DC-US-EAST`, для данных встречи — код ДЦ из `meeting_id`.
3. Если точка записи в другом ДЦ, запрос проксируется по gRPC с mTLS по межрегиональному каналу [52]. Это добавляет RTT между ДЦ — например, ≈ 153 мс Токио ↔ восточное побережье США [87], но только для редких записей.
4. После своей записи пользователь несколько секунд читает из primary (read-your-writes), пока асинхронная реплика не догонит.

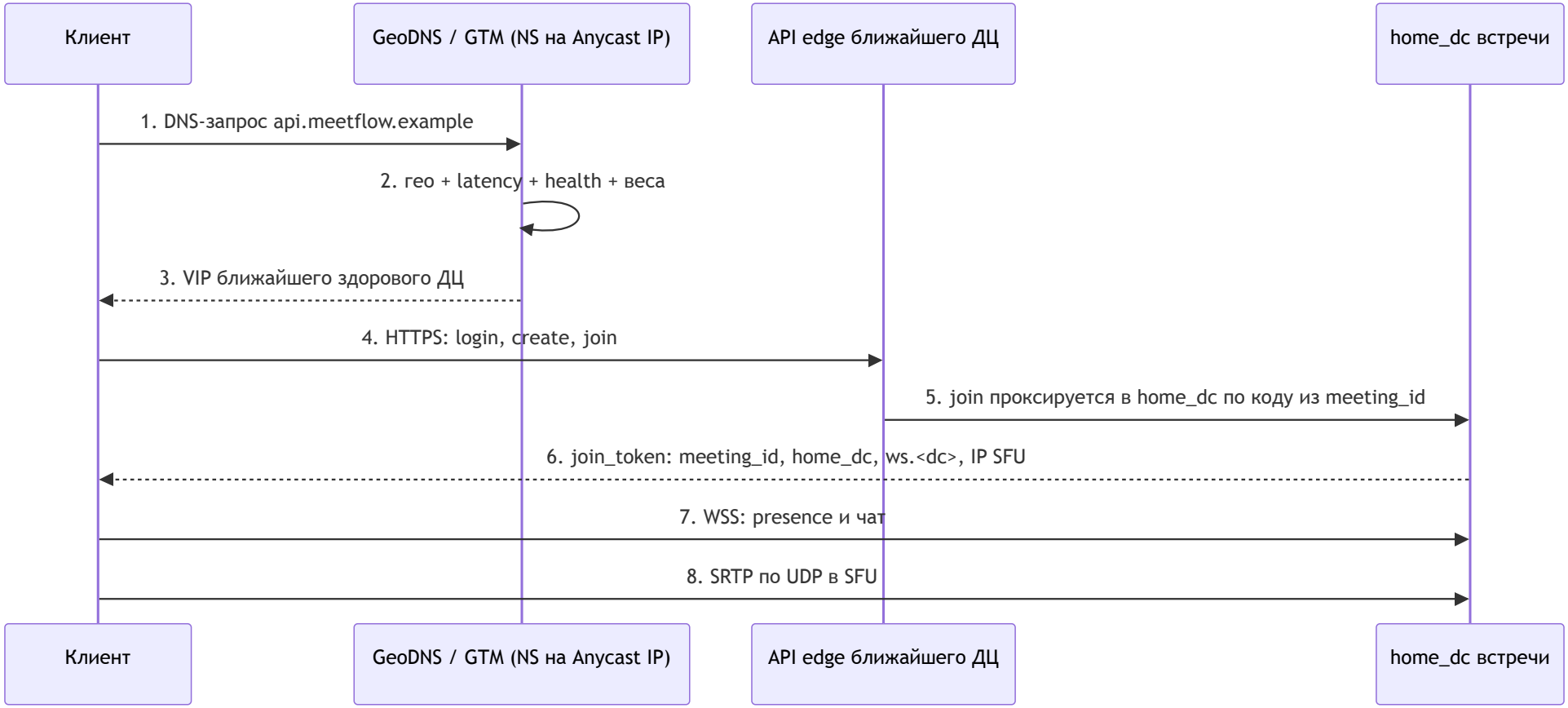
Отказ точки записи. Внутри `DC-US-EAST` primary `auth_pg` автоматически переключается на синхронную standby в соседней AZ без потери данных — через Patroni [86]. При отказе всего `DC-US-EAST` primary становится реплика в `DC-US-WEST`, но по команде дежурного (runbook), а не автоматически. Причина: в схеме с двумя площадками Patroni держит во втором ДЦ standby-кластер, который повышают только вручную — второй ДЦ не может сам отличить отказ первого от обрыва связи [97]. RPO — секунды лага асинхронной репликации. Логин продолжает работать: он читает локальную реплику и пишет сессию в локальный Redis. На время переключения блокируются только регистрация и изменение профиля.

3.5. Схема DNS-балансировки — основной механизм переключения

Ответ на вопрос «Anycast или DNS?»: ДЦ выбирается и переключается через DNS. Используется GeoDNS/GTM — managed-сервис с геолокацией, latency-based routing, health checks и весами пулов [15,16,17]. Anycast стоит только «под» DNS: авторитетные NS-серверы GTM-провайдера доступны по Anycast IP (п. 3.6).

DNS-имя	Что выбирает	Политика	TTL
`api.meetflow.example`	VIP API edge ближайшего здорового ДЦ	geo + latency + health + веса	30 с
`ws.<dc>.meetflow.example`	WebSocket edge конкретного `home_dc`	фиксированное региональное имя, приходит в ответе `join`	30 с
SFU, IP:port	конкретный SFU в `home_dc`	не DNS: адрес и ICE-кандидаты приходят в ответе `join`	—
`rec.<dc>.meetflow.example`	скачивание записей из object storage `home_dc`	региональное имя, presigned URL	300 с
`static.meetflow.example`	веб-клиент и статика	CDN провайдера	по CDN

GeoDNS определяет местоположение по IP рекурсивного резолвера или по подсети клиента из EDNS Client Subnet [82]. Поэтому ДЦ выбирается по тому, где пользователь находится сейчас, а не где он регистрировался.



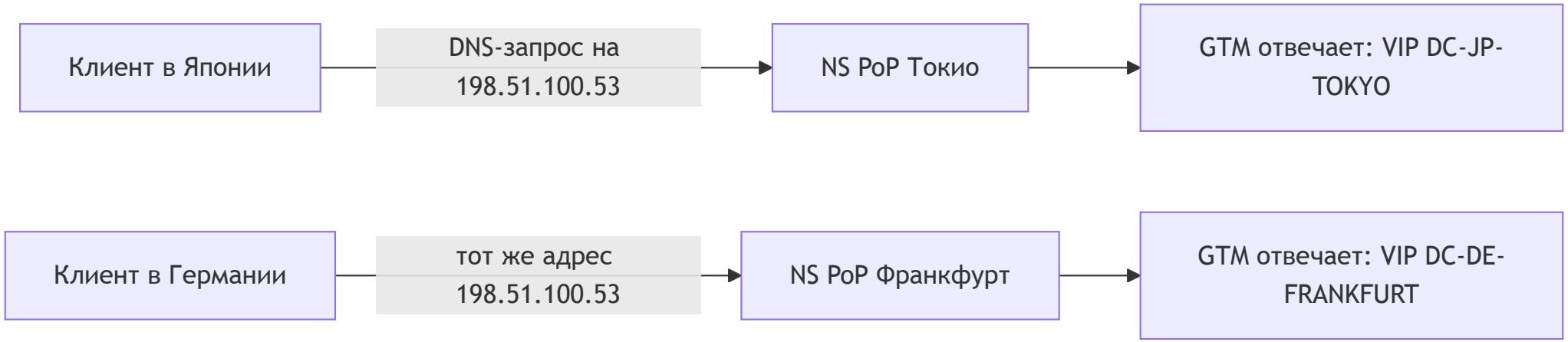
Время переключения через DNS. Monitor GTM опрашивает health-endpoint каждого ДЦ. Минимальный интервал зависит от тарифа: у Cloudflare это 60 с на Pro, 15 с на Business и 10 с на Enterprise; число неудач подряд до вывода ДЦ (`consecutive_down`) задаётся через API [17]. Берём Enterprise: интервал 10 с и 3 неудачи подряд — ДЦ выводится из ответов за ≈ 30 с, плюс TTL 30 с. Итого новые подключения уходят в здоровый ДЦ примерно за минуту. На Business-тарифе было бы ≈ 45 с + 30 с. Часть резолверов кеширует дольше TTL, поэтому у клиента есть свой fallback: при ошибке соединения он заново резолвит имя, а если это не помогло — идёт на резервный региональный адрес из конфигурации приложения.

3.6. Схема Anycast-балансировки

Anycast — один IP-адрес, который анонсируется по BGP из многих точек; пакет приходит в «ближайшую» по маршрутизации точку [83]. В MeetFlow Anycast используется **только там, где запросы короткие и не хранят состояние на сервере**:

Трафик	Anycast	Почему
DNS-запросы к	**Да**	UDP «запрос — ответ» без сессии: ответ из ближайшего PoP, устойчивость к отказу точки и DDoS [83]

авторитетным NS-серверам GTM		
Статика веб-клиента	Да, через CDN провайдера	короткие запросы, контент одинаковый во всех точках
API `api.meetflow.example`	Нет	BGP выбирает маршрут по AS-path, а не по задержке и загрузке ДЦ: нельзя задать веса, вывести ДЦ на обслуживание (drain), учесть ёмкость. Глобальный proxy-балансировщик с одним Anycast IP есть у крупных облаков [18], но он привязывает к сети одного облака
WebSocket	Нет	долгоживущее TCP-соединение: при перестроении BGP-маршрута пакеты уйдут в другой ДЦ, где нет состояния этого соединения [83]
RTP/SRTP медиа	Нет	встреча живёт на конкретном SFU в `home_dc`, смена маршрута рвёт медиасессию



Оба PoP анонсируют по BGP один и тот же адрес (в примере — документационный `198.51.100.53`), и маршрутизация приводит каждый запрос в ближайшую к клиенту точку.

Итог: Anycast ускоряет и защищает **получение DNS-ответа**, а **решение, в какой ДЦ идти, принимает GTM** — это и есть механизм переключения.

3.7. Регулировка трафика и переключение между дата-центрами

Регулировка идёт через веса пулов в GTM. Monitors регулярно проверяют endpoints и выводят нездоровый пул из ротации [16,17].

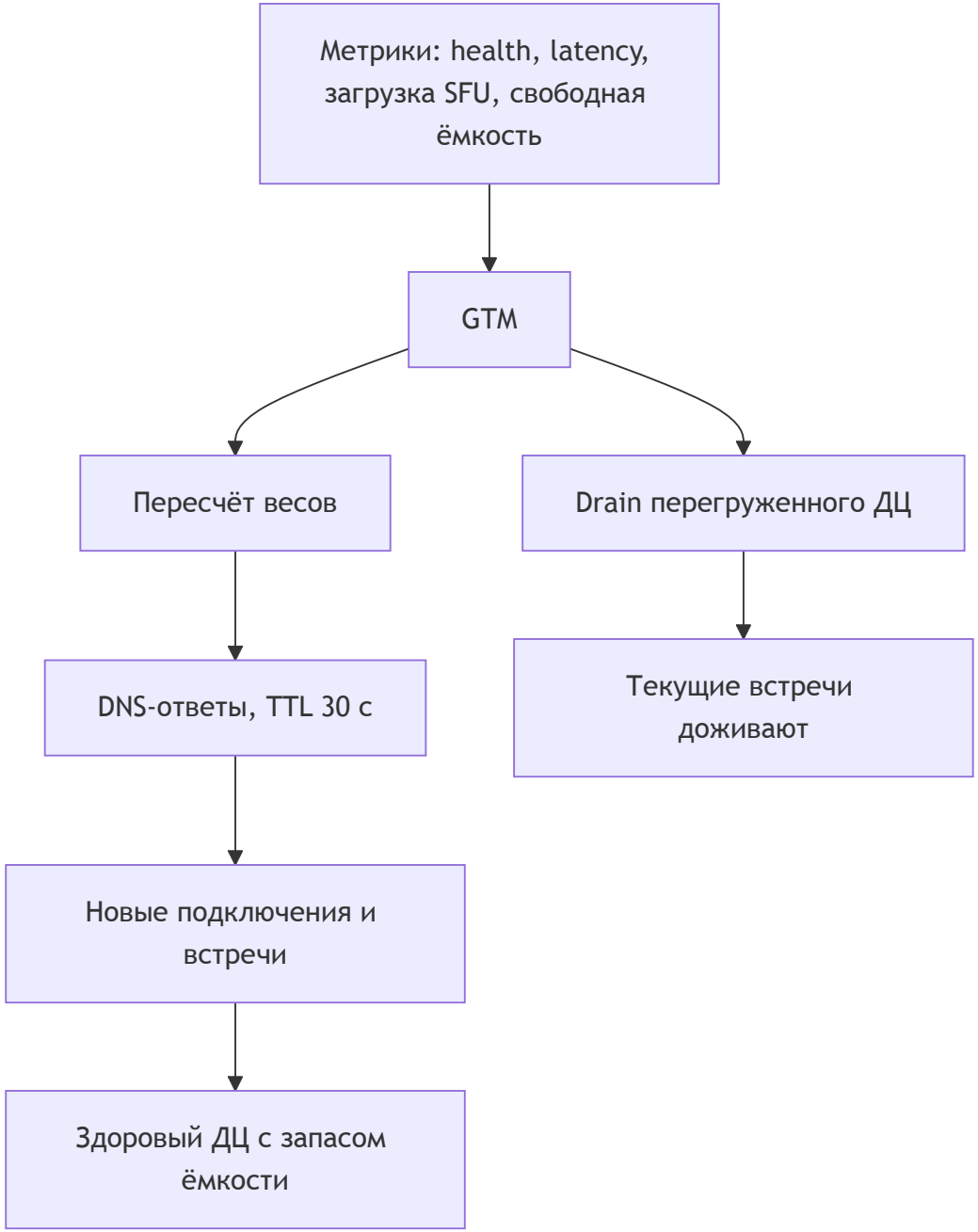
```
effective_weight = geo_match * health * capacity_weight * manual_weight
```

Ситуация	Что делаем	Кого затрагивает
Норма	Пользователь → ближайший здоровый ДЦ	—

ДЦ перегружен	Снижаем `capacity_weight` : новые пользователи и встречи уходят в резервные ДЦ	только новые
Плановые работы	`manual_weight = 0` (drain): новые встречи не создаются, текущие доживают	только новые
ДЦ недоступен	`health = 0` , ДЦ убирается из DNS-ответов	все, см. п. 9.2
Деградация пула SFU	Control plane не назначает этот пул новым встречам. При отказе SFU клиент получает адрес нового SFU по WebSocket и заново устанавливает с ним медиасоединение: новое ICE-согласование и DTLS-рукопожатие [84]	встречи на этом SFU

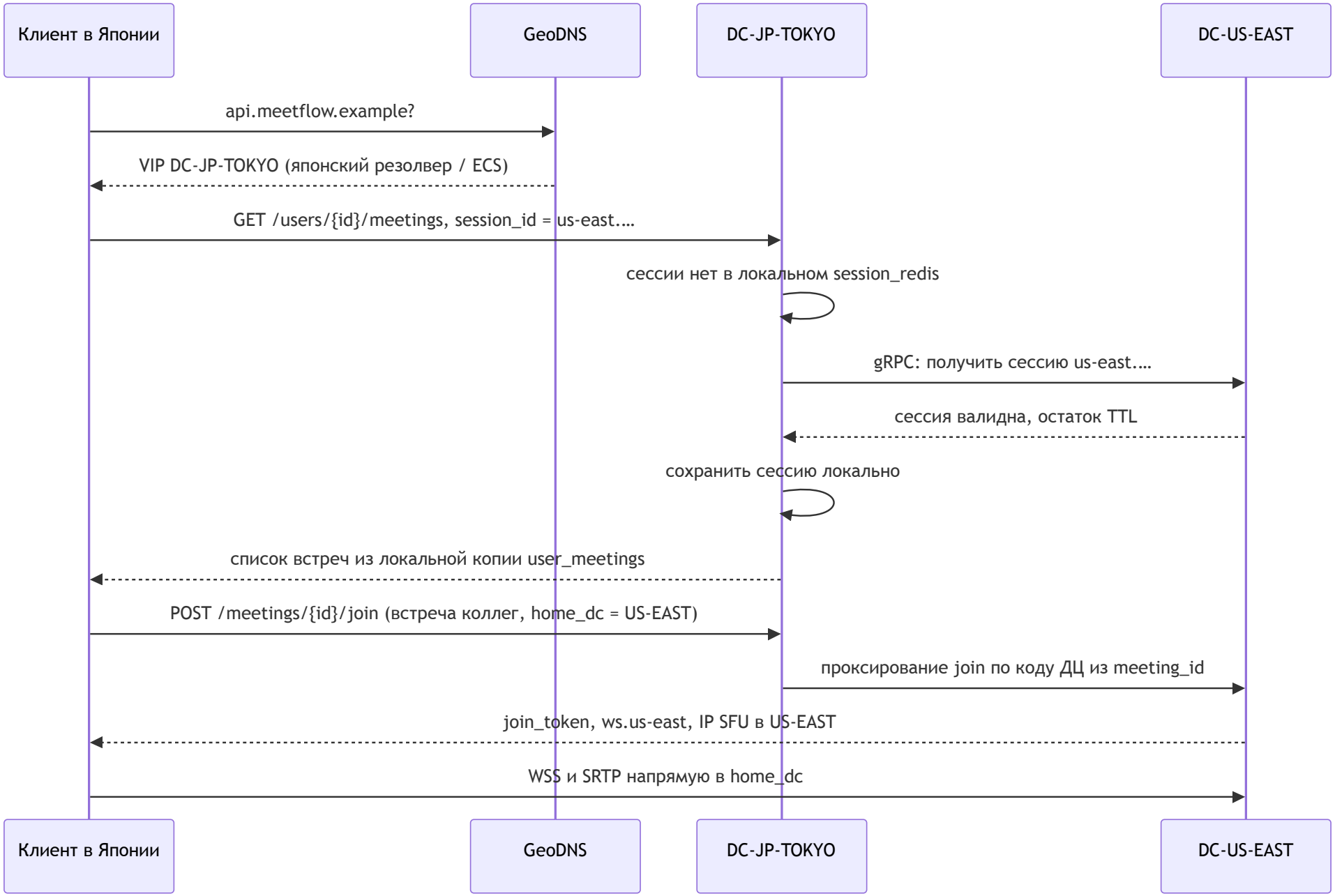
Основной ДЦ	Резерв 1	Резерв 2	Почему
`DC-US-EAST`	`DC-US-WEST`	`DC-UK-LONDON`	та же страна, затем ближайший ДЦ через Атлантику
`DC-US-WEST`	`DC-US-EAST`	`DC-JP-TOKYO`	та же страна, затем через Тихий океан
`DC-BR-SAOPAULO`	`DC-US-EAST`	`DC-DE-FRANKFURT`	прямые кабели в США и Европу
`DC-UK-LONDON`	`DC-DE-FRANKFURT`	`DC-US-EAST`	соседний европейский ДЦ
`DC-DE-FRANKFURT`	`DC-UK-LONDON`	`DC-US-EAST`	соседний европейский ДЦ, затем самый ёмкий ДЦ через Атлантику; страны Персидского залива GTM может отправить в `DC-IN-MUMBAI`
`DC-IN-MUMBAI`	`DC-SG-SINGAPORE`	`DC-DE-FRANKFURT`	ближайший ДЦ в Азии
`DC-SG-SINGAPORE`	`DC-JP-TOKYO`	`DC-IN-MUMBAI`	ближайшие ДЦ в Азии
`DC-JP-TOKYO`	`DC-SG-SINGAPORE`	`DC-US-WEST`	ближайший ДЦ в Азии, затем через Тихий океан

Резервный ДЦ может быть меньше основного: `DC-UK-LONDON` (4.06%) не вместит `DC-DE-FRANKFURT` (35.69%), а `DC-US-WEST` (7.40%) — `DC-US-EAST` (28.84%). Поэтому столбцы «Резерв» задают **порядок предпочтения**, а не единственный адрес. GTM раздаёт трафик отказавшего ДЦ по нескольким ДЦ пропорционально свободной ёмкости (`capacity_weight`). Расчёт того, что ёмкости хватит, — в п. 9.2.


Метрики: health, latency, загрузка SFU, свободная ёмкость
GTM
Пересчёт весов
Drain перегруженного ДЦ
DNS-ответы, TTL 30 с
Текущие встречи доживают
Новые подключения и встречи
Здоровый ДЦ с запасом ёмкости

3.8. Сценарий: вход в США, затем поездка в Японию

Вопрос: пользователь залогинился в США, а потом улетел в Японию. Что происходит?



Шаг	Что происходит	Почему так

1. Логин в США	Пароль проверяется по локальной реплике `auth_pg`. Сессия `us-east.<id>` создаётся в `session_redis` `DC-US-EAST`	Логину не нужна запись в primary
2. Прилёт в Японию	GeoDNS видит японский резолвер или подсеть клиента из ECS [82] и отдаёт VIP `DC-JP-TOKYO`	ДЦ выбирается по текущему местоположению; привязки к стране логина нет
3. Первый запрос в `DC-JP-TOKYO`	Сессии нет в локальном Redis. По префиксу `us-east` Auth один раз забирает её из `DC-US-EAST` (+≈ 153 мс [87]) и сохраняет у себя. **Повторный логин не нужен**	Сессии не копируются во все ДЦ заранее, а переносятся по запросу
4. Профиль	Чтение — из локальной реплики `auth_pg`, изменение — в primary `DC-US-EAST`	Одна точка записи для `users` (п. 3.4)
5. Список встреч	Читается из локальной копии `user_meetings` в `DC-JP-TOKYO`	Keyspace реплицирован во все ДЦ
6. Встреча американских коллег	`home_dc = DC-US-EAST`. `DC-JP-TOKYO` проксирует `join` в `DC-US-EAST` по коду ДЦ в `meeting_id`. WS и медиа идут в `DC-US-EAST`. RTT Токио ↔ Вашингтон ≈ 153 мс [87], то есть сеть в одну сторону ≈ 77 мс. По ITU-T G.114 большинство разговорных приложений не страдают, если сквозная задержка «от рта до уха» в одну сторону меньше 150 мс [88]. В этот бюджет входят и сеть до SFU (≈ 77 мс), и путь от SFU до собеседника в США, и кодек с джиттер-буфером. Поэтому разговор возможен, но близко к границе нормы	Все участники встречи должны быть на одном SFU. Улучшение после MVP — каскад SFU: клиент подключается к SFU в Токио, а тот пересылает потоки в SFU `home_dc` по магистрали
7. Новая встреча, созданная в Японии	`home_dc = DC-JP-TOKYO`; американские участники подключаются к SFU в Токио	`home_dc` выбирается по организатору
8. Пользователь остался жить в Японии	Мигрировать ничего не нужно: данные пользователя глобальные (реплики во всех ДЦ), а встречи привязаны к своему `home_dc`, а не к пользователю. Старая сессия в `DC-US-EAST` истечёт по TTL	В системе нет «домашнего ДЦ пользователя», который пришлось бы переносить
9. Logout	Сессия удаляется локально и в ДЦ из префикса, событие отзыва рассылается во все 8 ДЦ	Отозванная сессия не должна жить ни в одной копии

3.9. Вывод по глобальной балансировке

Решение	Значение
ДЦ	8 ДЦ в трёх макрорегионах. Доли — по ВВП стран (IMF WEO) без материкового Китая, США делятся по переписи 2020; абстрактного `OTHER` нет
Механизм выбора и переключения ДЦ	**DNS**: GeoDNS/GTM с health checks и весами, TTL 30 с, плюс клиентский fallback
Anycast	Только авторитетные DNS-серверы и CDN для статики
Переключение медиа	На уровне приложения: новый SFU от control plane, новое медиасоединение (ICE + DTLS)
Точки записи	`users` — один primary в `DC-US-EAST`; данные встречи — `home_dc`; `user_meetings` — локально с асинхронной репликацией во все ДЦ
Переезд пользователя	Без миграции и без повторного логина: сессия переносится по префиксу, данные пользователя читаются из локальной реплики

4. Локальная балансировка нагрузки

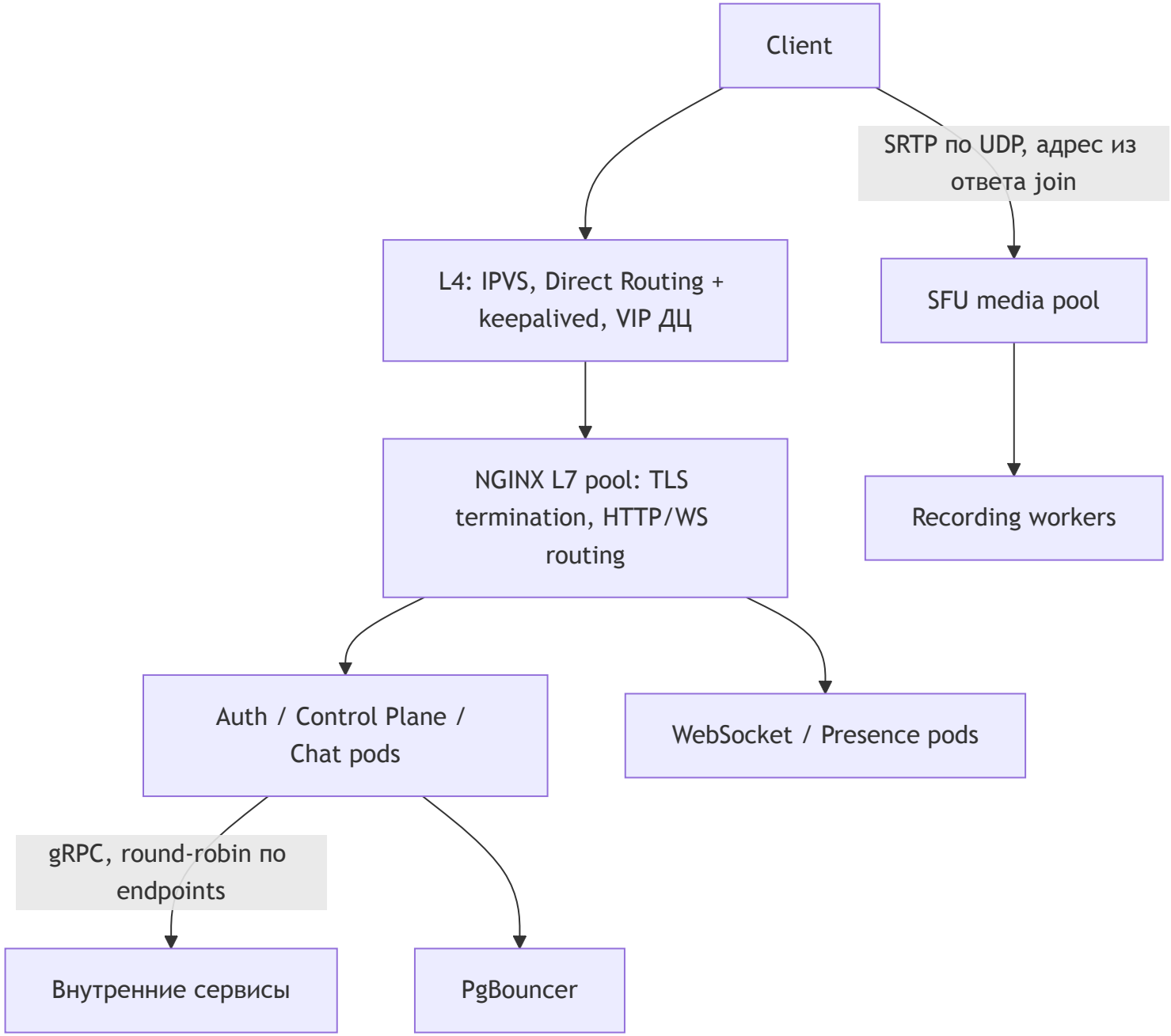
4.1. Границы локальной балансировки

Поток	Где балансируется	В расчёте NGINX L7
HTTPS API	L4 (IPVS) → NGINX L7 → поды сервисов	Да
WSS: heartbeat, доставка чата и presence-событий	L4 (IPVS) → NGINX L7 → WebSocket-поды	Да
Метаданные записей и ссылки на скачивание	L4 (IPVS) → NGINX L7	Да
RTP/SRTP медиа	Напрямую на публичный IP:порт SFU из ответа `join`. Балансирует control plane: он выбирает SFU для встречи	Нет
Скачивание файла записи	Напрямую из object storage `home_dc` по presigned URL	Нет
Поток записи SFU → recording workers	Внутри ДЦ, по адресу из control plane	Нет
Межсервисные вызовы внутри ДЦ	gRPC, клиентская балансировка по endpoints Kubernetes Service	Нет
Межрегиональные вызовы: прокси `join` в `home_dc`, перенос сессии	gRPC с mTLS между ДЦ	Нет

Медиа идёт мимо L4 и L7: клиент получает адрес конкретного SFU в ответе `join` (п. 3.5). Так встреча остаётся на одном SFU, а балансировщикам не нужно пропускать 78 Тбит/с.

Клиент за NAT и межсетевым экраном. У SFU публичный адрес, и он работает как ICE-Lite: сам соединение не начинает, а отвечает на проверки, которые присылает клиент [102]. Поэтому клиенту за домашним или офисным NAT не нужен ни STUN, ни TURN, чтобы достучаться до SFU. Если UDP закрыт межсетевым экраном, клиент подключается к тому же SFU по TCP: SFU слушает и UDP, и TCP [,102], а ICE умеет предлагать TCP-кандидаты [,103]. Медиа поверх TCP хуже переносит потери, но звонок не рвётся. Не поддерживаем в MVP только сети, где прокси пропускает исключительно TLS на порт 443: для них нужен TURN-релей по TLS, это улучшение после MVP.

4.2. Схема локальной и межсервисной балансировки


Client
SRTP по UDP, адрес из ответа join
L4: IPVS, Direct Routing + keepalived, VIP ДЦ
SFU media pool
NGINX L7 pool: TLS termination, HTTP/WS routing
Recording workers
Auth / Control Plane / Chat pods
WebSocket / Presence pods
gRPC, round-robin по endpoints
Внутренние сервисы
PgBouncer

L4 — свой, на IPVS. Директор IPVS принимает пакеты на VIP ДЦ и раздаёт TCP-соединения узлам NGINX. В режиме Direct Routing ответы идут от NGINX к клиенту напрямую, мимо директора [92], поэтому директор видит только входящий трафик. Директоров два, active/standby: keepalived переносит VIP на живой директор по VRRP и сам проверяет здоровье узлов NGINX [,93].

Почему не облачный балансировщик провайдера. У Selectel самый мощный тип — «Продвинутый с резервированием»: 2 инстанса по 4 vCPU / 2 GB, до 3 Гбит/с (до 5 по заявке), а производительность HTTPS провайдер приводит для 10 000 одновременных соединений [12]. В Kubernetes такой балансировщик создаётся для Service типа `LoadBalancer` [21]. В одном нашем ДЦ одновременно до 6 млн WSS-соединений, поэтому этот вариант не подходит.

Межсервисная балансировка: gRPC-клиент получает список подов через headless Kubernetes Service и распределяет вызовы round-robin [52,54]. Обычная L4-балансировка здесь не подходит: gRPC держит одно долгое HTTP/2-соединение, и все вызовы ушли бы в один под.

4.3. Механизмы резервирования

Уровень	Что делает	Резервирование	Формула
L4: IPVS + keepalived	TCP-вход HTTPS/WSS в ДЦ, VIP	Active/standby: при отказе директора VIP переезжает по VRRP [93]	`N+1`
NGINX L7 pool	HTTPS/WSS, TLS termination, routing	Active-active; keepalived исключает узел, не прошедший health check	`N+1`
NGINX L7 strict HA	Потеря AZ или стойки	Двойная ёмкость	`N*2`
SFU media pool	Видео, аудио, screen sharing	Sticky meeting + health checks; при отказе SFU клиент получает новый адрес и заново устанавливает медиасоединение	`N+1`

4.4. Ограничители для расчёта

Ограничитель	Значение	Источник / расчёт
`LB_HTTPS_RPS`	**341 232 RPS**	NGINX Ingress Controller, HTTPS, ответ 1 KB по уже открытым keep-alive-соединениям, 16 CPU [19]
`LB_TLS_CPS`	**6 676 новых TLS-соединений/с**	NGINX, HTTPS CPS на 16 CPU: полный TLS handshake на каждое соединение, RSA 2048, OpenSSL 1.0.1f [20]. С современным OpenSSL рукопожатия дешевле, так что значение консервативное
`LB_NET_GBPS`	**8.8 Gbps**	Throughput NGINX Ingress Controller; упор в 10GbE-карту тестового стенда, а не в CPU [19]
`MF_L7_MSG_SIZE`	**1 KB**	Размер ответа в RPS-тесте [19]; берём его как средний размер сообщения API и WebSocket
Почему значения для 16 CPU	—	В тестах NGINX «CPU» — физические ядра Xeon. Облачный узел на 24 vCPU — это ≈ 12 ядер с Hyper-Threading, поэтому берём ближайшую точку теста — 16 CPU, а не 24. Ни RPS, ни TLS всё равно не ограничивают расчёт (п. 4.6)
`MF_L7_CONN_MEM`	**54 KB на WSS-соединение**	NGINX держит проксируемое WebSocket-соединение (клиент + upstream) меньше чем в 20 KB: 50 000 соединений занимают < 1 GB [77]. Плюс до 34 KB TLS-буферов, если они не освобождены [,78]. Это худший случай
Узел L7	**24 vCPU / 64 GB / 100 GB SSD**	56 GB под соединения: `56 GB / 54 KB ≈ 1.04 млн` WSS-соединений на узел. Дескрипторы и порты настраиваются (`ulimit`, `ip_local_port_range`)
`IPVS_CONN_MEM`	**136 B на соединение**	Запись в таблице соединений IPVS занимает 128 B, элемент хеш-таблицы — 8 B [91]

Узел L4	**EL11-SSD-10GE: 6 ядер / 32 GB / 10GE**	Директор пропускает только входящий трафик (Direct Routing [92])

> **Исправлено после ревью.** В прошлой версии весь RPS, включая heartbeat, делился на SSL TPS. Но SSL TPS — это скорость **новых** TLS-рукопожатий, а heartbeat идёт по уже открытому WSS-соединению. Теперь у каждого ограничителя своя метрика. Для WebSocket главным ограничителем оказалась **память под одновременные соединения**, которая раньше не учитывалась.

4.5. Расчёт количества L7-балансировщиков

```
MF_L7_REST_RPS  = login + create + join + leave + list + recording_meta + recording_view
                  + chat + chat_history                                          = 51 215
MF_L7_WS_IN     = heartbeat                                                      = 675 000
MF_L7_WS_OUT    = (chat + join + leave) * (MF_MEET_SIZE - 1)                      = 328 125
MF_L7_PEAK_RPS  = MF_L7_REST_RPS + MF_L7_WS_IN + MF_L7_WS_OUT                    = 1 054 340
MF_L7_NEW_TLS   = MF_L7_REST_RPS + join                                          = 56 424
                  # худший случай: каждый REST-запрос без keep-alive + новое WSS-соединение на каждый join
MF_L7_PEAK_GBPS = MF_L7_PEAK_RPS * 1 KB * 8 / 10^6                               = 8.43
MF_L7_PEAK_CONN = MF_PEAK_WSS_CONNECTIONS                                        = 16 875 000

N_rps  = ceil(RPS_dc / 341 232)
N_tls  = ceil(NEW_TLS_dc / 6 676)
N_net  = ceil(GBPS_dc / 8.8)
N_conn = ceil(CONN_dc * 54 KB / 56 GB)
N_base = max(N_rps, N_tls, N_net, N_conn)

N+1 = N_base + 1
N*2 = N_base * 2
```

ДЦ	L7 peak RPS	Новых TLS/с	L7 Gbps	WSS-соединений
`DC-US-EAST`	**304 072**	16 273	2.43	**4 866 750**
`DC-US-WEST`	**78 021**	4 175	0.62	**1 248 750**
`DC-BR-SAOPAULO`	**49 027**	2 624	0.39	**784 688**
`DC-UK-LONDON`	**42 806**	2 291	0.34	**685 125**
`DC-DE-FRANKFURT`	**376 294**	20 138	3.01	**6 022 688**
`DC-IN-MUMBAI`	**53 033**	2 838	0.42	**848 813**
`DC-SG-SINGAPORE`	**73 488**	3 933	0.59	**1 176 188**
`DC-JP-TOKYO`	**77 599**	4 153	0.62	**1 242 000**

Сумма по ДЦ	1 054 340	56 424	8.43	16 875 000

ДЦ	N_rps	N_tls	N_net	N_conn	N+1	N*2
DC-US-EAST	1	3	1	5	**6**	10
DC-US-WEST	1	1	1	2	**3**	4
DC-BR-SAOPAULO	1	1	1	1	**2**	2
DC-UK-LONDON	1	1	1	1	**2**	2
DC-DE-FRANKFURT	2	4	1	6	**7**	12
DC-IN-MUMBAI	1	1	1	1	**2**	2
DC-SG-SINGAPORE	1	1	1	2	**3**	4
DC-JP-TOKYO	1	1	1	2	**3**	4
Сумма по ДЦ	**9**	**13**	**8**	**20**	**28**	**40**

4.6. Итоговое количество балансировщиков

Режим	Количество NGINX L7	Что закладываем
Рабочий вариант	**28**	`N+1` в каждом ДЦ
Жёсткое резервирование	**40**	`N*2` в каждом ДЦ: переживает потерю AZ

Директоры L4 считаются так же: по памяти под таблицу соединений и по входящему трафику, плюс резервный директор.

```
L4_RAM_dc = CONN_dc * 136 B
N_L4      = max(1, ceil(GBPS_dc / 10)) + 1        # active + standby (keepalived, VRRP)
```

ДЦ	Соединений в таблице IPVS	Память под таблицу	Входящий трафик, Gbps (не больше)	N	N+1
DC-US-EAST	4 866 750	0.66 GB	2.43	1	**2**
DC-US-WEST	1 248 750	0.17 GB	0.62	1	**2**
DC-BR-SAOPAULO	784 688	0.11 GB	0.39	1	**2**
DC-UK-LONDON	685 125	0.09 GB	0.34	1	**2**
DC-DE-FRANKFURT	6 022 688	0.82 GB	3.01	1	**2**
DC-IN-MUMBAI	848 813	0.12 GB	0.42	1	**2**
DC-SG-SINGAPORE	1 176 188	0.16 GB	0.59	1	**2**
DC-JP-TOKYO	1 242 000	0.17 GB	0.62	1	**2**

Сумма по ДЦ	**16 875 000**	**2.29 GB**		**8.43**	**8**	**16**

Итого **16 узлов L4** — по паре в каждом ДЦ. В самом большом ДЦ таблица соединений занимает 0.82 GB из 32 GB, а входящий трафик — не больше 3.01 Гбит/с при карте 10GE. Ни память, ни сеть директор не ограничивают: второй узел нужен ради отказоустойчивости.

Что ограничивает расчёт	Итог
Память под одновременные WSS-соединения	**Основной ограничитель**: 16.9 млн соединений в пик
RPS и TLS handshake	Не основной: 1–4 узла на ДЦ, TLS — в худшем случае без keep-alive
Пропускная способность сети	Не основной ограничитель для control plane
Media traffic	Не проходит ни через L4, ни через NGINX L7

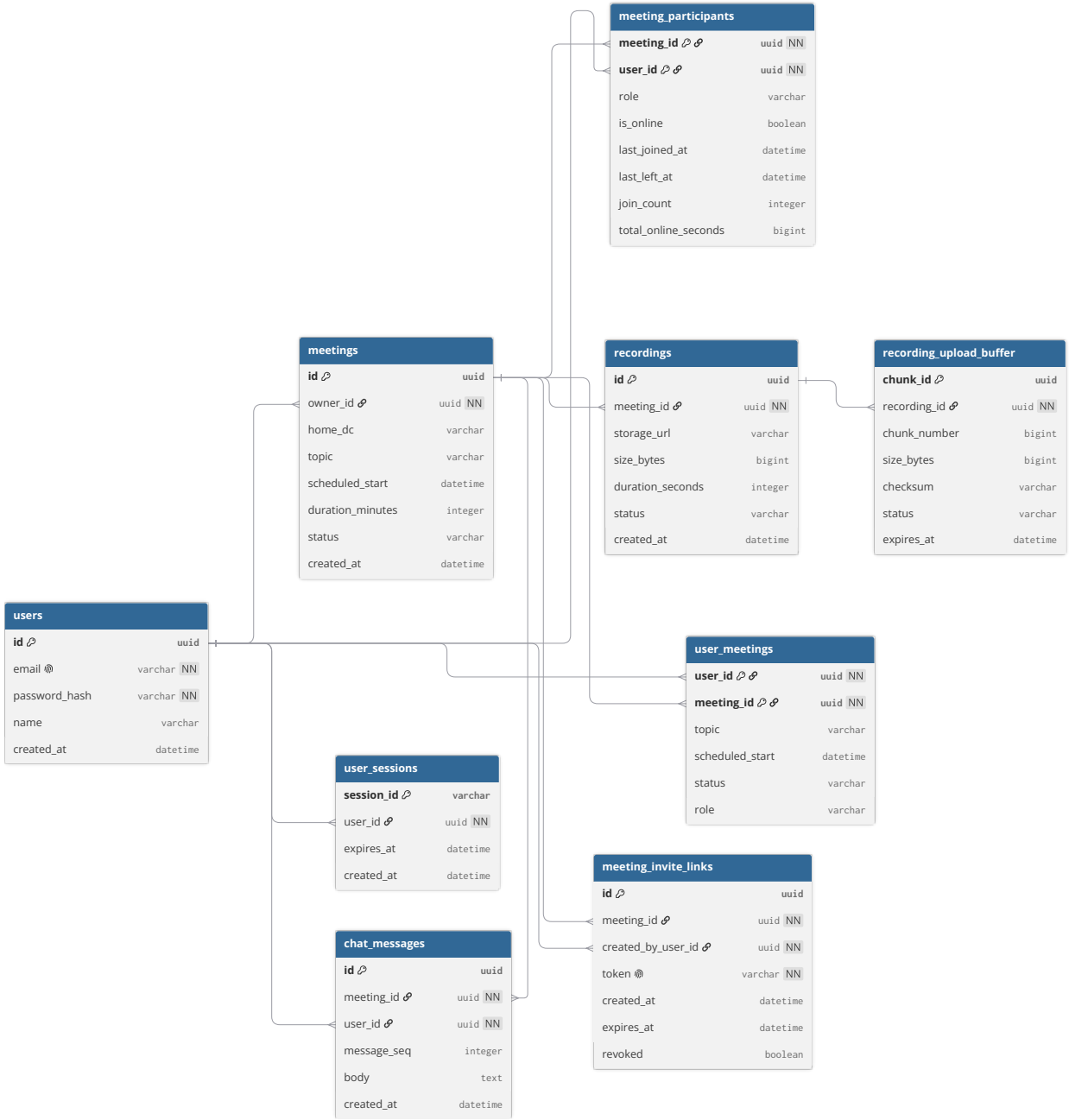
5. Логическая схема БД

5.1. Список логических таблиц

Логическая схема не привязана к конкретной СУБД, шардингу и физическим индексам. В схему включены только основные сущности сервиса и связи между ними.

Логическая схема

Для получения картинки использовался сервис dbdiagram с кодом из этого файла (интерактивная ссылка на работу).


meeting_participants
meeting_id uuid NN
user_id uuid NN
role varchar
is_online boolean
last_joined_at datetime
last_left_at datetime
join_count integer
total_online_seconds bigint
meetings
id uuid
owner_id uuid NN
home_dc varchar
topic varchar
scheduled_start datetime
duration_minutes integer
status varchar
created_at datetime
recordings
id uuid
meeting_id uuid NN
storage_url varchar
size_bytes bigint
duration_seconds integer
status varchar
created_at datetime
recording_upload_buffer
chunk_id uuid
recording_id uuid NN
chunk_number bigint
size_bytes bigint
checksum varchar
status varchar
expires_at datetime
users
id uuid
email varchar NN
password_hash varchar NN
name varchar
created_at datetime
user_meetings
user_id uuid NN
meeting_id uuid NN
topic varchar
scheduled_start datetime
status varchar
role varchar
user_sessions
session_id varchar
user_id uuid NN
expires_at datetime
created_at datetime
meeting_invite_links
id uuid
meeting_id uuid NN
created_by_user_id uuid NN
token varchar NN
created_at datetime
expires_at datetime
revoked boolean
chat_messages
id uuid
meeting_id uuid NN
user_id uuid NN
message_seq integer
body text
created_at datetime

Группы данных

Группа	Таблицы	Что хранит
Пользователи	`users`, `user_sessions`	Аккаунты и активные сессии
Встречи	`meetings`, `meeting_participants`, `meeting_invite_links`, `user_meetings`	Создание, вход, роли, список встреч пользователя
Чат	`chat_messages`	Сообщения внутри встречи
Файлы	`recordings`, `recording_upload_buffer`	Метаданные записей и временные чанки загрузки

Описание таблиц

Таблица	Тип данных	Назначение	Ключ	Основные связи
`users`	persistent	Аккаунт пользователя	`id`	Создаёт встречи, пишет чат, имеет сессии
`user_sessions`	session state	Авторизованные пользовательские сессии	`session_id`	N:1 к `users`
`meetings`	persistent	Основная карточка встречи	`id`	N:1 к `users`, 1:N к участникам, чату и записям
`meeting_participants`	persistent	Участники встречи и их состояние	`meeting_id + user_id`	N:1 к `meetings`, N:1 к `users`
`meeting_invite_links`	persistent	Ссылки-приглашения во встречу	`id`	N:1 к `meetings`, N:1 к `users`
`user_meetings`	read model	Список встреч пользователя	`user_id + meeting_id`	N:1 к `users`, N:1 к `meetings`
`chat_messages`	persistent	Сообщения чата встречи	`id`	N:1 к `meetings`, N:1 к `users`
`recordings`	persistent / file metadata	Записи встреч и ссылки на файлы	`id`	N:1 к `meetings`
`recording_upload_buffer`	upload buffer	Части multipart upload записи до её завершения	`chunk_id`	N:1 к `recordings`

Кеши, счётчики и буферы

Это производные от основных таблиц данные. На картинке их нет, но они хранятся и нагружают хранилища, поэтому описаны здесь, а их размеры и нагрузка — в п. 5.2.

Сущность	Тип данных	Что хранит	Ключ	Связь
`meetings_runtime`	hot state	Онлайн-состояние идущей встречи: статус, число участников онлайн, пул SFU, время последнего heartbeat	`meeting_id`	1:1 к `meetings`
`chat_seq`	счётчик	Последний выданный номер сообщения в чате встречи	`meeting_id`	1:1 к `meetings`
`chat_dedup`	ключ идемпотентности	Уже принятые `client_msg_id` за последние 60 с и выданный им `message_seq`	`meeting_id + client_msg_id`	N:1 к `meetings`
`active_invite_tokens`	кеш	Токены приглашений во встречи за последние сутки	`token`	кеш `meeting_invite_links`
`chat_last100`	буфер в памяти	Последние 100 сообщений идущей встречи	`meeting_id`	кеш `chat_messages`

5.2. Размеры данных и QPS

Допущения для таблиц

Допущение	Значение	Основание
Активная сессия	1 на `DAU`, создаётся при логине (`MF_LOGIN_PER_DAU = 1`)	Упрощённая модель входа
Проверка сессии	при `create`, `join`, открытии списка встреч и открытии записи	Трафик внутри встречи (`leave`, чат, история чата, WS, медиа) авторизуется подписанным `join_token` в `home_dc` без похода в `user_sessions`
Invite link	1 на встречу	Для MVP достаточно одной ссылки на встречу
Файлы записи	Видео, аудио и текст чата	Cloud recording у аналога хранит эти типы данных [7]
Данные по чатам	5 сообщений на участие, в среднем 1 KB	Допущение `MF_CHAT_PER_JOIN` из п. 2.1: публичной частоты сообщений нет, есть только лимит длины [6]
Список встреч	3 открытия на DAU в сутки, по 100 строк	Допущение `MF_LIST_PER_DAU` из п. 2.1
История чата	1 загрузка на участие, до 100 сообщений	Допущение `MF_CHAT_HISTORY_PER_JOIN`; отдаётся из буфера комнаты в памяти (п. 6.8), в ScyllaDB идут только промахи
Просмотр записи	не больше 1 на запись	Допущение `MF_REC_VIEWS_PER_REC` из п. 2.1
Часть записи	8 MB	Допущение `MF_REC_PART_MB`: запись в 0.761 GB — это 95 частей

Таблицы, объём и нагрузка

Таблица	Строки / окно	Байт / строка	Объём	Read QPS peak	Write QPS peak	Основание
`users`	320 000 000	256 B	81.92 GB	7 726	≤ 1 997	Чтение при `login`, `create`, `join`; запись — регистрации и профиль. Верхняя оценка записи: каждый DAU раз в сутки (п. 3.4)
`user_sessions`	115 000 000	128 B	14.72 GB	12 240	1 997	Проверка при `create`, `join`, `list` и открытии записи; запись при `login`
`meetings`	900 000 000 / 30 дней	256 B	230.4 GB	10 938	521	Чтение при `join`, `leave`, `recording_metadata`; запись при `create`
`meeting_participants`	9 000 000 000 / 30 дней	128 B	1.152 TB	5 208	10 417	`join + leave`
`meeting_invite_links`	900 000 000 / 30 дней	128 B	115.2 GB	5 208	521	1 ссылка на встречу

`user_meetings`	9 000 000 000 / 30 дней	128 B	1.152 TB, копия в каждом ДЦ	5 990 запросов, по 100 строк	10 938	Запись: строка организатора при `create`, строка участника при `join`, статус при `leave`
`chat_messages`	45 000 000 000 / 30 дней	1 KB в среднем, до 10 KB	45 TB	≤ 5 208 запросов по 100 строк, только промахи буфера комнаты	26 042	5 сообщений на участие (п. 2.1); новые сообщения доставляются через WS без чтения из БД
`recordings`	900 000 000 / 30 дней	256 B + файл	230.4 GB метаданных + 685.26 PB файлов	≤ 521	521	Чтение — при открытии записи, не больше одного на запись; запись — метаданные в конце встречи
`recording_upload_buffer`	≈ 80 млн частей в пик	8 MB	≈ 642 TB в пик, входит в 685.26 PB	—	49 570 PUT/c	Части незавершённых записей: 3 172.5 Гбит/с / 8 MB. Части не читаются: S3 сам собирает из них объект
`meetings_runtime`	1 687 500 в пик	512 B	0.864 GB	—	685 938	heartbeat 675 000 + `join` / `leave` 10 417 + `create` 521
`chat_seq`	1 687 500 в пик	один счётчик на встречу	входит в `meetings_runtime`	—	26 042 `INCR`	один номер на сообщение
`chat_dedup`	≈ 1.56 млн ключей в пик	≈ 100 B	≈ 0.16 GB	—	26 042 `SET NX`	ключ живёт 60 с: 26 042 × 60
`active_invite_tokens`	30 000 000	128 B	3.84 GB	5 208	521	токены встреч за сутки
`chat_last100`	168.75 млн сообщений в пик	1 KB	168.75 GB	≤ 5 208 запросов по 100 строк	26 042	1 687 500 встреч × 100 сообщений

5.3. Требования к консистентности

Таблица	Консистентность	Почему
`users`	Strong	Нельзя авторизовать несуществующего пользователя
`user_sessions`	Strong	Ошибка сессии ломает вход во встречу
`meetings`	Strong в `home_dc`	`join` должен видеть актуальный статус встречи
`meeting_participants`	Strong в `home_dc`	Нельзя дважды добавить одного участника
`meeting_invite_links`	Strong	Токен должен быть однозначно валиден или отозван
`user_meetings`	Eventual	Список встреч может обновиться с небольшой задержкой
`chat_messages`	Strong append внутри `meeting_id`	Внутри встречи важен порядок сообщений
`recordings`	Strong metadata + immutable file	UI должен видеть корректный статус записи
`recording_upload_buffer`	At-least-once + checksum	Часть можно перезалить под тем же номером, итоговый файл проверяется

`meetings_runtime`	Eventual, last write wins	Эфемерное состояние; после потери его восстанавливают heartbeat-ы клиентов
`chat_seq` , `chat_dedup`	Strong в одном шарде Redis: проверка и выдача номера — один атомарный скрипт	Номер выдаётся ровно один раз, повтор запроса не создаёт дубль

5.4. Распределение нагрузки по ключам

Ключ	Где используется	Распределение	Риск hot key	Что делаем логически
`user_id`	`users` , `user_sessions` , `user_meetings`	Почти равномерно	Низкий	Храним пользовательские данные отдельно от встреч
`meeting_id`	`meetings` , `participants` , `chat` , `recordings`	Неравномерно	Высокий на больших встречах	Встреча закрепляется за `home_dc` , код ДЦ записан в первом байте `meeting_id` (п. 3.1)
`meeting_id + user_id`	`meeting_participants` , `user_meetings`	Равномернее, чем один `meeting_id`	Средний	Участники пишутся отдельными строками
`session_id`	`user_sessions`	Равномерно	Низкий	Сессия ищется по прямому ключу
`token`	`meeting_invite_links`	Равномерно	Низкий	Уникальный токен приглашения
`message_id`	`chat_messages`	Равномерно	Низкий	Порядок задаётся внутри `meeting_id`
`recording_id`	`recordings` , `recording_upload_buffer`	По созданным записям	Средний	Файл после финализации не изменяется
`storage_url`	`recordings`	Может перекоситься по ДЦ/дате	Средний	Путь строится как `home_dc/date/hash(recording_id)`

6. Физическая схема БД

6.1. Физическая схема БД

Для получения картинки использовался сервис dbdiagram с кодом из этого файла (интерактивная ссылка на работу).


meeting_scylla.meeting_participants
ScyllaDB
meeting_id uuid NN
bucket smallint NN
user_id uuid NN
role varchar
is_online boolean
last_joined_at datetime
last_left_at datetime
join_count integer
total_online_seconds bigint
recording_obj.recording_upload_buffer
chunk_id uuid
recording_id uuid NN
chunk_number bigint NN
storage_key varchar NN
size_bytes bigint
checksum varchar
status varchar
expires_at datetime
recording_pg.recordings
id uuid
meeting_id uuid NN
storage_bucket varchar
storage_key varchar
storage_url varchar
size_bytes bigint
duration_seconds integer
status varchar
created_at datetime
meeting_pg.meetings
id uuid
owner_id uuid NN
home_dc varchar
topic varchar
scheduled_start datetime
duration_minutes integer
status varchar
created_at datetime
meeting_scylla.user_meetings
ScyllaDB
user_id uuid NN
scheduled_start datetime NN
meeting_id uuid NN
topic varchar
status varchar
role varchar
auth_pg.users
id uuid
email varchar NN
password_hash varchar NN
name varchar
created_at datetime
meeting_pg.meeting_invite_links
id uuid
meeting_id uuid NN
created_by_user_id uuid NN
token varchar NN
token_hash bigint NN
created_at datetime
expires_at datetime
revoked boolean
session_redis.user_sessions
session_id varchar
user_id uuid NN
expires_at datetime
created_at datetime
runtime_redis.meetings_runtime
meeting_id uuid
status varchar
participants_online_count integer
participants_total_count integer
sfu_pool varchar
updated_at datetime
chat_scylla.chat_messages
ScyllaDB
id uuid NN
meeting_id uuid NN
bucket smallint NN
message_seq integer NN
user_id uuid NN
body text
created_at datetime

Картинка показывает таблицы, поля, связи и привязку к хранилищам. Индексы и точные ключи dbdiagram не рисует: они — в DBML и в п. 6.4. В Citus первичный ключ и UNIQUE обязаны включать колонку распределения [94], поэтому физически ключи `recordings` и `meeting_invite_links` составные (п. 6.4), а на картинке показан логический идентификатор `id`.

6.2. Где хранятся данные

Таблица	Где хранится	Что хранится физически	Главный ключ доступа	Размещение по ДЦ (п. 3.4)
`users`	PostgreSQL, `auth_pg`	Аккаунты пользователей	`id`, `email`	primary в `DC-US-EAST`, реплики во всех ДЦ
`user_sessions`	Redis Cluster, `session_redis`	Активные сессии с TTL	`session_id = <dc>.<128 бит>`	ДЦ логина, перенос по запросу
`meetings`	PostgreSQL + Citus, `meeting_pg`	Метаданные встреч	`id = meeting_id`	`home_dc` + реплика в парном ДЦ
`meeting_participants`	ScyllaDB, `meeting_scylla`	Участники и состояние входа	`meeting_id + bucket + user_id`	`home_dc`
`meeting_invite_links`	PostgreSQL + Citus, `meeting_pg`	Токены приглашений	`token_hash`, `token`	`home_dc` + реплика в парном ДЦ
`user_meetings`	ScyllaDB, `meeting_scylla`	Список последних встреч пользователя	`user_id + scheduled_start`	все ДЦ, RF=3 в каждом
`meetings_runtime`	Redis Cluster, `runtime_redis`	Онлайн-состояние встречи	`meeting_id`	`home_dc`
`chat_messages`	ScyllaDB, `chat_scylla`	Append-only лог сообщений	`meeting_id + bucket + message_seq`	`home_dc`
`recordings`	PostgreSQL + Citus, `recording_pg`	Метаданные записей	`meeting_id`, `id`	`home_dc` + реплика в парном ДЦ
`recording_upload_buffer`	S3-compatible Object Storage	Части multipart upload незавершённых записей; их список держит воркер записи, после сбоя он читается обратно из S3 [46]	`recording_id + chunk_number`	`home_dc`
`chat_seq`, `chat_dedup`, `active_invite_tokens`	Redis Cluster, `runtime_redis`	Счётчик сообщений, ключи идемпотентности, кеш токенов	`meeting_id`, `token`	`home_dc`
`chat_last100`	Память Chat-сервиса	Буфер последних сообщений идущей встречи	`meeting_id`	`home_dc`

`meeting_pg` и `recording_pg` — две схемы одной базы на Citus-кластере ДЦ. Отдельные базы не подошли бы: co-location работает только между таблицами одной базы. У `meetings` и `recordings` одна колонка распределения — `meeting_id`, поэтому шарды встречи и её записей лежат на одном узле [30]. Нагрузка на один ДЦ при этом мала (п. 11.2).

6.3. Выбор СУБД по таблицам

Таблица	СУБД	Почему так
`users`	PostgreSQL	Нужны транзакции, уникальный `email`, индексы
`user_sessions`	Redis Cluster	TTL и быстрый lookup сессии
`meetings`	PostgreSQL + Citus	SQL-метаданные + шардирование по `meeting_id`
`meeting_participants`	ScyllaDB	Много записей `join/leave`, JOIN не нужен
`meeting_invite_links`	PostgreSQL + Citus	Точечный lookup по токену и TTL-индекс
`user_meetings`	ScyllaDB	Top-100 встреч пользователя без JOIN
`meetings_runtime`	Redis Cluster	≈ `738 000` ops/s в пике: heartbeat 675 000 + join/leave/create + `INCR chat_seq` + `SET NX` ключа идемпотентности чата
`chat_messages`	ScyllaDB	Append-only лог, чтение диапазона по `message_seq`
`recordings`	PostgreSQL + Citus	Статусы и метаданные записей
`recording_upload_buffer`	S3-compatible Object Storage	Чанки и большие файлы не хранятся в SQL

6.4. Индексы

Таблица	Индекс	Для какого запроса
`users`	`PK(id)`	Получение пользователя по `user_id`
`users`	`UNIQUE(email)`	Логин и проверка дубля
`users`	`INDEX(created_at)`	Админские выборки по дате
`user_sessions`	`KEY session:{session_id}`	Проверка сессии
`user_sessions`	`TTL(expires_at)`	Автоудаление сессий
`meetings`	`PK(id)`	`join`, `leave`, `recording_metadata`
`meetings`	`INDEX(owner_id, scheduled_start)`	Список встреч владельца
`meetings`	`INDEX(home_dc, status)`	Выбор активных встреч в ДЦ
`meeting_participants`	`PK(meeting_id, bucket, user_id)`	Вход/выход участника
`meeting_participants`	`INDEX(user_id, last_joined_at)`	История участника
`meeting_invite_links`	`PK(token_hash, id)`	Колонка распределения — `token_hash = hash(token)`: Citus сразу направляет запрос в один шард. PK обязан её включать [94]
`meeting_invite_links`	`UNIQUE(token_hash, token)`	Вход по ссылке. `token_hash` вычисляется из `token`, поэтому это та же уникальность токена
`meeting_invite_links`	`INDEX(expires_at)`	Удаление истёкших ссылок
`user_meetings`	`PK(user_id, scheduled_start, meeting_id)`	Top-100 встреч пользователя

`meetings_runtime`	`KEY runtime:{meeting_id}`	Heartbeat и счётчики
`meetings_runtime`	TTL ключа, его продлевает heartbeat	Брошенная встреча удаляется сама [22]. Вторичных индексов в Redis нет, поэтому поиск «по времени последнего heartbeat» не нужен
`chat_seq` , `chat_dedup`	`KEY chat_seq:{meeting_id}` , `KEY chat_dedup:{meeting_id}:{client_msg_id}`	Hash tag `{meeting_id}` кладёт ключи встречи в один слот Redis Cluster [31], поэтому проверка повтора и выдача номера — один атомарный скрипт
`chat_messages`	`PK(meeting_id, bucket, message_seq)`	Чтение сообщений по порядку
`chat_messages`	Ключ идемпотентности в Redis: `SET chat_dedup:{meeting_id}:{client_msg_id} <seq> NX EX 60` [98]	Повтор запроса с тем же `client_msg_id` получает уже выданный `message_seq` и не создаёт дубль [45]. В самой ScyllaDB нет UNIQUE: `INSERT` с тем же ключом перезаписывает строку [101], а повтор с новым `message_seq` стал бы дублем
`recordings`	`PK(meeting_id, id)`	Карточка записи. Таблица распределена по `meeting_id` , и PK обязан её включать [94]
`recordings`	`INDEX(meeting_id, created_at)`	Список записей встречи
`recordings`	`INDEX(status, created_at)`	Очередь обработки
`recording_upload_buffer`	`UNIQUE(recording_id, chunk_number)`	Номер части уникален в загрузке: повторная загрузка части под тем же номером её перезаписывает [46]
`recording_upload_buffer`	`TTL(expires_at)`	Брошенные загрузки удаляет правило lifecycle `AbortIncompleteMultipartUpload` [46]

Индекс в шардированной таблице работает внутри шарда. Поэтому запрос должен сначала попадать в правильный шард по `meeting_id` , `user_id` или `token_hash` .

6.5. Шардирование и размещение по дата-центрам

Шардирование двухуровневое. **Уровень 1 — по ДЦ:** всё, что относится к встрече, лежит в её `home_dc` , код которого записан в `meeting_id` (п. 3.1, 3.4). **Уровень 2 — внутри ДЦ:** по ключам из таблицы ниже.

Таблица	Что шардируем	Ключ шардинга	Резервирование	Почему
`users`	Не шардируем на MVP	`id` как partition key	Синхронная standby в другой AZ + асинхронная реплика в каждом ДЦ	`81.92 GB` , запись низкая; один primary на весь мир (п. 3.4)
`user_sessions`	Да	`session_id`	Redis Cluster: у каждого master есть replica на другом узле	`115 млн` активных сессий, Redis Cluster в каждом ДЦ
`meetings`	Да: по `home_dc` , затем по `meeting_id`	`meeting_id`	Citus: синхронная standby в другой AZ + асинхронная реплика в парном ДЦ	`900 млн` строк за 30 дней на весь мир, в самом большом ДЦ (`DC-DE-FRANKFURT`) ≈ 321 млн
`meeting_participants`	Да	`meeting_id + bucket`	ScyllaDB, RF=3 по 3 AZ `home_dc`	`9 млрд` строк за 30 дней
`meeting_invite_links`	Да	`token_hash`	Как у `meetings`	`join` начинается с токена
`user_meetings`	Да	`user_id`	ScyllaDB, RF=3 в каждом ДЦ	Пользователь читает свой сплошной список; копия в каждом

				ДЦ
`meetings_runtime`	Да	`meeting_id`	Redis replica; потерю восстанавливают heartbeat-ы клиентов	≈ `0.74 млн` ops/s на весь мир вместе с `chat_seq` и `chat_dedup`
`chat_messages`	Да	`meeting_id + bucket`	ScyllaDB, RF=3 по 3 AZ `home_dc` + снапшоты (п. 6.12)	Чат читается по встрече и диапазону `seq`
`recordings`	Да	`meeting_id`	Как у `meetings`	Записи читаются по встрече
`recording_upload_buffer`	Да	`recording_id`	S3: ≥ 3 копии в 3 AZ	Части собираются в одну запись

Объём метаданных одного ДЦ невелик: в самом большом, `DC-DE-FRANKFURT`, ≈ 206 GB и ≈ 5 950 чтений/с (встречи, ссылки и записи). Поэтому Citus в каждом ДЦ запускается в режиме single-node — координатор сам хранит шарды [90]. Distribution column `meeting_id` задан сразу, и при росте добавляются worker-узлы с ребалансировкой шардов без смены схемы [29,30].

Правило	Как применено
Репликация масштабирует чтение	Для чтения можно использовать read-replica
Репликация не масштабирует запись	Запись масштабируется шардингом
JOIN на больших RPS не делаем	Hot-path разбит на point queries
Таблицы в разных СУБД не JOIN-ятся	Связи логические, не физические FK
Top-100 списка пользователя	Шард по `user_id`, чтение без fan-out
Чат встречи	`message_seq integer`, чтение от `last_seen_seq` до `head_seq`

6.6. Нагрузка на строки

API / событие	Peak RPS	Что трогаем	Строк / ключей на запрос	Lines per second
`POST /auth/login`	**1 997**	`users` (чтение реплики), `user_sessions` (запись)	2	**3 993**
`POST /meetings`	**521**	`meetings`, `meeting_invite_links`, `user_meetings`, `meetings_runtime`	4	**2 083**
`POST /meetings/{id}/join`	**5 208**	`user_sessions`, `meeting_invite_links`, `meetings`, `meeting_participants`, `user_meetings`, `meetings_runtime`	6	**31 250**
`POST /meetings/{id}/leave`	**5 208**	`meeting_participants`, `user_meetings`, `meetings_runtime`	3	**15 625**
`WS /presence/heartbeat`	**675 000**	`meetings_runtime`	1	**675 000**
`POST /meeting-chat/messages`	**26 042**	`chat_dedup` и `chat_seq` (один скрипт Redis), `chat_messages`	3	**78 125**
`GET /meeting-chat/messages?after_seq=N`	**5 208**	`chat_last100` в памяти; `chat_messages` в ScyllaDB — только при промахе	100	**520 833**
`GET /users/{id}/meetings?`	**5 990**	`user_meetings`	100	**598 958**

`limit=100`				
`POST /recordings/metadata`	**521**	`recordings`, `meetings`	2	**1 042**
`GET /recordings/{id}`	**521**	`user_sessions`, `recordings`	2	**1 042**

Частота чата, истории чата, списка встреч и просмотров записей — допущения из п. 2.1. История чата отдаётся из буфера комнаты в памяти Chat-сервиса (п. 6.8): 520 833 строк/с — это чтение из памяти, в ScyllaDB идут только промахи. Heartbeat обновляет только `meetings_runtime`: WSS-соединение авторизуется один раз при подключении по `join_token`, поэтому `user_sessions` на каждом heartbeat не читается. В прошлой версии здесь стояло 2 ключа, а в п. 6.8 нагрузка на `user_sessions` считалась без heartbeat — теперь оба пункта согласованы.

6.7. API -> запрос -> БД

API	Запрос к БД	Где выполняется	Индекс
`POST /meetings`	`INSERT INTO meetings ...`	PostgreSQL + Citus	`PK(id)`
`POST /meetings`	`INSERT INTO meeting_invite_links ...`	PostgreSQL + Citus	`UNIQUE(token_hash, token)`
`POST /meetings`	`INSERT INTO user_meetings ...` — строка организатора	ScyllaDB	`user_id, scheduled_start`
`POST /meetings`	`HSET runtime:{meeting_id} ...`	Redis Cluster	`meeting_id`
`POST /meetings/{id}/join`	`GET session:{session_id}`; если сессии нет локально — один gRPC-запрос в ДЦ из префикса `session_id` (п. 3.8)	Redis Cluster ближайшего ДЦ	`session_id`
`POST /meetings/{id}/join`	`SELECT meeting_id FROM meeting_invite_links WHERE token_hash = hash(?) AND token = ?`	PostgreSQL + Citus	`UNIQUE(token_hash, token)`
`POST /meetings/{id}/join`	`SELECT id, status, home_dc FROM meetings WHERE id = ?`	PostgreSQL + Citus	`PK(id)`
`POST /meetings/{id}/join`	`INSERT INTO meeting_participants ...`	ScyllaDB	`meeting_id, bucket, user_id`
`POST /meetings/{id}/join`	`INSERT INTO user_meetings ...`	ScyllaDB	`user_id, scheduled_start`
`POST /meetings/{id}/join`	`HINCRBY runtime:{meeting_id} participants_online_count 1`	Redis Cluster	`meeting_id`
`POST /meetings/{id}/leave`	`UPDATE meeting_participants SET ...`	ScyllaDB	`meeting_id, bucket, user_id`
`POST /meetings/{id}/leave`	`UPDATE user_meetings SET status ...`	ScyllaDB	`user_id, scheduled_start`
`POST /meetings/{id}/leave`	`HINCRBY runtime:{meeting_id} participants_online_count -1`	Redis Cluster	`meeting_id`
`WS /presence/heartbeat`	`HSET runtime:{meeting_id} updated_at ...` и продление TTL	Redis Cluster	`meeting_id`
`POST /meeting-`	Скрипт: `SET chat_dedup:{meeting_id}:{client_msg_id} NX EX 60`, при успехе `INCR chat_seq:`	Redis Cluster, один	`meeting_id`

`chat/messages`	`{meeting_id}`	слот	
`POST /meeting-chat/messages`	`INSERT INTO chat_messages ...`	ScyllaDB	`meeting_id, bucket, message_seq`
`GET /meeting-chat/messages?after_seq=N`	Из буфера комнаты; при промахе — `SELECT ... WHERE meeting_id=? AND bucket=? AND message_seq>? LIMIT 100`	Память Chat-сервиса / ScyllaDB	`meeting_id, bucket, message_seq`
`GET /users/{id}/meetings?limit=100`	`SELECT ... WHERE user_id=? LIMIT 100`	ScyllaDB	`user_id, scheduled_start`
`POST /recordings/metadata`	`INSERT INTO recordings ...`	PostgreSQL + Citus	`PK(meeting_id, id)`
`GET /recordings/{id}`	`GET session:{session_id}`, затем `SELECT storage_bucket, storage_key FROM recordings WHERE meeting_id = ? AND id = ?` и выдача presigned URL	Redis Cluster, PostgreSQL + Citus	`PK(meeting_id, id)`
Воркер записи	`UploadPart`: часть `n` multipart upload, 8 MB	S3-compatible storage	`recording_id, chunk_number`

Запросы к таблицам встречи выполняются в `home_dc`. Если `join` пришёл в другой ДЦ, control plane проксирует его в `home_dc` по коду ДЦ в `meeting_id` (п. 3.4).

6.8. Кеш

Кеш	Raw size	Peak ops	Cache heat	Решение
`meetings_runtime`	**0.864 GB**	**685 938 ops/s**	Очень высокий	Обязательно Redis (`runtime_redis`)
`chat_seq` + `chat_dedup`	**≈ 0.16 GB**	**52 083 ops/s**	Высокий	Тот же `runtime_redis`, ключи встречи в одном слоте
`user_sessions`	**14.72 GB**	**14 236 ops/s**	Высокий	Обязательно Redis (`session_redis`)
`active_invite_tokens`	**3.84 GB**	**5 208 ops/s**	Высокий только на join	Можно Redis + PostgreSQL как source of truth
`user_meetings_top100`	**1.472 TB** для всех DAU	**598 958 rows/s**	Большой объём	Не кешировать полностью, хранить read model в ScyllaDB
`chat_last100`	**168.75 GB** raw	**≤ 520 833 rows/s** чтения, 26 042 записей/с	Средний	In-memory room buffer, не общий Redis

Формула	Значение
`meetings_runtime`	`1 687 500 active meetings * 512 B = 0.864 GB`; ops = heartbeat 675 000 + join 5 208 + leave 5 208 + create 521 = 685 938
`chat_seq` + `chat_dedup`	ops = `INCR` 26 042 + `SET NX` 26 042 = 52 083; ключей идемпотентности `26 042 * 60 с ≈ 1.56 млн * 100 B ≈ 0.16 GB`. Всего на `runtime_redis` — 738 021 ops/s
`user_sessions`	ops = проверки при create, join, list и открытии записи (12 240) + запись при login (1 997) = 14 236
`active_invite_tokens`	`30 000 000 meetings/day * 128 B = 3.84 GB`
`user_meetings_top100`	`115 000 000 DAU * 100 * 128 B = 1.472 TB`
`chat_last100`	`1 687 500 active meetings * 100 * 1 KB = 168.75 GB`; чтения — до `5 208 * 100` строк/с, записи — каждое новое сообщение

6.9. Денормализация и отказ от JOIN

Денормализация	Где хранится	Зачем
`topic`, `scheduled_start`, `status`, `role` в `user_meetings`	ScyllaDB	Список встреч пользователя без JOIN с `meetings`
`participants_online_count` в `meetings_runtime`	Redis	Не считать `COUNT(*)` по участникам
`home_dc` в `meetings`	PostgreSQL + Citus	Быстро направлять meeting traffic
`storage_bucket`, `storage_key` в `recordings`	PostgreSQL + Citus	UI получает ссылку без обхода object storage
`message_seq` в `chat_messages`	ScyllaDB	Клиент читает лог от `last_seen_seq` до `head_seq`

Сценарий	Физическая проекция	Ключ чтения	JOIN в hot-path
Top-100 актуальных встреч пользователя	`user_meetings`	`user_id`	Нет
События конкретной встречи	`meetings`, `meeting_participants`, `chat_messages`	`meeting_id`	Нет
Доступ пользователя к своим встречам	`user_meetings`	`user_id`	Нет
Доступ встречи к участникам	`meeting_participants`	`meeting_id + bucket`	Нет
Чтение новых сообщений	`chat_messages`	`meeting_id + bucket + message_seq`	Нет

6.10. Клиентские библиотеки

Хранилище	C++-клиент	Есть ли для C++	Источник
PostgreSQL	`libpqxx`, `libpq`	Да	`libpqxx` — C++ API, `libpq` — C API PostgreSQL [35,36]
PostgreSQL + Citus	`libpqxx` через PostgreSQL protocol	Да	Citus работает как расширение PostgreSQL [29]
Redis Cluster	`hiredis`, `redis-plus-plus`	Да	`hiredis` — официальный C-клиент, `redis-plus-plus` — C++ wrapper [37,38]
ScyllaDB	ScyllaDB CPP-RS Driver	Да	API-совместимая замена прежнего C/C++ драйвера, который больше не развивается [34]
S3-compatible storage	AWS SDK for C++	Да	Есть официальный AWS SDK for C++ [39]
PgBouncer	PostgreSQL protocol	Да	Для приложения это обычный PostgreSQL endpoint [32]

6.11. Балансировка запросов и мультиплексирование подключений

Хранилище	Как запрос попадает на нужный узел	Мультиплексирование подключений
PostgreSQL `auth_pg`	Запись — в primary `DC-US-EAST` через gRPC-прокси (п. 3.4), чтение — из реплики своего ДЦ	PgBouncer в режиме transaction pooling: серверное соединение выдаётся клиенту только на время транзакции [32]. Тысячи соединений подов сводятся к десяткам соединений с PostgreSQL
PostgreSQL + Citus	Координатор Citus направляет запрос в шард по колонке распределения: `meeting_id` или `token_hash` [29]	Тот же PgBouncer перед координатором

Redis Cluster	Клиент сам вычисляет слот ключа (16 384 слота) и идёт сразу в нужный master. Hash tag `{meeting_id}` держит ключи одной встречи в одном слоте [31]	Одно долгое соединение с каждым узлом из каждого пода
ScyllaDB	Драйвер маршрутизирует запрос по токену ключа прямо на узел с данными, а внутри узла — на нужное ядро CPU (shard-awareness). DC-aware политика держит запросы в своём ДЦ [100]	Пул соединений драйвера к каждому узлу
S3-compatible storage	Клиент скачивает запись по presigned URL напрямую из object storage `home_dc`; воркеры записи загружают части в `home_dc`	Обычный HTTPS, без отдельного балансировщика с нашей стороны

6.12. Резервное копирование

Хранилище	Как копируем	Частота и хранение	Куда	RPO при потере всего ДЦ
PostgreSQL `auth_pg`	Непрерывный архив WAL + базовая копия для восстановления на момент времени (PITR) [28]	WAL — непрерывно, принудительное переключение сегмента раз в 60 с (`archive_timeout`); базовая копия — раз в сутки; хранение 14 дней	S3 в `DC-US-WEST`	≈ 1 мин по архиву; обычный отказ закрывает реплика (п. 3.4)
PostgreSQL + Citus (`meeting_pg`, `recording_pg`)	То же: архив WAL и базовые копии каждого узла [28]	Так же, как у `auth_pg`	S3 парного ДЦ	≈ 1 мин по архиву; раньше срабатывает асинхронная реплика в парном ДЦ
ScyllaDB `meeting_scylla`, `chat_scylla`	Снапшоты узлов через ScyllaDB Manager с выгрузкой в S3-совместимое хранилище [96]	Раз в сутки, хранение 7 дней	S3 парного ДЦ	До 24 ч для истории участников и чата `home_dc`: осознанный компромисс, отказы узлов и AZ закрывает RF=3 (п. 9.1)
Redis `session_redis`, `runtime_redis`	Не копируем: данные эфемерные	—	—	Сессии — повторный логин; состояние встреч восстанавливают heartbeat-ы клиентов
S3, файлы записей	Отдельной копии нет: ≥ 3 копии внутри `home_dc` в 3 AZ	—	—	Записи `home_dc` теряются: копия 685 PB в другом регионе удвоила бы самую дорогую статью расходов (п. 3.4)

Объём бэкапов баз — сотни терабайт: это доли процента от 685 PB записей, на итоговую стоимость (п. 11.4) не влияет.

7. Алгоритмы

В раздел включены только алгоритмы, которые влияют на нагрузку, БД, media plane и взаимодействие сервисов.

7.1. Список алгоритмов

Алгоритм	Где используется	Коротко	На что влияет
Adaptive Bitrate	Media Plane / SFU	меняем качество под сеть участника	bandwidth, CPU клиента, SFU-routing
Выбор SFU-слоя видео	SFU	подписчику отдаётся нужный слой 720p/360p/180p	исходящий трафик SFU

7.2. Adaptive Bitrate

Параметр	Значение
Блок	`Media Plane / SFU`
Задача	Подобрать качество видео под текущую сеть участника
Вход	`packet_loss`, `RTT`, `jitter`, `available_bitrate`, `frames_dropped`, `audio_quality`
Ограничение	Звук важнее видео; понижение быстрое, повышение плавное
Результат	`target_profile = 720p / 360p / 180p / audio_only`

Метрики сети SFU получает из RTCP внутри самой медиасессии: отчёты о приёме с потерями и джиттером [44] и feedback о времени доставки пакетов для congestion control [41,][41]. Клиент видит те же метрики через WebRTC stats API [40,][40]. Отдельных запросов к API и WebSocket для статистики нет, поэтому в расчёт нагрузки (п. 2.4) она не входит. Верхний профиль — 720p: на нём построен расчёт трафика (п. 2.4) по bandwidth-оценке Zoom [5,][5]. 1080p в MVP не включаем: он требует 3.8/3.0 Мбит/с [5,][5] и увеличил бы трафик и число SFU.

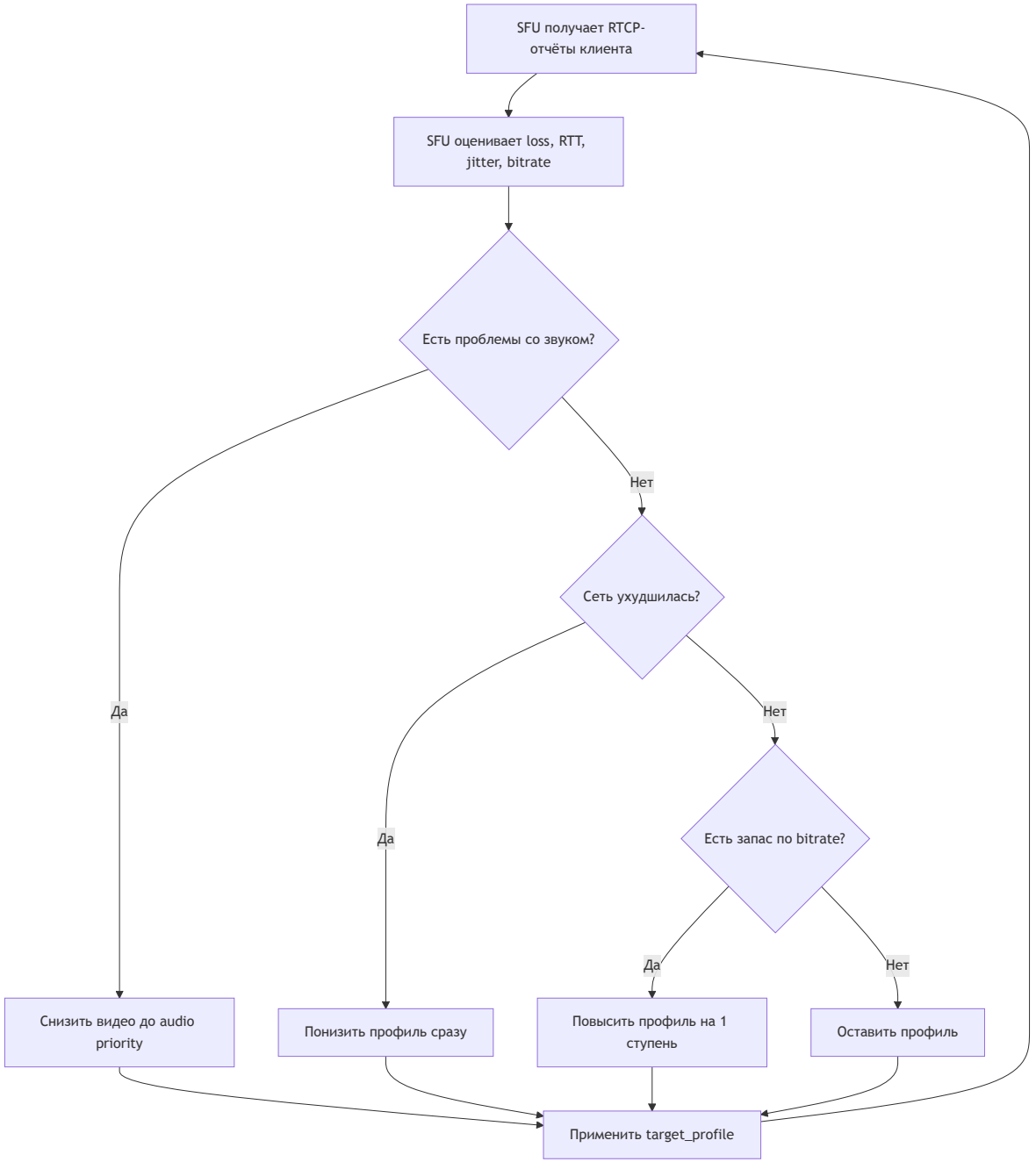
Альтернатива	Минус	Решение
Всегда 720p	При плохой сети звонок рвётся	Не подходит
Только клиент решает качество	SFU хуже управляет исходящим трафиком	Не выбираем
Server-guided ABR	SFU видит подписчиков и управляет слоями	**Выбрано**

Алгоритм:

1. SFU получает RTCP-отчёты клиента — обычно несколько раз в секунду, внутри медиасессии.
2. SFU считает состояние сети: потери, RTT, jitter, доступная полоса.
3. Если сеть плохая — профиль снижается сразу.
4. Если сеть стабильна — профиль повышается на одну ступень.
5. Если страдает audio — video снижается первым.

6. Новый профиль применяется к подписке участника.

Структура	Где	Поля
`participant_network_stats`	memory / metrics pipeline	`meeting_id`, `user_id`, `loss`, `rtt`, `jitter`, `bitrate`
`participant_media_profile`	SFU memory	`meeting_id`, `user_id`, `target_profile`, `updated_at`


SFU получает RTCP-отчёты клиента
SFU оценивает loss, RTT, jitter, bitrate
Есть проблемы со звуком?
Да
Нет
Сеть ухудшилась?
Да
Нет
Есть запас по bitrate?
Да
Нет
Снизить видео до audio priority
Понизить профиль сразу
Повысить профиль на 1 ступень
Оставить профиль
Применить target_profile

Влияние	Что меняется
БД	Постоянные таблицы не меняются
Redis	Не меняется: профили живут в памяти SFU, метрики качества уходят в Prometheus (п. 8)
SFU	Нужна per-participant media state
Нагрузка	Снижается video download при плохой сети

7.3. Выбор SFU-слоя видео

Параметр	Значение
Блок	`Media Plane / SFU`
Задача	Не транскодировать видео, а выбирать готовый слой
Вход	`target_profile`, `viewport`, `active_speaker`, доступные encodings
Ограничение	SFU не должен превращаться в MCU с высоким CPU
Результат	`selected_layer` для каждого подписчика

SFU-подход выбран потому, что media-сервер маршрутизирует RTP-потоки [44] между участниками, а не смешивает их в один общий поток [,14]. Для нескольких качеств используется simulcast / SVC: отправитель публикует несколько encodings, SFU выбирает нужный слой [,42].

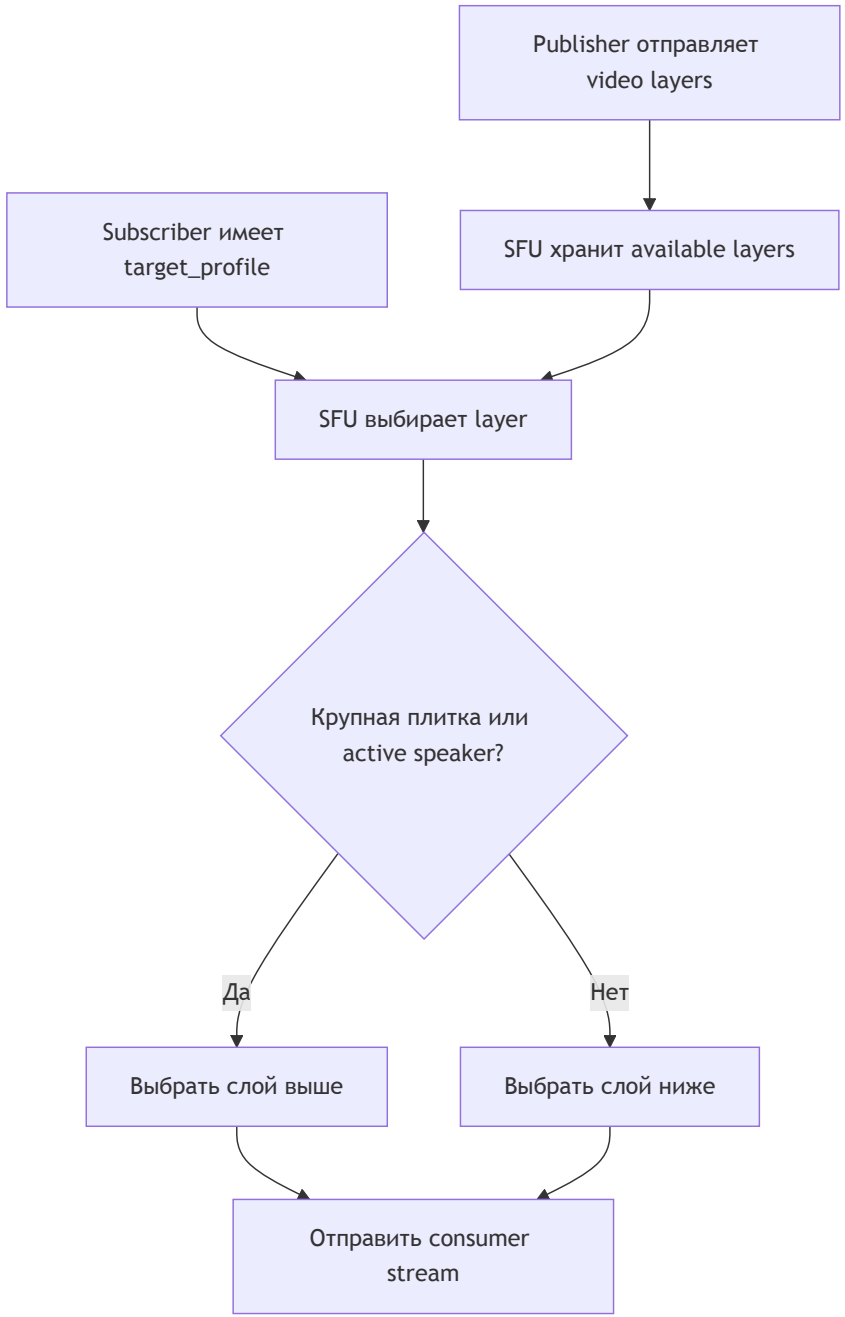
Альтернатива	Минус	Решение
Single stream	Всем отдаётся одно качество	Плохо для слабой сети
MCU transcoding	Высокая CPU-нагрузка	Не подходит для highload
SFU + simulcast/SVC	Больше upload от говорящего, зато меньше CPU на сервере	**Выбрано**

Алгоритм:

1. Клиент публикует несколько video layers.
2. SFU хранит список доступных слоёв для producer.
3. Для каждого consumer считается `target_profile`.
4. Активному говорящему (его определяет active speaker detection [43]) и крупной плитке даётся слой выше.
5. Миниатюрам даётся слой ниже.
6. При деградации сети SFU переключает consumer на слой ниже.

Структура	Где	Поля
`producer_layers`	SFU memory	`producer_id`, `rid`, `width`, `height`, `max_bitrate`

`consumer_subscription`	SFU memory	`consumer_id`, `producer_id`, `selected_layer`
`participant_media_profile`	SFU memory	`user_id`, `target_profile`, `reason`


Publisher отправляет video layers
Subscriber имеет target_profile
SFU хранит available layers
SFU выбирает layer
Крупная плитка или active speaker?
Да
Нет
Выбрать слой выше
Выбрать слой ниже
Отправить consumer stream

Влияние	Что меняется
БД	Не требуется новая persistent-таблица
SFU	Нужна таблица подписок `consumer -> producer -> layer`
Сеть	Снижается исходящий трафик SFU
CPU	Нет постоянного transcoding на сервере

8. Технологии

В раздел включены технологии, которые нужны для API, media plane, БД, деплоя и наблюдаемости.

8.1. Языки

Язык / стандарт	Где используется	Мотивационная часть
`C++23`	Backend, control-plane, SFU, recording workers	Основной язык: высокая производительность, строгая типизация, современный стандарт [47]
`TypeScript`	Web-клиент и админка	Типизированный frontend, удобно работать с WebRTC API [48]
`SQL`	PostgreSQL / Citus	Транзакционные метаданные: уровни изоляции транзакций [26], составные индексы [,25]
`CQL`	ScyllaDB	Запросы по partition key для чата и участников [62]

8.2. Сводная таблица технологий

Технология	Область применения	Мотивационная часть
`C++23`	Основной backend	Низкая latency, контроль памяти, удобен для media/control-plane [47]
`CMake`	Сборка C++ сервисов	Target-based сборка, удобно делить сервисы и библиотеки [49]
`Conan 2`	C++ зависимости	Фиксирует версии библиотек, упрощает сборку на разных окружениях [50]
`Boost.Asio / Boost.Beast`	HTTP / WSS / async networking	Асинхронная сеть в C++ без лишнего runtime [51]
`gRPC + Protobuf`	Внутренние RPC между сервисами	Строгие контракты, генерация C++ client/server кода [52]
`OpenSSL`	TLS / mTLS	Шифрование HTTPS/WSS и внутренних соединений [61]
`WebRTC`	Аудио, видео, screen sharing	Базовый стек real-time media, RTCP feedback и media stats [40,41]
`SFU`	Media Plane	Маршрутизация потоков без смешивания видео, поддержка simulcast/SVC [14,42]
`GeoDNS / GTM (managed)`	Выбор ДЦ и переключение между ДЦ	Geo + latency routing, health checks, веса пулов; NS-серверы провайдера на Anycast [15,16,17,83]
`IPVS + keepalived`	L4 в каждом ДЦ	Балансировка TCP в ядре Linux; в режиме Direct Routing ответы идут мимо директора; VIP переезжает на резервный директор по VRRP [91,92,93]
`NGINX L7`	HTTPS/WSS edge	SSL Termination, L7 routing, балансировка API [19,20]
`Kubernetes`	Оркестрация сервисов	Deployment, scaling, Service discovery, self-healing [54]
`Docker / OCI`	Упаковка сервисов	Один образ для dev/stage/prod, проще CI/CD [53]
`Helm`	Деплой в Kubernetes	Версионируемые шаблоны для сервисов и конфигов [55]
`PostgreSQL`	`users` , `meetings` , `recordings`	

	metadata	Транзакции с изоляцией (strong consistency) [26], составные индексы [25]
`Citus`	Шардирование PostgreSQL	Распределение таблиц по `meeting_id` / `token_hash` [29]
`PgBouncer`	Пул соединений PostgreSQL	Меньше постоянных соединений к PostgreSQL [32]
`Patroni`	Failover PostgreSQL (`auth_pg` , Citus)	Внутри ДЦ автоматически переключает primary на синхронную standby [86]; между ДЦ — standby-кластер, его повышают вручную [97]
`Redis Cluster`	`sessions` , `meetings_runtime` , counters	TTL, hot state, быстрый key-value доступ [22,31]
`ScyllaDB`	`participants` , `chat_messages` , `user_meetings`	Высокая write-нагрузка, чтение по ключу без JOIN, мультиДЦ-репликация `user_meetings` [33,34,80]
`S3-compatible Object Storage`	Записи встреч и чанки	Большие immutable-файлы, multipart upload [23,46]
`AWS SDK for C++`	Доступ к object storage	C++ клиент для S3-compatible API [39]
`Prometheus`	Метрики	Time-series мониторинг и alerting [56]
`Grafana`	Дашборды	Визуализация метрик, логов и traces [57]
`OpenTelemetry`	Instrumentation	Единый сбор traces, metrics, logs [58]
`Loki`	Логи	Централизованное хранение и поиск логов [59]
`Jaeger`	Distributed tracing	Поиск узких мест между микросервисами [60]

9. Обеспечение надёжности

9.1. Резервирование компонентов

Компонент системы	Что резервируем	Способ резервирования	Формула	При отказе
`GeoDNS / GTM`	выбор ДЦ и переключение между ДЦ	managed-сервис: health checks + geo/latency routing + веса пулов [15,16,17]	8 ДЦ, у каждого резерв по п. 3.7	ДЦ убирается из DNS-ответов, новые подключения уходят в здоровые ДЦ
`Anycast DNS`	авторитетные NS-серверы GTM	один Anycast IP из многих PoP провайдера [83]	`N_PoP >= 2`	DNS-запросы уходят в ближайший живой PoP
`L4: IPVS + keepalived`	TCP-вход HTTPS/WSS в ДЦ	пара директоров active/standby: VIP переезжает по VRRP, keepalived проверяет узлы NGINX [91,92,93]	`N+1` : 2 на ДЦ	VIP за секунды переходит на резервный директор; оборванные соединения клиенты открывают заново; узел NGINX с ошибками исключается из балансировки
`NGINX L7 pool`	HTTPS/WSS, TLS termination	active-active, health checks, запас по памяти под соединения [19,20,77]	`N+1` , для strict HA — `N*2`	клиенты переподключаются на живые L7-узлы
`Kubernetes`	pods, nodes, deployments	replicas, reschedule, self-healing [54,63]	`replicas >= 2` , `N+1 nodes`	pod пересоздаётся, workload переезжает на живую ноду
`Auth / Users API`	логин, профиль	stateless pods + локальная реплика `auth_pg` [27,54]	`N+1` в каждом ДЦ	запрос повторяется на другом поде
`Meeting Control Plane`	create/join/leave, прокси в `home_dc`	stateless pods [54]	`N+1` в каждом ДЦ	новые запросы идут на живые поды в том же ДЦ
`Meeting Chat`	отправка сообщений, история чата	stateless pods; буфер последних сообщений в памяти — производный, его можно собрать заново из ScyllaDB [54]	`N+1` в каждом ДЦ	запрос уходит на живой под, история догружается из ScyllaDB
`WebSocket / Presence`	heartbeat, доставка чата, online-state	stateless WS-поды + Redis Cluster для runtime-state [31,54]	`N+1` в каждом ДЦ	клиент переподключается, состояние читается из Redis
`SFU media pool`	audio/video/screen sharing	sticky meeting + health checks, SFU без смешивания потоков [14,42]	`N+1` в каждом ДЦ	новые встречи не назначаются на плохой SFU; при отказе SFU клиент получает новый SFU и заново устанавливает с ним медиасоединение (ICE + DTLS) [84]
`Recording workers`	сборка и финализация записи	worker pool + retry + multipart upload [46]	`N+1` в каждом ДЦ	незавершённую запись дописывает другой worker
`PostgreSQL auth_pg`	`users`	primary в `DC-US-EAST` + синхронная standby в другой AZ + 7 асинхронных реплик в остальных ДЦ, WAL/PITR, автоматический failover внутри ДЦ через Patroni [27,28,86,89]	`1 primary + 1 sync standby + 7 async`	standby становится primary без потери данных; при отказе всего `DC-US-EAST` дежурный повышает реплику `DC-US-WEST` [97]

`PostgreSQL + Citus` (`meeting_pg`, `recording_pg`)	встречи, ссылки, метаданные записей	в каждом ДЦ координатор с локальными шардами + синхронная standby; асинхронная реплика в парном ДЦ; WAL/PITR [27,29,90]	`2 в ДЦ + 1 в парном ДЦ`	standby в соседней AZ; при отказе ДЦ дежурный повышает реплику в парном ДЦ [97]
`PgBouncer`	пул соединений PostgreSQL	несколько PgBouncer-инстансов перед БД [32]	3 на ДЦ	приложение переключается на живой pooler
`Redis session_redis`	`user_sessions`	Redis Cluster в каждом ДЦ: 3 master + 3 replica [31]. Оба кластера ДЦ работают на 3 общих серверах: на каждом сервере master одного шарда и replica другого (п. 11.3)	`3 master + 3 replica` на ДЦ	replica становится master; потерянные сессии — повторный логин
`Redis runtime_redis`	`meetings_runtime`, `chat_seq`, `chat_dedup`	Redis Cluster в каждом ДЦ + TTL для hot-state [22,31]	`3 master + 3 replica` на ДЦ	heartbeat-state восстанавливают клиенты после переподключения
`ScyllaDB meeting_scylla`	`meeting_participants`, `user_meetings`	один кластер на все ДЦ: у participants отдельный keyspace в каждом ДЦ с RF=3 внутри него, `user_meetings` — RF=3 в каждом ДЦ, `LOCAL_QUORUM` [33,80,81]	`RF=3`	отказ узла не останавливает чтение и запись, пока жив кворум ДЦ
`ScyllaDB chat_scylla`	`chat_messages`	кластер в каждом ДЦ, append-only log, снапшоты в S3 парного ДЦ [33]	`RF=3`	сообщения читаются с реплик, запись работает при отказе узла
`S3-compatible Object Storage`	записи встреч и части загрузки	multipart upload, strong consistency, ≥ 3 копии в 3 AZ `home_dc` [23,24,46]	`>=3 copies`	файл доступен, незавершённые части можно дозагрузить
`Observability stack`	метрики, логи, traces	Prometheus/OpenTelemetry/Loki локально в каждом ДЦ, Grafana/Jaeger центрально [56,57,58,59,60]	2 узла на ДЦ + 2 центральных	сервис работает, но алерты и дашборды деградируют

9.2. Отказ целого дата-центра

Отказ одной AZ закрывается внутри ДЦ (п. 9.1). Отказ всего ДЦ — редкий сценарий, но для него есть явный план. Худший случай по ёмкости — отказ самого большого ДЦ, `DC-DE-FRANKFURT` (35.69% нагрузки). Особенность `DC-US-EAST`, где стоит primary `auth_pg`, разобрана после таблицы.

Шаг	Что происходит	Время
1	Monitors GTM фиксируют отказ, `DC-DE-FRANKFURT` убирается из DNS-ответов (п. 3.5)	≈ 30 с + TTL 30 с
2	Новые подключения GTM раздаёт по `capacity_weight`: сначала `DC-UK-LONDON`, затем `DC-US-EAST`, остальное — по свободной ёмкости других ДЦ (п. 3.7)	сразу после шага 1
3	Активные встречи `DC-DE-FRANKFURT` обрываются, клиенты переподключаются через `api` со случайной задержкой, чтобы не устроить шторм. Подписанный `join_token` (`meeting_id`, роль, срок) проверяется в любом ДЦ, поэтому комната пересоздаётся в резервном ДЦ без похода в недоступную БД	десятки секунд
4	Дежурный повышает асинхронную реплику Citus `DC-DE-FRANKFURT` в `DC-UK-LONDON` до primary [97]. Код ДЦ `de-frankfurt` из `meeting_id` начинает указывать на `DC-UK-LONDON`: в маршрутизации control plane и в GTM для регионального имени `ws.de-frankfurt`. Запланированные встречи снова доступны	минуты, RPO — секунды

5	Чат и история участников встреч `DC-DE-FRANKFURT` недоступны до восстановления ДЦ или до восстановления из снапшотов в S3 парного ДЦ (п. 6.12)	осознанный компромисс

Особенность `DC-US-EAST`. Шаги те же, реплику Citus повышают в `DC-US-WEST`. Дополнительно в этом ДЦ стоит primary `auth_pg`. Логин всё время работает на локальных репликах в каждом ДЦ, а регистрация и правка профиля ждут, пока дежурный повысит реплику в `DC-US-WEST` (п. 3.4) — это минуты.

Хватит ли ёмкости. Холодный резерв ×2 по всем ДЦ не закладываем: это +100% к стоимости SFU. Вместо него на время инцидента SFU во всех ДЦ, которые принимают нагрузку, ограничивают профиль видео до group «high-quality video»: 1.0/0.6 Мбит/с вместо 2.6/1.8 для 720p [5]. Поток в запись при этом не уменьшается.

```
трафик участника, 720p              = 2.6 + 1.8 + 0.08 + 0.15 = 4.63 Мбит/с
трафик участника, high-quality      = 1.0 + 0.6 + 0.08 + 0.15 = 1.83 Мбит/с  → 39.5% от 720p
с потоком в запись (1.88 / 10 = 0.188 Мбит/с на участника):
  720p = 4.818, high-quality = 2.018 Мбит/с                                  → 41.9% от 720p
ёмкость SFU без DC-DE-FRANKFURT     = 100% − 35.69% = 64.31% штатной
нагрузка при профиле high-quality   = 41.9% штатного пика  <  64.31%     → помещается
```

Это верно даже при худшем допущении, что пики всех ДЦ совпали. На деле они разнесены по часовым поясам. Отказ `DC-US-EAST` проверяется так же: 41.9% < 100% − 28.84% = 71.16%.

Stateless-слои (NGINX L7, WS-поды) работают на облачных VM и подах Kubernetes, поэтому досоздаются за минуты. Но и без этого запас есть:

- **Соединения.** У 21 узла L7 вне `DC-DE-FRANKFURT` — 21.8 млн соединений при пике 16.9 млн. У 22 узлов вне `DC-US-EAST` — 22.8 млн.
- **Шторм переподключений.** 6.0 млн клиентов Франкфурта открывают новые TLS-соединения: 6 022 688 / (21 узел × 6 676 в секунду) ≈ 43 с. Поэтому клиент переподключается со случайной задержкой в пределах минуты.

10. Схема проекта

10.1. Общая схема взаимодействия

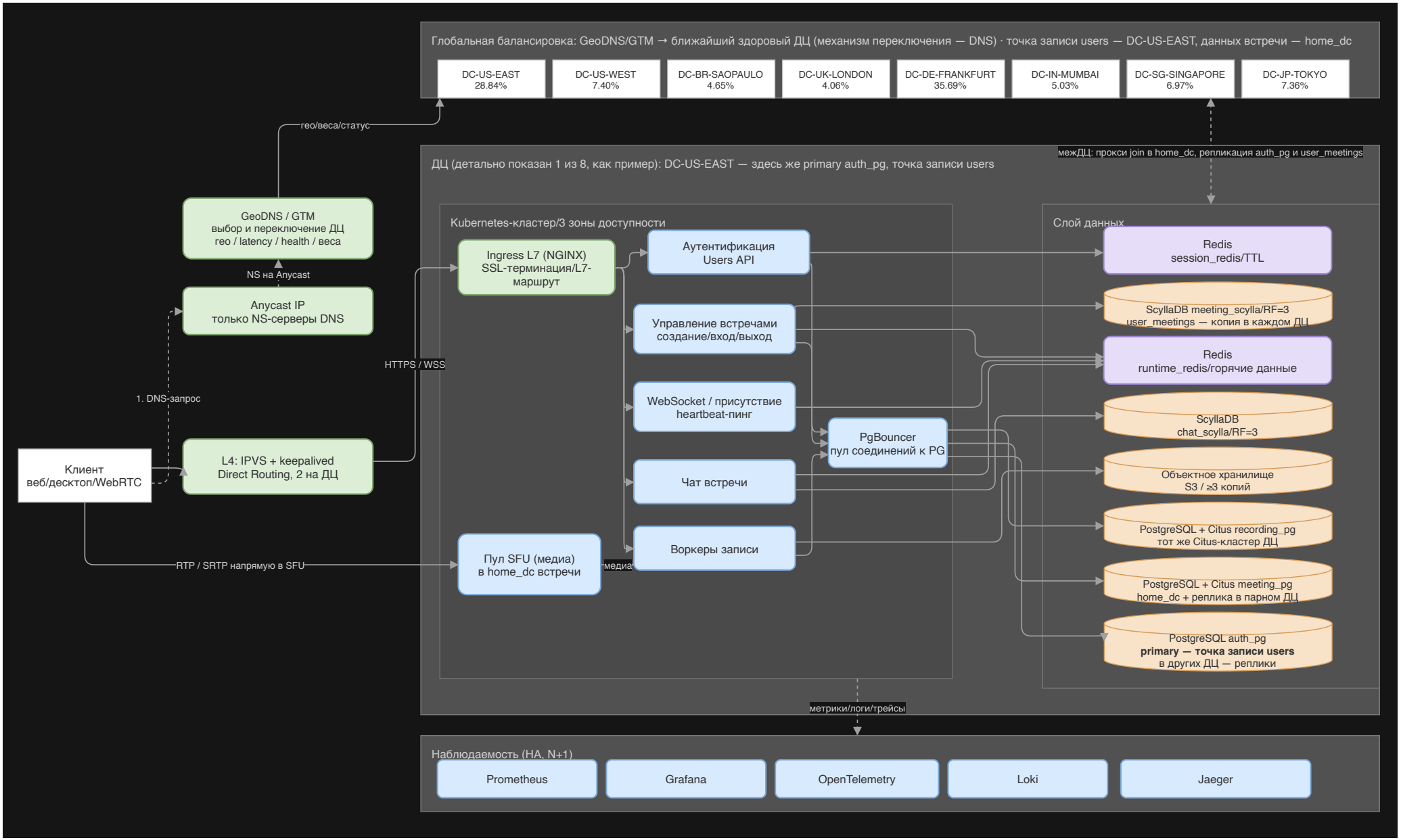


Схема открывается с масштабированием и слоями в diagrams.net: открыть для просмотра · открыть в редакторе · исходник `.drawio`. Все схемы работы со ссылками собраны в п. 10.5.

10.2. Как читать схему

Цвет	Что это	Компоненты
Зелёный	Сеть и балансировка	`GeoDNS / GTM`, `Anycast DNS`, `L4: IPVS + keepalived`, `Ingress L7`
Синий	Сервисы приложения (stateless, C++23)	`Auth/Users API`, `Meeting Control Plane`, `WebSocket/Presence`, `Meeting Chat`, `Recording workers`, `SFU`, `PgBouncer`
Оранжевый	Долговременные хранилища	`PostgreSQL`, `PostgreSQL + Citus`, `ScyllaDB`, `Object Storage`
Фиолетовый	Кеш и hot-state	`Redis session_redis`, `Redis runtime_redis`
Белый	Внешний участник и список ДЦ	`Клиент`, плашки 8 ДЦ
Серая стрелка	Поток данных	направление запроса или данных

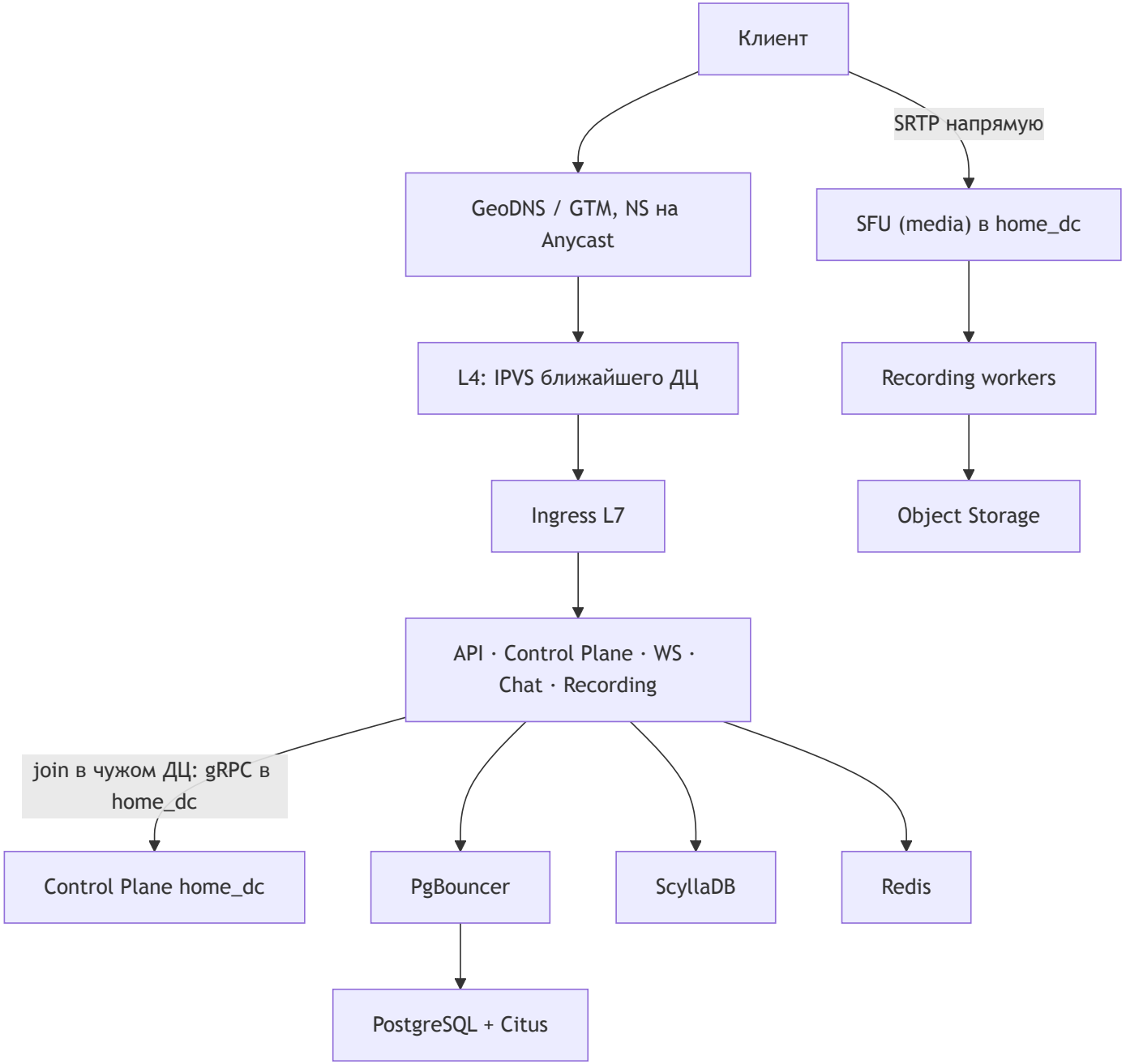
Детально показан один ДЦ — `DC-US-EAST`: в нём, кроме обычного набора, стоит primary `auth_pg` (точка записи `users`). Остальные 7 ДЦ устроены так же, но вместо primary `auth_pg` в них асинхронная реплика.

10.3. Потоки данных

Все потоки начинаются у клиента и расходятся по доменам. Media-поток (RTP/SRTP) намеренно идёт мимо L4 и `Ingress L7` — прямо на публичный адрес `SFU` из ответа `join`. Так балансировщикам не нужно пропускать 78 Тбит/с, мы не платим за TLS termination на видеотрафике, а встреча остаётся на одном SFU.

№	Поток	Путь	Хранилище
1	Авторизация	`Client -> L4 -> Ingress L7 -> Auth API` в ближайшем ДЦ	реплика `auth_pg` + `Redis session_redis`
2	Создание / вход во встречу	`Client -> L4 -> Ingress L7 -> Control Plane -> PgBouncer`; `join` выполняется в `home_dc`	`PG+Citus meeting_pg`, `ScyllaDB meeting_scylla`, `Redis runtime_redis`
3	Presence / heartbeat	`Client -> L4 -> Ingress L7 -> WebSocket/Presence` в `home_dc`	`Redis runtime_redis`
4	Чат встречи	`Client -> L4 -> Ingress L7 -> Meeting Chat` в `home_dc`, доставка через WS	`Redis` (seq, идемпотентность) + `ScyllaDB chat_scylla`
5	Media: видео / аудио / screen	`Client -> SFU` в `home_dc`, напрямую по адресу из ответа `join`	поток
6	Запись встречи	`SFU -> Recording workers -> Object Storage`; скачивание — `Client -> Object Storage` по presigned URL	`Object Storage` (файлы) + `PG+Citus recording_pg` (метаданные)
7	Межрегиональный прокси	`Control Plane ближайшего ДЦ -> gRPC/mTLS -> Control Plane home_dc`	—
8	Межрегиональная репликация	`auth_pg primary -> реплики во всех ДЦ`; `user_meetings -> все ДЦ`; `Citus -> парный ДЦ`	`auth_pg`, `meeting_scylla`, `meeting_pg`, `recording_pg`

Упрощённый поток:


Клиент
SRTP напрямую
GeoDNS / GTM, NS на Anycast
SFU (media) в home_dc
L4: IPVS ближайшего ДЦ
Recording workers
Ingress L7
Object Storage
API · Control Plane · WS · Chat · Recording
join в чужом ДЦ: gRPC в home_dc
Control Plane home_dc
PgBouncer
ScyllaDB
Redis
PostgreSQL + Citus

Запись использует multipart upload в object storage, метаданные пишутся отдельно в Citus [46,29]. Доступ к PostgreSQL во всех потоках идёт через PgBouncer, чтобы не держать тысячи прямых соединений [,32].

10.4. Балансировка: внешняя и внутренняя

Уровень	Назначение	Граница балансировки
Глобальный	Выбор ближайшего здорового ДЦ	внешняя: GeoDNS/GTM; Anycast только у DNS-серверов
ДЦ / Kubernetes	Приём трафика и stateless-сервисы	внутренняя: L4 (IPVS) → Ingress L7; медиа — напрямую в SFU, его выбирает control plane; gRPC round-robin между сервисами
Межрегиональный	Запись в точку записи другого ДЦ	gRPC с mTLS: прокси `join`, перенос сессии, репликация
Слой данных	Хранение и hot-state	шардинг по `home_dc` и ключам + репликация, не L7
Наблюдаемость	Метрики, логи, traces	side-stack, HA

10.5. Исходники и ссылки на схемы

Все схемы доступны без авторизации. Ссылки diagrams.net открывают файл из ветки `main` публичного репозитория, поэтому показывают новую схему после слияния этих изменений в `main`.

Схема	Где в работе	Открыть	Исходник
Общая схема проекта	10.1	diagrams.net viewer · SVG	`resource/schemes/full-scheme.drawio`
Логическая схема БД	5.1	dbdiagram · SVG	`resource/logic-database-sql.md` (DBML)
Физическая схема БД	6.1	dbdiagram · SVG	`resource/physical-database-sql.md` (DBML)
Домены и `home_dc`, расположение ДЦ, точки записи, DNS, Anycast, регулировка, сценарий «США → Япония»	3.1–3.8	рендерятся прямо в README на GitHub	Mermaid в тексте README; правится в mermaid.live
Алгоритмы ABR и выбор SFU-слоя	7.2, 7.3	рендерятся в README	Mermaid в тексте README

11. Список серверов

Серверы считаются **по каждому ДЦ**: данные встречи живут в её `home_dc` (п. 3.4), поэтому в каждом ДЦ свой набор кластеров. Общее правило — `ceil(нагрузка_ДЦ / ёмкость_узла) + 1` (`N+1` в каждом ДЦ) с минимальным размером кластера, которого требует технология.

11.1. Начальные нормативы для расчёта ресурсов

Компонент	Норматив / бенчмарк	Как используется	Источник
8 ДЦ	ДЦ из п. 3.2. Selectel работает в 4 странах (6 регионов, 11 зон доступности, 26 пулов), наших 8 городов среди них нет	Selectel — ценовой ориентир: его публичные цены берём для всех ДЦ, в доллары пересчитываем по курсу ЦБ РФ на 10.10.2026 — 84.90 ₽ за $1	64,65,99
`Kubernetes-кластер/3 зоны доступности`	Managed Kubernetes HA: 3 master-ноды	один HA-кластер на ДЦ	69
`L4: IPVS + keepalived`	EL11-SSD-10GE: 10GE, 32 GB; 136 B на соединение в таблице IPVS	2 директора на ДЦ, active/standby (п. 4.6)	66,91,93
`Ingress L7 (NGINX)`	341 232 HTTPS RPS; 6 676 новых TLS/с; 8.8 Gbps; 54 KB на WSS-соединение	расчёт узлов L7, п. 4.5	19,20,77,78
Ядро SFU	Jitsi Videobridge: 550.4 Мбит/с при 20.3% CPU 4-ядерного Xeon E5-1620 v2; по замеру CPU растёт почти линейно с битрейтом. Экстраполяция до 100% CPU — наше допущение: `550.4 / 0.203 / 4 ≈ 0.678 Гбит/с на ядро`	целевая загрузка CPU 70% → 0.4745 Гбит/с на ядро	74
`Пул SFU (медиа)`	EL11-SSD-10GE, 6 ядер: `min(10 Gbps NIC; 6 × 0.4745) = 2.847 Gbps` на узел	ограничитель — CPU, а не сетевая карта. SFU масштабируется по ядрам: один mediasoup worker = одно ядро [76]. Сверка с LiveKit — под таблицей	66,74,75,76
`Воркеры записи`	те же 2.847 Gbps на узел: расшифровка SRTP и упаковка потока без перекодирования — допущение по аналогии с SFU	расчёт по потоку в запись (п. 2.4)	66,74
WebSocket-под	4 CPU / 8 GB, 6 GB под соединения, ≤ 20 KB на соединение — по аналогии с замером NGINX → **300 000 соединений на под**	heartbeat на под ≈ 12 000/с	77
Поды Auth / Control Plane / Chat	1 000 / 1 500 / 5 000 запросов/с на под 4 CPU / 8 GB	проектный норматив для C++-сервиса с точечными запросами без JOIN	—
Redis	один шард ≈ 120 000 ops/s: redis-benchmark без пайплайнинга даёт 72–180 тыс. запросов/с	`max(3, ceil(ops_ДЦ / 120 000))` master	31,79
ScyllaDB	CL34-NVMe: 2×1 TB NVMe. Заполняем не больше 50%: при compaction со стратегией STCS в худшем случае нужна свободной половина диска [95] → ≤ 1 TB данных на узел	`max(3, ceil(объём_с_RF / 1 TB))`. Отдельный `+1` не добавляем: при RF=3 данные каждого узла есть ещё на двух (п. 9.1)	33,66,95
`Объектное`	685.26 PB / 30 дней; ледяное хранение — 0.89 ₽/GB в месяц, исходящий трафик — 1.96 ₽/GB	стоимость записей встреч	70

`хранилище S3 / ≥3 копий`			
`Cloud Server custom flavor`	1 vCPU = 735.19 ₽/мес; 1 GB RAM = 267.34 ₽/мес; 1 GB SSD = 10.48 ₽/мес	узлы L7	67,68
`EL11-SSD-10GE`	6 ядер, 32 GB RAM, 2×480 GB SSD, 10GE, 14 000 ₽/мес	SFU, recording workers, L4	66
`CL34-NVMe`	14 ядер, 64 GB RAM, 2×1 TB NVMe, 13 600 ₽/мес	app pool, Redis, ScyllaDB, наблюдаемость	66
`BL18-NVMe`	16 ядер, 128 GB RAM, 2×1920 GB + 1000 GB NVMe, 34 900 ₽/мес	PostgreSQL `auth_pg` и Citus	66
Трафик выделенного сервера	при порте больше 1 Гбит/с — 1 Гбит/с без учёта объёма или 300 ТВ в месяц без ограничения полосы. По умолчанию порт медный, до 1 Гбит/с; оптика — по заявке. При среднем потоке больше 100 Мбит/с — индивидуальные условия. Увеличение лимита — 336 ₽/мес за шаг 10 Мбит/с или 3 ТВ	оценка внешнего трафика SFU, п. 11.4	71,66
Актуальность цен	Прайс Selectel проверен 11.10.2026, цены с НДС 22%. По сравнению с ценами на 24.06.2026 облачные позиции и S3 подорожали на 10%, выделенные серверы — нет	оценка в п. 11.4 — на 11.10.2026	66,67

Конфигурации `CL62R-NVMe` (Redis сессий) и `PL70-NVMe-10GE` (Scylla чата) из прошлой версии больше не используются. Эти нагрузки упираются в объём данных и ops/s, а не в 128–256 GB RAM и 32 ядра. Например, чат — это 26 042 записи/с на весь мир, а под него стояло 150 серверов по 32 ядра.

Сверка ёмкости SFU с LiveKit. LiveKit тестировал свой SFU на 16-ядерном инстансе Google Cloud `c2-standard-16` [75]. В сценарии livestream пакеты размером с видео (≈ 950 байт): 4.25 Гбит/с при 92% CPU, то есть ≈ 0.29 Гбит/с на ядро инстанса. В сценарии «встреча 150 × 150» пакеты мелкие (≈ 120 байт), и выходит лишь ≈ 0.08 Гбит/с на ядро: SFU нагружают пакеты в секунду, а не биты. Во встрече на 10 человек основной трафик — пакеты видео 720p, поэтому за основу взят замер Jitsi. Это оптимистичная оценка: если реальные пакеты окажутся мельче, SFU-узлов понадобится больше.

11.2. Расчёт ресурсов по сервисам

Сервис	Пиковая нагрузка, весь мир	Правило на ДЦ	Итоговые ресурсы	Источник
`GeoDNS / GTM`	8 ДЦ	managed-сервис	0 серверов	16,17
`Anycast DNS`	NS-серверы GTM	managed у DNS-провайдера	0 серверов	83
`L4: IPVS + keepalived`	16 875 000 соединений (2.29 GB таблиц IPVS), входящий трафик ≤ 8.43 Gbps	`max(1, ceil(Gbps_ДЦ / 10)) + 1` , п. 4.6	16 узлов EL11-SSD-10GE	91,92,93
`Ingress L7 (NGINX)`	1 054 340 RPS, 56 424 новых TLS/с, 8.43 Gbps, 16 875 000 WSS	п. 4.5, `N+1`	28 узлов × `24 vCPU / 64 GB`	19,20,77,78
`Пул SFU (медиа)`	78 131.25 Gbps media + 3 172.5 Gbps поток в запись = **81 303.75 Gbps**	`ceil(Gbps_ДЦ / 2.847) + 1`	28 570 серверов × `6 CPU / 32 GB / 10GE`	66,74
`Воркеры записи`	3 172.5 Gbps	`ceil(Gbps_ДЦ / 2.847) + 1`	1 127 серверов × `6`	66,74

			CPU / 32 GB / 10GE	
Аутентификация Users API	7 726 RPS: login + чтение users при create/join	ceil(RPS_ДЦ / 1 000) + 1	20 подов × 4 CPU / 8 GB	расчёт по п. 5.2
Управление встречами	17 969 RPS: create + join + leave + list + метаданные записи + открытие записи	ceil(RPS_ДЦ / 1 500) + 1	23 пода × 4 CPU / 8 GB	расчёт по п. 2.4
WebSocket / присутствие	16 875 000 соединений, 675 000 heartbeat/c	ceil(соединений_ДЦ / 300 000) + 1	69 подов × 4 CPU / 8 GB	расчёт по п. 2.2
Чат встречи	31 250 запросов/с: сообщения 26 042 + история 5 208	ceil(RPS_ДЦ / 5 000) + 1	19 подов × 4 CPU / 8 GB	расчёт по п. 2.4
PgBouncer	соединения к auth_pg, meeting_pg, recording_pg	3 пода на ДЦ	24 пода × 2 CPU / 2 GB	32
Kubernetes app pool	572 CPU подов	ceil(CPU_ДЦ / 12) + 1 (2 из 14 ядер — ОС и kubelet)	59 узлов CL34-NVMe	66
Redis (session + runtime)	runtime: 738 021 ops/s; sessions: 14 236 ops/s. Память сессий — 14.72 GB raw, в самом большом ДЦ ≈ 5.3 GB, с репликой ≈ 10.5 GB	max(3, ceil(ops_ДЦ / 120 000)) узлов; оба кластера на одних серверах: на каждом master одного шарда и replica другого	24 узла CL34-NVMe	31,79
PostgreSQL auth_pg	81.92 GB, 7 726 чтений/с, ≤ 1 997 записей/с	primary + sync standby в DC-US-EAST, по одной реплике в остальных ДЦ	9 узлов BL18-NVMe	27,86
PostgreSQL + Citus (meeting_pg + recording_pg)	576 GB метаданных, 16 667 чтений/с; в самом большом ДЦ — 206 GB и ≈ 5 950 чтений/с	в ДЦ: координатор с локальными шардами + standby, плюс реплика парного ДЦ. При росте добавляются worker-узлы	24 узла BL18-NVMe	29,90
ScyllaDB meeting_scylla/RF=3	31.10 TB с репликацией: копия user_meetings в каждом ДЦ + participants своего ДЦ	max(3, ceil(TB_ДЦ / 1))	34 узла CL34-NVMe	33,80
ScyllaDB chat_scylla/RF=3	135 TB с репликацией	max(3, ceil(TB_ДЦ / 1))	138 узлов CL34-NVMe	33
Объектное хранилище S3 / ≥3 копий	685.26 PB / 30 дней	хранение в home_dc	Managed S3	70
Наблюдаемость	метрики, логи, traces	2 узла на ДЦ + 2 центральных в DC-US-EAST	18 узлов CL34-NVMe	56,57,58,59,60

11.3. Итоговая таблица серверов

Серверы по ДЦ

ДЦ	SFU	Запись	L4	L7	App	Redis	auth_pg	Citus	Scylla meeting	Scylla chat	Набл.	Всего

DC-US-EAST	8 238	323	2	6	12	3	2	4	5	39	4	**8 638**
DC-US-WEST	2 115	84	2	3	6	3	1	3	4	10	2	**2 233**
DC-BR-SAOPAULO	1 329	53	2	2	5	3	1	2	4	7	2	**1 410**
DC-UK-LONDON	1 161	47	2	2	5	3	1	3	4	6	2	**1 236**
DC-DE-FRANKFURT	10 194	399	2	7	14	3	1	3	5	49	2	**10 679**
DC-IN-MUMBAI	1 438	58	2	2	5	3	1	2	4	7	2	**1 524**
DC-SG-SINGAPORE	1 992	79	2	3	6	3	1	4	4	10	2	**2 106**
DC-JP-TOKYO	2 103	84	2	3	6	3	1	3	4	10	2	**2 221**
Итого	**28 570**	**1 127**	**16**	**28**	**59**	**24**	**9**	**24**	**34**	**138**	**18**	**30 047**

`Citus` = координатор + standby своего ДЦ + реплики парных ДЦ: в `DC-US-EAST` хранятся реплики `DC-US-WEST` и `DC-BR-SAOPAULO`, в `DC-SG-SINGAPORE` — `DC-IN-MUMBAI` и `DC-JP-TOKYO`.

Поды Kubernetes по ДЦ

ДЦ	Auth	Control Plane	WebSocket	Chat	PgBouncer	CPU подов	Узлы app pool
DC-US-EAST	4	5	18	3	3	126	12
DC-US-WEST	2	2	6	2	3	54	6
DC-BR-SAOPAULO	2	2	4	2	3	46	5
DC-UK-LONDON	2	2	4	2	3	46	5
DC-DE-FRANKFURT	4	6	22	4	3	150	14
DC-IN-MUMBAI	2	2	4	2	3	46	5
DC-SG-SINGAPORE	2	2	5	2	3	50	6
DC-JP-TOKYO	2	2	6	2	3	54	6
Итого	**20**	**23**	**69**	**19**	**24**	**572**	**59**

Объём ScyllaDB по ДЦ

ДЦ	meeting_scylla, с RF	Узлов	chat_scylla, с RF	Узлов
DC-US-EAST	4.45 TB	5	38.93 TB	39
DC-US-WEST	3.71 TB	4	9.99 TB	10
DC-BR-SAOPAULO	3.62 TB	4	6.28 TB	7
DC-UK-LONDON	3.60 TB	4	5.48 TB	6

`DC-DE-FRANKFURT`	4.69 TB	5	48.18 TB	49
`DC-IN-MUMBAI`	3.63 TB	4	6.79 TB	7
`DC-SG-SINGAPORE`	3.70 TB	4	9.41 TB	10
`DC-JP-TOKYO`	3.71 TB	4	9.94 TB	10
Итого	**31.10 TB**	**34**	**135.00 TB**	**138**

Сводка по конфигурациям

Пул / сервис	Размещение	Конфигурация одного сервера	Кол-во	Итого CPU	Итого RAM	Сеть	Что размещается	Источник
`Kubernetes-кластер/3 зоны доступности`	по одному на ДЦ	Managed Kubernetes HA	8	—	—	сеть провайдера	master-ноды Kubernetes	69
`L4: IPVS + keepalived`	по 2 на ДЦ	`EL11-SSD-10GE`: 6 CPU, 32 GB RAM, 10GE	16	96	512 GB	≤ 3.01 Gbps входящего на ДЦ при 10GE	директоры IPVS, keepalived	66,91,93
`Ingress L7 (NGINX)`	Kubernetes worker pool	`24 vCPU / 64 GB / 100 GB SSD`	28	672	1 792 GB	`8.43 Gbps / 28 ≈ 0.30 Gbps` на узел	`Ingress L7 (NGINX)`	19,20,67
`Kubernetes app pool`	Kubernetes worker pool	`CL34-NVMe`: 14 CPU, 64 GB RAM	59	826	3 776 GB	1 Gbps на узел достаточно для control plane	Auth, Control Plane, WebSocket, Chat, PgBouncer	66
`Пул SFU (медиа)`	media pool	`EL11-SSD-10GE`: 6 CPU, 32 GB RAM, 10GE	28 570	171 420	914 240 GB	2.847 Gbps на узел при NIC 10 Gbps; оптический порт по заявке [71]	`Пул SFU (медиа)`	66,74
`Воркеры записи`	recording pool	`EL11-SSD-10GE`: 6 CPU, 32 GB RAM, 10GE	1 127	6 762	36 064 GB	2.847 Gbps на узел	`Воркеры записи`	66,74
`Redis` (session + runtime)	data pool	`CL34-NVMe`: 14 CPU, 64 GB RAM	24	336	1 536 GB	1 Gbps на узел	`session_redis`, `runtime_redis`	31,66
`PostgreSQL auth_pg`	data pool	`BL18-NVMe`: 16 CPU, 128 GB RAM	9	144	1 152 GB	1 Gbps на узел	primary, standby, реплики	27,66
`PostgreSQL + Citus`	data pool	`BL18-NVMe`: 16 CPU, 128 GB RAM	24	384	3 072 GB	1 Gbps на узел	`meeting_pg`, `recording_pg`	27,29,66
`ScyllaDB meeting_scylla/RF=3`	data pool	`CL34-NVMe`: 14 CPU, 64 GB, 2×1 TB NVMe	34	476	2 176 GB	1 Gbps на узел	`meeting_participants`, `user_meetings`	33,66
`ScyllaDB chat_scylla/RF=3`	data pool	`CL34-NVMe`: 14 CPU, 64 GB, 2×1 TB NVMe	138	1 932	8 832 GB	1 Gbps на узел	`chat_messages`	33,66
`Объектное хранилище S3 / ≥3 копий`	Selectel S3	Managed S3	—	—	—	сеть object storage	записи встреч	70

`Наблюдаемость (HA)`	observability pool	`CL34-NVMe` : 14 CPU, 64 GB RAM	18	252	1 152 GB	1 Gbps на узел	Prometheus, OpenTelemetry, Loki в ДЦ; Grafana, Jaeger центрально	56,57,58,59,60,66

11.4. Оценка стоимости инфраструктуры

Цены Selectel на 11.10.2026 с НДС 22% (п. 11.1). В доллары пересчитано по курсу ЦБ РФ на 10.10.2026: 84.90 ₽ за $1 [99].

Пул / сервис	Кол-во	Требуемая конфигурация из расчёта	Ближайшая публичная конфигурация Selectel	Цена за 1 узел / мес.	Итого / мес.	Комментарий
`Kubernetes-кластер/3 зоны доступности`	8	HA master-ноды	Managed Kubernetes HA, 3 master-ноды	17 085.59 ₽	136 684.72 ₽	по одному HA-кластеру на ДЦ [69]
`L4: IPVS + keepalived`	16	2 директора на ДЦ, 10GE	`EL11-SSD-10GE`	14 000 ₽	224 000 ₽	active/standby в каждом ДЦ [66]
`Ingress L7 (NGINX)`	28	`24 vCPU / 64 GB / 100 GB SSD`	Cloud Server custom flavor	35 802.32 ₽	1 002 464.96 ₽	`24*735.19 + 64*267.34 + 100*10.48` [67]
`Kubernetes app pool`	59	572 CPU подов	`CL34-NVMe`	13 600 ₽	802 400 ₽	API, WS, chat, PgBouncer [66]
`Пул SFU (медиа)`	28 570	`81 303.75 Gbps` , 2.847 Gbps на узел	`EL11-SSD-10GE`	14 000 ₽ ≈ $164.90	399 980 000 ₽	без внешнего трафика, он оценён ниже [66]
`Воркеры записи`	1 127	`3 172.5 Gbps`	`EL11-SSD-10GE`	14 000 ₽ ≈ $164.90	15 778 000 ₽	поток в запись идёт внутри ДЦ [66]
`Redis` (session + runtime)	24	3 узла на ДЦ, перекрёстные реплики	`CL34-NVMe`	13 600 ₽	326 400 ₽	узкое место — ops/s [31,66]
`PostgreSQL auth_pg`	9	primary + standby + 7 реплик	`BL18-NVMe`	34 900 ₽	314 100 ₽	профили и auth [66]
`PostgreSQL + Citus`	24	2 узла в ДЦ + реплика парного ДЦ	`BL18-NVMe`	34 900 ₽	837 600 ₽	метаданные встреч и записей [66]
`ScyllaDB meeting_scylla/RF=3`	34	31.10 TB с RF	`CL34-NVMe`	13 600 ₽	462 400 ₽	participants + user meetings [33,66]
`ScyllaDB chat_scylla/RF=3`	138	135 TB с RF	`CL34-NVMe`	13 600 ₽	1 876 800 ₽	chat append-only log [66]
`Объектное хранилище S3 / ≥3 копий`	685 260 000 GB	`685.26 PB / 30 дней`	S3 ледяное хранение	0.89 ₽/GB	609 881 400 ₽	только хранение, без запросов и исходящего трафика [70]
`Наблюдаемость (HA)`	18	Prometheus, OpenTelemetry, Loki, Grafana, Jaeger	`CL34-NVMe`	13 600 ₽	244 800 ₽	2 узла на ДЦ + 2 центральных [66]
Итого: серверы и хранение	—	—	—	—	**1 031 867 049.68 ₽/мес**	**≈ $12 153 911.07/мес**

Структура расходов	Сумма / мес.	Доля

Хранение записей, S3	609.88 млн ₽	59.1%
Медиа, SFU	399.98 млн ₽	38.8%
Воркеры записи	15.78 млн ₽	1.5%
Всё остальное: балансировка, сервисы, БД, наблюдаемость	6.23 млн ₽	0.6%

Внешний трафик

Цены выделенных серверов не покрывают наш трафик. При порте больше 1 Гбит/с Selectel даёт 300 ТВ в месяц без ограничения полосы, увеличение лимита стоит 336 ₽/мес за шаг 3 ТВ. При среднем потоке больше 100 Мбит/с провайдер предлагает индивидуальные условия [71,66]. Через порты SFU проходит весь медиатрафик:

```
медиа за месяц     = 562 545 000 GB/сутки * 30 = 16 876 350 ТВ ≈ 16.9 EB  (≈ 591 ТВ на узел SFU)
входит в тарифы    = 300 ТВ * 28 570 узлов     = 8 571 000 ТВ
сверх лимита       = 8 305 350 ТВ → 2 768 450 шагов по 3 ТВ * 336 ₽ ≈ 930.20 млн ₽/мес
```

Это верхняя оценка по публичному тарифу: на таком объёме трафик покупают по индивидуальному договору (транзит и пиринг), и он выходит дешевле. Поэтому в итог выше трафик не включён. С ним по публичному тарифу итог — **≈ 1 962.07 млн ₽/мес (≈ $23.11 млн)**, и трафик становится крупнейшей статьёй — больше хранения записей. Исходящий трафик S3 при скачивании записей стоит 1.96 ₽/GB [70] и тоже не включён: частота просмотров неизвестна (п. 2.4).

Изменение относительно версии, которую смотрел ревьюер (800.08 млн ₽/мес):

- **Переход на `MF_K_PEAK = 1.5`** вместе с новыми долями ДЦ снизил итог примерно на 120 млн ₽. Пик определяет число серверов, но не объём записей, поэтому S3 от него не зависит.
- **Проверка SFU по CPU** добавила ≈ 297.16 млн ₽: без неё, при 10 Гбит/с на узел, итог был бы ≈ 734.70 млн ₽.
- **Рост цен Selectel с 01.07.2026** добавил ≈ 55 млн ₽, почти всё — S3.

Стоимость по-прежнему определяют те же блоки, что и нагрузку: хранение записей, медиа и внешний трафик медиа.

Список источников

1. Microsoft Investor Relations — Earnings Conference Call FY24 Q1 (транскрипт: «more than 320 million monthly active users» у Teams) ↩↩↩↩↩
2. Microsoft 365 Blog — Microsoft Teams reaches 115 million DAU ↩↩↩↩
3. Zoom Video Communications — Prospectus Supplement (Form 424B5), SEC, 13.01.2021: «In April 2020, our platform surpassed more than 300 million daily meeting participants»; run rate более 3.5 трлн минут встреч в год на 31.10.2020 ↩↩↩↩
4. Zoom Blog — Here's How You Zoomed Over the Past Year ↩↩↩↩↩
5. Zoom Support — Zoom system requirements: Zoom Web App ↩↩↩↩↩↩↩↩↩
6. Zoom Support — Zoom Chat specifications and limitations ↩↩↩↩
7. Zoom Support — Starting a cloud recording ↩↩
8. Zoom Support — Cloud recording storage capacity ↩↩↩

9. Socket.IO Documentation — Server options ↩
10. Zoom Investor Relations — Zoom Communications Reports Fourth Quarter and Fiscal Year 2026 Financial Results ↩↩
11. IMF — World Economic Outlook (April 2026): GDP, current prices (NGDPD), млрд долларов США, данные 2025 года ↩↩
12. Selectel Documentation — Cloud load balancers: типы балансировщиков и их производительность ↩
13. Zoom Support — Selecting data center for meetings, webinars, whiteboards, notes and docs ↩
14. mediasoup Documentation — SFU overview ↩↩↩
15. AWS Route 53 Documentation — Latency-based routing ↩↩↩
16. Cloudflare Docs — Global traffic steering policies ↩↩↩↩↩
17. Cloudflare Docs — Create a monitor (минимальный интервал: 60 с Pro, 15 с Business, 10 с Enterprise; consecutive_down через API) ↩↩↩↩↩↩
18. Google Cloud Documentation — Cloud Load Balancing overview ↩
19. NGINX Community Blog — Testing the Performance of NGINX Ingress Controller for Kubernetes ↩↩↩↩↩↩↩↩
20. NGINX Community Blog — Testing the Performance of NGINX and NGINX Plus Web Servers ↩↩↩↩↩↩
21. Kubernetes Documentation — Service type LoadBalancer ↩
22. Redis Docs — Key expiration ↩↩↩
23. Amazon S3 — Strong Consistency ↩↩
24. Google Cloud Storage — Consistency ↩
25. PostgreSQL Documentation — Multicolumn Indexes ↩↩
26. PostgreSQL Documentation — Transaction Isolation ↩↩
27. PostgreSQL Documentation — Replication ↩↩↩↩↩↩↩
28. PostgreSQL Documentation — Continuous Archiving and Point-in-Time Recovery ↩↩↩
29. Citus Documentation — Choosing Distribution Column ↩↩↩↩↩↩↩↩
30. Citus Documentation — Table Co-Location ↩↩
31. Redis Documentation — Redis Cluster Specification ↩↩↩↩↩↩↩↩↩↩
32. PgBouncer Documentation — Features ↩↩↩↩↩↩
33. ScyllaDB Documentation — Fault Tolerance ↩↩↩↩↩↩↩↩↩
34. ScyllaDB CPP-RS Driver — API-совместимая замена C/C++ Driver, который больше не развивается ↩↩
35. PostgreSQL Documentation — libpq C Library ↩
36. libpqxx — C++ connector for PostgreSQL ↩
37. Redis Documentation — hiredis guide ↩
38. redis-plus-plus — C++ Redis client ↩
39. AWS Documentation — AWS SDK for C++ ↩↩

40. W3C — Identifiers for WebRTC's Statistics API ↩↩
41. RFC 8888 — RTP Control Protocol Feedback for Congestion Control ↩↩
42. mediasoup Documentation — RTP Parameters and Capabilities ↩↩↩
43. mediasoup Documentation — ActiveSpeakerObserver API ↩
44. RFC 3550 — RTP: A Transport Protocol for Real-Time Applications ↩↩
45. Stripe Docs — Idempotent requests ↩
46. Amazon S3 Documentation — Multipart upload overview ↩↩↩↩↩↩↩↩
47. cppreference — Compiler support for C++23 ↩↩
48. TypeScript Documentation — The TypeScript Handbook ↩
49. CMake Documentation — cmake-buildsystem(7) ↩
50. Conan Documentation — Conan 2 C/C++ Package Manager ↩
51. Boost.Beast Documentation — WebSocket ↩
52. gRPC Documentation — C++ ↩↩↩
53. Docker Docs — Docker overview ↩
54. Kubernetes Documentation — Overview ↩↩↩↩↩↩↩
55. Helm Documentation — Docs Home ↩
56. Prometheus Documentation — Overview ↩↩↩↩
57. Grafana Documentation — Grafana OSS and Enterprise ↩↩↩↩
58. OpenTelemetry Documentation ↩↩↩↩
59. Grafana Loki OSS — Log aggregation system ↩↩↩↩
60. Jaeger Documentation — Introduction ↩↩↩↩
61. OpenSSL Documentation — openssl command summary ↩
62. ScyllaDB Documentation — CQL Reference ↩
63. Kubernetes Documentation — Self-Healing ↩
64. Selectel Documentation — Локации: страны, регионы, зоны доступности, пулы и адреса дата-центров ↩
65. Selectel Documentation — Product availability by location ↩
66. Selectel — Тарифы и цены на услуги: выделенные серверы ↩↩↩↩↩↩↩↩

71. Selectel Documentation — Dedicated server traffic consumption model ↩↩↩
72. Zoom Communications — Form 10-K за FY2026 (выручка по регионам: Americas 72.1%, APAC 12.1%, EMEA 15.8%) ↩
73. U.S. Census Bureau — Age and Sex Composition: 2020 (регион West — 78.6 млн, 23.7% населения) ↩
74. Jitsi — Jitsi Videobridge Performance Evaluation ↩↩↩↩↩↩↩
75. LiveKit Docs — Benchmarks ↩↩
76. mediasoup Documentation — Scalability ↩↩
77. NGINX (F5) Blog — NGINX WebSocket Performance ↩↩↩↩↩
78. OpenSSL Documentation — SSL_CTX_set_mode, SSL_MODE_RELEASE_BUFFERS ↩↩↩
79. Redis Docs — Redis benchmark ↩↩
80. ScyllaDB Documentation — Create a ScyllaDB Cluster: Multi Data Centers ↩↩↩↩
81. ScyllaDB Documentation — Production Readiness Guidelines ↩↩
82. RFC 7871 — Client Subnet in DNS Queries ↩↩
83. RFC 4786 — Operation of Anycast Services ↩↩↩↩↩↩
84. RFC 8445 — Interactive Connectivity Establishment (ICE) ↩↩
85. RFC 9562 — Universally Unique IDentifiers (UUIDs), UUIDv8 ↩
86. Patroni Documentation ↩↩↩↩
87. WonderNetwork — Ping time between Tokyo and Washington ↩↩↩
88. ITU-T Recommendation G.114 (05/2003) — One-way transmission time: при сквозной задержке в одну сторону меньше 150 мс большинство приложений не страдают ↩
89. PostgreSQL Documentation — Log-Shipping Standby Servers, synchronous replication ↩↩
90. Citus Documentation — Single-Node Citus ↩↩↩
91. Linux kernel — net/netfilter/ipvs/Kconfig: запись в таблице соединений IPVS занимает 128 B, элемент хеш-таблицы — 8 B ↩↩↩↩↩↩
92. Linux Virtual Server — Virtual Server via Direct Routing ↩↩↩↩↩
93. Keepalived Documentation — Introduction: IPVS, health checks, VRRP ↩↩↩↩↩↩↩
94. Citus Documentation — Creating and Modifying Distributed Objects (DDL): первичные ключи и UNIQUE должны включать колонку распределения ↩↩↩
95. ScyllaDB Documentation — Compaction Strategies: STCS в худшем случае требует свободной половины диска ↩↩
96. ScyllaDB Manager Documentation — Backup ↩
97. Patroni Documentation — HA multi datacenter: standby-кластер во втором ДЦ повышают вручную ↩↩↩↩↩
98. Redis Documentation — SET (опции NX, EX) ↩
99. Банк России — Официальные курсы валют на 10.10.2026: 84.9048 ₽ за $1 ↩↩
100. ScyllaDB CPP-RS Driver — Load balancing: token-aware, datacenter-aware, shard-awareness ↩
101. ScyllaDB Documentation — CQL INSERT: в отличие от SQL, INSERT не проверяет, существует ли строка ↩

102. mediasoup Documentation — API: WebRtcTransport (ICE Lite: транспорт не начинает ICE, а ждёт ICE Binding Requests от клиента; listenInfos с протоколом udp или tcp) ↩↩

103. RFC 6544 — TCP Candidates with Interactive Connectivity Establishment (ICE) ↩

Приложение Б. Ревью: что было неверно и что исправлено

Файл `docs/ревью.md`.

Ревью работы MeetFlow после защиты

Документ для автора работы: что было правильно, что нет, что исправлено в ветке `review-fixes` и как отвечать на вопросы. Сама работа — README.md. Разбор терминов и логики — разбор-работы.md.

1. Коротко

- Все 6 замечаний ревьюера закрыты; где в README что лежит — в разделе 2.
- Дополнительно найдено и исправлено 17 ошибок и несостыковок (раздел 3). Две из них меняют итоговые числа:
- **L7-балансировщики считались не той метрикой.** Весь RPS, включая heartbeat, делился на скорость новых TLS-рукопожатий. При этом совсем не учитывалась память под миллионы одновременных WebSocket-соединений.
- **Пропускная способность SFU не была проверена по CPU.** 10 Гбит/с на 6-ядерный сервер — в 2.5–6 раз выше реальных бенчмарков SFU.
- Итог изменился так: серверы и хранение стоят **≈ 1 031.87 млн ₽/мес (≈ $12.15 млн)** вместо 800.08. K = 1.5 снизил стоимость примерно на 120 млн, исправление ёмкости SFU добавило ≈ 297 млн, рост цен Selectel с 01.07.2026 — ≈ 55 млн. Отдельно оценён внешний трафик медиа: по публичному тарифу это ещё ≈ 930 млн ₽/мес, но это верхняя граница (раздел 8).
- Все числа теперь считает один скрипт `resource/calc/meetflow_calc.py`. Таблицы разделов 3, 4 и 11 вставлены из его вывода, поэтому суммы по ДЦ сходятся с глобальными значениями.
- Второй проход перепроверил источники, согласованность разделов и соответствие методичке курса. Главное: доли ДЦ больше не опираются на SimilarWeb, который методичка запрещает как первоисточник. Что ещё найдено и исправлено — в разделе 8.

2. Замечания ревьюера

2.1. «K = 1.5 сделать»

В чём была проблема. `MF_K_PEAK = 3`: пиковый час считался в 3 раза выше среднего. Через пик это раздувало всё: онлайн, RPS, трафик, число серверов.

Что сделано. `MF_K_PEAK = 1.5`, всё пересчитано скриптом. Обоснование в п. 2.1: сервис глобальный, часовые пояса сглаживают суточную кривую — когда в Америке день, в Азии ночь.

Как ответить устно. «K — отношение пикового часа к среднему за сутки. Для глобального сервиса кривая сглажена часовыми поясами, поэтому взяли 1.5. От K линейно зависят пиковые RPS, трафик и число одновременных соединений. Объёмы хранения от K не зависят — пик меняет скорость, а не суточный объём».

2.2. «other_pool не понравился (расписать)»

В чём была проблема. 42.5% мировой нагрузки — больше, чем у всех ДЦ, кроме США, — уходило в абстрактный `OTHER_REGIONAL_POOL`. У него не было ни места, ни состава стран, под него нельзя было поставить серверы. Он же был «глобальным fallback» для всех.

Что сделано (п. 3.2). Абстрактного пула больше нет: каждая страна привязана к одному из **8 ДЦ** с конкретными городами — Ашберн, Сан-Хосе, Сан-Паулу, Лондон, Франкфурт, Мумбаи, Сингапур, Токио. Доли считаются в три шага:

1. Доля страны — её доля в мировом ВВП 2025 года по IMF World Economic Outlook (апрель 2026). Видеовстречи — рабочий инструмент, поэтому их объём следует за экономикой. Материковый Китай не обслуживаем.
2. Каждую страну относим к ближайшему по задержке ДЦ. Полный список стран — в скрипте, значения ВВП — в `resource/calc/imf_weo_2026apr_gdp_2025.json`.
3. США делим на восток и запад по переписи 2020: на регион West приходится 23.7% населения.

Итог: Франкфурт 35.69%, US-EAST 28.84%, US-WEST 7.40%, Токио 7.36%, Сингапур 6.97%, Мумбаи 5.03%, Сан-Паулу 4.65%, Лондон 4.06%. Для каждого ДЦ указаны страны, доля нагрузки и резервный ДЦ.

> В первой версии исправлений доли считались от SimilarWeb (через перепечатку WallStreetZen). Методичка курса прямо запрещает SimilarWeb как первоисточник, поэтому во втором проходе метод заменён (раздел 8).

Как ответить устно. «Пользователей по странам первоисточники не публикуют, а SimilarWeb методичка запрещает. Поэтому распределили нагрузку по доле страны в мировом ВВП из базы IMF: видеосвязь — рабочий инструмент, и её объём следует за экономикой. Каждую страну отнесли к ближайшему ДЦ, США поделили по переписи. Проверка по аналогу: у Zoom Америка даёт 72% выручки — больше, чем 41% по ВВП, потому что Zoom сильнее всего в США. Мы проектируем сервис масштаба лидера рынка, поэтому берём нейтральную оценку. Точные доли уточняются по GeoIP после запуска, а GTM подстраивает веса».

2.3. «Зашарить схемы»

В чём была проблема. На общую схему проекта (раздел 10) не было ни ссылки, ни исходника — только картинка 1470×880 с мелким текстом. Ссылки dbdiagram на схемы БД уже были и открываются без авторизации (проверено).

Что сделано.

- Исходник общей схемы сохранён в `resource/schemes/full-scheme.drawio`. Добавлены ссылки «открыть для просмотра» и «открыть в редакторе» через diagrams.net — с масштабированием.
- Сама схема обновлена: 8 ДЦ вместо 5, Anycast только у DNS, primary `auth_pg` как точка записи, межрегиональные связи.
- В п. 10.5 собрана таблица всех схем работы со ссылками и исходниками.

Важно. Ссылки diagrams.net указывают на ветку `main` публичного репозитория и начнут показывать новую схему после слияния ветки `review-fixes` в `main`.

2.4. «Где записано в глобальной балансировке — находится точка записи»

В чём была проблема. Раздел 3 не говорил, куда идут **записи**. Для `Auth / Users` стояло «любой здоровый ДЦ». Но если писать в `users` можно в любом ДЦ, то уникальность `email` и конфликты одновременных записей не решены.

Что сделано (новый п. 3.4). Правило: у каждого объекта одна точка записи.

Данные	Точка записи
`users`	один primary на весь мир в `DC-US-EAST` + синхронная standby; в остальных ДЦ — реплики для чтения
Всё, что относится к встрече: `meetings`, участники, чат, runtime, записи	`home_dc` встречи. Его код зашит в `meeting_id`, поэтому любой ДЦ знает, куда направить запрос
`user_meetings`, список встреч пользователя	пишется в `home_dc` встречи, ScyllaDB асинхронно реплицирует его во все ДЦ
`user_sessions`	ДЦ, где был логин; код ДЦ — префикс `session_id`

Описано, как запрос на запись попадает в чужой ДЦ (gRPC-прокси), что такое read-your-writes и что происходит при отказе точки записи (Patroni, RPO).

2.5. Вопрос Быкова: «Пользователь залогинился в США, а потом поехал в Японию»

Что сделано (новый п. 3.8): sequence-диаграмма и таблица из 9 шагов.

Ответ одной фразой. GeoDNS отправит пользователя в Токио по текущему местоположению. Сессия переносится из США один раз по префиксу `session_id`, перелогиниваться не нужно. Профиль и список встреч читаются из локальных копий. Встречи американских коллег по-прежнему живут в `DC-US-EAST`: `join` проксируется туда, медиа идёт туда же. RTT Токио ↔ Вашингтон ≈ 153 мс, то есть сеть в одну сторону ≈ 77 мс. ITU-T G.114 нормирует сквозную задержку «от рта до уха» — до 150 мс в одну сторону, и в неё входят ещё кодек и джиттер-буфер. Разговор возможен, но близко к границе; улучшение — каскад SFU. Новые встречи, созданные в Японии, получают `home_dc = DC-JP-TOKYO`. Мигрировать ничего не нужно, потому что в системе нет «домашнего ДЦ пользователя».

2.6. «Механизм переключения какой? Anycast или DNS»

В чём была проблема. В прошлой версии было противоречие. В п. 3.4 `api.meetflow.example` выбирался DNS по latency, а в п. 3.5 тот же API шёл через Anycast — для одного имени заявлены два механизма. Если DNS возвращает один Anycast IP, ДЦ выбирает BGP, а не DNS.

Что сделано (п. 3.5–3.7, 3.9). Однозначный ответ: **DNS (GeoDNS/GTM)**.

- Anycast используется только для доставки DNS-запросов к авторитетным NS-серверам и в CDN для статики.
- Для API, WebSocket и медиа Anycast не подходит, и это объяснено. BGP выбирает маршрут по AS-path, а не по загрузке ДЦ, поэтому весов и drain нет. При перестроении маршрута долгие TCP- и UDP-сессии уходят в ДЦ, где нет их состояния.
- Медиа переключается на уровне приложения: control plane выдаёт новый SFU, клиент заново устанавливает с ним медиасоединение (ICE + DTLS).
- Посчитано время переключения: ≈ 1 минута при интервале проверок 10 с — у Cloudflare такой интервал доступен на Enterprise-плане, на Business минимум 15 с. Есть клиентский fallback на случай, если резолвер игнорирует TTL.

2.7. «В работе ориентироваться и понимать смысл»

Для этого написан разбор-работы.md: логика дисциплины, проход по всем 11 разделам с числами, словарь терминов, ответы на вероятные вопросы и шпаргалка чисел.

3. Что ещё было неверно или несогласованно

3.1. Ошибки, которые меняют результат

#	Где	Что было	Почему неверно	Что сделано
1	4.4–4.6	`N_ssl = ceil(RPS / 58 811 SSL TPS)`, где RPS включает 1.35 млн heartbeat	SSL TPS — это скорость **новых** TLS-рукопожатий, а heartbeat идёт по уже открытому WSS-соединению	Для RPS используется HTTPS RPS, для новых соединений — CPS, отдельно сеть. После второго прохода — значения для 16 CPU: 341 232 и 6 676 на узел (раздел 8)
2	4.5, 11	Не учитывались одновременные соединения	WebSocket держит соединение весь звонок: 16.9 млн соединений в пик. Каждое занимает память NGINX (< 20 КВ по замеру) и TLS (до 34 КВ)	Добавлен ограничитель `N_conn` — он и оказался главным. WS-поды тоже считаются по соединениям: 300 тыс. на под
3	11	10 Гбит/с на 6-ядерный SFU-узел, источники	Бенчмарки SFU: Jitsi ≈ 0.68 Гбит/с на ядро при 100% CPU, LiveKit — ≈ 0.29 на ядро инстанса в лучшем сценарии.	Узел SFU = `min(10, 6 × 0.678 × 0.7) = 2.847 Гбит/с`. SFU-серверов стало 28 570 вместо 15 632

		— прайс и тарифы трафика	Ограничитель — CPU, а не сетевая карта	
4	11	Серверы БД посчитаны одной суммой «на весь мир»	Архитектура с `home_dc` означает, что данные встречи лежат в её ДЦ. Значит, в каждом ДЦ нужен свой кластер	Расчёт по каждому ДЦ с минимальными размерами кластеров (Redis 3 узла, Scylla 3 узла, Citus 2 узла)
5	3.6 старая	Резерв для `DC-US-NORTHAMERICA` (44%) — `DC-UK-LONDON` (3.72%)	Резерв в 12 раз меньше основного ДЦ, принять нагрузку он не может	Резерв — порядок предпочтения, GTM раздаёт трафик по нескольким ДЦ. В п. 9.2 посчитано, что при деградации профиля видео ёмкости хватает
6	3.4–3.5 старые	`api` одновременно через DNS latency и через Anycast	Противоречие, см. 2.6	Исправлено
7	11	Чат: 150 серверов по 32 ядра (13.3 млн ₽/мес) на 26–52 тыс. записей/с	Нагрузка упирается в объём (135 TB с репликацией), а не в CPU	138 узлов CL34 с известными 2×1 TB NVMe — 1.88 млн ₽/мес
8	2.4, 11	Поток записи SFU → recording workers не входил в нагрузку SFU	SFU отправляет запись как ещё одному подписчику: +3 172.5 Гбит/с	Добавлен в п. 2.4 и в расчёт SFU

3.2. Несостыковки между разделами

#	Где	Что было	Что сделано
9	1.4, 2.4 ↔ 6.6, 11	В начале — «частота чата не выдумывается, оставлена формула», дальше — 5 сообщений на участие и 52 083 RPS	Допущение `MF_CHAT_PER_JOIN = 5` вынесено в п. 2.1 и используется везде
10	6.6	11 979 RPS для списка встреч нигде не определён	Допущение `MF_LIST_PER_DAU = 3` в п. 2.1, метод добавлен в таблицу RPS
11	5.2	3 993 записи сессий/с — неявное «1 логин на DAU × K»	Допущение `MF_LOGIN_PER_DAU = 1` и метод `POST /auth/login` в п. 2.4
12	6.6 ↔ 6.8	Heartbeat трогал 2 ключа, включая `user_sessions`, а нагрузка на `user_sessions` в 6.8 считалась без heartbeat	Heartbeat трогает только `meetings_runtime`: WSS авторизуется один раз по `join_token`
13	6.8 ↔ 6.7, 10.3	Нагрузка на `meetings_runtime` без `create` и без `INCR chat_seq`, хотя оба пишут в этот Redis	Учтены; вместе с ключами идемпотентности чата из второго прохода — 738 021 ops/s
14	1.1 ↔ 3.2	Целевые страны «США, Европа, Россия, Япония», а ДЦ стояли в US, JP, IN, UK + OTHER (Индии нет в целевых, у России и континентальной Европы нет ДЦ)	Целевые регионы — глобально, кроме материкового Китая; крупнейшие экономики — по данным IMF
15	Оглавление	10 ссылок вели на несуществующие заголовки (3.2–3.7, 6.1, 6.7, 10.4, 10.5); подразделов 3.7, 10.4, 10.5 не было	Оглавление пересобрано из заголовков, недостающие подразделы написаны
16	Источники	[^11], [^43], [^44], [^45] были в списке, но нигде не упоминались	Каждый привязан к утверждению, которое подтверждает
17	2.3	В «суммарное техническое хранение» не вошли чат (45 TB), ссылки-приглашения и `user_meetings`	Добавлены: 47.75 TB вместо 1.48 TB

Ещё одно неверное утверждение есть в сценариях выступления (они не в git): «Citus нужен, потому что 900 млн строк встреч не помещаются в один узел». Это неверно: 900 млн × 256 B = 230 GB, такой объём помещается на один сервер. Citus нужен для роста — distribution column `meeting_id` задан сразу, worker-узлы добавляются без смены схемы.

4. Что было правильно и осталось без изменений

- **Методика раздела 2.** Каждая метрика подписана источником и годом, формулы выписаны. Вся арифметика перепроверена скриптом: 30 млн встреч, 16.2 млрд participant-minutes, 11.25 млн онлайн в среднем, 38.88 млрд heartbeat, 0.761 GB на запись, 685.26 PB за 30 дней — всё сходится.
- **Разделение control plane и media plane.** Медиа идёт мимо L7 (а после второго прохода — и мимо L4), чтобы не терминировать TLS на десятках Тбит/с и держать встречу на одном SFU.
- **SFU вместо MCU, simulcast/SVC, server-guided ABR** (раздел 7) — стандартные и правильные решения для группового видео.
- **Выбор СУБД под паттерн доступа.** PostgreSQL — где нужны транзакции и уникальность, ScyllaDB — для write-heavy данных без JOIN, Redis — для горячего состояния с TTL, S3 — для файлов.
- **Денормализация и отказ от JOIN** в hot path, read model `user_meetings`, `message_seq` для порядка чата.
- **Индексы, таблица консистентности, идея `home_dc`.** Её не хватало только довести до точек записи и сценариев.
- **Подход к резервированию**: `N+1`, RF=3, WAL/PITR.

5. Что осталось допущением

Эти места не ошибки, но на них могут задать вопрос. Отвечать честно: «это допущение, вот чем оно ограничено».

Допущение	Где	Чем ограничено / как отвечать
Нагрузка по странам пропорциональна ВВП	3.2	Пользователей по странам первоисточники не публикуют. Ошибка доли меняет размер пулов в ДЦ, но не архитектуру; веса GTM и резерв компенсируют перекос
Участник сидит во встрече все 54 минуты	2.2	Верхняя граница: у Zoom реальный run rate — 3.5 трлн минут в год, то есть около 60% от нашей оценки. Ошибка в безопасную сторону
Каждая встреча записывается (worst-case)	2.3, 11	Это 59% бюджета. Реально записывается доля встреч, публичной цифры нет. Главный рычаг экономии — ретеншн и квоты
Ледяной класс S3 для свежих записей	11.4	Хранить дёшево, но запросы к ледяному классу дороже, чем к стандартному (документация Selectel). Корректнее держать первые дни в стандартном классе и переводить в ледяной правилом lifecycle
2.847 Гбит/с на SFU-узел	11.1	Экстраполяция замера Jitsi с 20% CPU до 100%. LiveKit даёт меньше, особенно на мелких пакетах, так что оценка оптимистичная
54 KB на WSS-соединение	4.4	Худший случай: TLS-буферы не освобождены. Реально меньше (NGINX освобождает буферы простаивающих соединений)
1 000 / 1 500 / 5 000 RPS на под	11.1	Проектные нормативы для C++-сервисов с точечными запросами, без бенчмарка
K = 1.5 одинаково для каждого ДЦ	2.1, 3.3	Внутри одного региона суточная кривая острее глобальной. Запас дают `N+1` и перераспределение весами GTM
Цены Selectel для зарубежных ДЦ	11	Ценовой ориентир: прайс на 11.10.2026, курс ЦБ 84.90 ₽ за $1. Городов наших ДЦ у Selectel нет

Внешний трафик не в итоге	11.4	По публичному тарифу это ≈ 930 млн ₽/мес — верхняя граница. На таком объёме трафик покупают по индивидуальному договору

6. Было → стало: ключевые числа

Таблица нужна в первую очередь для обновления слайдов и сценария выступления.

Метрика	Было	Стало
`MF_K_PEAK`	3	1.5
Участников онлайн в пик (= WSS-соединений)	33 750 000	16 875 000
Встреч одновременно в пик	3 375 000	1 687 500
Media в пик	156 262.5 Гбит/с	78 131.25 Гбит/с (+3 172.5 поток в запись)
Heartbeat в пик	1 350 000 RPS	675 000 RPS
`join` / `leave` в пик	10 417 RPS	5 208 RPS
`create` в пик	1 042 RPS	521 RPS
Чат в пик	52 083 msg/s	26 042 msg/s
Список встреч в пик	11 979 RPS	5 990 RPS
Логин в пик	—	1 997 RPS
Строк в секунду на `join`	62 502	31 250
ДЦ	4 + `OTHER_REGIONAL_POOL`	8; самый большой — Франкфурт, 35.69%
NGINX L7, `N+1` / `N*2`	31 / 52	28 / 40
L4	managed LB	16 директоров IPVS + keepalived, по 2 на ДЦ
SFU-серверов	15 632	28 570
Воркеров записи	640	1 127
Узлов app pool	36	59
Redis	6 + 24	24 (3 на ДЦ, оба кластера)
PostgreSQL `auth_pg`	2	9 (primary + standby + 7 реплик)
Citus	15 + 9	24
ScyllaDB meeting / chat	30 BL18 / 150 PL70	34 / 138 CL34
Kubernetes-кластеров	5	8
`runtime_redis`, ops/s	1 370 834	738 021 (с идемпотентностью чата)
`meetings_runtime`, объём	1.73 GB	0.864 GB
Техническое хранение без записей	1.48 TB	47.75 TB (с чатом)
Записи за 30 дней	685.26 PB	685.26 PB, не зависит от K
Стоимость серверов и хранения	**800.08 млн ₽ ≈ $10.0 млн**	**1 031.87 млн ₽ ≈ $12.15 млн**
Доля S3 / SFU	69% / 27%	59.1% / 38.8%
Внешний трафик медиа	не считался	≈ 930 млн ₽/мес по публичному тарифу, вне итога

7. Что не входит в эту ветку

Файлы `MeetFlow_presentation.pptx`, `MeetFlow_10min.pptx`, `MeetFlow_speaker_script.md` и `MeetFlow_script_10min.md` не отслеживаются git и не менялись. Числа в них устарели. Что обновить:

- K = 3 → 1.5; онлайн в пик 33.75 → 16.9 млн; медиа 156 → 78 Тбит/с; heartbeat 1.35 млн → 675 тыс.; join 10 тыс. → 5.2 тыс. RPS и 62 → 31 тыс. строк/с; чат 52 → 26 тыс. msg/s.
- «Пять логических ДЦ… общий пул 42%» → 8 ДЦ, доли по ВВП; самый большой — Франкфурт (35.69%); «показан один ДЦ из пяти» → «из восьми».
- L7: 31 → 28 узлов; ограничитель — память под соединения, а не SSL. L4 — IPVS + keepalived, 16 узлов; медиа идёт в SFU напрямую.
- SFU: «15 632 сервера по 10 Гбит/с» → «28 570 серверов по 2.847 Гбит/с, ограничитель — CPU».
- Redis: «12 master + 12 replica» → «по 3 узла на ДЦ, всего 24»; `meetings_runtime` 1.73 → 0.86 GB.
- Стоимость: ≈ 800 → ≈ 1 032 млн ₽/мес за серверы и хранение; S3 69% → 59%, SFU 27% → 39%; плюс внешний трафик ≈ 930 млн по публичному тарифу.
- Ответ про Citus — см. конец раздела 3.
- Добавить слайд или ответы на вопросы: точки записи, DNS vs Anycast, сценарий «США → Япония».

8. Второй проход: перепроверка источников и согласованности

Все ссылки работы открыты и сверены с утверждениями, арифметика пересчитана скриптом, разделы сверены между собой и с методичкой курса. Что найдено и исправлено:

8.1. Нарушения методички

#	Что было	Что сделано
1	Доли ДЦ построены на SimilarWeb (через перепечатку WallStreetZen). Методичка: «не разрешается использовать данные similarweb.com как первоисточник»	Доли по ВВП стран из IMF WEO (п. 3.2); SimilarWeb и WebbyCrown убраны из источников
2	Перепечатки вместо первоисточников: 320 млн MAU Teams — пост на форуме, 300 млн участников Zoom — статья Reuters	Microsoft: транскрипт звонка FY24 Q1; Zoom: проспект в SEC (Form 424B5)
3	Нет средних хранилища и действий на пользователя и сводной таблицы продуктовых метрик	Добавлены в п. 2.2
4	Объём хранения только в ТБ, без штук	Столбец «Штук» в п. 2.3
5	Нет схемы резервного копирования (обязательный пункт раздела 6)	п. 6.12
6	Нет балансировки запросов к БД и мультиплексирования подключений	п. 6.11
7	В логической модели нет кешей, счётчиков и буферов; в таблице 5.2 стояли заглушки «formula»	Сущности и числа в п. 5.1–5.2
8	L4 не посчитан («не считаем»)	IPVS + keepalived посчитан по памяти и трафику, п. 4.6

8.2. Ссылки, которые не подтверждали утверждение

#	Где	Что было	Что показал источник	Что сделано
1	3.5	Интервал проверки GTM «настраивается — 10 с»	У Cloudflare 10 с — только на Enterprise; Pro — 60 с, Business — 15 с	Указан тариф, ссылка на страницу создания monitor
2	3.4, 8, 9	Patroni «автоматически» переключает primary между ДЦ	Документация Patroni: standby-кластер во втором ДЦ повышают только вручную	Внутри ДЦ — автоматически, между ДЦ — дежурный по runbook
3	3.8	«76 мс меньше 150 мс по G.114»	G.114 нормирует сквозную задержку «от рта до уха», а не только сеть	Формулировка исправлена: разговор возможен, но близко к границе
4	4.4	«1 KB — размер запроса»; 24 CPU теста = 24 vCPU	1 KB — размер ответа; 8.8 Гбит/с упёрлись в сетевую карту стенда; CPU теста — физические ядра	Взяты значения для 16 CPU, подписи исправлены
5	4, 10,	Один управляемый L4 на	Балансировщик Selectel: до 3–5 Гбит/с,	Свой L4 на IPVS + keepalived;

	11	ДЦ, через него идёт и медиа	тесты на 10 000 соединений; в нашем ДЦ до 6 млн	медиа идёт в SFU напрямую, как и было описано в п. 3.5
6	6.4	`UNIQUE(token)` и `PK(id)` в Citus	Citus: PK и UNIQUE обязаны включать колонку распределения	`PK/UNIQUE` с `token_hash` и `meeting_id`
7	6.4	`UNIQUE(id)` в ScyllaDB	В ScyllaDB нет UNIQUE: `INSERT` с тем же ключом перезаписывает строку	Идемпотентность через ключ в Redis (`SET NX EX 60`)
8	6.4, DBML	Вторичные индексы в Redis	В Redis доступ только по ключу	Индексы убраны, вместо поиска по времени — TTL
9	6.10	ScyllaDB C/C++ Driver	Драйвер больше не развивается	ScyllaDB CPP-RS Driver
10	8	Сноски [^25], [^26] про транзакции	Страницы о составных индексах и партиционировании	[^26] заменён на «Transaction Isolation»
11	11.1	«≤ 50% заполнения под compaction» со ссылкой на Fault Tolerance	На этой странице этого нет	Ссылка на «Compaction Strategies»
12	11.1	LiveKit «≈ 0.3 Гбит/с на ядро» как подтверждение	0.29 — сценарий livestream; во «встрече 150 × 150» — ≈ 0.08	Описана зависимость от размера пакетов; оценка помечена как оптимистичная
13	11.1, 11.4	Цены от 24.06.2026, курс 80 ₽	С 01.07.2026 облако и S3 подорожали на 10%; курс ЦБ на 10.10.2026 — 84.90	Цены и курс обновлены
14	11.1	Сноска [^71] у «10 Gbps NIC»	Страница про тарифы трафика: сверх 1 Гбит/с трафик платный	Добавлена оценка внешнего трафика, п. 11.4

8.3. Несостыковки внутри работы

#	Где	Что было	Что сделано
1	3.5 ↔ 4.1, 10.3	Адрес SFU приходит в ответе `join` , но медиа шло «через Provider L4»	Медиа идёт напрямую в SFU
2	4.5	«REST без keep-alive», но чат в новые TLS-соединения не входил	Все REST-запросы входят в худший случай
3	5.2 ↔ 6.6	`create` пишет в `user_meetings` , но в 5.2 записи считались без него	10 938 записей/с вместо 10 417
4	6.5	«≈ 4 100 чтений/с» — только `meetings` , без ссылок и записей	Все чтения Citus: 16 667/с, в самом большом ДЦ ≈ 5 950
5	6.7 ↔ 2.4	История чата и скачивание записей были в API, но не в RPS	Добавлены в RPS, L7 и поды
6	7 ↔ 2.4	Профиль 1080p в ABR, хотя трафик посчитан для 720p	Верхний профиль 720p; статистика сети идёт в RTCP, без отдельных запросов
7	11.1	Формула SFU с 2.85, а скрипт считал с 2.8469	Везде 2.847
8	9.2	«Запас L7 — 28 млн соединений» включал узлы упавшего ДЦ	Считаются только живые ДЦ
9	6.2	`meeting_pg` и `recording_pg` — «две логические БД», хотя co-location работает только внутри одной базы	Две схемы одной базы
10	3.2	Корея и Тайвань обслуживались из Сингапура, хотя Токио ближе	Корея, Тайвань, Монголия — в Токио; Южная Азия — в Мумбаи

8.4. Что сделать самому

- **Залить ветку и слить её в `main` .** Ссылки diagrams.net в README открывают файл из `main` , поэтому новая схема появится по ним только после слияния.

- **Обновить документ в dbdiagram.** В `physical-database-sql.md` исправлены индексы (Redis, ScyllaDB, Citus), а документ на dbdiagram.io хранит старый код. Картинка от этого не меняется: dbdiagram не рисует индексы.

8.5. Дополнено после второго прохода

#	Где	Чего не хватало	Что сделано
1	4.1	Медиа идёт напрямую на публичный адрес SFU, но не сказано, что делает клиент за NAT или с закрытым UDP	SFU работает как ICE-Lite и слушает UDP и TCP: клиенту не нужен STUN/TURN, при закрытом UDP он подключается по TCP (ICE-TCP, RFC 6544). Сети, где прокси пропускает только TLS на 443, — ограничение MVP: для них нужен TURN по TLS
2	«Изменения после ревью», 3.7, 3.9, 4.3, 9.1	При отказе SFU было написано «клиент делает ICE restart». ICE restart ищет новый путь к тому же серверу; у другого SFU свои ключи DTLS, поэтому одного ICE restart мало	Клиент заново устанавливает медиасоединение с новым SFU: новое ICE-согласование и DTLS-рукопожатие

Приложение B. Исходный код схем БД (DBML)

Картинки схем в приложении A построены в dbdiagram.io из этого кода. dbdiagram не рисует индексы, поэтому они видны только здесь и в п. 6.4 работы.

B.1. Логическая схема — `resource/logic-database-sql.md`

```
Project MeetFlow {
  database_type: "Logical schema"
  Note: "Zoom-like video meetings service. Logical DB schema without physical sharding."
}

Table users {
  id uuid [pk]
  email varchar [unique, not null]
  password_hash varchar [not null]
  name varchar
  created_at datetime
}

Table user_sessions {
  session_id varchar [pk]
  user_id uuid [not null]
  expires_at datetime
  created_at datetime
}

Table meetings {
  id uuid [pk]
  owner_id uuid [not null]
  home_dc varchar
  topic varchar
  scheduled_start datetime
  duration_minutes integer
  status varchar
  created_at datetime
}

Table meeting_participants {
  meeting_id uuid [pk, not null]
  user_id uuid [pk, not null]
  role varchar
  is_online boolean
  last_joined_at datetime
  last_left_at datetime
  join_count integer
  total_online_seconds bigint
}

Table meeting_invite_links {
  id uuid [pk]
  meeting_id uuid [not null]
  created_by_user_id uuid [not null]
  token varchar [unique, not null]
  created_at datetime
  expires_at datetime
  revoked boolean
```

```
}

Table user_meetings {
  user_id uuid [pk, not null]
  meeting_id uuid [pk, not null]
  topic varchar
  scheduled_start datetime
  status varchar
  role varchar
}

Table chat_messages {
  id uuid [pk]
  meeting_id uuid [not null]
  user_id uuid [not null]
  message_seq integer
  body text
  created_at datetime
}

Table recordings {
  id uuid [pk]
  meeting_id uuid [not null]
  storage_url varchar
  size_bytes bigint
  duration_seconds integer
  status varchar
  created_at datetime
}

Table recording_upload_buffer {
  chunk_id uuid [pk]
  recording_id uuid [not null]
  chunk_number bigint
  size_bytes bigint
  checksum varchar
  status varchar
  expires_at datetime
}

Ref: users.id < meetings.owner_id
Ref: users.id < user_sessions.user_id
Ref: users.id < meeting_invite_links.created_by_user_id
Ref: users.id < chat_messages.user_id
Ref: users.id < user_meetings.user_id
Ref: users.id < meeting_participants.user_id

Ref: meetings.id < meeting_participants.meeting_id
Ref: meetings.id < meeting_invite_links.meeting_id
Ref: meetings.id < recordings.meeting_id
Ref: meetings.id < chat_messages.meeting_id
Ref: meetings.id < user_meetings.meeting_id

Ref: recordings.id < recording_upload_buffer.recording_id
```

B.2. Физическая схема — `resource/physical-database-sql.md`

```
Project MeetFlow_Physical {
  database_type: "PostgreSQL + Citus / Redis Cluster / ScyllaDB / S3-compatible Object Storage"
  Note: "Physical schema. Cross-storage refs are logical; hot-path JOIN is not used."
```

```
}

Table auth_pg.users {
  id uuid [pk]
  email varchar [unique, not null]
  password_hash varchar [not null]
  name varchar
  created_at datetime

  indexes {
    id [name: "pk_users_id"]
    email [unique, name: "ux_users_email"]
    created_at [name: "idx_users_created_at"]
  }
}

Table session_redis.user_sessions {
  session_id varchar [pk]
  user_id uuid [not null]
  expires_at datetime
  created_at datetime

  indexes {
    session_id [name: "key_session_id", note: 'Redis: доступ только по ключу, вторичных индексов
нет']
    expires_at [name: "ttl_sessions_expires_at", note: 'TTL ключа']
  }
}

Table meeting_pg.meetings {
  id uuid [pk]
  owner_id uuid [not null]
  home_dc varchar
  topic varchar
  scheduled_start datetime
  duration_minutes integer
  status varchar
  created_at datetime

  indexes {
    id [name: "pk_meetings_id"]
    (owner_id, scheduled_start) [name: "idx_meetings_owner_start"]
    (home_dc, status) [name: "idx_meetings_home_status"]
    (status, created_at) [name: "idx_meetings_status_created"]
  }
}

Table meeting_scylla.meeting_participants {
  meeting_id uuid [not null]
  bucket smallint [not null]
  user_id uuid [not null]
  role varchar
  is_online boolean
  last_joined_at datetime
  last_left_at datetime
  join_count integer
  total_online_seconds bigint

  indexes {
    (meeting_id, bucket, user_id) [pk, name: "pk_participants_meeting_bucket_user"]
    (user_id, last_joined_at) [name: "idx_participants_user_joined"]
  }
```

```
}

Table meeting_pg.meeting_invite_links {
  id uuid [pk]
  meeting_id uuid [not null]
  created_by_user_id uuid [not null]
  token varchar [not null]
  token_hash bigint [not null, note: 'hash(token), колонка распределения Citus']
  created_at datetime
  expires_at datetime
  revoked boolean

  indexes {
    (token_hash, token) [unique, name: "ux_invite_token_hash_token", note: 'Citus: UNIQUE и PK
включают колонку распределения; физически PK = (token_hash, id)']
    expires_at [name: "idx_invite_expires_at"]
    (meeting_id, created_at) [name: "idx_invite_meeting_created"]
  }
}

Table meeting_scylla.user_meetings {
  user_id uuid [not null]
  scheduled_start datetime [not null]
  meeting_id uuid [not null]
  topic varchar
  status varchar
  role varchar

  indexes {
    (user_id, scheduled_start, meeting_id) [pk, name: "pk_user_meetings_user_start_meeting"]
    (meeting_id, user_id) [name: "idx_user_meetings_meeting_user"]
  }
}

Table runtime_redis.meetings_runtime {
  meeting_id uuid [pk]
  status varchar
  participants_online_count integer
  participants_total_count integer
  sfu_pool varchar
  updated_at datetime

  indexes {
    meeting_id [name: "key_runtime_meeting_id", note: 'Redis: TTL ключа продлевает heartbeat,
вторичных индексов нет']
  }
}

Table chat_scylla.chat_messages {
  id uuid [not null, note: 'client_msg_id; идемпотентность — ключ chat_dedup в Redis (SET NX EX
60)']
  meeting_id uuid [not null]
  bucket smallint [not null]
  message_seq integer [not null]
  user_id uuid [not null]
  body text
  created_at datetime

  indexes {
    (meeting_id, bucket, message_seq) [pk, name: "pk_chat_meeting_bucket_seq"]
    (user_id, created_at) [name: "idx_chat_user_created"]
  }
```

```
}

Table recording_pg.recordings {
  id uuid [pk]
  meeting_id uuid [not null]
  storage_bucket varchar
  storage_key varchar
  storage_url varchar
  size_bytes bigint
  duration_seconds integer
  status varchar
  created_at datetime

  indexes {
    id [name: "pk_recordings_id", note: 'Citus: физически PK = (meeting_id, id), колонка
распределения — meeting_id']
    (meeting_id, created_at) [name: "idx_recordings_meeting_created"]
    (status, created_at) [name: "idx_recordings_status_created"]
    storage_key [name: "idx_recordings_storage_key"]
  }
}

Table recording_obj.recording_upload_buffer {
  chunk_id uuid [pk]
  recording_id uuid [not null]
  chunk_number bigint [not null]
  storage_key varchar [not null]
  size_bytes bigint
  checksum varchar
  status varchar
  expires_at datetime

  indexes {
    chunk_id [name: "pk_recording_chunk"]
    (recording_id, chunk_number) [unique, name: "ux_recording_chunk_number"]
    (recording_id, status) [name: "idx_recording_chunks_status"]
    expires_at [name: "ttl_recording_chunks_expires"]
  }
}

Ref: auth_pg.users.id < meeting_pg.meetings.owner_id
Ref: auth_pg.users.id < session_redis.user_sessions.user_id
Ref: auth_pg.users.id < meeting_pg.meeting_invite_links.created_by_user_id
Ref: auth_pg.users.id < meeting_scylla.meeting_participants.user_id
Ref: auth_pg.users.id < meeting_scylla.user_meetings.user_id
Ref: auth_pg.users.id < chat_scylla.chat_messages.user_id

Ref: meeting_pg.meetings.id < meeting_scylla.meeting_participants.meeting_id
Ref: meeting_pg.meetings.id < meeting_pg.meeting_invite_links.meeting_id
Ref: meeting_pg.meetings.id - runtime_redis.meetings_runtime.meeting_id
Ref: meeting_pg.meetings.id < meeting_scylla.user_meetings.meeting_id
Ref: meeting_pg.meetings.id < chat_scylla.chat_messages.meeting_id
Ref: meeting_pg.meetings.id < recording_pg.recordings.meeting_id

Ref: recording_pg.recordings.id < recording_obj.recording_upload_buffer.recording_id
```

Приложение Г. Скрипт расчёта — `resource/calc/meetflow_calc.py`

Скрипт считает все числа разделов 2–6, 9 и 11. Запуск: `python3 resource/calc/meetflow_calc.py`. ВВП стран для долей ДЦ он берёт из `resource/calc/imf_weo_2026apr_gdp_2025.json`.

```
#!/usr/bin/env python3
"""Единая модель расчёта MeetFlow.

Все числа в README (разделы 2–6, 9 и 11) получены этим скриптом, поэтому
при изменении любого допущения (например, MF_K_PEAK) достаточно поменять
константу и перезапустить:

    python3 resource/calc/meetflow_calc.py > /tmp/meetflow_tables.md

Единицы: десятичные (1 KB = 10^3 B, 1 GB = 10^9 B, 1 Gbps = 10^9 бит/с).
ВВП стран для долей ДЦ лежит рядом: imf_weo_2026apr_gdp_2025.json.
"""

import json
import math
import os

# ---------------------------------------------------------------------- 2. Исходные метрики
MAU = 320_000_000            # Teams, FY24 Q1 [^1]
DAU = 115_000_000            # Teams, 10.2020 [^2]
DMP = 300_000_000            # участий во встречах в сутки, Zoom 04.2020 [^3]
MEET_SIZE = 10               # участников в средней встрече [^4]
MEET_LEN_MIN = 54            # минут в средней встрече [^4]
V_UP, V_DOWN = 2.6, 1.8      # Мбит/с, 720p group video up/down [^5]
V_UP_HQ, V_DOWN_HQ = 1.0, 0.6  # Мбит/с, group "high-quality video" up/down [^5]
AUDIO, SCREEN = 0.08, 0.15   # Мбит/с, VoIP и screen sharing upper-bound [^5]
HB_SEC = 25                  # интервал heartbeat, Socket.IO pingInterval [^9]
K = 1.5                      # MF_K_PEAK — суточный пик к среднему (замечание ревьюера)

# Проектные допущения (раздел 2.1)
CHAT_PER_JOIN = 5            # сообщений чата на одно участие
CHAT_MSG_B = 1_000           # средний размер сообщения (лимит 4 000 символов [^6])
LIST_PER_DAU = 3             # открытий списка встреч на DAU в сутки
LOGIN_PER_DAU = 1            # созданий сессии (логинов) на DAU в сутки
CHAT_HIST_PER_JOIN = 1       # загрузок истории чата на одно участие (вход или переподключение)
REC_VIEWS_PER_REC = 1        # открытий записи на одну запись — верхняя оценка
REC_PART_B = 8_000_000       # часть multipart upload записи; S3 допускает до 10 000 частей на
объект [^46]
CHAT_DEDUP_TTL_SEC = 60      # сколько хранится ключ идемпотентности сообщения
RETENTION_DAYS = 30

DAY = 86_400


def peak(per_day):
    return per_day / DAY * K


# ---------------------------------------------------------------------- 2.2 продуктовые
MEETINGS_DAY = DMP / MEET_SIZE
PART_MIN_DAY = DMP * MEET_LEN_MIN
AVG_ONLINE = PART_MIN_DAY / 1440
PEAK_ONLINE = AVG_ONLINE * K                 # = пиковое число WSS-соединений
```

```
AVG_MEETINGS_ONLINE = AVG_ONLINE / MEET_SIZE
PEAK_MEETINGS_ONLINE = PEAK_ONLINE / MEET_SIZE
HB_DAY = PART_MIN_DAY * 60 / HB_SEC
CHAT_DAY = DMP * CHAT_PER_JOIN
LIST_DAY = DAU * LIST_PER_DAU
LOGIN_DAY = DAU * LOGIN_PER_DAU

# ---------------------------------------------------------------- 2.3 хранение
REC_SIZE_GB = (V_DOWN + AUDIO) * MEET_LEN_MIN * 60 / 8 / 1000
REC_PB_DAY = MEETINGS_DAY * REC_SIZE_GB / 1e6
REC_PB_30D = REC_PB_DAY * RETENTION_DAYS

STORAGE = [  # (блок, название, штук, байт на штуку)
    ("Профили пользователей", "MF_USERS_STORAGE_GB", MAU, 256),
    ("Метаданные встреч, 30 дней", "MF_MEETINGS_META_STORAGE_GB", MEETINGS_DAY * RETENTION_DAYS,
256),
    ("Ссылки-приглашения, 30 дней", "MF_INVITES_STORAGE_GB", MEETINGS_DAY * RETENTION_DAYS, 128),
    ("Участники встреч, 30 дней", "MF_PARTICIPANTS_STORAGE_TB", DMP * RETENTION_DAYS, 128),
    ("Списки встреч пользователей, 30 дней", "MF_USER_MEETINGS_STORAGE_TB", DMP * RETENTION_DAYS,
128),
    ("Активные сессии", "MF_SESSIONS_STORAGE_GB", DAU, 128),
    ("Чат встречи, 30 дней", "MF_CHAT_STORAGE_TB", CHAT_DAY * RETENTION_DAYS, CHAT_MSG_B),
]
TECH_TB = sum(n * b for _, _, n, b in STORAGE) / 1e12
USERS_GB = MAU * 256 / 1e9
MEET_META_GB = MEETINGS_DAY * RETENTION_DAYS * 256 / 1e9
INVITES_GB = MEETINGS_DAY * RETENTION_DAYS * 128 / 1e9
PARTICIPANTS_TB = DMP * RETENTION_DAYS * 128 / 1e12
USER_MEETINGS_TB = DMP * RETENTION_DAYS * 128 / 1e12
SESSIONS_GB = DAU * 128 / 1e9
CHAT_TB_30D = CHAT_DAY * RETENTION_DAYS * CHAT_MSG_B / 1e12

# ---------------------------------------------------------------- 2.4 трафик и RPS
BW_AVG = {  # Гбит/с, средний
    "Video upload": AVG_ONLINE * V_UP / 1000,
    "Video download": AVG_ONLINE * V_DOWN / 1000,
    "Audio VoIP": AVG_ONLINE * AUDIO / 1000,
    "Screen sharing upper-bound": AVG_ONLINE * SCREEN / 1000,
}
MEDIA_GB_DAY = sum(BW_AVG.values()) * DAY / 8           # Гбит/с * с / 8 = ГБ/сутки
REC_STREAM_AVG = AVG_MEETINGS_ONLINE * (V_DOWN + AUDIO) / 1000  # SFU -> recording, worst-case
MEDIA_PEAK = sum(BW_AVG.values()) * K
REC_STREAM_PEAK = REC_STREAM_AVG * K
SFU_PEAK = MEDIA_PEAK + REC_STREAM_PEAK

EVENTS_DAY = {  # метод -> событий в сутки
    "POST /auth/login": LOGIN_DAY,
    "POST /meetings": MEETINGS_DAY,
    "POST /meetings/{id}/join": DMP,
    "POST /meetings/{id}/leave": DMP,
    "WS /presence/heartbeat": HB_DAY,
    "POST /meeting-chat/messages": CHAT_DAY,
    "GET /meeting-chat/messages?after_seq=N": DMP * CHAT_HIST_PER_JOIN,
    "GET /users/{id}/meetings?limit=100": LIST_DAY,
    "POST /recordings/metadata": MEETINGS_DAY,
    "GET /recordings/{id}": MEETINGS_DAY * REC_VIEWS_PER_REC,
}
RPS = {k: peak(v) for k, v in EVENTS_DAY.items()}
R_LOGIN = RPS["POST /auth/login"]
R_CREATE = RPS["POST /meetings"]
R_JOIN = RPS["POST /meetings/{id}/join"]
R_LEAVE = RPS["POST /meetings/{id}/leave"]
```

```
R_HB = RPS["WS /presence/heartbeat"]
R_CHAT = RPS["POST /meeting-chat/messages"]
R_HIST = RPS["GET /meeting-chat/messages?after_seq=N"]
R_LIST = RPS["GET /users/{id}/meetings?limit=100"]
R_REC = RPS["POST /recordings/metadata"]
R_REC_VIEW = RPS["GET /recordings/{id}"]
WS_PUSH_DAY = (CHAT_DAY + DMP + DMP) * (MEET_SIZE - 1)   # доставка чата и событий join/leave

# ---------------------------------------------------------------- 3.2 ДЦ
# Доля ДЦ = доля его группы стран в мировом ВВП без материкового Китая (IMF WEO, 2025);
# США делим на восток и запад по населению (Census 2020 [^73]).
_HERE = os.path.dirname(os.path.abspath(__file__))
GDP = json.load(open(os.path.join(_HERE, "imf_weo_2026apr_gdp_2025.json"), encoding="utf-8"))
["gdp_busd"]
GROUPS = {  # группа стран -> ISO-коды IMF
    "США": "USA",
    "Канада": "CAN",
    "Мексика, Центральная Америка, Карибы":
        "MEX GTM BLZ SLV HND NIC CRI PAN DOM HTI JAM TTO BHS BRB PRI ABW ATG DMA GRD KNA LCA
VCT",
    "Южная Америка": "BRA ARG CHL COL PER VEN ECU BOL PRY URY GUY SUR",
    "Великобритания": "GBR",
    "Европа без Великобритании, СНГ, Ближний Восток, Африка": (
        "ALB AND ARM AUT AZE BLR BEL BIH BGR HRV CYP CZE DNK EST FIN FRA GEO DEU GRC HUN ISL IRL
ITA UVK "
        "LVA LIE LTU LUX MLT MDA MNE NLD MKD NOR POL PRT ROU RUS SMR SRB SVK SVN ESP SWE CHE TUR
UKR "
        "KAZ KGZ TJK TKM UZB BHR IRN IRQ ISR JOR KWT LBN OMN QAT SAU ARE YEM WBG DZA EGY LBY MAR
TUN "
        "AGO BEN BWA BFA BDI CPV CMR CAF TCD COM COD COG CIV DJI GNQ SWZ ETH GAB GMB GHA GIN GNB
KEN LSO "
        "LBR MDG MWI MLI MRT MUS MOZ NAM NER NGA RWA STP SEN SYC SLE SOM ZAF SSD SDN TZA TGO UGA
ZMB ZWE"),
    "Индия, Пакистан, Бангладеш, Шри-Ланка и соседи": "IND PAK BGD LKA NPL BTN MDV AFG",
    "Юго-Восточная Азия, Океания, Гонконг": (
        "IDN THA VNM PHL MYS SGP MMR KHM LAO BRN TLS HKG MAC AUS NZL PNG FJI SLB VUT WSM TON KIR
FSM "
        "MHL PLW NRU TUV"),
    "Япония, Корея, Тайвань, Монголия": "JPN KOR TWN MNG",
    "Материковый Китай (не обслуживаем)": "CHN",
}
GROUPS = {g: codes.split() for g, codes in GROUPS.items()}
EXCLUDED_GROUP = "Материковый Китай (не обслуживаем)"
_assigned = [c for codes in GROUPS.values() for c in codes]
assert len(_assigned) == len(set(_assigned)), "страна в двух группах"
assert set(GDP) <= set(_assigned), f"страна без группы: {set(GDP) - set(_assigned)}"
GROUP_GDP = {g: sum(GDP.get(c, 0.0) for c in codes) for g, codes in GROUPS.items()}
SERVED_GDP = sum(v for g, v in GROUP_GDP.items() if g != EXCLUDED_GROUP)
US_WEST_POP = 0.237                                  # доля населения West по Census 2020 [^73]

DC_GROUPS = {  # ДЦ -> (группа, доля группы)
    "DC-US-EAST": [("США", 1 - US_WEST_POP), ("Канада", 1), ("Мексика, Центральная Америка,
Карибы", 1)],
    "DC-US-WEST": [("США", US_WEST_POP)],
    "DC-BR-SAOPAULO": [("Южная Америка", 1)],
    "DC-UK-LONDON": [("Великобритания", 1)],
    "DC-DE-FRANKFURT": [("Европа без Великобритании, СНГ, Ближний Восток, Африка", 1)],
    "DC-IN-MUMBAI": [("Индия, Пакистан, Бангладеш, Шри-Ланка и соседи", 1)],
    "DC-SG-SINGAPORE": [("Юго-Восточная Азия, Океания, Гонконг", 1)],
    "DC-JP-TOKYO": [("Япония, Корея, Тайвань, Монголия", 1)],
}
DC_RAW = {d: sum(GROUP_GDP[g] * f for g, f in parts) / SERVED_GDP * 100 for d, parts in
```

```
DC_GROUPS.items()}


def round_shares(raw, nd=2):
    """Округление долей до nd знаков с сохранением суммы 100 (метод наибольшего остатка)."""
    scale = 10 ** nd
    floors = {k: math.floor(v * scale) for k, v in raw.items()}
    rest = round(100 * scale) - sum(floors.values())
    for k in sorted(raw, key=lambda k: raw[k] * scale - floors[k], reverse=True)[:rest]:
        floors[k] += 1
    return {k: v / scale for k, v in floors.items()}


DC = round_shares(DC_RAW)      # ДЦ -> доля нагрузки, %; в расчётах используются ровно эти доли
assert abs(sum(DC.values()) - 100) < 1e-9
MACRO = {
    "Америка": ["DC-US-EAST", "DC-US-WEST", "DC-BR-SAOPAULO"],
    "EMEA": ["DC-UK-LONDON", "DC-DE-FRANKFURT"],
    "APAC": ["DC-IN-MUMBAI", "DC-SG-SINGAPORE", "DC-JP-TOKYO"],
}
REV_AMER, REV_APAC, REV_EMEA = 72.1, 12.1, 15.8      # выручка Zoom FY2026 по регионам, % [^72]

# ---------------------------------------------------------------- 4. балансировка
L7_HTTPS_RPS = 341_232       # NGINX Ingress, HTTPS, ответ 1 KB, 16 CPU [^19]
L7_TLS_CPS = 6_676           # новые HTTPS-соединения (полный TLS handshake)/с, 16 CPU [^20]
L7_NET_GBPS = 8.8            # throughput NGINX Ingress, упор в 10GbE NIC стенда [^19]
L7_MSG_B = 1_000             # размер ответа в RPS-тесте [^19]
WS_CONN_B = 20_000           # NGINX держит WS-соединение (клиент+upstream) в < 20 KB [^77]
TLS_BUF_B = 34_000           # буферы TLS, если не освобождены (SSL_MODE_RELEASE_BUFFERS) [^78]
L7_CONN_B = WS_CONN_B + TLS_BUF_B
L7_NODE_RAM_GB, L7_RAM_RESERVED_GB = 64, 8

REST_RPS = (R_LOGIN + R_CREATE + R_JOIN + R_LEAVE + R_LIST + R_REC + R_REC_VIEW + R_CHAT +
R_HIST)
WS_IN = R_HB
WS_OUT = (R_CHAT + R_JOIN + R_LEAVE) * (MEET_SIZE - 1)   # доставка чата и presence-событий
L7_RPS = REST_RPS + WS_IN + WS_OUT
L7_NEW_TLS = REST_RPS + R_JOIN       # худший случай: каждый REST-запрос без keep-alive + WSS
connect на join
L7_GBPS = L7_RPS * L7_MSG_B * 8 / 1e9
CONN_PER_L7 = (L7_NODE_RAM_GB - L7_RAM_RESERVED_GB) * 1e9 / L7_CONN_B

IPVS_CONN_B = 128 + 8        # запись в таблице соединений IPVS + элемент хеш-таблицы [ipvs
Kconfig]
IPVS_NODE_GBPS = 10          # EL11-SSD-10GE: директор видит только входящий трафик (Direct
Routing)

# ---------------------------------------------------------------- 11. нормативы
JITSI_MBPS, JITSI_CPU, JITSI_CORES = 550.4, 0.203, 4   # [^74]
CORE_GBPS = JITSI_MBPS / (JITSI_CPU * JITSI_CORES) / 1000   # ≈ 0.678 Гбит/с на ядро при 100% CPU
CPU_TARGET = 0.7
SFU_CORES = 6                                          # EL11-SSD-10GE [^66]
SFU_NODE_GBPS = round(min(10.0, SFU_CORES * CORE_GBPS * CPU_TARGET), 3)   # 2.847

WS_POD_CONN = int(6e9 / WS_CONN_B)    # под 4 CPU / 8 GB, 6 GB под соединения
AUTH_POD_RPS, CP_POD_RPS, CHAT_POD_RPS = 1_000, 1_500, 5_000
NODE_CORES_USABLE = 12               # CL34-NVMe: 14 ядер, 2 под OC/kubelet
REDIS_SHARD_OPS = 120_000            # [^79]
SCYLLA_NODE_TB = 1.0                 # CL34-NVMe 2x1 TB, <= 50% заполнения под compaction

AUTH_RPS = R_LOGIN + R_CREATE + R_JOIN
CP_RPS = R_CREATE + R_JOIN + R_LEAVE + R_LIST + R_REC + R_REC_VIEW
```

```
CHAT_RPS = R_CHAT + R_HIST
RUNTIME_OPS = R_HB + R_JOIN + R_LEAVE + R_CREATE + R_CHAT + R_CHAT   # INCR chat_seq + SET NX
ключа идемпотентности
SESSION_OPS = (R_CREATE + R_JOIN + R_LIST + R_REC_VIEW) + R_LOGIN
CITUS_READS = (R_JOIN + R_LEAVE + R_REC) + R_JOIN + R_REC_VIEW   # meetings +
meeting_invite_links + recordings
CITUS_GB = MEET_META_GB + INVITES_GB + MEET_META_GB            # meetings + ссылки + recordings
(по 256 B)
USERS_READS = R_LOGIN + R_CREATE + R_JOIN

PRICE = {  # Selectel, цены на 11.10.2026 с НДС 22% [^66][^67][^69][^70]
    "k8s": 17_085.59, "el11": 14_000, "cl34": 13_600, "bl18": 34_900,
    "s3_gb": 0.89,
    "vcpu": 735.19, "ram_gb": 267.34, "ssd_gb": 10.48,
}
L7_NODE_PRICE = 24 * PRICE["vcpu"] + L7_NODE_RAM_GB * PRICE["ram_gb"] + 100 * PRICE["ssd_gb"]
USD = 84.90                    # курс ЦБ РФ на 10.10.2026
TRAFFIC_INCL_TB = 300          # ТБ/мес на сервер без ограничения полосы (порт > 1 Гбит/с) [^71]
TRAFFIC_STEP_TB, TRAFFIC_STEP_RUB = 3, 336   # увеличение лимита: шаг 3 ТБ (10 Мбит/с) за 336 ₽/
мес [^66]

# DR-пары для асинхронной реплики Citus (кто хранит реплику чьего кластера)
DR_HOST = {
    "DC-US-EAST": "DC-US-WEST", "DC-US-WEST": "DC-US-EAST", "DC-BR-SAOPAULO": "DC-US-EAST",
    "DC-UK-LONDON": "DC-DE-FRANKFURT", "DC-DE-FRANKFURT": "DC-UK-LONDON",
    "DC-IN-MUMBAI": "DC-SG-SINGAPORE", "DC-SG-SINGAPORE": "DC-JP-TOKYO",
    "DC-JP-TOKYO": "DC-SG-SINGAPORE",
}
AUTH_PRIMARY_DC = "DC-US-EAST"


def ceil(x):
    return int(math.ceil(x - 1e-9))


def per_dc():
    rows = {}
    for d, share in DC.items():
        s = share / 100
        r = {"share": share}
        r["conn"] = PEAK_ONLINE * s
        r["l7_rps"] = L7_RPS * s
        r["l7_tls"] = L7_NEW_TLS * s
        r["l7_gbps"] = L7_GBPS * s
        r["n_rps"] = ceil(r["l7_rps"] / L7_HTTPS_RPS)
        r["n_tls"] = ceil(r["l7_tls"] / L7_TLS_CPS)
        r["n_net"] = ceil(r["l7_gbps"] / L7_NET_GBPS)
        r["n_conn"] = ceil(r["conn"] / CONN_PER_L7)
        r["l7_base"] = max(r["n_rps"], r["n_tls"], r["n_net"], r["n_conn"])
        r["l7"] = r["l7_base"] + 1
        r["l7_x2"] = r["l7_base"] * 2
        r["ipvs_ram_gb"] = r["conn"] * IPVS_CONN_B / 1e9
        r["ipvs_base"] = max(1, ceil(r["l7_gbps"] / IPVS_NODE_GBPS))
        r["ipvs"] = r["ipvs_base"] + 1                # active + standby (keepalived, VRRP)
        r["sfu_gbps"] = SFU_PEAK * s
        r["sfu"] = ceil(r["sfu_gbps"] / SFU_NODE_GBPS) + 1
        r["rec_gbps"] = REC_STREAM_PEAK * s
        r["rec"] = ceil(r["rec_gbps"] / SFU_NODE_GBPS) + 1
        r["auth"] = ceil(AUTH_RPS * s / AUTH_POD_RPS) + 1
        r["cp"] = ceil(CP_RPS * s / CP_POD_RPS) + 1
        r["ws"] = ceil(r["conn"] / WS_POD_CONN) + 1
        r["chat"] = ceil(CHAT_RPS * s / CHAT_POD_RPS) + 1
```

```
        r["pgb"] = 3
        r["app_cpu"] = 4 * (r["auth"] + r["cp"] + r["ws"] + r["chat"]) + 2 * r["pgb"]
        r["app_ram"] = 8 * (r["auth"] + r["cp"] + r["ws"] + r["chat"]) + 2 * r["pgb"]
        r["app_nodes"] = ceil(r["app_cpu"] / NODE_CORES_USABLE) + 1
        r["runtime_ops"] = RUNTIME_OPS * s
        r["runtime_masters"] = max(3, ceil(r["runtime_ops"] / REDIS_SHARD_OPS))
        r["redis_nodes"] = r["runtime_masters"]  # перекрёстная схема: на узле master + replica
        r["auth_pg"] = 2 if d == AUTH_PRIMARY_DC else 1
        r["citus"] = 2 + sum(1 for v in DR_HOST.values() if v == d)
        r["scylla_meet_tb"] = 3 * (PARTICIPANTS_TB * s + USER_MEETINGS_TB)
        r["scylla_meet"] = max(3, ceil(r["scylla_meet_tb"] / SCYLLA_NODE_TB))
        r["scylla_chat_tb"] = 3 * CHAT_TB_30D * s
        r["scylla_chat"] = max(3, ceil(r["scylla_chat_tb"] / SCYLLA_NODE_TB))
        r["obs"] = 2 + (2 if d == AUTH_PRIMARY_DC else 0)
        r["s3_pb"] = REC_PB_30D * s
        rows[d] = r
    return rows


def fmt(x, nd=0):
    return f"{x:,.{nd}f}".replace(",", " ")


def b(x):
    return f"**{x}**"


def table(head, rows, align=None):
    align = align or ["---"] * len(head)
    out = ["| " + " | ".join(head) + " |", "| " + " | ".join(align) + " |"]
    out += ["| " + " | ".join(str(c) for c in r) + " |" for r in rows]
    return "\n".join(out)


def totals(p):
    keys = [k for k in next(iter(p.values())) if k != "share"]
    return {k: sum(r[k] for r in p.values()) for k in keys}


def servers_dc(r):
    return (r["sfu"] + r["rec"] + r["ipvs"] + r["l7"] + r["app_nodes"] + r["redis_nodes"] +
r["auth_pg"]
            + r["citus"] + r["scylla_meet"] + r["scylla_chat"] + r["obs"])


def cost_lines(tot, sfu_n=None, rec_n=None):
    n_dc = len(DC)
    return [
        ("Kubernetes-кластер/3 зоны доступности", n_dc, PRICE["k8s"]),
        ("L4 (IPVS + keepalived)", tot["ipvs"], PRICE["el11"]),
        ("Ingress L7 (NGINX)", tot["l7"], L7_NODE_PRICE),
        ("Kubernetes app pool", tot["app_nodes"], PRICE["cl34"]),
        ("Пул SFU (медиа)", tot["sfu"] if sfu_n is None else sfu_n, PRICE["el11"]),
        ("Воркеры записи", tot["rec"] if rec_n is None else rec_n, PRICE["el11"]),
        ("Redis (session + runtime)", tot["redis_nodes"], PRICE["cl34"]),
        ("PostgreSQL auth_pg", tot["auth_pg"], PRICE["bl18"]),
        ("PostgreSQL + Citus (meeting_pg + recording_pg)", tot["citus"], PRICE["bl18"]),
        ("ScyllaDB meeting_scylla", tot["scylla_meet"], PRICE["cl34"]),
        ("ScyllaDB chat_scylla", tot["scylla_chat"], PRICE["cl34"]),
        ("Объектное хранилище S3", REC_PB_30D * 1e6, PRICE["s3_gb"]),
        ("Наблюдаемость", tot["obs"], PRICE["cl34"]),
    ]
```

```
def main():
    p = per_dc()
    tot = totals(p)
    sh = {d: r["share"] / 100 for d, r in p.items()}
    total_servers = sum(servers_dc(r) for r in p.values())

    print("<!-- 2.2 хранилище на пользователя (на MAU за 30 дней) -->")
    per_user = [
        ("Профиль", 1, 256),
        ("Созданные встречи", MEETINGS_DAY * RETENTION_DAYS / MAU, 256),
        ("Ссылки-приглашения", MEETINGS_DAY * RETENTION_DAYS / MAU, 128),
        ("Участия во встречах", DMP * RETENTION_DAYS / MAU, 128),
        ("Строки списка встреч", DMP * RETENTION_DAYS / MAU, 128),
        ("Сообщения чата", CHAT_DAY * RETENTION_DAYS / MAU, CHAT_MSG_B),
        ("Записи встреч, worst-case", MEETINGS_DAY * RETENTION_DAYS / MAU, REC_SIZE_GB * 1e9),
    ]
    for name, n, bb in per_user:
        print(f"{name}: {n:.2f} шт × {bb:,.0f} B = {n*bb:,.0f} B".replace(",", " "))
    print(f"итого без записей: {sum(n*bb for nm, n, bb in per_user if 'Запис' not in nm)/1e3:.1f}
KB")

    print("\n<!-- 2.2 действий на DAU в сутки -->")
    for k_, v in EVENTS_DAY.items():
        print(f"{k_}: {v/DAU:.2f}")
    print(f"WS push (доставка другим участникам): {WS_PUSH_DAY/DAU:.1f}")

    print("\n<!-- 2.3 хранение: штук и объём -->")
    for blk, nm, n, bb in STORAGE:
        print(f"{blk} | {nm} | {fmt(n)} шт | {n*bb/1e9:,.2f} GB".replace(",", " "))
    print(f"TECH_TB={TECH_TB:.3f}  REC_SIZE_GB={REC_SIZE_GB:.4f}  REC_PB_DAY={REC_PB_DAY:.3f}
REC_PB_30D={REC_PB_30D:.2f}  "
          f"TOTAL_30D_PB={REC_PB_30D + TECH_TB/1000:.3f}")

    print("\n<!-- 2.4 RPS -->")
    names = {
        "POST /auth/login": "MF_LOGIN_RPS", "POST /meetings": "MF_CREATE_MEETING_RPS",
        "POST /meetings/{id}/join": "MF_JOIN_RPS", "POST /meetings/{id}/leave": "MF_LEAVE_RPS",
        "WS /presence/heartbeat": "MF_HEARTBEAT_RPS", "POST /meeting-chat/messages":
"MF_CHAT_RPS",
        "GET /meeting-chat/messages?after_seq=N": "MF_CHAT_HISTORY_RPS",
        "GET /users/{id}/meetings?limit=100": "MF_LIST_RPS", "POST /recordings/metadata":
"MF_RECORDING_META_RPS",
        "GET /recordings/{id}": "MF_RECORDING_VIEW_RPS",
    }
    for k_, v in EVENTS_DAY.items():
        print(f"| `{k_}` | `{names[k_]}` | {fmt(v)} | **{fmt(v/DAY)}** | **{fmt(RPS[k_])}** |")
    print(f"| WS push | `MF_WS_PUSH_RPS` | {fmt(WS_PUSH_DAY)} | **{fmt(WS_PUSH_DAY/DAY)}** | **
{fmt(WS_PUSH_DAY/DAY*K)}** |")
    print(f"MEDIA_GB_DAY={fmt(MEDIA_GB_DAY)}  MEDIA_PEAK={fmt(MEDIA_PEAK, 2)}  REC_STREAM_PEAK=
{fmt(REC_STREAM_PEAK, 1)}")

    print("\n<!-- 3.2 группы стран по ВВП -->")
    rows = []
    for g, v in GROUP_GDP.items():
        share = "—" if g == EXCLUDED_GROUP else f"{v / SERVED_GDP * 100:.2f}%"
        rows.append([g, fmt(v, 1), share])
    rows.append(["**Итого без Китая**", b(fmt(SERVED_GDP, 1)), "**100%**"])
    print(table(["Группа стран", "ВВП 2025, млрд $", "Доля"], rows, ["---", "---:", "---:"]))
    print(f"\nWEO world (сумма стран) = {sum(GDP.values()):.1f}")
    print("DC raw:", {d: round(v, 4) for d, v in DC_RAW.items()})
```

```
    print("DC rounded:", DC)
    for m, ds in MACRO.items():
        print(f"  {m}: {sum(DC[d] for d in ds):.2f}%")
    us_east_us = GROUP_GDP["США"] * (1 - US_WEST_POP) / SERVED_GDP * 100
    us_west_us = GROUP_GDP["США"] * US_WEST_POP / SERVED_GDP * 100
    print(f"  US east part={us_east_us:.2f}  US west part={us_west_us:.2f}  US total=
{GROUP_GDP['США']/SERVED_GDP*100:.2f}")
    print("  Zoom revenue: Americas 72.1, EMEA 15.8, APAC 12.1")

    print("\n<!-- 3.3 продуктовая нагрузка -->")
    rows = [[f"`{d}`", f"{r['share']:.2f}%", b(fmt(DAU*sh[d])), b(fmt(DMP*sh[d])),
b(fmt(MEETINGS_DAY*sh[d])),
            b(fmt(PEAK_ONLINE*sh[d]))] for d, r in p.items()]
    rows.append(["**Итого**", "**100%**", b(fmt(DAU)), b(fmt(DMP)), b(fmt(MEETINGS_DAY)),
b(fmt(PEAK_ONLINE))])
    print(table(["ДЦ", "Доля", "DAU", "Участий/сутки", "Встреч/сутки", "Peak online = WSS-
соединений"], rows,
                ["--", "---:", "--:", "------------:", "-----------:", "----------:"]))

    print("\n<!-- 3.3 пиковый RPS -->")
    keys = list(EVENTS_DAY)
    short = ["`login`", "`create`", "`join`", "`leave`", "`heartbeat`", "`chat`", "`chat
history`", "`list`",
             "`recording meta`", "`recording view`"]
    rows = [[f"`{d}`"] + [b(fmt(RPS[k_]*sh[d])) for k_ in keys] for d in p]
    rows.append(["**Итого**"] + [b(fmt(RPS[k_])) for k_ in keys])
    print(table(["ДЦ"] + short, rows, ["--"] + ["--:"] * len(keys)))

    print("\n<!-- 3.3 пиковый media bandwidth -->")
    rows = []
    for d in p:
        vals = [v * K * sh[d] for v in BW_AVG.values()]
        rows.append([f"`{d}`"] + [b(fmt(v, 1) + " Gbps") for v in vals] + [b(fmt(sum(vals), 1) +
" Gbps"),
                     fmt(REC_STREAM_PEAK*sh[d], 1) + " Gbps"])
    vals = [v * K for v in BW_AVG.values()]
    rows.append(["**Итого**"] + [b(fmt(v, 2) + " Gbps") for v in vals] + [b(fmt(sum(vals), 2) + "
Gbps"),
                 b(fmt(REC_STREAM_PEAK, 1) + " Gbps")])
    print(table(["ДЦ", "Video upload", "Video download", "Audio", "Screen sharing", "Media
суммарно", "SFU → запись"],
                rows, ["--"] + ["-----------:"] * 6))

    print("\n<!-- 3.3 записи -->")
    rows = [[f"`{d}`", b(f"{REC_PB_DAY*sh[d]:.3f} PB/сутки"), b(f"{REC_PB_30D*sh[d]:.2f} PB")]
for d in p]
    rows.append(["**Итого**", b(f"{REC_PB_DAY:.3f} PB/сутки"), b(f"{REC_PB_30D:.2f} PB")])
    print(table(["ДЦ", "Worst-case запись/сутки", "Worst-case запись за 30 дней"], rows, ["--",
"------:", "------:"]))

    print("\n<!-- 4.5 L7 формулы -->")
    print(f"REST={fmt(REST_RPS)} WS_IN={fmt(WS_IN)} WS_OUT={fmt(WS_OUT)} L7_RPS={fmt(L7_RPS)}
NEW_TLS={fmt(L7_NEW_TLS)} "
          f"GBPS={L7_GBPS:.2f} CONN={fmt(PEAK_ONLINE)} conn/node={fmt(CONN_PER_L7)}
({CONN_PER_L7/1e6:.3f} млн)")

    print("\n<!-- 4.5 L7 нагрузка -->")
    rows = [[f"`{d}`", b(fmt(r["l7_rps"])), fmt(r["l7_tls"]), f"{r['l7_gbps']:.2f}",
b(fmt(r["conn"]))] for d, r in p.items()]
    rows.append(["**Сумма по ДЦ**", b(fmt(tot["l7_rps"])), b(fmt(tot["l7_tls"])), b(f"
{tot['l7_gbps']:.2f}"), b(fmt(tot["conn"]))])
    print(table(["ДЦ", "L7 peak RPS", "Новых TLS/с", "L7 Gbps", "WSS-соединений"], rows, ["--",
```

```
"----------:", "----------:", "------:", "-------------:"]))
    print()
    rows = [[f"`{d}`", r["n_rps"], r["n_tls"], r["n_net"], r["n_conn"], b(r["l7"]), r["l7_x2"]]
for d, r in p.items()]
    rows.append(["**Сумма по ДЦ**", b(tot["n_rps"]), b(tot["n_tls"]), b(tot["n_net"]),
b(tot["n_conn"]), b(tot["l7"]), b(tot["l7_x2"])])
    print(table(["ДЦ", "`N_rps`", "`N_tls`", "`N_net`", "`N_conn`", "`N+1`", "`N*2`"], rows, ["--
"] + ["----:"] * 6))

    print("\n<!-- 4.6 L4 IPVS -->")
    rows = [[f"`{d}`", fmt(r["conn"]), f"{r['ipvs_ram_gb']:.2f} GB", f"{r['l7_gbps']:.2f}",
r["ipvs_base"], b(r["ipvs"])]
            for d, r in p.items()]
    rows.append(["**Сумма по ДЦ**", b(fmt(tot["conn"])), b(f"{tot['ipvs_ram_gb']:.2f} GB"), b(f"
{tot['l7_gbps']:.2f}"),
                 b(tot["ipvs_base"]), b(tot["ipvs"])])
    print(table(["ДЦ", "Соединений в таблице IPVS", "Память под таблицу", "Входящий трафик, Gbps
(не больше)", "`N`", "`N+1`"],
                rows, ["--", "--:", "--:", "--:", "--:", "--:"]))

    print("\n<!-- 5.2 / 6 ключевые нагрузки -->")
    print(f"users reads={fmt(USERS_READS)} writes<= {fmt(R_LOGIN)}")
    print(f"sessions reads={fmt(R_CREATE + R_JOIN + R_LIST + R_REC_VIEW)} writes={fmt(R_LOGIN)}
SESSION_OPS={fmt(SESSION_OPS)}")
    print(f"meetings reads={fmt(R_JOIN + R_LEAVE + R_REC)} writes={fmt(R_CREATE)}")
    print(f"participants reads={fmt(R_JOIN)} writes={fmt(R_JOIN + R_LEAVE)}")
    print(f"invites reads={fmt(R_JOIN)} writes={fmt(R_CREATE)}")
    print(f"user_meetings reads={fmt(R_LIST)} x100 writes={fmt(R_CREATE + R_JOIN + R_LEAVE)}")
    print(f"chat reads<= {fmt(R_HIST)} x100 writes={fmt(R_CHAT)}")
    print(f"recordings reads<= {fmt(R_REC_VIEW)} writes={fmt(R_REC)}")
    parts_peak = REC_STREAM_PEAK * 1e9 / 8 / REC_PART_B
    inflight_gb_peak = PEAK_MEETINGS_ONLINE * REC_SIZE_GB / 2
    inflight_parts_peak = inflight_gb_peak * 1e9 / REC_PART_B
    print(f"upload_buffer: PUT/s peak={fmt(parts_peak)} avg={fmt(parts_peak / K)}; in-flight
peak={inflight_gb_peak/1e3:.1f} TB, "
          f"parts={fmt(inflight_parts_peak)}; parts per recording=
{REC_SIZE_GB*1e9/REC_PART_B:.1f}")
    print(f"CITUS_READS={fmt(CITUS_READS)} CITUS_GB={CITUS_GB:.1f}")
    big = max(DC, key=DC.get)
    print(f"largest DC {big}: meetings rows={fmt(MEETINGS_DAY*RETENTION_DAYS*sh[big])} citus GB=
{CITUS_GB*sh[big]:.1f} "
          f"citus reads={fmt(CITUS_READS*sh[big])}")

    print("\n<!-- 6.6 нагрузка на строки -->")
    lines = [
        ("POST /auth/login", R_LOGIN, 2), ("POST /meetings", R_CREATE, 4), ("POST
/meetings/{id}/join", R_JOIN, 6),
        ("POST /meetings/{id}/leave", R_LEAVE, 3), ("WS /presence/heartbeat", R_HB, 1),
        ("POST /meeting-chat/messages", R_CHAT, 3), ("GET /meeting-chat/messages?after_seq=N",
R_HIST, 100),
        ("GET /users/{id}/meetings?limit=100", R_LIST, 100), ("POST /recordings/metadata", R_REC,
2),
        ("GET /recordings/{id}", R_REC_VIEW, 2),
    ]
    for nm, r_, n in lines:
        print(f"{nm}: {fmt(r_)} x {n} = {fmt(r_*n)}")

    print("\n<!-- 6.8 кеш -->")
    print(f"meetings_runtime: {PEAK_MEETINGS_ONLINE*512/1e9:.3f} GB, ops={fmt(RUNTIME_OPS)}")
    print(f"user_sessions: {SESSIONS_GB:.2f} GB, ops={fmt(SESSION_OPS)}")
    print(f"active_invite_tokens: {MEETINGS_DAY*128/1e9:.2f} GB, ops={fmt(R_JOIN)}")
    print(f"user_meetings_top100: {DAU*100*128/1e12:.3f} TB, rows/s={fmt(R_LIST*100)}")
```

```
    print(f"chat_last100: {PEAK_MEETINGS_ONLINE*100*CHAT_MSG_B/1e9:.2f} GB, rows/s<=
{fmt(R_HIST*100)}")
    print(f"chat_dedup: {R_CHAT*CHAT_DEDUP_TTL_SEC*100/1e9:.2f} GB (100 B x TTL
{CHAT_DEDUP_TTL_SEC}s), ops={fmt(R_CHAT)}")
    print(f"sessions per DC max: raw {SESSIONS_GB*sh[big]:.2f} GB, with replica
{SESSIONS_GB*sh[big]*2:.2f} GB")

    print("\n<!-- 9.2 отказ ДЦ -->")
    rec_pp = (V_DOWN + AUDIO) / MEET_SIZE
    full = V_UP + V_DOWN + AUDIO + SCREEN
    hq = V_UP_HQ + V_DOWN_HQ + AUDIO + SCREEN
    print(f"full={full:.2f} hq={hq:.2f} ratio={hq/full*100:.2f}%; with rec:
{(hq+rec_pp)/(full+rec_pp)*100:.2f}% "
          f"(rec per participant {rec_pp:.3f})")
    for d in [big, AUTH_PRIMARY_DC]:
        rest = 100 - DC[d]
        l7_left = sum(r["l7"] for dd, r in p.items() if dd != d)
        print(f"fail {d} ({DC[d]:.2f}%): remaining={rest:.2f}%  L7 nodes left={l7_left} -> "
              f"{l7_left*CONN_PER_L7/1e6:.1f} млн соединений vs peak {PEAK_ONLINE/1e6:.2f} млн")
    print("receivers for", big, ": UK, IN by order; capacity UK+IN =", DC["DC-UK-LONDON"] +
DC["DC-IN-MUMBAI"])

    print("\n<!-- 11.2 сводные -->")
    print(f"core={CORE_GBPS:.4f} Gbps; at {CPU_TARGET:.0%}: {CORE_GBPS*CPU_TARGET:.4f}; node=
{SFU_NODE_GBPS} Gbps; "
          f"SFU_PEAK={fmt(SFU_PEAK, 2)}; REC_STREAM_PEAK={fmt(REC_STREAM_PEAK, 1)}")
    print(f"AUTH_RPS={fmt(AUTH_RPS)} CP_RPS={fmt(CP_RPS)} CHAT_RPS={fmt(CHAT_RPS)} WS_POD_CONN=
{fmt(WS_POD_CONN)} "
          f"hb per pod={fmt(R_HB / PEAK_ONLINE * WS_POD_CONN)} RUNTIME_OPS={fmt(RUNTIME_OPS)}
SESSION_OPS={fmt(SESSION_OPS)}")
    print(f"pods total: auth={tot['auth']} cp={tot['cp']} ws={tot['ws']} chat={tot['chat']} pgb=
{tot['pgb']} "
          f"cpu={tot['app_cpu']} ram={tot['app_ram']} nodes={tot['app_nodes']}")

    print("\n<!-- 11.3 поды по ДЦ -->")
    rows = [[f"`{d}`", r["auth"], r["cp"], r["ws"], r["chat"], r["pgb"], r["app_cpu"],
r["app_nodes"]] for d, r in p.items()]
    rows.append(["**Итого**", b(tot["auth"]), b(tot["cp"]), b(tot["ws"]), b(tot["chat"]),
b(tot["pgb"]), b(tot["app_cpu"]), b(tot["app_nodes"])])
    print(table(["ДЦ", "Auth", "Control Plane", "WebSocket", "Chat", "PgBouncer", "CPU подов",
"Узлы app pool"], rows,
                ["--"] + ["--:"] * 7))

    print("\n<!-- 11.3 серверы по ДЦ -->")
    rows = []
    for d, r in p.items():
        rows.append([f"`{d}`", fmt(r["sfu"]), fmt(r["rec"]), r["ipvs"], r["l7"], r["app_nodes"],
r["redis_nodes"], r["auth_pg"],
                     r["citus"], r["scylla_meet"], r["scylla_chat"], r["obs"],
b(fmt(servers_dc(r)))])
    rows.append(["**Итого**", b(fmt(tot["sfu"])), b(fmt(tot["rec"])), b(tot["ipvs"]),
b(tot["l7"]), b(tot["app_nodes"]),
                 b(tot["redis_nodes"]), b(tot["auth_pg"]), b(tot["citus"]),
b(tot["scylla_meet"]), b(tot["scylla_chat"]),
                 b(tot["obs"]), b(fmt(total_servers))])
    print(table(["ДЦ", "SFU", "Запись", "L4", "L7", "App", "Redis", "auth_pg", "Citus", "Scylla
meeting", "Scylla chat",
                 "Набл.", "Всего"], rows, ["--"] + ["--:"] * 12))

    print("\n<!-- 11.3 Scylla объём -->")
    rows = [[f"`{d}`", f"{r['scylla_meet_tb']:.2f} TB", r["scylla_meet"], f"
{r['scylla_chat_tb']:.2f} TB", r["scylla_chat"]] for d, r in p.items()]
```

```
    rows.append(["**Итого**", b(f"{tot['scylla_meet_tb']:.2f} TB"), b(tot["scylla_meet"]), b(f"
{tot['scylla_chat_tb']:.2f} TB"), b(tot["scylla_chat"])])
    print(table(["ДЦ", "meeting_scylla, с RF", "Узлов", "chat_scylla, с RF", "Узлов"], rows, ["--
", "--:", "--:", "--:", "--:"]))

    print("\n<!-- 11.3 сводка по конфигурациям: CPU / RAM -->")
    cfg = [("L4 (IPVS)", tot["ipvs"], 6, 32), ("Ingress L7", tot["l7"], 24, 64), ("app pool",
tot["app_nodes"], 14, 64),
           ("SFU", tot["sfu"], 6, 32), ("rec", tot["rec"], 6, 32), ("Redis", tot["redis_nodes"],
14, 64),
           ("auth_pg", tot["auth_pg"], 16, 128), ("Citus", tot["citus"], 16, 128), ("Scylla
meet", tot["scylla_meet"], 14, 64),
           ("Scylla chat", tot["scylla_chat"], 14, 64), ("obs", tot["obs"], 14, 64)]
    for nm, n, c, ram in cfg:
        print(f"{nm}: {fmt(n)} x {c} CPU = {fmt(n*c)} CPU; RAM {fmt(n*ram)} GB")
    print(f"L7 Gbps per node = {L7_GBPS / tot['l7']:.2f}")

    print("\n<!-- 11.4 стоимость -->")
    total = 0
    for name, n, price in cost_lines(tot):
        total += n * price
        print(f"{name} | {fmt(n)} | {price:,.2f} | {n*price:,.2f}".replace(",", " "))
    print(f"L7_NODE_PRICE = {L7_NODE_PRICE:.2f}")
    print(f"ИТОГО {total:,.2f} ₽/мес ≈ ${total/USD:,.2f}".replace(",", " "))
    for name, n, price in cost_lines(tot):
        print(f"  share {name}: {n*price/total*100:.2f}%  ({n*price/1e6:.2f} млн)")
    s3 = REC_PB_30D * 1e6 * PRICE["s3_gb"]
    sfu = tot["sfu"] * PRICE["el11"]
    rec = tot["rec"] * PRICE["el11"]
    print(f"  structure: S3 {s3/1e6:.2f} ({s3/total*100:.1f}%), SFU {sfu/1e6:.2f}
({sfu/total*100:.1f}%), "
          f"rec {rec/1e6:.2f} ({rec/total*100:.1f}%), other {(total-s3-sfu-rec)/1e6:.2f}
({(total-s3-sfu-rec)/total*100:.1f}%)")
    old_sfu = sum(ceil(SFU_PEAK * s / 10) + 1 for s in sh.values())
    old_rec = sum(ceil(REC_STREAM_PEAK * s / 10) + 1 for s in sh.values())
    what_if = sum(n * pr for _, n, pr in cost_lines(tot, old_sfu, old_rec))
    print(f"what-if SFU 10 Gbps/узел без проверки CPU: SFU={fmt(old_sfu)} rec={fmt(old_rec)}
total={what_if/1e6:,.2f} млн "
          f"(разница {(total-what_if)/1e6:,.2f} млн)".replace(",", " "))

    print("\n<!-- 11.4 внешний трафик SFU (оценка по публичному тарифу) -->")
    media_tb_month = MEDIA_GB_DAY * 30 / 1000
    incl_tb = TRAFFIC_INCL_TB * tot["sfu"]
    extra_tb = max(0.0, media_tb_month - incl_tb)
    steps = ceil(extra_tb / TRAFFIC_STEP_TB)
    traffic_rub = steps * TRAFFIC_STEP_RUB
    print(f"media per month = {fmt(media_tb_month)} TB ({media_tb_month/1e6:.2f} EB); per SFU
node = {media_tb_month/tot['sfu']:.0f} TB")
    print(f"included = {fmt(incl_tb)} TB; extra = {fmt(extra_tb)} TB; steps = {fmt(steps)}; cost
= {traffic_rub/1e6:,.2f} млн ₽/мес")
    print(f"ИТОГО с трафиком {(total + traffic_rub)/1e6:,.2f} млн ₽/мес ≈ ${(total +
traffic_rub)/USD/1e6:,.2f} млн")
    print(f"servers total = {fmt(total_servers)}")


if __name__ == "__main__":
    main()
```